

DOMAINS

GORDON PLOTKIN

Department of Computer Science

University of Edinburgh

1983

CONTENTS

Notes to the $\text{\TeX}$ Edition and Acknowledgements	ii
1. Complete Partial Orders and Continuous Functions	1
Exercises	8
2. Products and Function Spaces	11
Exercises	21
3. Other Constructions	25
Exercises	32
4. The Category of CPOs Considered as a CPO Itself	35
Exercises	44
5. Solving Recursive Domain Equations	48
6. Algebraic Domains	59
Continuous Functions	61
Completion by Ideals	61
Completion	61
Completeness	67
Function Spaces	68
7. Computability	70
Computability	71
Computable Functions	73
Least Fixed-Points	74
Products	75
Function Spaces	76
Embeddings	78
Direct Limits	79
Algebras	81
8. Nondeterminism and Parallelism	82
Discrete Powerdomains	82
The Relational View	82
The Egli-Milner Order	84
The Smyth Order	87
Predicate Transformers	87
General Powerdomains	88
How to Order Subset of $\mathbf{D}$	88
Denotational Semantics	92
Synchronisation Methods	94
Communicating Processes	95
General Powerdomains: The Plotkin Powerdomain	98
Finitely Generable Sets	101
How to Order Our Sets	102
Canonical Representatives and Scott-Compact Sets	105
The Lawson Topology	106
General Powerdomains: The Smyth Powerdomain	109
Duality	111
The Vietoris Topology	113
Exercises	114

NOTES TO THE T_EX EDITION AND ACKNOWLEDGEMENTS

This is a T_EX*-ed edition of the course notes “Domains” by Professor Gordon Plotkin and the copyright of this edition belongs to him⁺.

This edition was prepared by Yugo Kashiwagi[†] and Hidetaka Kondoh[‡] using Prof. Donald Knuth’s plain T_EX with some macros (especially for the picture environment) from Dr. Leslie Lamport’s L^AT_EX macro package after some modifications for use in plain T_EX and Prof. Michael Barr’s macro package for category theoretic diagrams. Misprints in the original notes were corrected without notices, and some notations were changed uniformly due to typographical difficulties imposed by T_EX.

The editors of this edition wish to express their deepest thanks to Dr. Tatsuya Hagino for lending them his copy of the original Prof. Plotkin’s course notes so that they could photocopy it, without which the present edition could never be produced, and his memos in his copy were very helpful in correcting misprints. They also thank to Prof. Michael Barr for e-mailing one of them (HK) his diagram macro packages, which enable them to typeset many diagrams found in the present notes.

The present edition was typeset as carefully as possible. The editors, however, are afraid that there may be some misprints not appeared in the original course notes.

Therefore comments and information on misprints and/or typographical problems/advice are welcome. They are preferably notified by e-mail to one of the editors (HK[‡]).

June, 1992 HK

After the first version of this T_EX edition was released, the editors have received more than 170 direct requests to this edition, and they have been very much encouraged by these so many responses. The editors wish to deeply thank to the readers for their encouragements. When the editors prepared the first version, regrettably however, they had no spell-checker available at hand, hence there remained so many misspellings in the first version. They sincerely apologise the readers as well as Prof. Plotkin for the low quality of the first version.

In the present second version many misspellings as well as several misprints in mathematical formulae are corrected. In this new version, the editors have changed a few typographical conventions in order to increase the typographical consistency, have changed the original spellings in the first version according to the direction of the original author, Prof. Plotkin, have recovered an original notation which seemed to be difficult for them to typeset in T_EX due to their illiteracy about T_EX when they prepared the first version of this T_EX edition, and have made use of an $\mathcal{AMS}$ -font to get better output for a family of symbols appeared in Chapter 8.

First of all, the editors wish to express their deepest thanks to Prof. Plotkin for his warm encouragements and his generous permission to the distribution of this T_EX edition. The editors also wish to acknowledge (in alphabetical order) Yutaka Kikuchi at Tokyo Institute of Technology, Ralf Treinen at Univ. des Saarlandes, and David Wald at Massachusetts Institute of Technology for supporting ftp services of the T_EX edition, which substantially decreased the editors’ efforts required for the distribution, and without their supports, the number of people accessible to this T_EX edition would be significantly less than as it is, and David Wald also have repositioned the dvi file of the T_EX edition in A4 size so that it is appropriate for US letter-size papers.

Last but not least, the editors wish to sincerely thank Professor John C. Mitchell, who informed them that there remained many misspellings in the first version and strongly advised them to use some spell checker, and also have kindly spent his important time to pass the TeX source files to a spell-checker and told them the results with the original author’s comments and directions concerning misspellings and several original (British) spellings in the notes, and Marcelo Fiore at Univ. of Edinburgh and Daisuke Suzuki at Univ. of Tokyo for notifying them of many misprints.

November, 1992 HK

* T_EX is a trademark of the American Mathematical Society.

+ © Gordon D. Plotkin, 1983.

† kashiwagi@hcr1gw.hitachi.co.jp

‡ kondoh@har1.hitachi.co.jp

1. Complete Partial Orders and Continuous Functions

It is possible to carry out a good deal of denotational semantics without invoking any mathematical apparatus other than sets, simple constructions on them and ordinary functions over them. We illustrate this with an extended example and then see what problems arise to force us out of the comfortable world of sets. Please note that we are assuming a standard approach to denotational semantics in which we ascribe meanings to syntactical objects, following the syntactic structure of these objects. The meanings are elements of some abstract domain, which is often a set. In computer science, however, it seems that sets are not enough. First, here is a simple imperative language (or, rather, a family of such languages) called *Imp*.

Syntax. There are three sets of syntactic items.

1. **BExp** — a given set of *boolean expressions*, ranged over by the metavariable, b .
2. **Act** — a given set of primitive *actions*, ranged over by the metavariable, a .
3. **Stat** — a set of *statements*, ranged over by the metavariable, s and given in terms of **BExp** and **Act** by the grammar:

$$s ::= a \mid \mathbf{dummy} \mid (s; s) \mid (\mathbf{if } b \mathbf{ then } s \mathbf{ else } s).$$

Semantic Domains. There are three semantic domains used to provide denotations for syntactic items.

1. **T** — the set, $\{tt, ff\}$, of truth-values, ranged over by t .
2. **S** — a given set of *states*, ranged over by σ .
3. **C** = $(\mathbf{S} \rightarrow \mathbf{S})$ — the set of *commands*, ranged over by θ .

Denotations. There are three denotational functions, one for each syntactic class.

1. $\mathcal{B}: \mathbf{BExp} \rightarrow (\mathbf{S} \rightarrow \mathbf{T})$ — a given denotational function.
2. $\mathcal{A}: \mathbf{Act} \rightarrow \mathbf{C}$ — a given denotational function.
3. $\mathcal{C}: \mathbf{Stat} \rightarrow \mathbf{S}$ — defined by structural induction on statements by:

- (i) $\mathcal{C}[a](\sigma) = \mathcal{A}[a](\sigma);$
- (ii) $\mathcal{C}[\mathbf{dummy}](\sigma) = \sigma;$
- (iii) $\mathcal{C}[s_1; s_2](\sigma) = \mathcal{C}[s_2](\mathcal{C}[s_1](\sigma));$
- (iv) $\mathcal{C}[(\mathbf{if } b \mathbf{ then } s_1 \mathbf{ else } s_2)](\sigma) = \mathit{Cond}(\mathcal{B}[b](\sigma), \mathcal{C}[s_1](\sigma), \mathcal{C}[s_2](\sigma))$

where the function $\mathit{Cond}: \mathbf{T} \times \mathbf{S} \times \mathbf{S} \rightarrow \mathbf{S}$ is given by:

$$\mathit{Cond}(t, \sigma, \sigma') = \begin{cases} \sigma & (\text{if } t = tt), \\ \sigma' & (\text{if } t = ff). \end{cases}$$

The reader will note how these definitions follow his computational intuitions. It is not our intention to follow up these connections in detail here, although the connection between denotational and operational semantics is of course an important topic. However, we shall rely heavily on our computational intuitions when formulating semantics. It could even be argued that we should concentrate first on an analysis of computability and then describe (or even design) programming languages accordingly. This is advice we will not take: existing languages are used to develop our ideas on computability in the first instance and so we begin with them.

The next example is a simple expressional language (or rather family of such languages) called *Dec*.

Syntax.

1. **Id** is a given set of *identifiers* ranged over by x .
2. **Fun** $_n$ is a given set of *function symbols* ranged over by f_n ($n = 1, 2, \dots$).
3. **Op** $_n$ is a given set of *operations* ranged over by o_n ($n = 1, 2, \dots$).
4. **BExp** is a given set of *boolean expressions* ranged over by b .
5. **Exp** is a set of *expressions* ranged over by e and given by:

$$e ::= x \mid o_0() \mid o_1(e) \mid o_2(e, e) \mid \dots \mid (\mathbf{if } b \mathbf{ then } e \mathbf{ else } e) \mid (\mathbf{let } x \mathbf{ be } e \mathbf{ in } e) \mid (\mathbf{let } f_1(x) \mathbf{ be } e \mathbf{ in } e) \mid \dots$$

Semantic Domains.

1. **V** is a given set of *values* ranged over by v .
2. **IEnv** = $\mathbf{V}^{\mathbf{Id}}$ is the set of *identifier environments* ranged over by ρ .
3. **FEnv** = $\prod_{n \geq 1} (\mathbf{V}^n \rightarrow \mathbf{V})^{\mathbf{Fun}_n}$ is the set of *function environments*, ranged over by φ .

Denotations.

1. $\mathcal{I}_n: \mathbf{Op}_n \rightarrow (\mathbf{V}^n \rightarrow \mathbf{V})$ is given.
2. $\mathcal{B}: \mathbf{BExp} \rightarrow (\mathbf{IEnv} \times \mathbf{FEnv} \rightarrow \mathbf{T})$ is given.
3. $\mathcal{E}: \mathbf{Exp} \rightarrow (\mathbf{IEnv} \times \mathbf{FEnv} \rightarrow \mathbf{V})$ is given by structural induction:

- (i) $\mathcal{E}[\![x]\!](\rho, \varphi) = \rho[\![x]\!]$;
- (ii)_n $\mathcal{E}[\![o_n(e_1, \dots, e_n)]\!](\rho, \varphi) = \mathcal{I}_n[\![o_n]\!](\mathcal{E}[\![e_1]\!](\rho, \varphi), \dots, \mathcal{E}[\![e_n]\!](\rho, \varphi))$;
- (iii) $\mathcal{E}[\![\text{if } b \text{ then } e_1 \text{ else } e_2]\!](\rho, \varphi) = \text{Cond}(\mathcal{B}[\![b]\!](\rho, \varphi), \mathcal{E}[\![e_1]\!](\rho, \varphi), \mathcal{E}[\![e_2]\!](\rho, \varphi))$;
- (iv) $\mathcal{E}[\![\text{let } x \text{ be } e_1 \text{ in } e_2]\!](\rho, \varphi) = \mathcal{E}[\![e_2]\!](\rho[\mathcal{E}[\![e_1]\!](\rho, \varphi)/x], \varphi)$

where $\cdot[\cdot/\cdot]: \mathbf{IEnv} \times \mathbf{V} \times \mathbf{Id} \rightarrow \mathbf{IEnv}$ is given by:

$$\rho[v/x][\![x']\!] = \begin{cases} v & (\text{if } x = x'), \\ \rho[\![x']\!] & (\text{if } x \neq x'); \end{cases}$$

- (v)_n $\mathcal{E}[\![\text{let } f_n(x_1, \dots, x_n) \text{ be } e_1 \text{ in } e_2]\!](\rho, \varphi) =$
 $\mathcal{E}[\![e_2]\!](\rho, \varphi[\lambda v_1 \in \mathbf{V}, \dots, v_n \in \mathbf{V}. \mathcal{E}[\![e_1]\!](\rho[v_1/x_1] \cdots [v_n/x_n], \varphi)/f_n])$

where $\cdot[\cdot/\cdot]: \mathbf{FEnv} \times (\mathbf{V}^n \rightarrow \mathbf{V}) \times \mathbf{Fun}_n \rightarrow \mathbf{FEnv}$ is given by:

$$(\varphi[g/f_n])_{n'}[\![f'_{n'}]\!] = \begin{cases} g & (\text{if } n' = n \text{ and } f'_{n'} = f_n), \\ \varphi_{n'}[\![f'_{n'}]\!] & (\text{otherwise}). \end{cases}$$

Both languages can be put together by giving **Act** by the little grammar: $a ::= (x := e)$, then giving **S** as $\mathbf{IEnv} \times \mathbf{FEnv}$ and giving **A** by:

$$\mathcal{A}[\![x := e]\!](\rho, \varphi) = (\rho[\mathcal{E}[\![e]\!](\rho, \varphi)/x], \varphi).$$

Notice the various methods we have used. First we pick out various sets either as being somehow given (parameters) or given by some syntactic means (and we are not worrying about details of syntax) or given by constructions such as Cartesian product, or powers, or sets of functions. Next we then pick out some denotational functions over these sets either as given or else as defined by ordinary mathematical expression (if you allow the typed λ -calculus). When we use more complicated objects than sets, we will have to consider these methods anew.

To see that sets are not enough and that instead we need “partial” or “incomplete” entities as well as the usual “total” ones, suppose we added a recursive declaration facility to *Dec* as for example:

$$e ::= (\text{letrec } f(x) \text{ be } e \text{ in } e).$$

So for example if $\mathbf{V} = \mathbf{N}$, the set of integers, and $\mathbf{Op}$, includes the successor function, $(\cdot + 1)$ then the intention of the expression **letrec** $f(x)$ **be** $f(x) + 1$ **in** $f(0)$ is to declare f to be a function $g: \mathbf{N} \rightarrow \mathbf{N}$ such that $g(n) = g(n) + 1$ for all integers n and then to give as result $g(0)$. (We feel free to use infix and other notations rather than the strict function applications considered above.) However there is no such function as g and computational intuition suggests that the expression does not terminate and indeed after such a declaration any expression of the form $f(e)$ will not terminate. So we expect that the value of the expression is undefined and we will have to use partial functions on the integers.

Again iteration will, like recursion, introduce non-termination. Suppose we extend *Imp* by adding the following clause to the definition of **Stat**:

$$s ::= (\text{while } b \text{ do } n).$$

Then we expect the meaning of **while** b **do** s to be the same as that of

$$\text{if } b \text{ then } s; (\text{while } b \text{ do } s) \text{ else dummy.}$$

So if this meaning is θ and that of b is $p: \mathbf{S} \rightarrow \mathbf{T}$ and that of s is $\theta': \mathbf{S} \rightarrow \mathbf{S}$ we expect that:

$$\theta(\sigma) = \text{Cond}(p(\sigma), \theta(\theta'(\sigma)), \sigma) \tag{1}$$

and if, for example b is **true**, the constantly true Boolean expression, and s is **dummy** then we have:

$$\theta(\sigma) = \theta(\sigma)$$

and θ can be any function at all; but again computational intuition suggests it ought to be always undefined.

As is well-known, Scott introduced an element, $\perp$, into the value domain, $\mathbf{V}$, rather than worrying about partial functions and undefined results: to coin a phrase, he objectifies (some of) the metalanguage. This is particularly appropriate as it is possible for a subexpression to be undefined but for the whole expression to have a value, as in:

$$e = (\text{letrec } f(x) \text{ be } f(x) + 1 \text{ in } (\text{let } g(x) \text{ be } 0 \text{ in } g(f(0))))$$

which under Algol's copy rule (call-by-name) has value 0 although $f(0)$ would have been undefined. So we want to consider partiality in the domain as well as the codomain. So in the integer example we will use $\mathbf{N}_\perp$ ($\stackrel{\text{def}}{=} \mathbf{N} \cup \{\perp\}$) and define $\perp + 1 = \perp$ in accord with computational intuition and then the equation $g(n) = g(n) + 1$ has the (unique) solution $g(n) = \perp$. But having introduced $\perp$ we cannot allow arbitrary functions $w: \mathbf{N}_\perp \rightarrow \mathbf{N}_\perp$ as then we would just get the same problems over again. Once more, computational intuition comes to the rescue and we expect that if $w(\perp) \neq \perp$ then $w(\perp) = w(n)$ for all integers, n . This can be put much better: since $\perp$ is undefined it is less defined than anything else and we introduce a *partial order*, $\sqsubseteq$, into $\mathbf{N}_\perp$ by:

$$x \sqsubseteq y \stackrel{\text{def}}{=} (x = \perp) \vee (x = y).$$

Then the condition on w is just that it is *monotonic* i.e. that if $x \sqsubseteq y$ then $w(x) \sqsubseteq w(y)$. Here is a picture of $\mathbf{N}_\perp$:

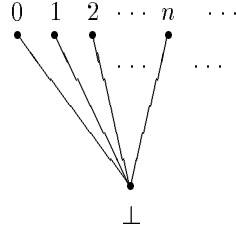


Figure I: The integer domain, $\mathbf{N}_\perp$.

Similarly we would introduce $\mathbf{S}_\perp$ and then expect that $\theta(\sigma) = \perp$, always; we also introduce $\mathbf{T}_\perp$ which looks like:

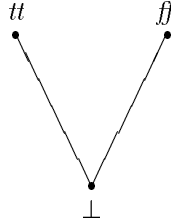


Figure II: The truth-value domain, $\mathbf{T}_\perp$.

These simple partial orders are called *flat* or *discrete* being characterised by the fact that if $x \sqsubseteq y \sqsubseteq z$ then $x = y$ or $y = z$. More complicated ones arise if we consider functions of several arguments such as $\text{Cond}: \mathbf{T}_\perp \times \mathbf{S}_\perp \times \mathbf{S}_\perp \rightarrow \mathbf{S}_\perp$. Again computational experience suggests that Cond is to be defined by: $\text{Cond}(\perp, \sigma, \sigma') = \perp$, $\text{Cond}(tt, \sigma, \sigma') = \sigma$ and $\text{Cond}(ff, \sigma, \sigma') = \sigma'$. So we expect that Cond — and all other computable functions of several arguments — will be monotonic in each of their arguments. We can rephrase this by defining the partial order on the Cartesian product to be the coordinatewise one:

$$\langle t, \sigma_0, \sigma_1 \rangle \sqsubseteq \langle t', \sigma'_0, \sigma'_1 \rangle \stackrel{\text{def}}{=} (t \sqsubseteq t') \wedge (\sigma_0 \sqsubseteq \sigma'_0) \wedge (\sigma_1 \sqsubseteq \sigma'_1)$$

and then noting that monotonicity in each argument is exactly monotonicity on the newly defined partial order. Here is an “aerial” picture of $\mathbf{T}_\perp \times \mathbf{T}_\perp$ which shows that the orders are no longer flat:

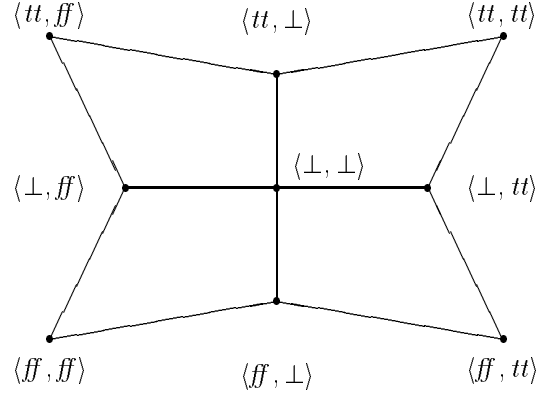


Figure III.

Thus we are led to the general idea of considering our domains to be partial orders rather than sets: the relation $x \sqsubseteq y$ means that the abstract object, x , viewed as the result of a computation (or more generally when involved in one) contains less information than y does. This general idea makes it reasonably clear that $\sqsubseteq$ is a partial order: reflexivity and transitivity are obvious and the point of antisymmetry is that if two elements contain exactly the same amount of information then they have the same uses and cannot be told apart — so we identify them. (It does not make much difference to the theory if we drop antisymmetry.) What is more there is always a least element, $\perp$, which corresponds to no information at all. So we have the following thesis (axiom) for all domains where we will take our denotations to lie:

Thesis 1. *All domains are partial orders with a least element.*

Our experience with functions now suggests a thesis about the computable functions over domains:

Thesis 2. *All computable functions are monotonic.*

This is the reasonable requirement that the amount of information in the output of a function grows as we increase the amount of information in the input.

Of course the above discussion is not a knockdown argument and many other structures than partial orders might fit the bill just as well. Let us look for further examples. Returning to equation (1) we see that it can be rewritten as:

$$\theta = F(\theta)$$

where the second-order functional, $F: (\mathbf{S}_\perp \rightarrow_{\mathbf{M}} \mathbf{S}_\perp) \rightarrow (\mathbf{S}_\perp \rightarrow_{\mathbf{M}} \mathbf{S}_\perp)$ is defined by:

$$F(\theta)(\sigma) = \text{Cond}(p(\sigma), \theta(\theta'(\sigma)), \sigma).$$

Here $\mathbf{S}_\perp \rightarrow_{\mathbf{M}} \mathbf{S}_\perp$ is the set of all monotonic functions from $\mathbf{S}_\perp$ to $\mathbf{S}_\perp$. Similarly recursive function declarations can be read as needing the fixed-point of a second-order functional, of a slightly different type. Now arbitrary functionals do not always have fixed-points and we return again to our computational intuitions. If $F(\theta)(\sigma)$ is defined ($\sigma \in \mathbf{S}$) then in the corresponding computation finitely many (one!) values of θ will be needed (namely the value at $\theta'(\sigma)$). So if we replace θ by another function $\bar{\theta}$ which always returns a result with more information, for any given argument, $F(\bar{\theta})(\sigma)$ will be defined too, and have the same value. This observation is just that F is monotonic, if we define the natural partial order on $\mathbf{S} \rightarrow_{\mathbf{M}} \mathbf{S}$ by:

$$\theta \sqsubseteq \bar{\theta} \equiv \forall \sigma \in \mathbf{S}_\perp. \theta(\sigma) \sqsubseteq \bar{\theta}(\sigma).$$

So we have another example, $\mathbf{S}_\perp \rightarrow_{\mathbf{M}} \mathbf{S}_\perp$ of a partial order and another example, $F: (\mathbf{S}_\perp \rightarrow_{\mathbf{M}} \mathbf{S}_\perp) \rightarrow_{\mathbf{M}} (\mathbf{S}_\perp \rightarrow_{\mathbf{M}} \mathbf{S}_\perp)$ of a monotonic function. However we have now hit a second obstacle: not all monotonic functions have fixed-points. The reader is invited to follow out the corresponding considerations on recursive function declarations where he will encounter exactly the same difficulty.

Now while $\mathbf{S}_\perp$ is a simple partial order, $(\mathbf{S}_\perp \rightarrow_M \mathbf{S}_\perp)$ is not, as it contains nontrivial countably infinite increasing chains if $\mathbf{S}$ is infinite. All these chains have least upper bounds (lubs) defined pointwise by: $(\bigsqcup_n \theta_n)(\sigma) = \bigsqcup_n \theta_n(\sigma)$. Furthermore F preserves the lubs of such increasing chains. Let $\langle \theta_n \rangle_{n \in \omega}$ be an increasing chain in $(\mathbf{S}_\perp \rightarrow_M \mathbf{S}_\perp)$. Then $F(\bigsqcup_n \theta_n)(\sigma) = \sigma'$ where σ and σ' are in $\mathbf{S}$. Now the value of $F(\bigsqcup_n \theta_n)(\sigma)$ depends, as remarked above, on finitely many values of $(\bigsqcup_n \theta_n)$ which therefore must also be given by one of the θ_n and then $F(\theta_n)(\sigma) = \sigma'$ for that θ_n showing $(\bigsqcup_n F(\theta_n))(\sigma) = \sigma'$. Thus we have proved the reverse inclusion (the other cases are trivial), albeit using a rather informal argument.

These facts make it easy to show that F has a fixed-point. For the sequence $\langle F^n(\perp) \rangle_{n \in \omega}$ is increasing as is easily shown by induction using the monotonicity of F . Let its lub be fix_F . Then:

$$F(fix_F) = F(\bigsqcup_n F^n(\perp)) = \bigsqcup_n F^{n+1}(\perp) = fix_F.$$

As is well-known, fix_F is the *least* fixed-point of F for if θ is any other one one easily shows by induction that $F^n(\perp) \sqsubseteq \theta$. Intuitively this minimality corresponds to the computational situation where we only expect θ to be defined when that can be proved — forced to be true by virtue of the computation rules. Indeed let us consider the iterates, $F^n(\perp)$. Writing them using the definition of F we find:

$$\begin{aligned} F^0(\perp)(\sigma) &= \perp, \\ F^1(\perp)(\sigma) &= Cond(p(\sigma), \perp, \sigma), \\ F^2(\perp)(\sigma) &= Cond(p(\sigma), Cond(p(\theta'(\sigma)), \perp, \theta'(\sigma)), \sigma), \\ F^3(\perp)(\sigma) &= Cond(p(\sigma), Cond(p(\theta'(\sigma)), Cond(p(\theta'(\theta'(\sigma))), \perp, \theta'(\theta'(\sigma))), \theta'(\sigma)), \sigma). \end{aligned}$$

So $F^n(\perp)$ corresponds to going around the loop up to n times at most and so it does seem that fix_F is the right fixed-point to choose. Essentially the same considerations apply to recursive declaration of functions.

All this suggests that we ought to consider domains which are partial orders with a least element which have lubs of increasing ω -chains; these partial orders are called *complete partial orders* (cpo):

Thesis 3. *Every domain is a cpo.*

What is more we ought to consider only these monotonic functions between cpo which preserve the lubs of increasing ω -chains; these functions are called *continuous* functions:

Thesis 4. *Every computable function is continuous.*

It turns out that we can do a great deal just with these two simple notions of cpo and of the continuous functions between them. We study them both together as a *category*:

Definition 1. The category, $\mathcal{CP}\mathcal{O}$ has as objects the cpo and as morphisms the continuous functions between them. Composition of morphisms is ordinary function composition.

One easily checks that the identity function, $id_{\mathbf{D}}: \mathbf{D} \rightarrow \mathbf{D}$ is continuous and that the composition $\mathbf{D} \xrightarrow{f} \mathbf{E} \xrightarrow{g} \mathbf{F}$ of two continuous functions is continuous, showing that $\mathcal{CP}\mathcal{O}$ is indeed a category.

Note that all flat partial orders are cpo (as is any partial order, such as $\mathbf{T}_\perp \times \mathbf{T}_\perp$, with a least element which only has eventually constant increasing ω -chains); further all monotonic functions over such partial orders are continuous.

The above considerations on fixed-point are quite general. Let $\mathbf{D} \xrightarrow{f} \mathbf{D}$ be a continuous function. A *prefixed-point* of f is an element d of $\mathbf{D}$ such that $f(d) \sqsubseteq d$.

Theorem 1 The Fixed-point Theorem. *Let $\mathbf{D}$ be a cpo and let $f: \mathbf{D} \rightarrow \mathbf{D}$ be a continuous function. Then $fix_f \stackrel{\text{def}}{=} \bigsqcup_n f^n(\perp_{\mathbf{D}})$ is the least prefixed-point of f .*

Proof. This is a straightforward generalisation of the particular case considered in the text and is left to the reader. ■

The applications of this theorem are so widespread and important in the present approach to the denotational semantics of programming languages, that it is sometimes called the fixed-point approach. Let us now consider cpo and continuous functions at the general level of information content, before giving further examples. In a cpo $\mathbf{D}$, the lub, $\bigsqcup_{\mathbf{D}} d_n$ of an increasing chain, $\langle d_n \rangle_{n \in \omega}$ of elements of $\mathbf{D}$ is just all the information in the d_n . To make it feasible that such lubs exist we consider a certain idea of finite information which is implicit in some of the above discussion, and which will be studied more thoroughly in the chapter on algebraicity.

We suppose that some elements in $\mathbf{D}$ are finite, that is, contain only a finite amount of information, while the others are infinite. For example in this sense every element of $\mathbf{N}_\perp$ (or any other flat cpo) is finite while the finite elements of, say, $\mathbf{N}_\perp \rightarrow_{\mathbf{M}} \mathbf{T}_\perp$ are those monotonic functions which have value $\perp$ almost everywhere. Now consider the following axioms on finiteness for a partial order, $\mathbf{D}$, with least element $\perp$.

1. Let d be finite and suppose $\langle x_n \rangle$ is an increasing ω -chain whose lub, $\bigsqcup_{\mathbf{D}} x_n$, exists. Then if $d \sqsubseteq \bigsqcup_{\mathbf{D}} x_n$ it follows that for some n , $d \sqsubseteq x_n$.
2. Each infinite element, x , is the lub of an increasing ω -chain of finite elements.
3. Every increasing ω -chain of finite elements has a lub in $\mathbf{D}$.

We believe Axioms 1 and 2 are evident, Axiom 3 says that all “potential” infinities are allowed. It is a consequence of these axioms that $\mathbf{D}$ is a cpo. For let $\langle x_n \rangle$ be an increasing ω -chain in $\mathbf{D}$ and suppose that, by Axiom 2, for every n , $\langle e_{nm} \rangle_{m \in \omega}$ is an increasing chain of finite elements of $\mathbf{D}$ with lub x_n . By Axiom 1 for each e_{nm} and $n' \geq n$, as $e_{nm} \sqsubseteq x_n \sqsubseteq x_{n'} = \bigsqcup_{m'} e_{n'm'}$, there is an $e_{n'm'}$ so that $e_{nm} \sqsubseteq e_{n'm'}$. It follows that we can find an increasing chain $\langle e_n \rangle$ of finite elements such that each e_n is some e_{nm} , e_0 is e_{00} , and for all $n' < n$, $m' < n$, $e_{n'm'} \sqsubseteq e_n$. Then by Axiom 3, $\bigsqcup_{\mathbf{D}} e_n$ exists and is easily seen to be $\bigsqcup_{\mathbf{D}} x_n$.

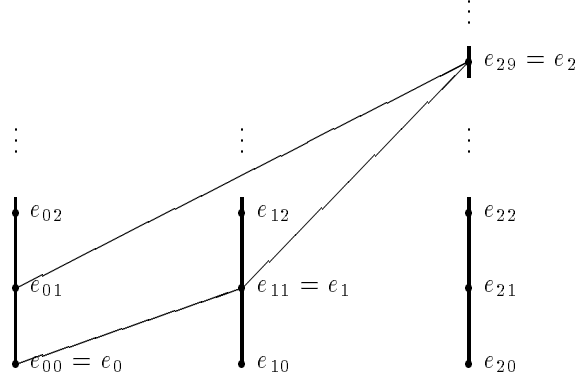


Figure IV: Construction of the chain $\langle e_n \rangle$.

Similarly we can give a general version of the above argument for the continuity of the second-order functional, F . Suppose $\mathbf{D}$ and $\mathbf{E}$ obey the above axioms on finiteness and $f: \mathbf{D} \rightarrow \mathbf{E}$ is monotonic. Then f is continuous iff wherever $e \sqsubseteq f(x)$ and e is finite there is a finite d such that $d \sqsubseteq x$ and $e \sqsubseteq f(d)$. What this condition says is that any finite amount of information in the output only depends on a finite amount of information in the input. Suppose the condition holds and $\langle d_n \rangle$ is an increasing chain of elements in $\mathbf{D}$. Then by monotonicity $f(\bigsqcup_n d_n) \sqsupseteq \bigsqcup_n f(d_n)$. Now if $e \sqsubseteq f(\bigsqcup_n d_n)$ where e is finite, it follows by the condition that there is a finite d in $\mathbf{D}$ such that $d \sqsubseteq \bigsqcup_n d_n$ and $e \sqsubseteq f(d)$. But then by Axiom 1, $d \sqsubseteq d_n$ for some n and so $e \sqsubseteq \bigsqcup_n f(d_n)$. Now by Axiom 2 $f(\bigsqcup_n d_n) = \bigsqcup_n e_n$, where $\langle e_n \rangle$ is an increasing ω -chain of finite elements of $\mathbf{E}$. So by the above reasoning $e_n \sqsubseteq \bigsqcup_n f(d_n)$ for each n and so $f(\bigsqcup_n d_n) \sqsubseteq \bigsqcup_n f(d_n)$ which completes the proof that f is continuous. Conversely suppose f is continuous and $e \sqsubseteq f(x)$ where $\langle d_n \rangle$ is an increasing ω -chain of finite elements of $\mathbf{D}$ and so $e \sqsubseteq f(x) = f(\bigsqcup_n d_n) = \bigsqcup_n f(d_n)$ (by continuity) and so, by Axiom 1, $e \sqsubseteq f(d_n)$ for some n , which establishes the condition. (Note that Axiom 3 was not used in this argument.)

We would like to suggest another thesis on continuous functions which refines Thesis 2. There are many monotonic but noncomputable functions in, for example, $\mathbf{N}_\perp \rightarrow_{\mathbf{M}} \mathbf{N}_\perp$, but they are all computable by infinite programs, or more generally by machines with infinitely many internal states. Of course such computation is not effective or even mechanical in the usual sense of a finite machine. Nonetheless it is physically feasible. And indeed at higher types continuity is a real restriction. Again with the above characterisation of continuous functions we could imagine an infinite machine which stored (codes for) all the pairs (d, e) of finite elements such that $e \sqsubseteq f(d)$ used this store to give out the information in each e whenever it had received all the information in the corresponding d . Of course, this depends on the input and output domains being themselves physically feasible (and we would dearly like to be able to put down the right axioms for that notion), but let us state a thesis for such domains:

Thesis 5. *A monotonic function is continuous iff it is physically feasible.*

This implies Thesis 4 as surely every computable function is physically feasible. Since the thesis is an equivalence it may be easier to give more convincing arguments for its truth than Thesis 4, in some cases at least.

One kind of information is to do with (occurrences of) events: namely the information that those events occurred. For example in the case of $\mathbf{N}_\perp$, $\perp$ might mean that no event occurred and an integer n , might mean that the event occurred of the integer n being output (or, in another circumstance being input). For an example with more structure, suppose we have a Turing machine (TM) with an input tape and an output tape. The tape squares can be blank, or, after printing, contain a 0 or a 1. The events that can occur are the printing of a 0 or a 1 on the (next) output tape square by the TM or the inputting (by some unspecified agency) of a 0 or a 1 on the (next) input tape square. Thus the output domain is just the collection of all possible sets of events that could occur, ordered by the subset ordering; equivalently this is the set of all finite or infinite binary sequences with the subsequence ordering. For example 0111 means that 0 was printed on the first tape square, and one on the second, third and fourth tape squares, and thereafter there was no more output. The same (isomorphic) domain is associated with the input. This cpo is called **Tapes**, not unnaturally, and the reader will see that the function $f: \mathbf{Tapes} \rightarrow \mathbf{Tapes}$ computed by the TM must be continuous.

By taking a different view of what constitutes an output event, we can obtain a different cpo for the same physical situation. Let us agree that outputting a 0 followed by n 1's followed by a 0 constitutes an outputting of the integer n (and so the same event may occur several times). Then the appropriate cpo is $\mathbf{P}\omega$, the set of all subsets of $\omega (= \mathbf{N})$ with the subset ordering. It must be admitted that what can legitimately be called an event is not a very clear notion, although there must be some rules. For example it does not seem correct to allow as an event “the printing of infinitely many 1's”. Again if we regard the input tape also as specifying an element of $\mathbf{P}\omega$, the TM may not compute a function from $\mathbf{P}\omega$ to $\mathbf{P}\omega$ at all — we should insist either that the input has a special form or that the TM has a certain kind of behaviour.

Just as more than one domain may be appropriate when describing a physical situation, so the same domain may be used to describe different situations. For example suppose we have infinitely many lights and a computational mechanism which is switching them on (and never off). Then “the switching on of the n th light” would be a bona fide event and the appropriate cpo would then be $\mathbf{P}\omega$ again.

In all those last examples we have not considered any problem of giving a denotational semantics; however the above domains would clearly arise if we wanted to consider the semantics of languages for outputting or inputting on tapes or for switching on infinite arrays of lights.

In terms of events a finite amount of information should just be a finite set of events. Then if $\bigsqcup_n x_n = \bigcup_n x_n$ (as seems reasonable) the above axioms seem reasonable enough and we also believe that it is reasonably self-evident that any physically feasible function must be monotonic and obey the condition proved equivalent to continuity as an output event could hardly depend on infinitely many input events as a machine should only be able to do a finite amount of computation before causing a given output event. However further analysis of events is required to show that every continuous function is feasible. But in particular cases, such as the above interpretation of **Tapes**, this is not difficult.

In the above we have argued for the use of the category, $\mathcal{CP}\mathcal{O}$ of cpos and continuous functions, rather than the category of sets and (ordinary) functions. We have come across a few examples of cpos. Each set, $\mathbf{X}$, gives us a flat cpo, $\mathbf{X}_\perp = \mathbf{X} \cup \{\perp\}$ ordered by: $x \sqsubseteq y$ iff $x = \perp$ or $x = y$. Particular examples are given in Figures I, II above and we should also mention $\mathbf{U} \stackrel{\text{def}}{=} \emptyset_\perp$ and $\mathbf{O} \stackrel{\text{def}}{=} \{\top\}_\perp$. We have considered such continuous functions as $(+1): \mathbf{N}_\perp \rightarrow \mathbf{N}_\perp$ and should also mention $(-1): \mathbf{N}_\perp \rightarrow \mathbf{N}_\perp$ and $Z: \mathbf{N}_\perp \rightarrow \mathbf{T}_\perp$ where:

$$x - 1 = \begin{cases} n & (\exists n \in \mathbf{N}. x = n + 1), \\ \perp & (\text{otherwise}); \end{cases}$$

$$Z(x) = \begin{cases} tt & (\text{if } x = 0), \\ ff & (\text{if } x \neq 0, \perp), \\ \perp & (\text{otherwise}). \end{cases}$$

We have also seen nonflat cpos such as **Tapes** and $\mathbf{P}\omega$. Some basic continuous functions for these cpos are $(0 \cdot -)$, $(1 \cdot -)$, $tl: \mathbf{Tapes} \rightarrow \mathbf{Tapes}$, $hd: \mathbf{Tapes} \rightarrow \mathbf{T}_\perp$, $(+1)$, $(-1): \mathbf{P}\omega \rightarrow \mathbf{P}\omega$ and $Z: \mathbf{P}\omega \rightarrow \mathbf{T}_\perp$ (we don't mind introducing a small amount of ambiguity for names of functions), where $(0 \cdot -)$ and $(1 \cdot -)$ append 0 (respectively 1) to a tape and tl and hd are characterised by the equations, $tl(0 \cdot x) = tl(1 \cdot x) = x$, $tl(\perp) = \perp$, $hd(0 \cdot x) = ff$, $hd(1 \cdot x) = tt$, $hd(\perp) = \perp$ and the functions over $\mathbf{P}\omega$ are given by:

$$x + 1 = \{n + 1 \mid n \in x\};$$

$$x - 1 = \{n \mid n + 1 \in x\};$$

$$Z(x) = \begin{cases} tt & (0 \in x), \\ \perp & (\text{otherwise}). \end{cases}$$

We will consider other cpos and functions of several arguments when we discuss the construction in $\mathcal{CP}\mathcal{O}$.

Let us conclude with some discussion of some choices we have not made. First we have not used complete lattices, that is we have not assumed that every subset has a lub (although, of course, the lattices are included). For the presence of the top element, $\top (= \bigsqcup_{\mathbf{D}} \mathbf{D})$ causes problems, leading to extra decisions when extending functions and cases when making proofs, in an unsystematic way. For example suppose we make $\mathbf{T}$ into a lattice by adding a top element, giving:

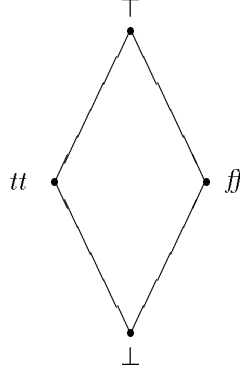


Figure V: The truth-value lattice.

Now, even apart from the fact that $\top$ has no computational interpretation, when using $\mathbf{T}_{\perp}$ in the cases discussed above, it is not clear how to extend the definition of $Cond: \mathbf{T}_{\perp} \times \mathbf{N}_{\perp} \times \mathbf{N}_{\perp} \rightarrow \mathbf{N}_{\perp}$ to a function on the lattices. By monotonicity we must make $\mathbf{N}_{\perp}$ into a lattice too, say by adding a top element, and then we must choose between a minimum and a maximum extension:

$$Cond(\top, x, y) = x \sqcup y$$

or

$$Cond(\top, x, y) = \top.$$

Either choice means that one of the following program equivalence will fail in general, although they are correct from the computational point of view:

$$\begin{aligned} &(\text{if } b \text{ then } (\text{if } b \text{ then } e_0 \text{ else } e_1) \text{ else } (\text{if } b \text{ then } e_2 \text{ else } e_3)) \equiv (\text{if } b \text{ then } e_0 \text{ else } e_3), \\ &(\text{if } b_0 \text{ then } (\text{if } b_1 \text{ then } e_0 \text{ else } e_1) \text{ else } (\text{if } b_1 \text{ then } e_2 \text{ else } e_3)) \equiv \\ &(\text{if } b_1 \text{ then } (\text{if } b_0 \text{ then } e_0 \text{ else } e_2) \text{ else } (\text{if } b_0 \text{ then } e_1 \text{ else } e_3)). \end{aligned}$$

One often sees another definition of cpo in which it is assumed that every directed set has a lub, where a set X is *directed* iff it is nonempty and any two elements have an upper bound in X . Let us call such partial orders *dcpo*s to distinguish them from ours. We decided to make the choice of assuming only lubs of increasing ω -chains because lubs of general directed sets are not so easily justified from a computational point of view and because it is a weaker assumption. But we shall see that all the cpos we are interested in turn out to be dcpo, so this choice is of little importance in the end.

As to functions, we have not assumed that our functions are strict (preserve the least element). For then all least fixed-points would be $\perp$, which is certainly wrong. But on the other hand from the categorical point of view the strict continuous functions are important as they preserve all the cpo structure. So we will often consider the subcategory $\mathcal{CPO}_{\perp}$ of all strict continuous functions and write $f: \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ to mean f is strict and continuous. Other axioms and kinds of functions will be considered later.

Exercises

1. Add a syntax for **BExp** to the specification of *Dec*, and extend the semantics accordingly.
2. In the semantics given above for nonrecursive function declarations, the free variables (other than the parameters) in the body of the function declaration receive their values from the “definition-time” environment. Can you change the semantics so that this value is taken from the “call-time” environment instead?

3. Suppose we add a class **Proc** of (parameterless) procedure variables to the syntax of *Imp* and add procedure declaration and calling facilities to **Stat** by including the extra clauses:

$$s ::= (\text{proc } p \text{ begin } s \text{ end}) \mid (\text{call } p).$$

What difficulties do you discover when trying to do the denotational semantics?

4. Add some facilities to *Imp* to allow reading a boolean value from an input channel and outputting one to an output channel. What happens with the denotational semantics? (Repeated input requests and outputs are allowed.)

5. Let $\mathbf{S}$ be a set, $p: \mathbf{S} \rightarrow \mathbf{T}$ be a predicate, $\theta': \mathbf{S} \rightarrow \mathbf{S}$ be a command as above. What commands, θ , satisfy the fixed-point equation, $\theta(\sigma) = \text{Cond}(p(\sigma), \theta \circ \theta'(\sigma), \sigma)$.

6. Write Turing machines to compute the functions $(0 \cdot -)$, $(1 \cdot -)$, $tl: \mathbf{Tapes} \rightarrow \mathbf{Tapes}$, $hd: \mathbf{Tapes} \rightarrow \mathbf{T}_\perp$, $(+1)$, $(-1): \mathbf{P}\omega \rightarrow \mathbf{P}\omega$ and $Z: \mathbf{P}\omega \rightarrow \mathbf{T}_\perp$.

7. Let $\mathbf{D}$ be a cpo and let $f: \mathbf{D} \rightarrow \mathbf{D}$ be continuous. A *prefixed-point* (*fixed-point*, *postfixed-point*) of f is an element, d of $\mathbf{D}$ such that $fd \sqsubseteq d$ ($fd = d$, $fd \sqsupseteq d$). Show that the set of prefixed-points (respectively fixed-points, postfixed-points) of f form a cpo under the partial order inherited from $\mathbf{D}$.

8. Show that every monotonic function, $f: \mathbf{D} \rightarrow \mathbf{D}$ where $\mathbf{D}$ is a dcpo has a least fixed-point. Give an example of a monotonic function $f: \mathbf{D} \rightarrow \mathbf{D}$, where $\mathbf{D}$ is a cpo, which does not have any fixed-point.

9. The cpo, $\mathbf{Pred}$, is the set of partial function $f: \mathbf{N} \rightarrow \mathbf{P} \mathbf{T}$ ordered by:

$$f \sqsubseteq g \quad \text{iff} \quad \forall n \in \mathbf{N}. f(n) \text{ defined} \Rightarrow (g(n) \text{ defined and } f(n) = g(n));$$

the partial order, $(\mathbf{T}_\perp)^\omega$ is the Cartesian Product of denumerably many copies of $\mathbf{T}_\perp$; the partial order $\mathbf{N}_\perp \rightarrow_\perp \mathbf{T}_\perp$ is the subset of strict functions of $\mathbf{N}_\perp \rightarrow_{\mathbf{M}} \mathbf{T}_\perp$ with the inherited ordering. Show that all these cpos are isomorphic to the cpo,

$$\{\langle x, y \rangle \mid x \subseteq \mathbf{N}, y \subseteq \mathbf{N}, x \cap y = \emptyset\}$$

under the ordering: $\langle x, y \rangle \sqsubseteq \langle u, v \rangle$ iff $x \subseteq u$ and $y \subseteq v$.

10. Why is the function $\text{Tot}: \mathbf{O} \rightarrow \mathbf{T}$ not intuitively computable, where: $\text{Tot}(\top) = tt$, $\text{Tot}(\perp) = ff$? Find some other, similar examples.

11. Find an example of a monotonic but intuitively non-computable function from $\mathbf{Pred}$ (see Exercise 9) to $\mathbf{T}_\perp$. Show it is not continuous. Find some other, similar examples.

12. The function $\partial_{\mathbf{D}}: \mathbf{D} \rightarrow \mathbf{O}$ where $\mathbf{D}$ is a cpo defined by:

$$\partial_{\mathbf{D}}(d) = \begin{cases} \top & (d \neq \perp), \\ \perp & (d = \perp). \end{cases}$$

Show $\partial_{\mathbf{D}}$ is continuous. Give an argument to show that $\partial_{\mathbf{Tapes}}$ is intuitively computable for at least one interpretation of $\mathbf{Tapes}$; do the same for $\partial_{\mathbf{Pred}}$. A function $f: \mathbf{D} \rightarrow \mathbf{E}$ where $\mathbf{D}$, $\mathbf{E}$ are partial orders is *multiplicative* iff whenever the glb, $x \sqcap y$ of two elements x and y of $\mathbf{D}$ exists then $f(x \sqcap y) = f(x) \sqcap f(y)$; it is *dual-continuous* iff whenever $\langle d_n \rangle_{n \in \omega}$ is a decreasing sequence in $\mathbf{D}$ with glb $\bigcap_{\mathbf{D}} x_n$ then $f(\bigcap_{\mathbf{D}} x_n) = \bigcap_{\mathbf{E}} f(x_n)$. Is $\partial_{\mathbf{D}}$ always multiplicative? Is it always dual-continuous?

13. A function is *additive* iff it preserves binary lubs (cf. Exercise 12). Give an example of a continuous, intuitively computable function between cpos which is not additive. Why is the functional $F: (\mathbf{S}_\perp \rightarrow \mathbf{S}_\perp) \rightarrow (\mathbf{S}_\perp \rightarrow \mathbf{S}_\perp)$ involved in the above discussion on while-loops additive?

14. A function $\bar{\mu}: \mathbf{Pred} \rightarrow \mathbf{N}$ is a *minimalisation operator* iff $\bar{\mu}(f) = n \neq \perp$ implies $f(n) = tt$ and $\forall m < n. f(m) \neq tt$. Show there is a greatest monotonic minimalisation operator and that is continuous. Is the case $\bar{\mu}: (\mathbf{N}_\perp \rightarrow_{\mathbf{M}} \mathbf{T}_\perp) \rightarrow \mathbf{N}_\perp$ much different?

15. A function $\exists: (\mathbf{N}_\perp \rightarrow_{\mathbf{M}} \mathbf{T}_\perp) \rightarrow \mathbf{T}_\perp$ is an *existential quantifier* iff

$$\begin{aligned} \exists f = tt &\Rightarrow \text{there is an element, } d \text{ of } \mathbf{N} \text{ s.t. } f(d) = tt, \\ \exists f = ff &\Rightarrow \text{for all elements, } d \text{ of } \mathbf{N} \text{ s.t. } d \neq \perp, f(d) = ff. \end{aligned}$$

What are the maximal monotonic and continuous existential quantifiers, respectively? How is the continuous one intuitively computable? What happens if you replace $(\mathbf{N}_\perp \rightarrow_{\mathbf{M}} \mathbf{T}_\perp)$ by $\mathbf{Pred}$?

16. Draw the following partial orders: $\mathbf{0} \rightarrow_M \mathbf{0}$, $\mathbf{0} \rightarrow_M \mathbf{T}_\perp$, $\mathbf{T}_\perp \rightarrow_M \mathbf{0}$, $\mathbf{T}_\perp \rightarrow_M \mathbf{T}_\perp$, $(\mathbf{0} \rightarrow_M \mathbf{0}) \rightarrow_M (\mathbf{0} \rightarrow_M \mathbf{0})$, $\mathbf{0} \times \mathbf{0}$, $(\mathbf{0} \times \mathbf{0}) \rightarrow_M \mathbf{0}$, **Tapes**.

17. The definition of **Tapes** of tapes uses $\{0, 1\}$ as a parameter. Define X^∞ similarly for any set X so that $\mathbf{Tapes} = \{0, 1\}^\infty$ and $\omega \stackrel{\text{def}}{=} \{0\}^\infty$ looks like:

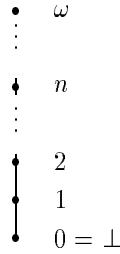


Figure VI: The cpo ω .

18. Define the partial order $\mathbf{R}$ (closed intervals on the real line) as the set $\{[\underline{x}, \overline{x}] \mid -\infty \leq \underline{x} \leq \overline{x} \leq +\infty\}$ where the intervals $[\underline{x}, \overline{x}] \stackrel{\text{def}}{=} \{y \in \mathbf{Reals} \mid \underline{x} \leq y \leq \overline{x}\}$ are ordered by reverse inclusion, i.e. $[\underline{x}, \overline{x}] \sqsubseteq [\underline{x}', \overline{x}']$ iff $\underline{x} \leq \underline{x}' \leq \overline{x}' \leq \overline{x}$. Show that $\mathbf{R}$ is a cpo. How would you use $\mathbf{R}$ when computing functions over reals?

19. Cantor's Theorem shows there is no nontrivial set X which is isomorphic to (in bijection with) its own function space, X^X . Show there is no nontrivial partial order, $\mathbf{D}$, such that $\mathbf{D} \cong \mathbf{D} \rightarrow_M \mathbf{D}$.

This gives yet another argument for restricting attention to the continuous functions, as we will see that the answer for the corresponding question on cpos and continuous functions is positive. [Hint Consider $\mathbf{D} \rightarrow \mathbf{0}$ and the cardinalities of sets of incomparable elements.]

20. Show that every *denumerable* directed subset of a cpo has a least upper bound. Let $\mathbf{D}, \mathbf{E}$ be cpos. Show that $f: \mathbf{D} \rightarrow \mathbf{E}$ is continuous iff f preserves lubs of denumerable directed sets (i.e. iff for every denumerable directed subset, X , of $\mathbf{D}$, $f(X)$ is directed and $f(\bigsqcup_{\mathbf{D}} X) = \bigsqcup_{\mathbf{E}} f(X)$). Let $\mathbf{D}$ be a cpo and let $\langle d_{m,n} \rangle_{m,n \in \omega}$ be a double increasing sequence (i.e. $m \leq m', n \leq n'$ implies $d_{m,n} \sqsubseteq d_{m',n'}$). Show that $\bigsqcup_m (\bigsqcup_n d_{m,n}) = \bigsqcup_{m,n} d_{m,n} = \bigsqcup_n d_{n,n}$.

21. Let $\mathbf{D}$ be a cpo. Show that a T_0 -topology can be defined in $\mathbf{D}$ by taking a subset, O , to be open iff:

- (1) Whenever x, y are elements of $\mathbf{D}$ such that $x \sqsubseteq y$ and $x \in O$ then $y \in O$.
- (2) Whenever $\langle x_n \rangle_{n \in \omega}$ is an increasing sequence of elements of $\mathbf{D}$ s.t. $(\bigsqcup_n x_n) \in O$, then for some n , $x_n \in O$.

Show that the order can be recovered from the topology by: $x \sqsubseteq y$ iff $\forall$ open sets O . ($x \in O \Rightarrow y \in O$). Show too that if $\mathbf{D}, \mathbf{E}$ are cpos then $f: \mathbf{D} \rightarrow \mathbf{E}$ is continuous iff it is topologically continuous with respect to the above topology. Show that this is the only assignment of topologies to cpos which makes topological continuity coincide with order continuity. [Hint Consider $\mathbf{0}, \omega$ (Exercise 17) and then an arbitrary cpo.]

22. Let $\mathbf{D}, \mathbf{E}$ be dcpos. A function $f: \mathbf{D} \rightarrow \mathbf{E}$ is (*directed*) *continuous* iff for every directed subset, X , of $\mathbf{D}$, $f(X)$ is directed and $f(\bigsqcup_{\mathbf{D}} X) = \bigsqcup_{\mathbf{E}} f(X)$. Give a T_0 -topology for dcpos so that this definition of continuity coincides with topological continuity. Give a topology for partial orders so that topological continuity coincides with monotonicity.

2. Products and Function Spaces

Now that we have an idea of what our domains are, we need some language (metalanguage) to denote elements of and continuous functions between domains. This will be a more or less normal mathematical language of terms, including a typed λ -calculus, and looks much like the language used to describe the set-theoretic semantics of *Imp* and *Dec* in Chapter 1. We will treat this language and its semantics in a semi-formal fashion, leaving the reader, if he cares, to provide a formal definition in set theory.

In fact we expand the language as we go along, beginning with:

- E1.** There are constants, $a, \dots$ denoting elements of cpos. Their types and what they denote are given when they are introduced. For example, $\perp_{\mathbf{D}}$ denotes the least element of $\mathbf{D}$ (and we just write $\perp$ when $\mathbf{D}$ is understood from the context). We also use tt, ff for elements of $\mathbf{T}_{\perp}$, numerals for elements of $\mathbf{N}_{\perp}$, $\top_{\mathbf{D}}$ for the top element of $\mathbf{D}$, if it exists, all as in Chapter 1.
- E2.** There are variables, $x, \dots$ denoting elements of cpos. Their types and what they denote are determined by the context of their use.
- E3.** There are constants, $id_{\mathbf{D}}, (+1), (-1), Z, \dots$ denoting morphisms, that are continuous functions between cpos. Their types and what they denote are given when they are introduced. Again we have introduced a few of these in Chapter 1. Again we tolerate a certain amount of ambiguity — as long as this can be disambiguated by the context; for example for each constant a , denoting an element of a domain $\mathbf{E}$, we have the constant K_a denoting the function $f: \mathbf{D} \rightarrow \mathbf{E}$ where $f(d) = a$, and $\mathbf{D}$ is to be understood from the context.
- E4.** If α, β are expressions denoting morphisms of types $\mathbf{A} \rightarrow \mathbf{B}, \mathbf{B} \rightarrow \mathbf{C}$ respectively then $(\beta \circ \alpha)$ is an expression of type $\mathbf{A} \rightarrow \mathbf{C}$ which denotes the composition of β and α . Because function composition is associative we can always omit brackets.
- E5.** If α is an expression denoting a morphism $f: \mathbf{A} \rightarrow \mathbf{B}$ and e is an expression denoting an element, a of $\mathbf{A}$, then $\alpha(e)$ denotes $f(a)$. Actually we won't stick to this rigid applicative syntax, but will also infix and postfix operators and so on.

We will write $e: \mathbf{D}$ to mean that the expression e has type $\mathbf{D}$ and also $\alpha: \mathbf{D} \rightarrow \mathbf{E}$ to mean that the expression α has type $\mathbf{D} \rightarrow \mathbf{E}$. Note that there is a correspondence between expressions denoting elements and expressions denoting morphisms. If e is an expression of type $\mathbf{E}$ with one free variable, x , of type $\mathbf{D}$ then we can define a function, f , from $\mathbf{D}$ to $\mathbf{E}$ by: $f(x) = e$. This is continuous as can be shown by a little inductive proof to the effect that each such function can be denoted by an expression: a variable x of type $\mathbf{D}$ gives the function $id_{\mathbf{D}}$; the expression $\alpha(e)$ gives the function $\alpha \circ \beta$ where e gives β .

What we need next are functions of several variables and they go hand in hand with products in $\mathcal{CPO}$. Let $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ be cpos. Their *Cartesian product*, $\prod_{i < n} \mathbf{D}_i$ is the set $\{\langle x_0, \dots, x_{n-1} \rangle \mid \forall i < n. x_i \in \mathbf{D}_i\}$ of n -tuples ordered coordinatewise by:

$$\langle x_0, \dots, x_{n-1} \rangle \sqsubseteq \langle x'_0, \dots, x'_{n-1} \rangle \equiv \forall i < n. x_i \sqsubseteq_{\mathbf{D}_i} x'_i.$$

(We have already seen an example of this construction in Chapter 1, namely $\mathbf{T}_{\perp} \times \mathbf{S}_{\perp} \times \mathbf{S}_{\perp}$.) Sometime we write $\mathbf{D}_0 \times \dots \times \mathbf{D}_n$ for $\prod_{i < n} \mathbf{D}_i$ or $\mathbf{D}^n$ when the $\mathbf{D}_i$ are all the same. Note that the Cartesian product is a cpo with the least element, $\langle \perp_{\mathbf{D}_0}, \dots, \perp_{\mathbf{D}_{n-1}} \rangle$ and with lubs of increasing sequence $\langle x^{(m)} \rangle_{m \in \omega}$ determined coordinatewise as: $\bigsqcup_m x^{(m)} = \langle \bigsqcup_m \mathbf{D}_0 x_0^{(m)}, \dots, \bigsqcup_m \mathbf{D}_{n-1} x_{n-1}^{(m)} \rangle$, (and in fact all lubs are determined coordinatewise).

This product is also the normal categorical product. We have the natural projection maps,

$$\pi_i: \left(\prod_{i < n} \mathbf{D}_i \right) \rightarrow \mathbf{D}_i \quad (i < n)$$

where $\pi_i(x) = x_i$ — and these are continuous as can be seen from the remark on lubs. Sometimes $\pi_i(x)$ is written as $(x \downarrow i)$ or even $(x)_i$. The projection maps have the universal property that if $f_i: \mathbf{F} \rightarrow \mathbf{D}_i$ ($i < n$) are n continuous maps then there is a unique (mediating) morphism, $h: \mathbf{F} \rightarrow (\prod_{i < n} \mathbf{D}_i)$ which makes all the following diagrams commute:

$$\begin{array}{ccc}
\mathbf{F} & \xrightarrow{h} & \prod_{i < n} \mathbf{D}_i \\
& \searrow f_i & \downarrow \pi_i \\
& & \mathbf{D}_i
\end{array} \quad (i < n)$$

To see this, suppose h is such a morphism. Then for any element, x , of F :

$$\begin{aligned}
h(x) &= \langle h(x)_0, \dots, h(x)_{n-1} \rangle \\
&= \langle \pi_0 \circ h(x), \dots, \pi_{n-1} \circ h(x) \rangle \\
&= \langle f_0(x), \dots, f_{n-1}(x) \rangle,
\end{aligned}$$

showing uniqueness. Conversely we can use the formula to define h , proving continuity by:

$$\begin{aligned}
h(\bigsqcup_m x_m) &= \langle f_0(\bigsqcup_m x_m), \dots, f_{n-1}(\bigsqcup_m x_m) \rangle \\
&= \langle \bigsqcup_m f_0(x_m), \dots, \bigsqcup_m f_{n-1}(x_m) \rangle && \text{the } f_i \text{ are continuous} \\
&= \bigsqcup_m \langle f_0(x_m), \dots, f_{n-1}(x_m) \rangle && \text{lubs are determined coordinatewise} \\
&= \bigsqcup_m h(x_m) && \text{by definition of } h.
\end{aligned}$$

We write h as $\langle f_0, \dots, f_{n-1} \rangle$ and the double use of pointed brackets should not cause any confusion. Note the particular case, $n = 0$, when $\prod_{i < 0} \mathbf{D}_i$ is just $\mathbf{U}$ and we see that for any cpo, $\mathbf{F}$, there is a unique morphism $\mathbf{F} \rightarrow \mathbf{U}$, showing that $\mathbf{U}$ is the *final* object in $\mathcal{CP}\mathcal{O}$. Note, too, that as usual the universal property determines $\prod_{i < n} \mathbf{D}_i$ up to isomorphism.

We can now expand our stock of expressions:

- E6.** If $e_0, \dots, e_{n-1}$ are expressions of types $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ respectively, then $\langle e_0, \dots, e_{n-1} \rangle$ is an expression of type $\mathbf{D}_0 \times \dots \times \mathbf{D}_{n-1}$ and denotes the n -tuple of the values denoted by the $e_0, \dots, e_{n-1}$, taken in that order. When $\alpha: (\mathbf{D}_0 \times \dots \times \mathbf{D}_{n-1}) \rightarrow \mathbf{E}$ we will often write $\alpha(e_0, \dots, e_{n-1})$ instead of the more pedantic $\alpha(\langle e_0, \dots, e_{n-1} \rangle)$; this is just the usual notation for functions of several arguments.
- E7.** If $\alpha_0, \dots, \alpha_{n-1}$ are expressions of types $(\mathbf{F} \rightarrow \mathbf{D}_0), \dots, (\mathbf{F} \rightarrow \mathbf{D}_{n-1})$ respectively then $\langle \alpha_0, \dots, \alpha_{n-1} \rangle$ is an expression of type $\mathbf{F} \rightarrow (\prod_{i < n} \mathbf{D}_i)$, and it denotes the evident mediating morphism, as described above.

One reason for interest in the categorical properties of the Cartesian product is that they serve as a guide for the right axioms, which we can give in terms of either the morphisms or the elements. We call the first kind of axiomatisation *external* and the second *internal*. Unfortunately, we will not have time to pursue questions of proof much further than informally stating a few axioms.

External Axiomatisation.

1. $\forall f_0: \mathbf{F} \rightarrow \mathbf{D}_0 \dots \forall f_{n-1}: \mathbf{F} \rightarrow \mathbf{D}_{n-1}. \forall i < n. \pi_i \circ \langle f_0, \dots, f_{n-1} \rangle = f_i.$
2. $\forall h: \mathbf{F} \rightarrow (\prod_{i < n} \mathbf{D}_i). h = \langle \pi_0 \circ h, \dots, \pi_{n-1} \circ h \rangle.$

These axioms are clearly true. Let us see that they imply the requisite universal property. Let $f_i: \mathbf{F} \rightarrow \mathbf{D}_i$ ($i < n$) be n morphisms then let the mediating morphism be $h \stackrel{\text{def}}{=} \langle f_0, \dots, f_{n-1} \rangle$. By Axiom 1 all the required diagrams commute. Now let h' be any morphism which makes all the diagrams commute. Then $h = \langle f_0, \dots, f_{n-1} \rangle = \langle \pi_0 \circ h', \dots, \pi_{n-1} \circ h' \rangle$ (h' makes all the diagrams commute) $= h'$ (by Axiom 2).

Internal Axiomatisation.

1. $\forall x_0: \mathbf{D}_0 \dots \forall x_{n-1}: \mathbf{D}_{n-1}. \pi_i(x_0, \dots, x_{n-1}) = x_i.$
2. $\forall x: \prod_{i < n} \mathbf{D}_i. x = \langle \pi_0(x), \dots, \pi_{n-1}(x) \rangle.$

Again the axioms are true and they imply the external ones (using the extensionality of functions, the definition of composition and the definition of the tupling operator on functions). Note that the external axiomatisation refers to “external” cpos, such as $\mathbf{F}$, whereas the internal one refers only to the $\mathbf{D}_i$ (and their product).

Returning to expressions we note an extended version of the correspondence between expressions for elements and expressions for morphisms. For any expression $e: \mathbf{D}$ suppose $x_0: \mathbf{D}_0, \dots, x_{n-1}: \mathbf{D}_{n-1}$ is a list (without repetition) of variables of the indicated types which includes all the free variables of e (i.e. *all* the variables). Then we can define a function $f: \prod_{i < n} \mathbf{D}_i \rightarrow \mathbf{D}$ by $f(x_0, \dots, x_{n-1}) = e$. This is a continuous function as it can be given by an expression $\alpha[e; x_0, \dots, x_{n-1}]$ which is defined by structural induction on expressions e , thus:

- (i) $\alpha[a; x_0, \dots, x_{n-1}] = K_a,$
- (ii) $\alpha[x_i; x_0, \dots, x_{n-1}] = \pi_i,$
- (iii) $\alpha[\beta(e); x_0, \dots, x_{n-1}] = \beta \circ (\alpha[e; x_0, \dots, x_{n-1}]),$
- (iv) $\alpha[\langle e_0, \dots, e_{m-1} \rangle; x_0, \dots, x_{n-1}] = \langle \alpha[e_0; x_0, \dots, x_{n-1}], \dots, \alpha[e_{m-1}; x_0, \dots, x_{n-1}] \rangle.$

It follows if we fix some arguments of a continuous function $f: \prod_{i < n} \mathbf{D}_i \rightarrow \mathbf{D}$ we still obtain a continuous function. For suppose we fix the i th argument to be the element d_i in $\mathbf{D}$ then the function $g: \mathbf{D}_0 \times \dots \times \mathbf{D}_{i-1} \times \mathbf{D}_{i+1} \times \dots \times \mathbf{D}_{n-1} \rightarrow \mathbf{D}$ where $g(x_0, \dots, x_{i-1}, x_{i+1}, \dots, x_{n-1}) = f(x_0, \dots, x_{i-1}, d_i, x_{i+1}, \dots, x_{n-1})$ is continuous as the expression on the right of the equation is one of the expression we have considered above. Now it is clear that the same holds if we fix any subset of the arguments of f . As it happens the converse is true: if a function is continuous in each of its arguments it is continuous (where a function is continuous in its i th argument means that the function obtained by fixing all the other arguments is continuous). We illustrate this for binary functions. Certainly monotonicity in each argument implies monotonicity; for continuity suppose $f: \mathbf{D} \times \mathbf{E} \rightarrow \mathbf{F}$ and $\langle d_n, e_n \rangle_{n \in \omega}$ is an increasing chain of elements in $\mathbf{D} \times \mathbf{E}$. Then:

$$\begin{aligned}
 f\left(\bigsqcup_n \langle d_n, e_n \rangle\right) &= f\left(\left\langle \bigsqcup_n d_n, \bigsqcup_n e_n \right\rangle\right) && \text{lubs are determined coordinatewise} \\
 &= \bigsqcup_n f(d_n, \bigsqcup_n e_n) && f \text{ is continuous in its first argument} \\
 &= \bigsqcup_n \bigsqcup_m f(d_n, e_m) && f \text{ is continuous in its second argument} \\
 &= \bigsqcup_n f(d_n, e_n) && \text{by Exercise 1.20, as } f \text{ is monotonic.}
 \end{aligned}$$

Example. Any function $f: X_0 \times \dots \times X_{n-1} \rightarrow Y$ on sets extends to a monotonic and so continuous function, $f_\perp: (X_0)_\perp \times \dots \times (X_{n-1})_\perp \rightarrow Y_\perp$ where

$$f_\perp(d_0, \dots, d_{n-1}) = \begin{cases} \perp & \text{(if some } d_i \text{ is } \perp), \\ f(d_0, \dots, d_{n-1}) & \text{(otherwise).} \end{cases}$$

This is the minimal extension and is the one we will normally consider. For example the resulting truth-table for $or_\perp: \mathbf{T}_\perp \times \mathbf{T}_\perp \rightarrow \mathbf{T}_\perp$ is:

$or_\perp$	tt	ff	$\perp$
tt	tt	tt	$\perp$
ff	tt	ff	$\perp$
$\perp$	$\perp$	$\perp$	$\perp$

A program for $or_\perp$ is: **if x then (if y then true else true) else (if y then true else false)**. It is the “call-by-value” or. However there are three other monotonic extensions which we might call or_L , or_R , or_P with truth tables:

or_L	tt	ff	$\perp$
tt	tt	tt	tt
ff	tt	ff	$\perp$
$\perp$	$\perp$	$\perp$	$\perp$

or_R	tt	ff	$\perp$
tt	tt	tt	$\perp$
ff	tt	ff	$\perp$
$\perp$	tt	$\perp$	$\perp$

or_P	tt	ff	$\perp$
tt	tt	tt	tt
ff	tt	ff	$\perp$
$\perp$	tt	$\perp$	$\perp$

One occasionally sees programming constructs corresponding to or_L . One can write a program for it as: **if x then true else (if y then true else false)** and a similar one for or_R . However one can't write a sequential program for or_P — parallel or. This distinction raises the question where one can axiomatise the *sequential* continuous functions, as the others play no role in most programming languages.

The conditional function $cond: \mathbf{T}_\perp \times \mathbf{N}_\perp \times \mathbf{N}_\perp \rightarrow \mathbf{N}_\perp$ defined in Chapter 1 is also not the minimal extension of the set-theoretic condition, and this is in accord with the computational intuition that, according to the value of the first argument, either the second or the third argument is not evaluated. There is also a larger extension, $parcond: \mathbf{T}_\perp \times \mathbf{N}_\perp \times \mathbf{N}_\perp \rightarrow \mathbf{N}_\perp$ given by:

$$parcond(t, x, y) = \begin{cases} x & (t = tt \text{ or } x = y), \\ y & (t = ff \text{ or } x = y), \\ \perp & (\text{otherwise}). \end{cases}$$

Note that $parcond(\perp, x, x) = x$ showing its parallel nature. More generally we define $cond_{\mathbf{D}}: \mathbf{T}_\perp \times \mathbf{D} \times \mathbf{D} \rightarrow \mathbf{D}$ by:

$$cond_{\mathbf{D}}(t, x, y) = \begin{cases} x & (t = tt), \\ y & (t = ff), \\ \perp & (t = \perp). \end{cases}$$

We will often write $(e \rightarrow e' \mid e'')$ instead of $cond_{\mathbf{D}}(e, e', e'')$. When $\mathbf{D}$ has a continuous glb operation, $\sqcap$ (as is the case with flat domains) we define:

$$parcond_{\mathbf{D}}(t, x, y) = \begin{cases} x & (t = tt), \\ y & (t = ff), \\ x \sqcap y & (t = \perp) \end{cases}$$

and it is easily verified that $cond_{\mathbf{D}}$ and $parcond_{\mathbf{D}}$ are continuous. Note that $or_{\mathbf{P}}$ can be defined in terms of $parcond_{\mathbf{T}_\perp}$ by:

$$or_{\mathbf{P}}(x, y) = parcond_{\mathbf{T}_\perp}(x, tt, cond_{\mathbf{T}_\perp}(y, tt, ff)).$$

With the above exceptions, and unless otherwise stated, we will always use the minimal extensions hereafter and will drop the suffix, $\perp$ — similarly we will drop the suffices on $\mathbf{T}_\perp, \mathbf{N}_\perp, \dots$ when it is clear we intend a cpo rather than a set. So for example when e, e' are expressions of type $\mathbf{X}_\perp$, we write the expression $=_\perp(e, e')$ of type $\mathbf{T}$ in the less pedantic form $(e = e')$. Note this is different from the *assertion* $e = e'$ which means that e and e' denote the same value: so for example the assertion $\perp = \perp$ is true but the expression $(\perp = \perp)$ denotes $\perp$, as does $(e = \perp)$ or $(\perp = e)$.

Finally we consider the example of finite and infinite binary trees possibly with integers on their tips, ordered by the sub-tree ordering.

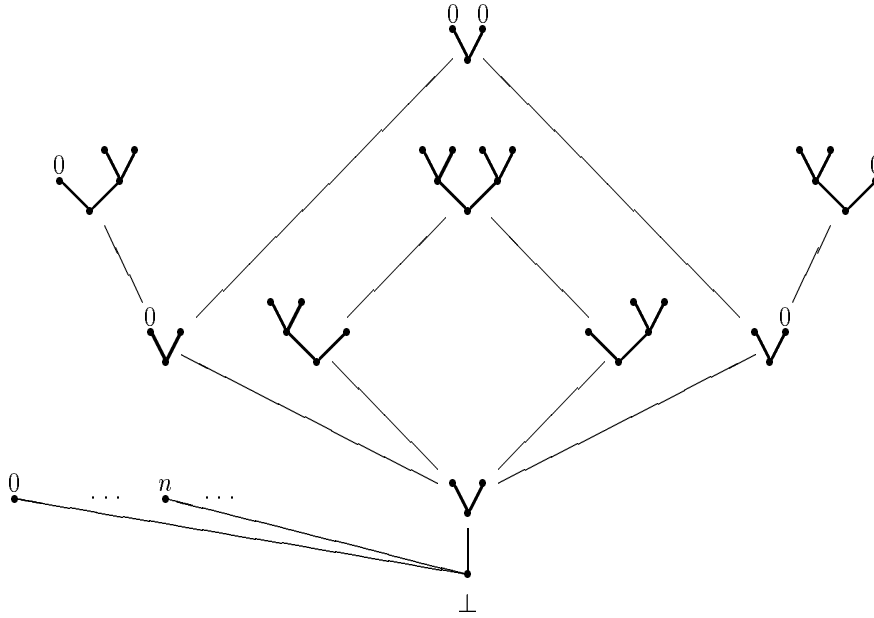


Figure VII: The cpo Btrees.

Some of this domain, **Btrees**, is drawn in the diagram. It is only one of a great variety of domains of trees. We will give a construction of **Btrees** later, but intuitively it can be thought of as generated by the following rules:

1. Any element of $\mathbf{N}_\perp$ is a Btree: if t_0, t_1 are elements so is $\bigvee_{\bullet}^{t_0 t_1}$.
2. For any element, t , we have $\perp \sqsubseteq t$; if $t_0 \sqsubseteq t'_0$ and $t_1 \sqsubseteq t'_1$ then $\bigvee_{\bullet}^{t_0 t_1} \sqsubseteq \bigvee_{\bullet}^{t'_0 t'_1}$, for the finite elements, and then adding the infinite ones as limits of ω -chains with the rule: $\bigsqcup_n t_n \sqsubseteq \bigsqcup_n t'_n \equiv \forall n. \exists m. t_n \sqsubseteq t'_m$ to decide the approximation order between two chains.

The appropriate functions are $tip: \mathbf{N} \rightarrow \mathbf{Btrees}$, $mktree: \mathbf{Btrees} \times \mathbf{Btrees} \rightarrow \mathbf{Btrees}$, $val: \mathbf{Btrees} \rightarrow \mathbf{N}$, $ltree, rtree: \mathbf{Btrees} \rightarrow \mathbf{Btrees}$ and $istip: \mathbf{Btrees} \rightarrow \mathbf{T}$ where:

$$\begin{aligned} tip(n) &= \begin{cases} \perp & (n = \perp), \\ \eta & (n \neq \perp). \end{cases} \\ mketree(t_0, t_1) &= \bigvee_{\bullet}^{t_0 t_1}. \\ val(t) &= \begin{cases} n & (t = \eta), \\ \perp & (\text{otherwise}). \end{cases} \\ ltree(t) &= \begin{cases} t_0 & (t = \bigvee_{\bullet}^{t_0 t_1}), \\ \perp & (\text{otherwise}). \end{cases} \\ rtree(t) &= \begin{cases} t_1 & (t = \bigvee_{\bullet}^{t_0 t_1}), \\ \perp & (\text{otherwise}). \end{cases} \\ istip(t) &= \begin{cases} tt & (t = \eta), \\ ff & (t = \bigvee_{\bullet}^{t_0 t_1}), \\ \perp & (t = \bullet). \end{cases} \end{aligned}$$

We conclude the discussion of the Cartesian product construction by noting that there is a product operation on the morphisms too. Suppose $f: \mathbf{D}_i \rightarrow \mathbf{E}_i$ ($i < n$) are continuous functions. Then $\prod_{i < n} f_i: \prod_{i < n} \mathbf{D}_i \rightarrow \prod_{i < n} \mathbf{E}_i$ is defined by:

$$\prod_{i < n} f_i = \langle f_0 \circ \pi_0, \dots, f_{n-1} \circ \pi_{n-1} \rangle$$

so that $(\prod_{i < n} f_i)(x_0, \dots, x_{n-1}) = \langle f(x_0), \dots, f(x_{n-1}) \rangle$. Sometimes it is written as $f_0 \times \dots \times f_{n-1}$ instead. The two product operations (on objects and morphisms) together make Cartesian product into a covariant functor, $\prod: \mathcal{CPO}^n \rightarrow \mathcal{CPO}$, which just means that the following equations are true:

- (i) $\prod_{i < n} id_{\mathbf{D}_i} = id_{\prod_{i < n} \mathbf{D}_i},$
- (ii) $\prod_{i < n} (g_i \circ f_i) = (\prod_{i < n} g_i) \circ (\prod_{i < n} f_i).$

It will prove very useful when we come to solving recursive domain equations (try Exercise 1.3) to consider domain constructions, such as Cartesian product, as functors.

In applications of the fixed-point theorem we will usually want to use domains of functions, as illustrated in Chapter 1 and so we now consider the function-space construction (exponentiation). Let $\mathbf{D}$ and $\mathbf{E}$ be cpos. Their function space, $\mathbf{D} \rightarrow \mathbf{E}$ in the set of all continuous functions from $\mathbf{D}$ to $\mathbf{E}$ with the pointwise ordering:

$$f \sqsubseteq g \quad \text{iff} \quad \forall x \in \mathbf{D}. f(x) \sqsubseteq_{\mathbf{E}} g(x).$$

Note that $(\mathbf{D} \rightarrow \mathbf{E})$ is a cpo with least element $K_{\perp_{\mathbf{E}}}$ and with lubs of increasing sequence determined pointwise by: $(\bigsqcup_n f_n)(x) = \bigsqcup_n (f_n(x)).$

We can characterise exponentiation by a universal property relating it to Cartesian product. Note that there is a natural application (evaluation) function, $apply: ((\mathbf{D} \rightarrow \mathbf{E}) \times \mathbf{D}) \rightarrow \mathbf{E}$ defined by:

$$apply(f, x) = f(x).$$

This is continuous as it is continuous in each argument separately. We have already seen in our discussion of expressions that $f(x)$ is continuous in x , when f is a fixed function (the value of some constant). The formula for lubs of increasing chains of continuous functions shows that the expression $f(x)$ is continuous in its first argument too.

The universal property is that if $f: \mathbf{F} \times \mathbf{D} \rightarrow \mathbf{E}$ is any continuous function (so that $\mathbf{F}$ is a “trial” function-space) there is a unique mediating morphism $g: \mathbf{F} \rightarrow (\mathbf{D} \rightarrow \mathbf{E})$ so that the following diagram commutes:

$$\begin{array}{ccc} (\mathbf{D} \rightarrow \mathbf{E}) \times \mathbf{D} & \xleftarrow{g \times id_{\mathbf{D}}} & \mathbf{F} \times \mathbf{D} \\ \downarrow apply & \searrow f & \\ \mathbf{E} & & \end{array}$$

To see this suppose g is such a morphism. Then for any elements v of $\mathbf{F}$ and d of $\mathbf{D}$ we have:

$$\begin{aligned} g(v)(d) &= apply(g(v), d) \\ &= apply(g \times id_{\mathbf{D}}(v, d)) \\ &= apply \circ (g \times id_{\mathbf{D}})(v, d) \\ &= f(v, d) \end{aligned}$$

showing that $g(v)$ is uniquely determined as the function $z_v: (\mathbf{D} \rightarrow \mathbf{E})$ such that $z_v(d) = f(v, d)$, which is continuous with a fixed f and v , by the above remarks on expressions. To see that g itself is continuous, let $\langle v_n \rangle_{n \in \omega}$ be an increasing sequence of elements of $\mathbf{F}$ and calculate:

$$\begin{aligned} g(\bigsqcup_n v_n)(d) &= f(\bigsqcup_n v_n, d) \\ &= \bigsqcup_n f(v_n, d) && f \text{ is continuous and so continuous in its first argument} \\ &= \bigsqcup_n (g(v_n)(d)) \\ &= (\bigsqcup_n g(v_n))(d) && \text{limits being determined pointwise} \end{aligned}$$

which shows that $g(\bigsqcup_n v_n) = \bigsqcup_n g(v_n)$ by extensionality. In category-theoretic jargon, we have shown that $\mathcal{CPO}$ is Cartesian closed.

We write g as $Curry(f)$ and expand our stock of expressions for continuous functions:

E8. If α is an expression of type $(\mathbf{F} \times \mathbf{D}) \rightarrow \mathbf{E}$ then $Curry(\alpha)$ is one of type $\mathbf{F} \rightarrow (\mathbf{D} \rightarrow \mathbf{E})$, with denotation as specified above.

External Axiomatisation.

1. $\forall f: (\mathbf{F} \times \mathbf{D}) \rightarrow \mathbf{E}. f = apply \circ (Curry(f) \times id_{\mathbf{D}}).$
2. $\forall g: \mathbf{F} \rightarrow (\mathbf{D} \rightarrow \mathbf{E}). g = Curry(apply \circ (g \times id_{\mathbf{D}})).$

These axioms are easily seen to be true. Axiom 1 implies the existence part of the universal property: Axiom 2 implies uniqueness as if $g: \mathbf{F} \rightarrow (\mathbf{D} \rightarrow \mathbf{E})$ makes the above diagram commute, for a given $f: (\mathbf{F} \times \mathbf{D}) \rightarrow \mathbf{E}$,

then $g = \text{Curry}(\text{apply} \circ (g \times \text{id}_{\mathbf{D}}))$ (by Axiom 2) $= \text{Curry}(f)$. For the internal axiomatisation we have to expand our language further by introducing the *typed λ -calculus*:

E9. If e is an expression of type $\mathbf{E}$ and x is a variable of type $\mathbf{D}$ then $(\lambda x:\mathbf{D}.e)$ is an expression of type $(\mathbf{D} \rightarrow \mathbf{E})$.

Note that $\lambda x:\mathbf{D}.$ is a binding operator and so x is not a free variable in $\lambda x:\mathbf{D}.e$. We define the denotation of $(\lambda x:\mathbf{D}.e)$ whenever $e:\mathbf{E}$ satisfies the following conditions:

- A) the denotation of e is defined for all values of its free variables.
- B) if $x_0:\mathbf{D}_0, \dots, x_{m-1}:\mathbf{D}_{m-1}$ is a list of variables of the indicated types which includes all the free variables of e then the function $f: \prod_{i < m} \mathbf{D}_i \rightarrow \mathbf{E}$ associated with e by: $f(x_0, \dots, x_{m-1}) = e$ can be defined by an expression, $\alpha[e; x_0, \dots, x_{m-1}]$ built up using E3, E4, E7, E8.

Then if the free variables of $(\lambda x:\mathbf{D}.e)$ are $x_0, \dots, x_{n-1}$, the value of $(\lambda x:\mathbf{D}.e)$ at $x_0, \dots, x_{n-1}$ is the function $f:\mathbf{D} \rightarrow \mathbf{E}$ given by $f(x) = e$. This is a continuous function since it is obtained from either $\alpha[e; x_0, \dots, x_{n-1}]$ or $\alpha[e; x_0, \dots, x_{n-1}]$ by fixing the values of $x_0, \dots, x_{n-1}$. So we see that $(\lambda x:\mathbf{D}.e)$ satisfies condition A. We now check it also satisfies condition B. Let $x_0:\mathbf{D}_0, \dots, x_{m-1}:\mathbf{D}_{m-1}$ be a list of variables including all the free variables of $(\lambda x:\mathbf{D}.e)$. Now let $\mathbf{D}_m \stackrel{\text{def}}{=} \mathbf{D}$ and let $\theta: (\prod_{i < m} \mathbf{D}_i) \times \mathbf{D} \rightarrow \prod_{i \leq m} \mathbf{D}_i$ be the isomorphism $\langle \pi_0 \circ \pi_0, \dots, \pi_{m-1} \circ \pi_0, \pi_1 \rangle$ (which has inverse $\langle \langle \pi_0, \dots, \pi_{m-1} \rangle, \pi_m \rangle$). Then we can take $\alpha[\lambda x:\mathbf{D}.e; x_0, \dots, x_{m-1}]$ to be $\text{Curry}(\alpha[e; x'_0, \dots, x'_{m-1}, x] \circ \theta)$ where $x'_0:\mathbf{D}_0, \dots, x'_{m-1}:\mathbf{D}_{m-1}, x:\mathbf{D}_m$ are a list of distinct variables such that x'_i is x_i unless x_i is x .

It follows from these remarks and (i) to (iv) above that any expression e for an element built up by E1–E9 obeys conditions A and B. On the other hand we can now see that the expression for morphisms can be viewed as abbreviations for expressions for elements, now that the function-space construction allows us to think of morphisms as elements. Suppose we just allow an expression-forming rules E1, E2, E5' (which is E5 restricted to the case where α is a constant), E6, and E9. Then if $\alpha:\mathbf{B} \rightarrow \mathbf{C}$ and $\beta:\mathbf{A} \rightarrow \mathbf{B}$ are abbreviations for e' and e respectively, $\alpha \circ \beta$ is one for $\lambda x:\mathbf{A}.\text{apply}(e', \text{apply}(e, x))$; if $\alpha:\mathbf{A} \rightarrow \mathbf{B}$ abbreviates e' then $\alpha(e)$ abbreviates $\text{apply}(e', e)$; if $\alpha_0:(\mathbf{F} \rightarrow \mathbf{D}_0), \dots, \alpha_{n-1}:(\mathbf{F} \rightarrow \mathbf{D}_{n-1})$ abbreviates $e_0, \dots, e_{n-1}$ respectively $\langle \alpha_0, \dots, \alpha_{n-1} \rangle$ abbreviates $\lambda v:\mathbf{F}.\langle \text{apply}(e_0, v), \dots, \text{apply}(e_{n-1}, v) \rangle$; if $\alpha:\mathbf{F} \times \mathbf{D} \rightarrow \mathbf{E}$ abbreviates e then $\text{Curry}(\alpha)$ abbreviates $\lambda v:\mathbf{F}.\lambda d:\mathbf{D}.\text{apply}(e, \langle v, d \rangle)$.

The rules E1, E2, E5', E6, E9 give us a *typed λ -calculus* and it is convenient to introduce a few more abbreviations. For expressions $e': \prod_{i < n} \mathbf{A}_i \rightarrow \mathbf{B}$, $e_0:\mathbf{A}_0, \dots, e_{n-1}:\mathbf{A}_{n-1}$ $e'(e_0, \dots, e_{n-1})$ abbreviates $\text{apply}(e', \langle e_0, \dots, e_{n-1} \rangle)$; when $\mathbf{D}$ can be understood from the context $(\lambda x.e)$ abbreviates $(\lambda x:\mathbf{D}.e)$; the expression $(\lambda x_0:\mathbf{D}_0, x_1:\mathbf{D}_1, \dots, x_{n-1}:\mathbf{D}_{n-1}.e)$ abbreviates

$$(\lambda x: \prod_{i < n} \mathbf{D}_i. (\lambda x_0:\mathbf{D}_0 (\dots (\lambda x_{n-1}:\mathbf{D}_{n-1}.e) \dots)) (\pi_0(x)) \dots (\pi_{n-1}(x))).$$

Now the internal axiomatisation of the function space construction just consists of the familiar rules for the typed λ -calculus.

Internal Axiomatisation.

1. β -reduction: $(\lambda x.\alpha[e; x])e' = \alpha(e')$.
2. η -reduction: $(\lambda x.e(x)) = e$ (x not a free variable of e).

The above axioms are asserted for all expressions e and all values of their free variables. They are evidently true and we check that they imply the external axioms (together with our other axioms and a few elementary ones and principles of logic). Suppose $f: (\mathbf{F} \times \mathbf{D}) \rightarrow \mathbf{E}$ and calculate for each w in $\mathbf{F} \times \mathbf{D}$.

$$\begin{aligned} (\text{apply} \circ (\text{Curry}(f) \times \text{id}_{\mathbf{D}}))(w) &= \text{apply}((\text{Curry}(f) \times \text{id}_{\mathbf{D}})(w)) \\ &= \text{apply}(\text{Curry}(f)(\pi_0(w)), \pi_1(w)) \\ &= (\lambda v:\mathbf{F}.\lambda d:\mathbf{D}.\text{apply}(f, \langle v, d \rangle))(\pi_0(w))(\pi_1(w)) \\ &= (\lambda d:\mathbf{D}.f(\pi_0(w), d))(\pi_1(w)) && \beta\text{-reduction} \\ &= f(\pi_0(w), \pi_1(w)) && \beta\text{-reduction} \\ &= f(w). \end{aligned}$$

Then Axiom 1 follows by extensionality. For Axiom 2, suppose $g: \mathbf{F} \rightarrow (\mathbf{D} \rightarrow \mathbf{E})$ and calculate for each v in $\mathbf{F}$:

$$\begin{aligned}
Curry(apply \circ (g \times id_{\mathbf{D}}))(v) &= (\lambda v: \mathbf{F}. \lambda d: \mathbf{D}. apply(apply \circ (g \times id_{\mathbf{D}}), \langle v, d \rangle))(v) \\
&= \lambda d: \mathbf{D}. apply((g \times id_{\mathbf{D}})(v, d)) && \beta\text{-reduction} \\
&= \lambda d: \mathbf{D}. g(v)(d) \\
&= g(v) && \eta\text{-reduction.}
\end{aligned}$$

and then Axiom 2 follows by extensionality. Note by the way that all the operations such as composition, $\circ$, the two tupling operators, $\langle \cdot, \dots, \cdot \rangle$, and Currying (*Curry*) are all continuous as we have $\circ = \lambda f. \lambda g. g \circ f$ etc; also $\times$ acts continuously on morphisms.

As a functor exponentiation, $\rightarrow$, is defined on morphisms by defining $f \rightarrow f': (\mathbf{A} \rightarrow \mathbf{A}') \rightarrow (\mathbf{B} \rightarrow \mathbf{B}')$ where $f: \mathbf{B} \rightarrow \mathbf{A}$, $f': \mathbf{A}' \rightarrow \mathbf{B}'$ by:

$$f \rightarrow f' = \lambda x: \mathbf{A} \rightarrow \mathbf{A}'. f' \circ x \circ f.$$

Note that $f: \mathbf{B} \rightarrow \mathbf{A}$ rather than $f: \mathbf{A} \rightarrow \mathbf{B}$ which indicates that $\rightarrow: \mathcal{CPO}^2 \rightarrow \mathcal{CPO}$ is a bifunctor which is contravariant in its first argument and covariant in its second argument, which is to say that the following equations are true, where $f: \mathbf{B} \rightarrow \mathbf{A}$, $g: \mathbf{C} \rightarrow \mathbf{B}$, $f': \mathbf{A}' \rightarrow \mathbf{B}'$, $g': \mathbf{B}' \rightarrow \mathbf{C}'$:

- (i) $id_{\mathbf{D}} \rightarrow id_{\mathbf{E}} = id_{\mathbf{D} \rightarrow \mathbf{E}}$,
- (ii) $(f \circ g) \rightarrow (g' \circ f') = (g \rightarrow g') \circ (f \rightarrow f')$.

Again, exponentiation acts continuously on morphisms.

Now that we have the basic constructions on cpos and a language for expressing elements and morphisms we return to fixed-points and then see how to give a denotational semantics for *Imp* and *Dec* when extended by adding iteration and recursive declaration facilities. The least fixed-point, fix_f , of a continuous function $f: \mathbf{D} \rightarrow \mathbf{D}$ is itself a continuous function of f , $fix: (\mathbf{D} \rightarrow \mathbf{D}) \rightarrow \mathbf{D}$ as we have:

$$fix_f = \bigsqcup_n f^n(\perp) = \bigsqcup_n (\lambda f. f^n(\perp))(f) = (\bigsqcup_n \lambda f. f^n(\perp))(f)$$

showing that $fix = \bigsqcup_n \lambda f. f^n(\perp)$, a continuous function. Often fix is called, Y , after the paradoxical combinator of combinatory logic; we will often use the abbreviation $(\mu x: \mathbf{D}. e)$ (or just $\mu x. e$ when $\mathbf{D}$ is understood from the context) to stand for $fix(\lambda x. e)$ — clearly $\mu x. e$ is the least element, x such that $x = e$. Sometimes we just define elements, x , recursively by writing $x = e$, meaning $x = \mu x. e$, or even $f(x_0, \dots, x_{n-1}) = e$ for functions meaning $f = \mu f. (\lambda x_0, \dots, x_{n-1}. e)$.

Examples

1. In the case of the statement **while** b **do** s we will be given a meaning $p_{\perp}: \mathbf{S}_{\perp} \rightarrow \mathbf{T}_{\perp}$ for b (in the case that evaluation of b always terminates) and a meaning $\theta': \mathbf{S}_{\perp} \rightarrow \mathbf{S}_{\perp}$ for s and take the meaning of the statement to be $\theta = fix(F)$ where $F: (\mathbf{S}_{\perp} \rightarrow \mathbf{S}_{\perp}) \rightarrow (\mathbf{S}_{\perp} \rightarrow \mathbf{S}_{\perp})$ is $F = \lambda \theta: (\mathbf{S}_{\perp} \rightarrow \mathbf{S}_{\perp}). \lambda \sigma: \mathbf{S}_{\perp}. cond(p_{\perp}(\sigma), \theta(\theta'(\sigma)), \sigma)$.
2. In the case of the recursive declaration **letrec** $f(x)$ **be** e **in** e' we shall take the meaning of f to be $\mu g. G(g)$ where $G(g)(v)$ is the value of e in the current environment, altered so that f and x have values g and v respectively.
3. Of course we can just define elements without using any particular programming language. So addition can be defined by taking over the usual primitive recursive definition as:

$$+ = \mu f: (\mathbf{N}_{\perp} \times \mathbf{N}_{\perp} \rightarrow \mathbf{N}_{\perp}). \lambda m: \mathbf{N}_{\perp}. \lambda n: \mathbf{N}_{\perp}. (n = 0 \rightarrow m \mid f(m, n - 1) + 1).$$

We can also define functions by minimalisation. Suppose we have $p_{\perp}: \mathbf{N}_{\perp}^2 \rightarrow \mathbf{T}_{\perp}$ and wish to define $f: \mathbf{N}_{\perp} \rightarrow \mathbf{N}_{\perp}$ as $f(m) =$ the smallest integer n such that $p_{\perp}(m, n) = tt$. Then

$$f = \lambda n. (\mu h. \lambda m. (p(n, m) \rightarrow m \mid h(m + 1)))(0).$$

It should now be clear that in this way we can define (the minimal extension of) any partial recursive function or predicate over the integers. We can also define many functionals at higher types. For instance suppose we want to define $sum: ((\mathbf{N}_{\perp} \rightarrow \mathbf{N}_{\perp}) \times \mathbf{N}_{\perp}) \rightarrow \mathbf{N}_{\perp}$ so that $sum(f, n) = f(0) + \dots + f(n - 1)$, then sum is defined recursively by: $sum(f, n) = (n = 0 \rightarrow 0 \mid f(n - 1) + sum(f, n - 1))$, or, in other words: $sum = \mu sum. \lambda f. \lambda n. (n = 0 \rightarrow 0 \mid f(n - 1) + sum(f, n - 1))$. Later on we will see that all the functions and functionals we can define are computable, in a sense to be described.

4. In the case of domains such as **Tapes** we can define some infinite elements by recursion: for example $010101\dots$ is just $\mu x.0.1.x$. One way to obtain nonperiodic sequence is to take $p: \mathbf{N} \rightarrow \{0, 1\}$ and define $g: \mathbf{N} \rightarrow \mathbf{Tapes}$ recursively by:

$$g(n) = (p(n) = 0 \rightarrow 0.g(n+1) \mid 1.g(n+1)).$$

Then $g(0)$ is $p(0)p(1)p(2)p(3)\dots$.

Similarly in the case of $\mathbf{P}\omega$ take $p: \mathbf{N}_\perp \rightarrow \mathbf{T}_\perp$ and define $g: \mathbf{N}_\perp \rightarrow \mathbf{P}\omega$ by:

$$g(n) = \{g(n+1) + 1\} \cup (p(n) \rightarrow \{0\} \mid \perp)$$

where $\cup: \mathbf{P}\omega^2 \rightarrow \mathbf{P}\omega$ is the (continuous) union function and $\{0\}$ is the singleton zero set. Then $g(0) = \{n \in \omega \mid p(n) = tt\}$.

5. Simultaneous fixed-points are easily reduced to single ones. Suppose $f_i: (\prod_{i < n} \mathbf{D}_i) \rightarrow \mathbf{D}_i$ ($i < n$) are continuous functions. The elements $d_0, \dots, d_{n-1}$ of $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ respectively are simultaneous fixed-points of $f_0, \dots, f_{n-1}$ iff

$$\begin{aligned} d_0 &= f_0(d_0, \dots, d_{n-1}), \\ &\vdots \\ d_{n-1} &= f_{n-1}(d_0, \dots, d_{n-1}). \end{aligned}$$

Then $d_0, \dots, d_{n-1}$ are simultaneous fixed-points of $f_0, \dots, f_{n-1}$ iff $\langle d_0, \dots, d_{n-1} \rangle$ is a fixed-point of $\langle f_0, \dots, f_{n-1} \rangle$ and we see that the least simultaneous fixed-points are $d_i = (fix(\langle f_0, \dots, f_{n-1} \rangle))_i$.

For instance we can define functions $g: \mathbf{N}^2 \rightarrow \mathbf{N}$ recursively by:

$$\begin{aligned} g(x, y) &= (x = 0 \rightarrow 7 \mid g(x-1, h(y))), \\ h(x) &= (x = 0 \rightarrow 6 \mid g(x-1, x+1)) \end{aligned}$$

meaning that g, h should be the least simultaneous fixed points of the functions F_0, F_1 where $F_0 = \lambda g: (\mathbf{N}^2 \rightarrow \mathbf{N}). h: \mathbf{N} \rightarrow \mathbf{N}. \lambda x: \mathbf{N}. \lambda y: \mathbf{N}. (x = 0 \rightarrow 7 \mid g(x-1, h(y)))$ and $F_1 = \lambda g, h. \lambda x. (x = 0 \rightarrow 6 \mid g(x-1, x+1))$.

The idea of making definitions with the aid of the least *fixed-point* operator, *fix*, leads to a major proof rule, due to Scott, called *fixed-point induction* (sometimes *computational induction*).

Definition 2. Let $\mathbf{D}$ be a cpo. A property $\mathbf{P}$ (a subset of $\mathbf{D}$) is ω -*inductive* (= *inclusive*) iff whenever $\langle x_n \rangle_{n \in \omega}$ is an increasing sequence of elements in $\mathbf{P}$, its lub, $\bigsqcup_n x_n$ is also in $\mathbf{P}$.

Fixed-point Induction. Let $\mathbf{P} \subseteq \mathbf{D}$ be ω -inductive and let $f: \mathbf{D} \rightarrow \mathbf{D}$ be continuous. Then we have: $[\mathbf{P}(\perp) \wedge (\forall x: \mathbf{D}. \mathbf{P}(x) \Rightarrow \mathbf{P}(fx))] \Rightarrow \mathbf{P}(fix(f))$.

The validity of this principle is clear: note that it can be rephrased as every subcpo closed under a continuous function contains fix_f . One idea behind fixed-point induction is to avoid integer reasoning to show $\forall n \in \omega. \mathbf{P}(f^n(\perp))$ when proving $\mathbf{P}(fix_f)$.

Examples. For $f: \mathbf{D} \rightarrow \mathbf{D}$ prove $fix_f \sqsubseteq f(fix_f)$ by using $\mathbf{P}(x) \stackrel{\text{def}}{=} x \sqsubseteq fx$. The opposite inclusion needs a separate axiom in any formal system.

It is important to be able to manufacture inclusive predicates.

- Basic Relations.** Let $\mathbf{D}$ be a cpo. Then $\emptyset, \mathbf{D}$ are inclusive predicates on $\mathbf{D}$; $x = y$ and $x \sqsubseteq y$ are inclusive relations on $\mathbf{D} \times \mathbf{D}$.
- Inverse Image.** Let $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ be cpos and let $\mathbf{R} \subseteq \prod_{i < n} \mathbf{D}_i$ be inclusive. Then if $f_i: \prod_{j < m} \mathbf{E}_j \rightarrow \mathbf{D}_i$ are i continuous functions ($i < n$) then

$$\mathbf{Q}(y_0, \dots, y_{m-1}) \stackrel{\text{def}}{=} \mathbf{R}(f_0(y_0, \dots, y_{m-1}), \dots, f_{n-1}(y_0, \dots, y_{m-1}))$$

is inclusive too. In particular taking $\mathbf{D}_0 = \mathbf{D}_1$ and $\mathbf{R}(x_0, x_1) \equiv x_0 \sqsubseteq x_1$ we find that inclusions of the form $f_0(y_0, \dots, y_{m-1}) \sqsubseteq f_1(y_0, \dots, y_{m-1})$ are inclusive.

- Logical.** Let $\mathbf{D}$ be a cpo and if $\mathbf{P}, \mathbf{Q} \subseteq \mathbf{D}$ are inclusive so are $\mathbf{P} \cap \mathbf{Q}$ and $\mathbf{P} \cup \mathbf{Q}$: If $\mathbf{E}$ is another cpo and $\mathbf{R} \subseteq \mathbf{D} \times \mathbf{E}$ is inclusive with respect to its first argument then $\forall y: \mathbf{E}. \mathbf{R}(x, y)$ is inclusive.

4. **Products.** Let $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ be cpos and let $\mathbf{P}_0 \subseteq \mathbf{D}_0, \dots, \mathbf{P}_{n-1} \subseteq \mathbf{D}_{n-1}$ be inclusive. Then $\prod_{i < n} \mathbf{P}_i \subseteq \prod_{i < n} \mathbf{D}_i$ is inclusive where $\prod_{i < n} \mathbf{P}_i = \{x \in \prod_{i < n} \mathbf{D}_i \mid \forall i < n. \mathbf{P}_i((x)_i)\}$.

5. **Exponentiation.** Let $\mathbf{D}, \mathbf{E}$ be cpos and let $\mathbf{P} \subseteq \mathbf{D}, \mathbf{Q} \subseteq \mathbf{E}$ be inclusive. Then $(\mathbf{P} \rightarrow \mathbf{Q}) \subseteq (\mathbf{D} \rightarrow \mathbf{E})$ is inclusive where: $(\mathbf{P} \rightarrow \mathbf{Q}) = \{f: \mathbf{D} \rightarrow \mathbf{E} \mid \forall d: \mathbf{D}. \mathbf{P}(d) \Rightarrow \mathbf{Q}(fd)\}$.

Unfortunately, as remarked above, we do not have time to pursue issues connected with proof much beyond stating the relevant axioms. Next we see that it is now possible to give the semantics of *Dec* and *Imp* with the added iterative and recursive declaration features.

For *Imp* we have the syntax of Chapter 1, including the iteration statement. The semantic domains are $\mathbf{T}_\perp, \mathbf{S}_\perp$, and now $\mathbf{C} = (\mathbf{S}_\perp \rightarrow \mathbf{S}_\perp)$ the function space cpo. The denotational functions are $\mathcal{B}: \mathbf{BExp} \rightarrow (\mathbf{S}_\perp \rightarrow \mathbf{T}_\perp)$ (where the range of $\mathbf{BExp}$ is the cpo $(\mathbf{S}_\perp \rightarrow \mathbf{T}_\perp)$), $\mathcal{A}: \mathbf{Act} \rightarrow \mathbf{C}$ and $\mathcal{C}: \mathbf{Stat} \rightarrow \mathbf{C}$ which is defined by structural induction on statements by:

- (i) $\mathcal{C}[a] = \mathcal{A}[a]$;
- (ii) $\mathcal{C}[\mathbf{dummy}] = \lambda\sigma: \mathbf{S}_\perp. \sigma$;
- (iii) $\mathcal{C}[(s_1; s_2)] = \mathcal{C}[s_2] \circ \mathcal{C}[s_1]$;
- (iv) $\mathcal{C}[\mathbf{if } b \mathbf{ then } s_1 \mathbf{ else } s_2] = \lambda\sigma: \mathbf{S}_\perp. (\mathcal{B}[b](\sigma) \rightarrow \mathcal{C}[s_1](\sigma) \mid \mathcal{C}[s_2](\sigma))$;
- (v) $\mathcal{C}[\mathbf{while } b \mathbf{ do } s] = \mu\theta: \mathbf{C}. \lambda\sigma: \mathbf{S}_\perp. (\mathcal{B}[b](\sigma) \rightarrow \theta(\mathcal{C}[s](\sigma)) \mid \sigma)$.

We have abstracted the free variables on the right-hand side of the definitions in Chapter 1 to make it a little clearer that the meaning of a statement is a function of the meanings of its components. Note that our discussion on the typed λ -calculus we are using as a metalanguage show that, by structural induction, each statement receives a meaning expressible in this typed λ -calculus providing each primitive expression of the form $\mathcal{A}[a]$ or $\mathcal{B}[b]$ is taken as a constant. This is, perhaps, a little awkward; ideally we might like to have a single expression in the metalanguage for the function $\mathcal{C}$, and that would require us to take the syntax as a cpo too. It is most convenient to leave this idea for the time being and return to it when considering recursively specified domain equations.

Equations (i)–(iv) are just taken over from the equations we had when using ordinary sets to give the semantics. Equation (v) should be exactly what the reader expects given the discussion of the iteration statement in Chapter 1 and Example 1 above. Notice that the cpo $\mathbf{C}$ is too large in the sense that commands should never be nonstrict. This point will be taken up when we consider the strict function space construction in Chapter 3.

Turning to *Dec*, we have the syntax of Chapter 1, including the recursive declaration facility extended to functions of several arguments. We assume that there are fixed limits (m and n) on the arity of the function and operation symbols, as we have only considered finite products so far. For the semantic domains we take $\mathbf{V}$ to be a fixed cpo of values, $\mathbf{IEnv}$ to be $(\mathbf{Id}_\perp \rightarrow \mathbf{V})$, $\mathbf{FEnv}$ to be $(\mathbf{Fun}_j)_\perp \rightarrow (\mathbf{V}^j \rightarrow \mathbf{V})$. Again we may question why nonstrict environments are allowed. The denotational functions are $\mathcal{I}_i: \mathbf{Op}_i \rightarrow (\mathbf{V}^i \rightarrow \mathbf{V})$ ($i < n$), $\mathcal{B}: \mathbf{BExp} \rightarrow ((\mathbf{IEnv} \times \mathbf{FEnv}) \rightarrow \mathbf{T})$ and $\mathcal{E}: \mathbf{Exp} \rightarrow ((\mathbf{IEnv} \times \mathbf{FEnv}) \rightarrow \mathbf{V})$ which is defined by structural induction:

- (i) $\mathcal{E}[x] = \lambda\rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. \rho[x]$;
- (ii) $\mathcal{E}[o_i(e_0, \dots, e_{i-1})] = \lambda\rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. \mathcal{I}_i[o_i](\mathcal{E}[e_0](\rho, \varphi), \dots, \mathcal{E}[e_{i-1}](\rho, \varphi))$ ($i < n$);
- (iii) $\mathcal{E}[\mathbf{if } b \mathbf{ then } e_0 \mathbf{ else } e_1] = \lambda\rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. (\mathcal{B}[b](\rho, \varphi) \rightarrow \mathcal{E}[e_0](\rho, \varphi) \mid \mathcal{E}[e_1](\rho, \varphi))$;
- (iv) $\mathcal{E}[\mathbf{let } x \mathbf{ be } e_0 \mathbf{ in } e_1] = \lambda\rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. \mathcal{E}[e_1](\rho[\mathcal{E}[e_0](\rho, \varphi)/x], \varphi)$;
- (v)_j $\mathcal{E}[\mathbf{let } f_j(x_0, \dots, x_{j-1}) \mathbf{ be } e_0 \mathbf{ in } e_1] =$
 $\lambda\rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. \mathcal{E}[e_1](\rho, \varphi[\lambda w: \mathbf{V}^j. \mathcal{E}[e_0](\rho[(w)_0/x_0] \dots [(w)_{j-1}/x_{j-1}], \varphi)/f_j])$;
- (vi)_j $\mathcal{E}[\mathbf{letrec } f_j(x_0, \dots, x_{j-1}) \mathbf{ be } e_0 \mathbf{ in } e_1] =$
 $\lambda\rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. \mathcal{E}[e_1](\rho, \varphi[\mu f: \mathbf{V}^j \rightarrow \mathbf{V}. \lambda w: \mathbf{V}^j. \mathcal{E}[e_0](\rho[(w)_0/x_0] \dots [(w)_{j-1}/x_{j-1}],$
 $\varphi[f/f_j]/f_j])$.

Here $[\cdot/\cdot]: \mathbf{IEnv} \times \mathbf{V} \times \mathbf{Id}_\perp \rightarrow \mathbf{IEnv}$ is defined by:

$$\rho[v/x] = \lambda y: \mathbf{Id}_\perp. ((y = x) \rightarrow v \mid \rho[y]),$$

and $[\cdot/\cdot]: \mathbf{FEnv} \times (\mathbf{V}^j \rightarrow \mathbf{V}) \times (\mathbf{Fun}_j)_\perp \rightarrow \mathbf{FEnv}$ is defined by:

$$\varphi[g/f] = \langle \varphi_0, \dots, \varphi_{j-1}, \lambda f_j: (\mathbf{Fun}_j)_\perp. (f_j = f \rightarrow g \mid (\varphi)_j(f_j)), \varphi_{j+1}, \dots, \varphi_{m-1} \rangle.$$

Again equations (i)–(v) are taken over from Chapter 1 and equation (vi) should be as expected from previous discussions. Again there is a slight awkwardness in treating syntax in terms of sets. Further this semantics corresponds to call-by-name (see Exercise 33, below) and the reader may wonder how call-by-value can be catered for in the present framework. Finally we note that the semantic equation for assignments in Chapter 1 carries over to the present situation without any alteration.

Exercises

1. Let $\mathbf{D}$ be a cpo. Use the external axiomatisation of Cartesian product to show that $\langle f_0, \dots, f_{n-1} \rangle \circ g = \langle f_0 \circ g, \dots, f_{n-1} \circ g \rangle$ and then prove the functor laws for $\prod$. Define $\Delta_{\mathbf{D}}: \mathbf{D} \rightarrow \mathbf{D} \times \dots \times \mathbf{D}$ (the diagonal map) by: $\Delta_{\mathbf{D}} = \langle \text{id}_{\mathbf{D}}, \dots, \text{id}_{\mathbf{D}} \rangle$ and show that: $\langle f_0, \dots, f_{n-1} \rangle = (f_0 \times \dots \times f_{n-1}) \circ \Delta_{\mathbf{D}}$.
2. Prove the isomorphisms: $\mathbf{U} \times \mathbf{D} \cong \mathbf{D} \times \mathbf{U} \cong \mathbf{D}$; $\mathbf{D} \times \mathbf{E} \cong \mathbf{E} \times \mathbf{D}$; $\mathbf{D} \times (\mathbf{E} \times \mathbf{F}) \cong (\mathbf{D} \times \mathbf{E}) \times \mathbf{F}$. Formulate and prove general versions of these commutative and associative laws for arbitrary finite products.
3. For any map $\delta: [m] \rightarrow [n]$ (where $[m] = \{j \in \mathbf{N} \mid j < m\}$ for any m in $\mathbf{N}$) and cpos $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ and $\mathbf{E}_0, \dots, \mathbf{E}_{m-1}$ where $\mathbf{E}_j = \mathbf{D}_{\delta(j)}$ define the natural *substitution* map $\mathcal{S}_\delta: \prod_{i < n} \mathbf{D}_i \rightarrow \prod_{j < m} \mathbf{E}_j$. Show that $\mathcal{S}_\delta \circ \mathcal{S}_{\delta'} = \mathcal{S}_{\delta' \circ \delta}$. Show that for any expression, e , of the metalanguage whose free variables are included in the list $x_0: \mathbf{D}_0, \dots, x_{n-1}: \mathbf{D}_{n-1}$ and also in the list $y_0: \mathbf{E}_0, \dots, y_{m-1}: \mathbf{E}_{m-1}$ (given via a map $\delta: [m] \rightarrow [n]$ as above) we have: $\alpha[e; x_0, \dots, x_{n-1}] = \alpha[e; y_0, \dots, y_{m-1}] \circ \mathcal{S}_\delta$.
4. Investigate products $\prod_{i \in S} \mathbf{D}_i$ over any set S . Is it the case that a function of infinitely many arguments is continuous iff it is continuous in each of its arguments separately? [*Hint* Consider the case $S = \omega$ ($= \mathbf{N}$) and $\mathbf{D}_i = \mathbf{O}$.] When the $\mathbf{D}_i$ are all the same cpo, $\mathbf{D}$, we obtain a power, $\mathbf{D}^S$. Show that $\mathbf{T}^\omega \cong \mathbf{T} \times \mathbf{T}^\omega$. Why are $\mathbf{T}^\omega$ and **Tapes** not isomorphic?
5. Let $\mathbf{D}$ and $\mathbf{E}_0, \dots, \mathbf{E}_{n-1}$ be cpos. Show that for any continuous function $f: \mathbf{D} \rightarrow \prod_{i < n} \mathbf{E}_i$ there are unique maps $f_i: \mathbf{D} \rightarrow \mathbf{E}_i$ ($i < n$) such that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{D} & & \\ \Delta \downarrow & \searrow f & \\ \mathbf{D}^n & \xrightarrow{\prod_{i < n} f_i} & \prod_{i < n} \mathbf{E}_i \end{array}$$

6. Show that $\prod_{i < n} \mathbf{D}_i$ is also the product in $\mathcal{CPO}_\perp$. [*Hint* Check everything is strict.]
7. Check the isomorphisms: $\langle \cdot, \dots, \cdot \rangle: \prod_{i < n} (\mathbf{D} \rightarrow \mathbf{E}_i) \cong \mathbf{D} \rightarrow \prod_{i < n} \mathbf{E}_i$; $(\mathbf{D} \rightarrow \mathbf{U}) \cong \mathbf{U}$; $(\mathbf{U} \rightarrow \mathbf{D}) \cong \mathbf{D}$; $\mathbf{D} \rightarrow \mathbf{E}^S \cong (\mathbf{D} \rightarrow \mathbf{E})^S$.
8. Let $\mathbf{C}, \mathbf{D}, \mathbf{E}$ be cpos. Define $\text{pair}: \mathbf{C} \rightarrow (\mathbf{D} \rightarrow (\mathbf{C} \times \mathbf{D}))$ by $\text{pair} = \text{Curry}(\text{id}_{\mathbf{C} \times \mathbf{D}}) = \lambda x. \lambda y. \langle x, y \rangle$. Show that for any continuous $f: \mathbf{C} \rightarrow (\mathbf{D} \rightarrow \mathbf{E})$ there is a unique $g: \mathbf{C} \times \mathbf{D} \rightarrow \mathbf{E}$ (namely $\text{Curry}^{-1}(f)$) such that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{C} & & \\ \text{pair} \downarrow & \searrow f & \\ \mathbf{D} \rightarrow (\mathbf{C} \times \mathbf{D}) & \xrightarrow{(\text{id}_{\mathbf{D}} \rightarrow g)} & (\mathbf{D} \rightarrow \mathbf{E}) \end{array}$$

Show $\text{Curry} = \lambda f. (\text{id} \times f) \circ \text{pair}$ and $\text{Curry}^{-1} = \lambda g. \text{apply} \circ \langle g \circ \pi_0, \pi_1 \rangle$.

9. Show that exponentiation preserves strictness giving a bifunctor $\rightarrow: \mathcal{CPO}_\perp^2 \rightarrow \mathcal{CPO}_\perp$ which is contravariant in its first argument and covariant in its second argument.

10. Show that every continuous function, $p: \mathbf{T}^n \rightarrow \mathbf{T}$, can be defined from tt , ff and $parcond$ as the only constants using the typed λ -calculus (you will only need first-order terms — those with no λ 's). A collection of type symbols so are $(\sigma \times \tau)$ and $(\sigma \rightarrow \tau)$; the cpos $\mathbf{T}_\sigma$ are defined by: $\mathbf{T}_0 = \mathbf{T}$, $\mathbf{T}_{\sigma \times \tau} = \mathbf{T}_\sigma \times \mathbf{T}_\tau$, $\mathbf{T}_{\sigma \rightarrow \tau} = \mathbf{T}_\sigma \rightarrow \mathbf{T}_\tau$. Show that every element of every $\mathbf{T}_\sigma$ can be defined from tt , ff and $parcond$ using the typed λ -calculus. [Warning This last part is a good deal harder.]

11. Let $\mathbf{X}_0, \dots, \mathbf{X}_{n-1}, \mathbf{Y}$ be sets. Show any function $f: \mathbf{X}_0 \times \dots \times \mathbf{X}_{n-1} \rightarrow \mathbf{Y}$ has a *maximal* extension to a continuous function $g: \prod_{i < n} (\mathbf{X}_i)_\perp \rightarrow \mathbf{Y}_\perp$. Is $parcond_{\mathbf{D}}$ always such a maximal extension?

12. Both Milner and Vuillemin have given definitions of sequentiality. These depend on viewing a function $f: \prod_{i < n} \mathbf{D}_i \rightarrow \mathbf{E}$ as being of n arguments (viewed as being more or less arguments may change its character according to these definitions!). Such a function is *M-sequential* if either it is constant or there is an integer i (with $0 \leq i < n$) such that f is strict in its i -th argument ($(x)_i = \perp$ implies $f(x) = \perp$) and the function obtained by fixing its i -th argument (to, say, a_i)

$$(\lambda x_0: \mathbf{D}_0, \dots, x_{i-1}: \mathbf{D}_{i-1}, x_{i+1}: \mathbf{D}_{i+1}, \dots, x_{n-1}: \mathbf{D}_{n-1}. f(x_0, \dots, x_{i-1}, a_i, x_{i+1}, \dots, x_{n-1}))$$

is M-sequential. Also such a function is *V-sequential* iff either it is a constant or there is an integer i (with $0 \leq i < n$) such that $y \sqsupseteq x$ and $(y)_i = (x)_i$ implies $f(y) = f(x)$. Which of the functions of several arguments we have considered are sequential in one or other of these senses? Show that if $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}, \mathbf{E}$ are all flat cpos then the two definitions are equivalent. Show that all sequential functions $p: \mathbf{T}^n \rightarrow \mathbf{T}$ are λ -definable using as constants tt , ff and $cond$.

13. Define the minimalisation operator $\bar{\mu}: (\mathbf{N} \rightarrow \mathbf{T}) \rightarrow \mathbf{T}$ in the typed λ -calculus, using the primitive sequential functions on $\mathbf{N}$ and $\mathbf{T}$. What about the function $\partial: (\mathbf{N} \rightarrow \mathbf{T}) \rightarrow \mathbf{T}$? What is the greatest existential quantifier $\bar{\exists}: (\mathbf{N} \rightarrow \mathbf{T}) \rightarrow \mathbf{T}$ you can define using the primitive functions and $parcond$? (See Exercise 1.14, 1.15).

14. We can define elements of **Tapes** by simultaneous fixed-points using functions defined by first-order terms with $(0 \cdot -)$, $(1 \cdot -)$ as consists such as:

$$x = 0 \cdot 0 \cdot 1 \cdot y,$$

$$y = 1 \cdot 0 \cdot 0 \cdot x.$$

In this way only ultimately periodic functions can be defined. What happens in the case of **Btrees** if we allow as constants numerals (for the corresponding numbers), *tip* and *mktree*?

15 (Bekič). Let $f: \mathbf{D} \times \mathbf{E} \rightarrow \mathbf{D}$, $g: \mathbf{D} \times \mathbf{E} \rightarrow \mathbf{E}$ be continuous functions. We can solve the simultaneous fixed-point equations:

$$x = f(x, y),$$

$$y = g(x, y).$$

by a method of elimination. First define a parameterised solution to the first equation by: $h(y) = \mu x. f(x, y)$, then substitute in the second to obtain the equation $y = g(h(y), y) = k(y)$, say. Then show that $\bar{x}$, $\bar{y}$ are the least simultaneous fixed-points of the above equations where $\bar{y} = \mu y. k(y)$ and $\bar{x} = h(\bar{y})$. What does this amount to when $f(x, y)$ is independent of y ? What if, further, g does not depend on x ?

16. Theorem 1 in Chapter 1 shows the validity of the following principle of *recursion induction* (McCarthy, Park). Let $f: \mathbf{D} \rightarrow \mathbf{D}$ be continuous. Then:

$$\forall x: \mathbf{D}. (f(x) \sqsubseteq x \Rightarrow Y f \sqsubseteq x).$$

Establish this principle using fixed-point induction.

17. Give examples of non-inclusive predicates on $\mathbf{P}\omega$, **Fun**. Show that if $\mathbf{P} \subseteq \mathbf{D}$ is inclusive $\mathbf{D} - \mathbf{P}$ need not be; if $\mathbf{R} \subseteq \mathbf{D} \times \mathbf{E}$ is inclusive $\forall y. \mathbf{R}(x, y)$ need not be; if $\mathbf{R} \subseteq \mathbf{D} \times \mathbf{E}$, $\mathbf{S} \subseteq \mathbf{E} \times \mathbf{F}$ are inclusive, $\mathbf{S} \circ \mathbf{R}$ need not be.

18. Show that $\mathbf{R} \subseteq \prod_{i < n} \mathbf{D}_i$ is inclusive iff it is inclusive in each argument taken separately.

19. Show, using fixed-point induction, the following principle of simultaneous fixed-point induction. Let $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ be cpos, $\mathbf{R} \subseteq \prod_{i < n} \mathbf{D}_i$ be inclusive and $f_i: \prod_{i < n} \mathbf{D}_i \rightarrow \mathbf{D}_i$ ($i < n$) be continuous functions. Then:

$$[\mathbf{R}(\perp, \dots, \perp) \wedge (\forall x_0: \mathbf{D}_0, \dots, x_{n-1}: \mathbf{D}_{n-1}. \mathbf{R}(x_0, \dots, x_{n-1}) \Rightarrow \mathbf{R}(f_0(x_0, \dots, x_{n-1}), \dots, f_{n-1}(x_0, \dots, x_{n-1})))] \\ \Rightarrow \mathbf{R}(Y(\langle f_0, \dots, f_{n-1} \rangle)).$$

20. Let $f: \mathbf{D} \rightarrow \mathbf{D}$, $g: \mathbf{D} \rightarrow \mathbf{D}$ be continuous functions. Show $f(Y(g \circ f)) = Y(f \circ g)$. Show that if $f\perp = g\perp$ and f and g commute ($f \circ g = g \circ f$) then $Yf = Y(f \circ g) = Yg$. This and the next few exercises should be tried using fixed-point induction or recursion induction, as appropriate. (Try Exercise 15 this way too).

21. Let $f: \mathbf{D} \rightarrow \mathbf{D}$, $g: \mathbf{D} \rightarrow \mathbf{D}$ be continuous functions. Show that if $f \circ f \circ g = g \circ f$ and $f\perp = g\perp$ then $Yf = Yg$. [Warning This is hard by fixed-point induction. The only known proof needs an inclusive predicate defined by the conjunction of three inequalities (order relations of the form $t \sqsubseteq u$).]

22. Define $h: \mathbf{N} \rightarrow \mathbf{N}$ recursively by: $h(x) = h(x) + 1$. Show $h = \perp$. For $a: \mathbf{D}$, $h: \mathbf{D} \rightarrow \mathbf{D}$, and $p: \mathbf{D} \rightarrow \mathbf{T}$ define $g: \mathbf{D}^2 \rightarrow \mathbf{D}$ recursively by: $g(x, y) = (p(x) \rightarrow g(x, h(y)) \mid a)$. Show $g = \perp$.

23. For $p: \mathbf{D} \rightarrow \mathbf{T}$, $h: \mathbf{D} \rightarrow \mathbf{D}$, $k: \mathbf{D} \rightarrow \mathbf{D}$ define $f, g: \mathbf{D}^2 \rightarrow \mathbf{D}$ recursively by:

$$f(x, y) = (p(x) \rightarrow y \mid h(f(k(x), y))), \\ g(x, y) = (p(x) \rightarrow y \mid g(k(x), h(y))).$$

Show $h(f(x, y)) = f(x, hy)$ and then show $f = g$.

24. Suppose $a: \mathbf{D}$, $p: \mathbf{D} \rightarrow \mathbf{T}$, $d: \mathbf{D} \rightarrow \mathbf{D}$, $b: \mathbf{D} \times \mathbf{D} \rightarrow \mathbf{D}$ are continuous functions which satisfy the following axioms:

<i>Associativity</i>	$\forall x, y, z: \mathbf{D}. b(x, b(y, z)) = b(b(x, y), z).$
<i>Commutativity</i>	$\forall x, y: \mathbf{D}. b(x, y) = b(y, x).$
<i>Left Identity</i>	$\forall x: \mathbf{D}. b(a, x) = x.$
<i>Bistrictness</i>	$\forall x: \mathbf{D}. b(\perp, x) = b(x, \perp) = \perp.$

Define $f: \mathbf{D} \rightarrow \mathbf{D}$, $g: \mathbf{D}^2 \rightarrow \mathbf{D}$ recursively by:

$$f(x) = (p(x) \rightarrow a \mid b(x, f(d(x)))), \\ g(x, y) = (p(x) \rightarrow y \mid g(d(x), b(x, y))).$$

Show $g(x, y) = b(f(x), y)$ and deduce $f(x) = g(x, a)$. Interpret this result when $\mathbf{D} = \mathbf{N}$, $a = 1$, $p = Z$, $d = (\cdot - 1)$ and $b = \times$. Why does fixed-point induction on f not help to show that $\partial \circ f(x) = \partial(x)$ for all x in $\mathbf{D}$ (termination)? What does help?

25. Let $\mathbf{D}$ be a cpo. Show that $Y_{\mathbf{D}} = \mu F: ((\mathbf{D} \rightarrow \mathbf{D}) \rightarrow \mathbf{D}). \lambda f: (\mathbf{D} \rightarrow \mathbf{D}). f(Ff)$.

26. If $\mathbf{D}$ and $\mathbf{E}$ are partial orders, we say a function $f: \mathbf{D} \rightarrow \mathbf{E}$ is *continuous* iff it is monotonic and preserves lubs of increasing ω -chains (that is, if $\langle x_n \rangle_{n \in \omega}$ is an increasing ω -chain in $\mathbf{D}$ such that $\bigsqcup_{\mathbf{D}} x_n$ exists then $f(\bigsqcup_{\mathbf{D}} x_n) = \bigsqcup_{\mathbf{E}} f(x_n)$). Is it the case that if $\mathbf{D}$ is any partial order with a least element, but which is not a cpo then there is a continuous function $f: \mathbf{D} \rightarrow \mathbf{D}$ with no fixed-point? This seems to be an open question — a solution would help show the necessity of considering cpos.

27. A *fixed-point operator of type $\mathbf{D}$* is a function $F_{\mathbf{D}}: (\mathbf{D} \rightarrow \mathbf{D}) \rightarrow \mathbf{D}$ (not necessarily continuous) such that for all continuous $f: \mathbf{D} \rightarrow \mathbf{D}$ we have $f(Ff) = Ff$; it is *invariant* iff for every automorphism, $\theta: \mathbf{D} \rightarrow \mathbf{D}$, we always have: $Ff = \theta^{-1}(F(\theta \circ f \circ \theta^{-1}))$. What are the (continuous, invariant) fixed-point operators on $\mathbf{T}^\omega$? Show $Y_{\mathbf{D}}$ is always invariant.

28. A strict *monic* is a monic in $\mathcal{CPO}_{\perp}$, i.e. a strict continuous function, $m: \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ such that for any $f, g: \mathbf{C} \rightarrow_{\perp} \mathbf{D}$, $m \circ f = m \circ g$ implies $f = g$. A strict *order monic* is a strict continuous function $m: \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ such that for any $f, g: \mathbf{C} \rightarrow_{\perp} \mathbf{D}$, $m \circ f \sqsubseteq m \circ g$ implies $f \sqsubseteq g$. Show that $m: \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ is a strict monic iff for all x, y in $\mathbf{D}$, $m(x) = m(y)$ implies $x = y$; show it is a strict order monic iff for all x, y in $\mathbf{D}$, $m(x) \sqsubseteq m(y)$ implies $x \sqsubseteq y$. Let $\mathbf{P} \subseteq \mathbf{D}$ be an inclusive predicate such that $\mathbf{P}(\perp)$. Show that $\mathbf{P}$ is a cpo under the order inherited from $\mathbf{D}$ and that the natural inclusion map $\iota: \mathbf{P} \rightarrow_{\perp} \mathbf{D}$ is a strict order monic. Show that $m: \mathbf{C} \rightarrow_{\perp} \mathbf{D}$ is a strict order monic iff it factors as $\mathbf{C} \xrightarrow{\theta}_{\perp} \mathbf{P} \xrightarrow{\iota}_{\perp} \mathbf{D}$ (i.e. $m = \iota \circ \theta$) where $\mathbf{P} \subseteq \mathbf{D}$ is an inclusive predicate on $\mathbf{D}$ such that $\mathbf{P}(\perp)$ and θ is an isomorphism. We conclude that the strict order monics are the categorical equivalent of inclusive predicates containing $\perp$. What about the inclusive predicates?

29. Show that fixed-point induction can be reformulated in the following way: For every $f: \mathbf{D} \rightarrow \mathbf{D}$, if $m: \mathbf{C} \rightarrow \mathbf{D}$ is a strict order monic and $g: \mathbf{C} \rightarrow \mathbf{C}$ is a (necessarily unique) continuous function such that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{C} & \xrightarrow{g} & \mathbf{C} \\ m \downarrow & & \downarrow m \\ \mathbf{D} & \xrightarrow{f} & \mathbf{D} \end{array}$$

then $m(Yg) = Yf$.

30. A collection, $F_{\mathbf{D}}$, of fixed-point operators (one for each cpo $\mathbf{D}$) is *uniform* iff wherever $f: \mathbf{D} \rightarrow \mathbf{D}$, $g: \mathbf{E} \rightarrow \mathbf{E}$ and $h: \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ are such that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{D} & \xrightarrow{f} & \mathbf{D} \\ h \downarrow & & \downarrow h \\ \mathbf{E} & \xrightarrow{g} & \mathbf{E} \end{array}$$

then $h(F_{\mathbf{D}}f) = F_{\mathbf{D}}g$. Show that the collection, $Y_{\mathbf{D}}$, is uniform and that it is the only uniform collection. [Hint Use Exercise 29.] This is further justification of our choice of least fixed-points. Show that invariance of each of a collection of fixed-point operators (one for each cpo) can be rephrased as uniformly w.r.t. automorphisms. It is an open question whether Y is the only collection of fixed-point operators invariant over the embeddings (see Chapter 4).

31. Show that $\mathcal{C}[\text{while } b \text{ do } s] = \mathcal{C}[\text{if } b \text{ then } (s; \text{while } b \text{ do } s) \text{ else dummy}]$. Define the semantics of a construct **until** b **do** s and show the equivalence, $\mathcal{C}[\text{while } b \text{ do } s] = \mathcal{C}[\text{until } \neg b \text{ do } s]$ (with the evident semantics for $\neg b$).

32. Define a syntactic substitution, $[e_0/x]e_1$ of expressions for identifiers in expressions and prove that $\mathcal{E}[\text{let } x \text{ be } e_0 \text{ in } e_1] = \mathcal{E}[[e_0/x]e_1]$. Define a syntactic substitution $[e_0/f_n(x_0, \dots, x_{n-1})]e_1$ of expressions for function symbols with indicated identifiers as parameters in expressions so that for example, $[e_0/f_1(x_0)](f_1(e_1)) = [[e_0/f_1(x_0)]e_1/x_0]e_0$. Show that

$$\mathcal{E}[\text{let } f_n(x_0, \dots, x_{n-1}) \text{ be } e_0 \text{ in } e_1] = \mathcal{E}[[e_0/f_n(x_0, \dots, x_{n-1})]e_1]$$

and also that

$$\mathcal{E}[\text{letrec } f_n(x_0, \dots, x_{n-1}) \text{ be } e_0 \text{ in } e_1] = \mathcal{E}[\text{letrec } f_n(x_0, \dots, x_{n-1}) \text{ be } e_0 \text{ in } [e_0/f_n(x_0, \dots, x_{n-1})]e_1].$$

These last two equations show why our semantics is “call-by-name” — that Algol copy rule is valid. [Warning Make sure that free identifiers or function symbols are not captured by binding mechanisms when defining your syntactic substitution.]

33. Extend *Imp* to allow assignments and recursive assignments to functions thus:

$$s ::= f_n(x_0, \dots, x_{n-1}) := e \mid f_n(x_0, \dots, x_{n-1}) \Rightarrow e.$$

This is only possible because **FEnv** is part of the state. Given an alternative semantics of *Imp* without this extension in which **FEnv** is part of the environment so that $\mathbf{S} = \mathbf{IEnv}$ and $\mathcal{C}$ has type $\mathcal{C}: \mathbf{Stat} \rightarrow (\mathbf{FEnv} \rightarrow \mathbf{C})$ (where $\mathbf{C} = \mathbf{S} \rightarrow \mathbf{S}$).

34. Write a few example programs and prove that they denote what you think they ought to.

35. Consider the combination of *Imp* and *Dec* given in Chapter 1. Can you define a syntactic substitution of expressions for identifiers in statements so that $\mathcal{C}[x := e; s]$ and $\mathcal{C}[[e/x]s]$ bear a suitably close relationship?

3. Other Constructions

In the last chapter we were able to give the semantics of recursively defined functions which take their arguments by call-by-name; this was accomplished with the aid of the product and function-space constructions. Now we will consider call-by-value as a prop for introducing the strict versions of these constructions. This will entail consideration of $\mathcal{CPO}_\perp$ rather than $\mathcal{CPO}$.

Under the call-by-value regime, if the evaluation of an expression, e , in **Exp** does not terminate then neither does that of $f_1(e)$. So we expect that f_1 should denote a *strict* function. To ensure this we will define the cpo, $(\mathbf{V} \rightarrow_\perp \mathbf{V})$ of the strict functions and when f_1 is recursively defined we will take its denotation as the least fixed-point of a suitable functional, $F: (\mathbf{V} \rightarrow_\perp \mathbf{V}) \rightarrow (\mathbf{V} \rightarrow_\perp \mathbf{V})$. In the case of functions of n arguments, if the evaluation of any one of the expressions $e_0, \dots, e_{n-1}$ does not terminate then, under the call-by-value regime, neither does the evaluation of $f_n(e_0, \dots, e_{n-1})$ terminate. Thus the denotation of f_n should, perhaps, be an *n-strict* function, $f_n: \mathbf{V}^n \rightarrow_\perp \mathbf{V}$ (that is, if for some i , with $i < n$, $(x)_i = \perp$ then $f_n(x) = \perp$). For recursively defined functions this would lend us to consider cpos of *n-strict* functions of n arguments. In the case of arbitrary continuous functions of n -arguments we consider products such as $\mathbf{V}^n$, and unary functions $\mathbf{V}^n \rightarrow \mathbf{V}$ rather than create special cpos of functions of n -arguments. In just the same way we will consider strict products and unary strict functions.

Let $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ be n cpos. Their *strict product* (= *smash product*), $\bigotimes_{i < n} \mathbf{D}_i$ is the subset, $\{x \in \prod_{i < n} \mathbf{D}_i \mid (\forall i < n. (x)_i \neq \perp) \vee (x = \perp)\}$, of $\prod_{i < n} \mathbf{D}_i$ with the coordinatewise ordering inherited from $\prod_{i < n} \mathbf{D}_i$. Sometimes we write $\mathbf{D}_0 \otimes \dots \otimes \mathbf{D}_{n-1}$ for $\bigotimes_{i < n} \mathbf{D}_i$ or $\mathbf{D}^\otimes$ when the $\mathbf{D}_i$ are all the same cpo, $\mathbf{D}$. The smash product is a cpo; indeed it has the same least element and the same lubs of increasing sequences as the Cartesian product. Thus it is even a subcpo of the Cartesian product and so the natural inclusion map $\iota: \bigotimes_{i < n} \mathbf{D}_i \rightarrow_\perp \prod_{i < n} \mathbf{D}_i$ is continuous. There is another natural continuous map in the opposite direction, namely $\otimes: \prod_{i < n} \mathbf{D}_i \rightarrow_\perp \bigotimes_{i < n} \mathbf{D}_i$ where:

$$\otimes(x) = \begin{cases} x & (\forall i < n. (x)_i \neq \perp), \\ \perp & (\text{otherwise}). \end{cases}$$

Note that $\otimes$ is *n-strict*. When we look at the universal property for the smash product we shall see that $\otimes$ is the universal *n-strict* map.

Let $\mathbf{D}$ and $\mathbf{E}$ be cpos. Their *strict function space*, $\mathbf{D} \rightarrow_\perp \mathbf{E}$, is the set $\{f: \mathbf{D} \rightarrow \mathbf{E} \mid f(\perp) = \perp\}$ of strict continuous functions from $\mathbf{D}$ to $\mathbf{E}$, with the pointwise order inherited from $\mathbf{D} \rightarrow \mathbf{E}$. Again we find that $(\mathbf{D} \rightarrow_\perp \mathbf{E})$ is a subcpo of $(\mathbf{D} \rightarrow \mathbf{E})$, meaning that it has the same least element and lubs of increasing sequences. Again the natural inclusion map $\iota: (\mathbf{D} \rightarrow_\perp \mathbf{E}) \rightarrow (\mathbf{D} \rightarrow \mathbf{E})$ is continuous. We now introduce our last piece of metalanguage:

E10. If e is an expression of type $\mathbf{E}$ and x is a variable of type $\mathbf{D}$ then $(\lambda_\perp x: \mathbf{D}. e)$ is an expression of type $(\mathbf{D} \rightarrow_\perp \mathbf{E})$ which is to have the same denotation as $(\lambda x: \mathbf{D}. (\partial x \rightarrow e \mid \perp))$ (but note the types are different). When $\mathbf{D}$ is clear from the context $(\lambda_\perp x: \mathbf{D}. e)$ may be abbreviated to $(\lambda_\perp x. e)$.

For example this strict kind of abstraction allows us to define a map *strict*: $(\mathbf{D} \rightarrow \mathbf{E}) \rightarrow (\mathbf{D} \rightarrow_\perp \mathbf{E})$ by:

$$\text{strict} = \lambda f: \mathbf{D} \rightarrow \mathbf{E}. \lambda_\perp d: \mathbf{D}. f(d).$$

Whenever $\mathbf{A}$ is a subcpo of $\mathbf{B}$ we will include an anonymous constant, ι , for the inclusion map $\iota: \mathbf{A} \rightarrow \mathbf{B}$ in our metalanguage. Often we will write e instead of $\iota(e)$ when no confusion thereby results. For example when $f: \mathbf{D} \rightarrow_\perp \mathbf{E}$ and $x: \mathbf{D}$, we might write $f(x)$ instead of $\iota(f)(x)$. Again we introduce the abbreviation $(\lambda_\perp x_0: \mathbf{D}_0, \dots, x_{n-1}: \mathbf{D}_{n-1}. e)$ to stand for:

$$\lambda_\perp w: \bigotimes_{i < n} \mathbf{D}_i. ((\lambda_\perp x_0: \mathbf{D}_0. (\dots (\lambda_\perp x_{n-1}: \mathbf{D}_{n-1}. e) \dots))((w)_0) \dots ((w)_{n-1}));$$

when the $\mathbf{D}_i$ are clear from the context we might just write $(\lambda_\perp x_0, \dots, x_{n-1}. e)$ instead. Thus, for example, we can define the composition map, $\circ$, of strict functions, of type $((\mathbf{E} \rightarrow_\perp \mathbf{F}) \otimes (\mathbf{D} \rightarrow_\perp \mathbf{E})) \rightarrow_\perp (\mathbf{D} \rightarrow_\perp \mathbf{F})$ by $(\lambda_\perp g, f. \lambda_\perp d. g(f(d)))$. Finally, we will often write $e_0 \otimes \dots \otimes e_{n-1}$ as an abbreviation for $\otimes(e_0, \dots, e_{n-1})$.

Turning to categorical considerations we first observe that $\otimes: \prod_{i < n} \mathbf{D}_i \rightarrow \perp \otimes_{i < n} \mathbf{D}_i$ is universal in the sense that if $f: \prod_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{E}$ is any n -strict continuous map then there is a unique strict continuous $g: \otimes_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{E}$ such that the following diagram commutes:

$$\begin{array}{ccc} \prod_{i < n} \mathbf{D}_i & & \\ \downarrow \otimes & \searrow f & \\ \otimes_{i < n} \mathbf{D}_i & \xrightarrow{g} & \mathbf{E} \end{array}$$

For if this diagram commutes for a given g we have $g = f \circ \iota$ as: $g(w) = g(\otimes(\iota(w)))$ (as $\otimes \circ \iota = id$) $= f(\iota(w))$. This proves uniqueness. For existence let g be the strict function $f \circ \iota$. Then if $w \in \prod_{i < n} \mathbf{D}_i$ then either $(w)_i = \perp$ for some i , with $i < n$, or not. In the first case $f(w) = \perp$ and $g \circ \otimes(w) = g(\perp) = \perp$; in the second case $g \circ \otimes(w) = (f \circ \iota)(w) = f(w)$, proving existence. This universal property shows how n -strict functions $f: \prod_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{E}$ are to be replaced by strict functions, $g: \otimes_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{E}$ (see Exercise 1).

One now easily sees that the following external axioms characterise this universal property:

External Axiomatisation.

1. $\forall f: \prod_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{E}. (f \text{ } n\text{-strict}) \Rightarrow f = (f \circ \iota) \circ \otimes.$
2. $\forall g: \otimes_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{E}. g = (g \circ \otimes) \circ \iota.$

Internal Axiomatisation.

1. $\forall x_0: \mathbf{D}_0, \dots, \forall x_{n-1}: \mathbf{D}_{n-1}. \pi_i(x_0 \otimes \dots \otimes x_{n-1}) = (((\partial x_0) \text{ and } \dots \text{ and } (\partial x_{n-1})) \rightarrow x_i \mid \perp).$
2. $\forall w: \otimes_{i < n} \mathbf{D}_i. w = \pi_0(w) \otimes \dots \otimes \pi_{n-1}(w).$

These axioms are clearly true. To see they imply the external ones we begin by showing that they imply $\otimes$ is n -strict. It is given that $\otimes$ is strict (which is also a consequence of 2). Suppose $x_0: \mathbf{D}_0, \dots, x_{n-1}: \mathbf{D}_{n-1}$ and some x_i is $\perp$. Then by part 1, $\pi_i(x_0 \otimes \dots \otimes x_{n-1}) = \perp$ for all i ($i < n$) and so by part 2, $x_0 \otimes \dots \otimes x_{n-1} = \perp$. For part 1, suppose $x_0: \mathbf{D}_0, \dots, x_{n-1}: \mathbf{D}_{n-1}$. If some x_i is $\perp$ then: $(f \circ \iota) \circ \otimes(x_0, \dots, x_{n-1}) = f \circ \iota(\perp) = \perp = f(x_0, \dots, x_{n-1})$ as f is n -strict. Otherwise:

$$\begin{aligned} (f \circ \iota) \circ \otimes(x_0, \dots, x_{n-1}) &= f(\pi_0(x_0 \otimes \dots \otimes x_{n-1}), \dots, \pi_{n-1}(x_0 \otimes \dots \otimes x_{n-1})) \\ &= f(x_0, \dots, x_{n-1}) \end{aligned} \quad \text{by Axiom 1.}$$

Finally we show that $\otimes \circ \iota = id$, to prove the second external axiom. We have:

$$\otimes \circ \iota(w) = \otimes(\pi_0(w), \dots, \pi_{n-1}(w)) = w \quad \text{by Axiom 2.}$$

Note that if $\mathbf{D}$ and $\mathbf{E}$ are flat cpos so is $\mathbf{D} \otimes \mathbf{E}$. For example $\mathbf{T} \otimes \mathbf{T}$ is drawn in Figure VIII:

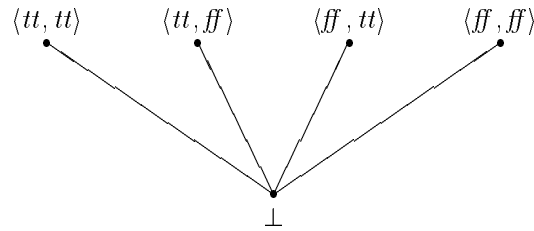


Figure VIII: The cpo $\mathbf{T} \otimes \mathbf{T}$.

This idea will help us to consider syntax as a cpo, in a systematic way, as mentioned above. Note too, that it follows that when $f: \mathbf{X}_0 \times \dots \times \mathbf{X}_{n-1} \rightarrow \mathbf{Y}$ is a function on sets, then f has a *unique* extension to a continuous function, $\hat{f}: (\mathbf{X}_0)_\perp \otimes \dots \otimes (\mathbf{X}_{n-1})_\perp \rightarrow \perp \mathbf{Y}_\perp$ and the extension, $f_\perp$, considered in Chapter 2 is just $\iota(f) \circ \otimes$. Conversely, every n -strict function on flat cpos is the extension of an ordinary function on the corresponding sets. So we see that the strict constructions allow us to import functions of several arguments over sets more accurately than the ordinary product and function space constructions. But, of course, we still need many continuous functions other than the n -strict ones; it is absolutely necessary to have $cond: \mathbf{T} \times \mathbf{D} \times \mathbf{D} \rightarrow \mathbf{D}$ for instance.

We conclude the discussion of the smash product construction by showing that it can be considered as a covariant functor, $\bigotimes : \mathcal{CPO}_\perp^n \rightarrow \mathcal{CPO}_\perp$ by defining its action on morphisms $f_i : \mathbf{D}_i \rightarrow_\perp \mathbf{E}_i$ ($i < n$) by:

$$\bigotimes_{i < n} f_i = \bigotimes \circ \left(\prod_{i < n} f_i \right) \circ \iota$$

where we are considering $\prod$ as a functor, $\prod : \mathcal{CPO}_\perp^n \rightarrow \mathcal{CPO}_\perp$ (see Exercise 2.6). In this case the functor laws are:

- (i) $\bigotimes_{i < n} id_{\mathbf{D}_i} = id_{\bigotimes_{i < n} \mathbf{D}_i},$
- (ii) $\bigotimes_{i < n} (g_i \circ f_i) = \left(\bigotimes_{i < n} g_i \right) \circ \left(\bigotimes_{i < n} f_i \right).$

The universal property for the strict function space, $\mathbf{D} \rightarrow_\perp \mathbf{E}$ is specified in exactly the same way as that for the function space, $\mathbf{D} \rightarrow \mathbf{E}$ but using the smash product in place of the Cartesian one. Define $apply : ((\mathbf{D} \rightarrow_\perp \mathbf{E}) \otimes \mathbf{D}) \rightarrow_\perp \mathbf{E}$ by:

$$apply = \lambda_\perp f : \mathbf{D} \rightarrow_\perp \mathbf{E}. x : \mathbf{D}. f(x).$$

Then if $f : \mathbf{F} \otimes \mathbf{D} \rightarrow_\perp \mathbf{E}$ is any continuous function there is a unique mediating $g : \mathbf{F} \rightarrow_\perp (\mathbf{D} \rightarrow_\perp \mathbf{E})$ such that the following diagram commutes:

$$\begin{array}{ccc} (\mathbf{D} \rightarrow_\perp \mathbf{E}) \otimes \mathbf{D} & \xleftarrow{g \otimes id_{\mathbf{D}}} & \mathbf{F} \otimes \mathbf{D} \\ \downarrow apply & \searrow f & \\ \mathbf{E} & & \end{array}$$

In fact g must be $Curry(f)$ where $Curry : ((\mathbf{F} \otimes \mathbf{D}) \rightarrow_\perp \mathbf{E}) \rightarrow_\perp (\mathbf{F} \rightarrow_\perp (\mathbf{D} \rightarrow_\perp \mathbf{E}))$ is defined by:

$$Curry = \lambda_\perp f : (\mathbf{F} \otimes \mathbf{D}) \rightarrow_\perp \mathbf{E}. \lambda_\perp v : \mathbf{F}. \lambda_\perp d : \mathbf{D}. f(v \otimes d).$$

External Axiomatisation.

1. $\forall f : (\mathbf{F} \otimes \mathbf{D}) \rightarrow_\perp \mathbf{E}. f = apply \circ (Curry(f) \otimes id_{\mathbf{D}}).$
2. $\forall g : \mathbf{F} \rightarrow_\perp (\mathbf{D} \rightarrow_\perp \mathbf{E}). g = Curry(apply \circ (g \otimes id_{\mathbf{D}})).$

Internal Axiomatisation.

1. $\beta_\perp\text{-reduction} : (\lambda_\perp x. e)(x) = (\partial x \rightarrow e \mid \perp).$
2. $\eta_\perp\text{-reduction} : \lambda_\perp x. f(x) = f \quad (f : \mathbf{D} \rightarrow_\perp \mathbf{E}, x : \mathbf{D}).$

Note that 1 and 2 are largely axioms on ι (assuming ordinary β -reduction and η -reduction). Thus 1 may be rephrased as: $\iota(\lambda_\perp x. e) = \lambda x. (\partial x \rightarrow e \mid \perp)$ and 2 may be rephrased as: $\iota(f)(\perp) = \perp$.

As a functor, $\rightarrow_\perp$, is defined just like $\rightarrow$. For $f : \mathbf{B} \rightarrow_\perp \mathbf{A}$, $f' : \mathbf{A}' \rightarrow_\perp \mathbf{B}'$ we define $(f \rightarrow_\perp f') : (\mathbf{A} \rightarrow_\perp \mathbf{A}') \rightarrow_\perp (\mathbf{B} \rightarrow_\perp \mathbf{B}')$ by:

$$(f \rightarrow_\perp f') = \lambda_\perp x : \mathbf{A} \rightarrow_\perp \mathbf{A}'. f' \circ x \circ f.$$

We obtain a functor, $\rightarrow_\perp : \mathcal{CPO}_\perp^2 \rightarrow \mathcal{CPO}_\perp$ which is contravariant in its first argument and covariant in its second argument so that the functor laws are:

- (i) $id_{\mathbf{D}} \rightarrow_\perp id_{\mathbf{E}} = id_{(\mathbf{D} \rightarrow_\perp \mathbf{E})},$
- (ii) $(f \circ g) \rightarrow_\perp (g' \circ f') = (g \rightarrow_\perp g') \circ (f \rightarrow_\perp f').$

for f, f', g, g' of the appropriate types. In category-theoretic jargon what we have essentially shown about $\otimes, \rightarrow_\perp$ and their relation to each other is that $\mathcal{CPO}_\perp$ equipped with $\otimes$ and $\rightarrow_\perp$ is a *closed* category.

It is now quite straightforward to change the semantics of *Dec* to correspond to call-by-value. We keep the same syntax as in Chapter 2. For the semantic domains we again take $\mathbf{V}$ to be a fixed cpo of values; we can now model $\mathbf{IEnv}$ more accurately as $(\mathbf{Id}_\perp \rightarrow_\perp \mathbf{V}_\perp)$ and $\mathbf{FEnv}$ should now be $\prod_{j < m} (\mathbf{Fun}_j)_\perp \rightarrow_\perp (\mathbf{V}^\odot \rightarrow_\perp \mathbf{V})$. We assume the basic operation also take their arguments by value and so we have $\mathcal{I}_i: \mathbf{Op}_i \rightarrow (\mathbf{V}^\odot \rightarrow_\perp \mathbf{V})$ ($i < n$). Next $\mathcal{B}: \mathbf{BExp} \rightarrow ((\mathbf{IEnv} \times \mathbf{FEnv}) \rightarrow \mathbf{T})$ and finally $\mathcal{E}: \mathbf{Exp} \rightarrow ((\mathbf{IEnv} \times \mathbf{FEnv}) \rightarrow \mathbf{V})$ is defined by structural induction:

- (i) (as in Chapter 2);
- (ii)_i $\mathcal{E}[\![o_i(e_0, \dots, e_{n-1})]\!] = \lambda \rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. \mathcal{I}[o_i](\mathcal{E}[\![e_0]\!](\rho, \varphi) \otimes \dots \otimes \mathcal{E}[\![e_{i-1}]\!](\rho, \varphi))$ ($i < n$);
- (iii) (as in Chapter 2);
- (iv) (as in Chapter 2);
- (v)_j $\mathcal{E}[\![\text{let } f_j(x_0, \dots, x_{j-1}) \text{ be } e_0 \text{ in } e_1]\!] =$
 $\lambda \rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. \mathcal{E}[\![e_1]\!](\rho, \varphi[(\lambda_\perp w: \mathbf{V}^\odot. \mathcal{E}[\![e_0]\!](\rho[(w)_0/x_0] \dots [(w)_{j-1}/x_{j-1}], \varphi))/f_j])$ ($j < m$);
- (vi)_j $\mathcal{E}[\![\text{letrec } f_j(x_0, \dots, x_{j-1}) \text{ be } e_0 \text{ in } e_1]\!] =$
 $\lambda \rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv}. \mathcal{E}[\![e_1]\!](\rho, \varphi[\mu f: \mathbf{V}^\odot \rightarrow_\perp \mathbf{V}. (\lambda_\perp w: \mathbf{V}^\odot. \mathcal{E}[\![e_0]\!](\rho[(w)_0/x_0] \dots [(w)_{j-1}/x_{j-1}],$
 $\varphi[f/f_j]))/f_j])$ ($j < m$).

Here $[\![\cdot/\cdot]\!]: \mathbf{IEnv} \times \mathbf{V} \times \mathbf{Id}_\perp \rightarrow \mathbf{IEnv}$ is defined by:

$$\rho[v/x] = \lambda_\perp y: \mathbf{Id}_\perp. ((v = x) \rightarrow v \mid \rho[y])$$

and the definition of $[\![\cdot/\cdot]\!]: (\mathbf{FEnv} \times (\mathbf{V}^\odot \rightarrow_\perp \mathbf{V}) \times (\mathbf{Fun}_j)_\perp) \rightarrow \mathbf{FEnv}$ is left to the reader.

In order to introduce our next construction suppose we want to specify $\mathbf{V}$ so that expressions can have either integers or truth-values as their values and then we can identify $\mathbf{BExp}$ with $\mathbf{Exp}$. Then we expect that an expression evaluates to an integer, or to a truth-values or else evaluation does not terminate. So we expect $\mathbf{V}$ to be as in Figure IX:

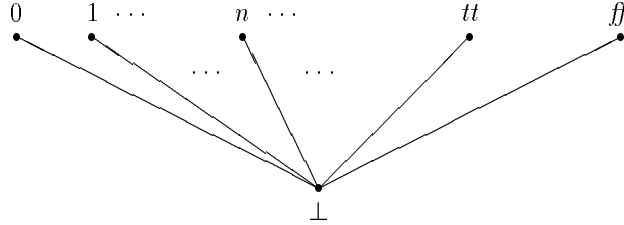


Figure IX: The cpo $\mathbf{N} + \mathbf{T}$.

This cpo is called the (coalesced) sums of $\mathbf{N}$ and $\mathbf{T}$ and is useful whenever values of identifiers can be in any of several different cpos — that is they are, in a sense, of ambiguous type. We now turn to the general construction.

Let $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ be cpos. Their *coalesced sum* (= *amalgamated sum*), $\sum_{i < n} \mathbf{D}_i$ is the set:

$$(\bigcup_{i < n} \{(i, d) \mid d \in \mathbf{D}_i \wedge d \neq \perp\}) \cup \{\perp\}$$

with the order

$$x \sqsubseteq y \quad \text{iff} \quad (x = \perp) \text{ or } (\exists i < n. \exists d, d' \in \mathbf{D}_i. d \sqsubseteq d' \wedge x = \langle i, d \rangle \wedge y = \langle i, d' \rangle).$$

Thus the order is inherited from the $\mathbf{D}_i$ with the additional requirement that the $\perp_{\mathbf{D}_i}$ are identified. A picture for the case $n = 2$ should help:

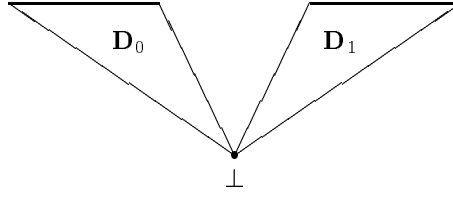


Figure X: The cpo $\mathbf{D}_0 + \mathbf{D}_1$.

Sometimes we write $\mathbf{D}_0 + \dots + \mathbf{D}_{n-1}$ for $\sum_{i < n} \mathbf{D}_i$. Note that the sum is a cpo as all non-trivial increasing chains essentially lie within one or other of the $\mathbf{D}_i$. This is also the reason why the natural injection functions, $in_i: \mathbf{D}_i \rightarrow \perp \sum_{i < n} \mathbf{D}_i$ ($i < n$) are continuous where:

$$in_i(d) = \begin{cases} \perp & (d = \perp), \\ \langle i, d \rangle & (\text{otherwise}). \end{cases}$$

These injections (sometimes written Π_i) show how the sum is a compound type of cpo which contains all its summands; this leads to the following universal property. Let $f_i: \sum_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{E}$ such that for all i with $i < n$ the following diagram commutes.

$$\begin{array}{ccc} \mathbf{D}_i & & \\ \downarrow in_i & \searrow f_i & \\ \sum_{i < n} \mathbf{D}_i & \xrightarrow{h} & \mathbf{E} \end{array}$$

What this says is that $\sum_{i < n} \mathbf{D}_i$ is the categorical sum in $\mathcal{CP}\mathcal{O}_\perp$; it is dual to the product. Clearly h is unique as we must have $h(\perp) = \perp$ and $h(\langle i, d \rangle) = h(in_i(d)) = f_i(d)$, when $d \neq \perp$, and this determines h as a strict continuous function which makes the diagram commute. We write h as $[f_0, \dots, f_{n-1}]$ and one easily checks that $[\cdot, \dots, \cdot]: (\prod_{i < n} (\mathbf{D}_i \rightarrow \perp \mathbf{E})) \rightarrow ((\sum_{i < n} \mathbf{D}_i) \rightarrow \perp \mathbf{E})$ is continuous. Note the case $n = 0$ when the sum $\sum_{i < 0} \mathbf{D}_i$ is just $\mathbf{U}$ and we see that for any cpo $\mathbf{E}$ there is a unique morphism $[\cdot]: \mathbf{U} \rightarrow \mathbf{E}$ (namely $\perp$) showing that $\mathbf{U}$ is the *initial* object in $\mathcal{CP}\mathcal{O}_\perp$.

External Axiomatisation.

1. $\forall f_0: \mathbf{D}_0 \rightarrow \perp \mathbf{E}. \dots \forall f_{n-1}: \mathbf{D}_{n-1} \rightarrow \perp \mathbf{E}. \forall i < n. [f_0, \dots, f_{n-1}] \circ in_i = f_i$ ($0 \leq i < n$).
2. $\forall h: \sum_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{E}. h = [h \circ in_0, \dots, h \circ in_{n-1}]$.

To give the internal axiomatisation it is convenient to introduce a few more functions. First projection functions $out_i: \sum_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{D}_i$ ($i < n$) are defined by:

$$out_i = [\perp, \dots, \perp, \lambda x: \mathbf{D}_i. x, \perp, \dots, \perp]$$

and next discriminator functions, $is_i: \sum_{i < n} \mathbf{D}_i \rightarrow \perp \mathbf{T}$ are defined by:

$$is_i = [\lambda x: \perp. \text{ff}, \dots, \lambda x: \perp. \text{ff}, \lambda x: \mathbf{D}_i. \text{tt}, \lambda x: \perp. \text{ff}, \dots, \lambda x: \perp. \text{ff}].$$

Sometimes out_i is written as $(\cdot \mid \mathbf{D}_i)$ when the $\mathbf{D}_i$ are all different.

Internal Axiomatisation.

1. $\forall x: \mathbf{D}_i. x = out_i(in_i(x))$ ($0 \leq i < n$).
2. $\forall x: \sum_{i < n} \mathbf{D}_i. x = (is_0(x) \rightarrow in_0(out_0(x)) \mid (is_1(x) \rightarrow in_1(out_1(x))$
 $\mid \dots \mid (is_{n-1}(x) \rightarrow in_{n-1}(out_{n-1}(x)) \mid \perp) \dots).$

To obtain the external axioms from the internal ones we must define $[\cdot, \dots, \cdot]$ in terms of the functions out_i , is_i by:

$$[\cdot, \dots, \cdot] = \lambda w: (\prod_{i < n} (\mathbf{D}_i \rightarrow_{\perp} \mathbf{E})) . \lambda_{\perp} x . \sum_{i < n} \mathbf{D}_i . (is_0(x) \rightarrow (w \downarrow 0)(out_0(x)) \mid (is_1(x) \rightarrow (w \downarrow 1)(out_1(x)) \mid \dots \mid (is_{n-1}(x) \rightarrow (w \downarrow n-1)(out_{n-1}(x)) \mid \perp) \dots)).$$

Finally to consider $\sum$ as a covariant functor $\sum: \mathcal{CPO}_{\perp}^n \rightarrow \mathcal{CPO}_{\perp}$ we define, for $f_i: \mathbf{D}_i \rightarrow_{\perp} \mathbf{E}_i$ ($i < n$):

$$\sum_{i < n} f_i = [in_0 \circ f_0, \dots, in_{n-1} \circ f_{n-1}]$$

where $in_i: \mathbf{E}_i \rightarrow_{\perp} \sum \mathbf{E}_i$ and note the appropriate functor laws:

$$\begin{aligned} \text{(i)} \quad & \sum_{i < n} in_{\mathbf{D}_i} = id_{\sum \mathbf{D}_i}, \\ \text{(ii)} \quad & \sum_{i < n} (g_i \circ f_i) = (\sum_{i < n} g_i) \circ (\sum_{i < n} f_i). \end{aligned}$$

Returning to our example where $\mathbf{V} = \mathbf{N} + \mathbf{T}$ we can now see how to specify Dec in a bit more detail. We could take $\mathbf{Op}_0 = \{0, \text{true}, \text{false}\}$, $\mathbf{Op}_1 = \{\text{succ}, \text{pred}, \text{zero}\}$ and $\mathbf{Op}_n = \emptyset$ for $n > 1$. Then $\mathbf{BExp}$ could be given by the trivial grammar: $\mathbf{BExp} ::= \mathbf{Exp}$. The functions $\mathcal{I}_n$ ($n < 2$) are given by:

$$\begin{aligned} \mathcal{I}[\mathbf{0}] &= inl(0); \\ \mathcal{I}[\mathbf{true}] &= inr(tt); \\ \mathcal{I}[\mathbf{false}] &= inr(ff); \\ \mathcal{I}[\mathbf{succ}] &= inl \circ (+1) \circ outl; \\ \mathcal{I}[\mathbf{pred}] &= inl \circ (-1) \circ outl; \\ \mathcal{I}[\mathbf{zero}] &= inr \circ (z) \circ outl. \end{aligned}$$

Note that when there are just two summands we often use (as here) inl , inr , $outl$, $outr$, isl , isr instead of respectively in_0 , in_1 , out_0 , out_1 , is_0 , is_1 . Finally $\mathcal{B}: \mathbf{BExp} \rightarrow ((\mathbf{IEnv} \times \mathbf{FEnv}) \rightarrow \mathbf{T})$ is specified by:

$$\text{(i)} \quad \mathcal{B}[e] = \lambda \rho: \mathbf{IEnv}, \varphi: \mathbf{FEnv} . outr(\mathcal{E}[e](\rho, \varphi)).$$

The other details are filled in as usual with whatever parameter-calling discipline is desired. Note that $\mathcal{B}$ and $\mathcal{E}$ are mutually recursive, and are still well-defined by structural induction on (the parse trees of) expressions.

The final construction considered here — lifting — does not seem to be of great significance by itself but, rather, is often useful in combination with other constructions in order to achieve an accurate model of computation. Categorically, lifting allows continuous functions to be turned into strict continuous functions.

Let $\mathbf{D}$ be a cpo. Its *lifting* $\mathbf{D}_{\perp}$ is the set $\{\langle o, d \rangle \mid d \in \mathbf{D}\} \cup \{\perp\}$ with the ordering:

$$x \sqsubseteq y \quad \text{iff} \quad (x = \perp) \text{ or } (\exists d, d' \in \mathbf{D} . d \sqsubseteq d' \wedge x = \langle o, d \rangle \wedge y = \langle o, d' \rangle).$$

Clearly this makes $\mathbf{D}_{\perp}$ a cpo; roughly speaking one notes that all non-trivial chains in $\mathbf{D}_{\perp}$ are (eventually) in $\mathbf{D}$ and the lub is then inherited from $\mathbf{D}$. This observation also shows why the function $up: \mathbf{D} \rightarrow \mathbf{D}_{\perp}$ is continuous where:

$$up(d) = \langle o, d \rangle \quad (d \in \mathbf{D}).$$

The cpo $\mathbf{D}_{\perp}$ can be pictured thus:

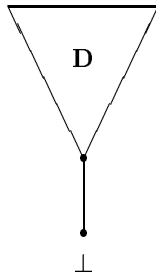


Figure XI: The cpo $\mathbf{D}_{\perp}$.

The universal property for $\mathbf{D}_\perp$ says that $up: \mathbf{D} \rightarrow \mathbf{D}_\perp$ is the basic example of a continuous function from $\mathbf{D}$: all others can be obtained from it via a factorisation through a strict function. For if $f: \mathbf{D} \rightarrow \mathbf{E}$ is any continuous function then there is a unique strict continuous function $h: \mathbf{D}_\perp \rightarrow_\perp \mathbf{E}$ such that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{D} & & \\ \downarrow up & \searrow f & \\ \mathbf{D}_\perp & \xrightarrow{h} & \mathbf{E} \end{array}$$

For we must have $h(\perp) = \perp$ and, for each member, d , of $\mathbf{D}$, $h(\langle o, d \rangle) = h(up(d)) = f(d)$. This proves uniqueness and existence too should now be clear. We write $lift(f)$ for h and indeed $lift: (\mathbf{D} \rightarrow \mathbf{E}) \rightarrow (\mathbf{D}_\perp \rightarrow_\perp \mathbf{E})$ is a continuous function.

External Axiomatisation.

1. $\forall f: \mathbf{D} \rightarrow \mathbf{E}. f = lift(f) \circ up.$
2. $\forall h: \mathbf{D}_\perp \rightarrow_\perp \mathbf{E}. h = lift(h \circ up).$

For the internal axiomatisation it is convenient to define as auxiliary functions, $down: \mathbf{D}_\perp \rightarrow_\perp \mathbf{D}$ and the “termination predicate” $\partial: \mathbf{D}_\perp \rightarrow_\perp \mathbf{T}$ by:

$$\begin{aligned} down &= lift(id_{\mathbf{D}}), \\ \partial &= lift(\lambda x: \mathbf{D}. tt). \end{aligned}$$

Internal Axiomatisation.

1. $\forall x: \mathbf{D}. x = down(up(x)).$
2. $\forall y: \mathbf{D}_\perp. y = (\partial y \rightarrow up(down(y)) \mid \perp).$

To establish the external axiomatisation using the internal ones one notes that:

$$lift = \lambda f: \mathbf{D} \rightarrow \mathbf{E}. \lambda \perp d: \mathbf{D}_\perp. (\partial d \rightarrow f(down(d)) \mid \perp).$$

To view $(\cdot)_\perp$ as a covariant functor: $(\cdot)_\perp: \mathcal{CPO} \rightarrow \mathcal{CPO}_\perp$ we define for $f: \mathbf{D} \rightarrow \mathbf{E}$:

$$f_\perp = lift(up \circ f).$$

The functor laws are:

- (i) $(id_{\mathbf{D}})_\perp = (id_{\mathbf{D}_\perp})$
- (ii) $(g \circ f)_\perp = g_\perp \circ f_\perp$

A typical use of lifting occurs when we do not want to amalgamate the bottom elements of the summands of $\sum_{i < n} \mathbf{D}_i$ in the case where they play different computational roles. Instead we might use the *separated* sum $\sum_{i < n} (\mathbf{D}_i)_\perp$; on occasion we might prefer *semi-separated* sums such as $\mathbf{D}_\perp + \mathbf{E}$.

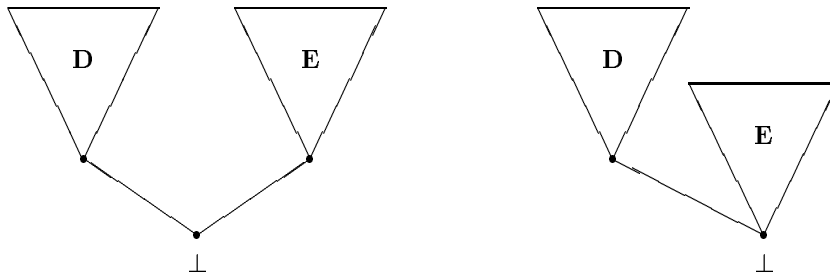


Figure XII: The cpos $\mathbf{D}_\perp + \mathbf{E}_\perp$ and $\mathbf{D}_\perp + \mathbf{E}$.

An example can be found in Exercise 21.

Exercises

1. Let $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}, \mathbf{E}$ be cpos. Define the cpo, $Strict_n(\mathbf{D}_0, \dots, \mathbf{D}_{n-1}; \mathbf{E})$ of n -strict continuous functions and establish the isomorphism:

$$Strict_n(\mathbf{D}_0, \dots, \mathbf{D}_{n-1}; \mathbf{E}) \cong (\bigotimes_{i < n} \mathbf{D}_i) \rightarrow_{\perp} \mathbf{E}.$$

Prove the functor laws for $\bigotimes$. What difficulties arise if you consider infinite smash products?

2 (**Lehmann-Smyth**). A morphism $f: \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ is *very* strict if $f(x) = \perp$ iff $x = \perp$. Show that $\bigotimes$ is the categorical product in the subcategory of the very strict morphisms.

3. Show that $\bigotimes$ obeys commutative and associative laws. Show $\mathbf{U} \otimes \mathbf{D} \cong \mathbf{U}$ and $\mathbf{0} \otimes \mathbf{D} \cong \mathbf{D}$.

4. Let $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}, \mathbf{E}$ be cpos. A continuous function, $f: \prod_{i < n} \mathbf{D}_i \rightarrow \mathbf{E}$ is (m, n) strict iff whenever any m of $x_0, \dots, x_{n-1}$ are $\perp$ then $f(x_0, \dots, x_{n-1})$ is $\perp$ ($x_0: \mathbf{D}_0, \dots, x_{n-1}: \mathbf{D}_{n-1}$). Define a cpo $Strict_{m,n}(\mathbf{D}_0, \dots, \mathbf{D}_{n-1}; \mathbf{E})$ and a product $Prod_{m,n}(\mathbf{D}_0, \dots, \mathbf{D}_{n-1})$ and prove:

$$Strict_{m,n}(\mathbf{D}_0, \dots, \mathbf{D}_{n-1}; \mathbf{E}) \cong Prod_{m,n}(\mathbf{D}_0, \dots, \mathbf{D}_{n-1}) \rightarrow_{\perp} \mathbf{E}.$$

5. Fill in the details on the connection between the external and internal axiomatisations of $\rightarrow_{\perp}$. Show that $f \rightarrow_{\perp} g = Curry(g \circ apply \circ (id \otimes f))$ and use this to prove the functor laws for $\rightarrow_{\perp}$. Why can't $\rightarrow_{\perp}$ be extended to a bifunctor over $\mathcal{CPO}$?

6. Establish the following isomorphisms: $Curry: (\mathbf{D} \otimes \mathbf{E}) \rightarrow_{\perp} \mathbf{F} \cong \mathbf{D} \rightarrow_{\perp} (\mathbf{E} \rightarrow_{\perp} \mathbf{F})$; $\mathbf{X}_{\perp} \rightarrow_{\perp} \mathbf{D} \cong \mathbf{D}^{\mathbf{X}}$ ($\mathbf{X}$ is a set); $\mathbf{U} \rightarrow_{\perp} \mathbf{D} \cong \mathbf{D} \rightarrow_{\perp} \mathbf{U} \cong \mathbf{U}$; $\mathbf{0} \rightarrow_{\perp} \mathbf{D} \cong \mathbf{D}$; $\mathbf{D} \rightarrow_{\perp} (\prod_{i < n} \mathbf{E}_i) \cong \prod_{i < n} (\mathbf{D} \rightarrow_{\perp} \mathbf{E}_i)$.

7. Formulate and prove the analogue of Exercise 2.9, but with $\rightarrow_{\perp}$ in place of $\rightarrow$.

8. Make the necessary changes in the denotational semantics of *Imp* when we use the more accurate $\mathbf{S}_{\perp} \rightarrow_{\perp} \mathbf{S}_{\perp}$ in place of $\mathbf{S}_{\perp} \rightarrow \mathbf{S}_{\perp}$.

9. Show that, with the semantics of call-by-value the following equivalence holds:

$$\mathcal{E}[\llbracket \text{let } f_j(x_0, \dots, x_{j-1}) \text{ be } e \text{ in } f_j(a_0(), \dots, a_{j-1}()) \rrbracket] = \mathcal{E}[\llbracket [a_0()/x_0] \dots [a_{j-1}()/x_{j-1}] e \rrbracket]$$

where the a_i are constants (elements of $\mathbf{Op}_0$) such that $\mathcal{I}_0[a_i()] \neq \perp$. Show that the more general equivalence in Exercise 2.32 fails. What is the appropriate equivalence for calls of recursively defined functions?

10. Fill in the details on the connection between the external and internal axiomatisations of $\sum$. Show that $h \circ [f_0, \dots, f_{n-1}] = [h \circ f_0, \dots, h \circ f_{n-1}]$ and use this result to establish the functor laws for $\sum$. Formulate and prove the analogue of Exercise 2.5 for $\sum$ (the analogue of Δ is written as ∇ and is called the codiagonal function).

11. Show that $\mathcal{CPO}$ does not have any initial object or a binary categorical sum. What goes wrong if you attempt to extend $\sum$ to a functor over $\mathcal{CPO}$?

12. Show that $\sum$ obeys suitable commutative and associative laws. Establish the isomorphisms: $\mathbf{U} + \mathbf{D} \cong \mathbf{D}$; $\mathbf{T} \cong \mathbf{0} + \mathbf{0}$; $\mathbf{N} \cong \mathbf{0} + \mathbf{N}$; $\mathbf{D} \otimes (\mathbf{E} + \mathbf{F}) \cong (\mathbf{D} \otimes \mathbf{E}) + (\mathbf{D} \otimes \mathbf{F})$; $\mathbf{T} \otimes \mathbf{D} \cong \mathbf{D}_{\perp} + \mathbf{D}_{\perp}$. Why do the analogous isomorphisms for $\times$ fail?

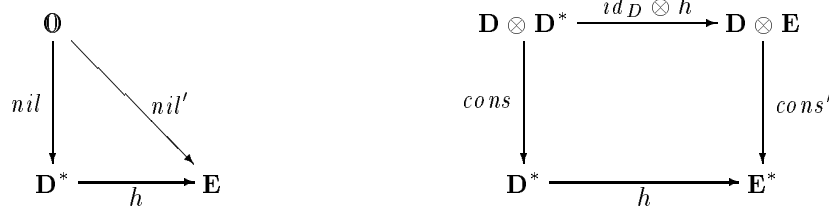
13. Show that the flat cpo, $\mathbf{X}_{\perp}$, associated with any set $\mathbf{X}$ can be characterised by the following universal property of the natural injection function, $\iota: \mathbf{X} \rightarrow \mathbf{X}_{\perp}$: if $f: \mathbf{X} \rightarrow \mathbf{D}$ is any function from $\mathbf{X}$ to the carrier of a cpo $\mathbf{D}$ then there is a unique $h: \mathbf{X}_{\perp} \rightarrow_{\perp} \mathbf{D}$ such that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{X} & & \\ \downarrow \iota & \searrow f & \\ \mathbf{X}_{\perp} & \xrightarrow{h} & \mathbf{D} \end{array}$$

In the category of sets, the categorical sum of two sets $\mathbf{X}$ and $\mathbf{Y}$ is their disjoint union, $\mathbf{X} + \mathbf{Y} \stackrel{\text{def}}{=} \{\langle 0, x \rangle \mid x \in \mathbf{X}\} \cup \{\langle 1, y \rangle \mid y \in \mathbf{Y}\}$ and the categorical product is the ordinary Cartesian product. Show that for any sets $\mathbf{X}$ and $\mathbf{Y}$: $(\mathbf{X} + \mathbf{Y})_{\perp} \cong \mathbf{X}_{\perp} + \mathbf{Y}_{\perp}$ and $(\mathbf{X} \times \mathbf{Y})_{\perp} \cong \mathbf{X}_{\perp} \otimes \mathbf{Y}_{\perp}$.

14. Investigate infinite sums, $\sum_{i \in S} \mathbf{D}_i$.

15. Let $\mathbf{D}$ be a cpo. Let $\mathbf{D}^* \stackrel{\text{def}}{=} \sum_{i \in \omega} \mathbf{D}^{\odot i}$, which is just the infinite sum, $\mathbf{0} + \mathbf{D} + (\mathbf{D} \otimes \mathbf{D}) + \dots$. What is $(\mathbf{N})^*$? Define the natural functions, $\text{nil}: \mathbf{0} \rightarrow \mathbf{D}^*$, $\text{cons}: \mathbf{D} \otimes \mathbf{D}^* \rightarrow \mathbf{D}^*$. Show that they are universal in the sense that if $\mathbf{E}$ is a cpo and we have $\text{nil}': \mathbf{0} \rightarrow_{\perp} \mathbf{E}$, $\text{cons}': \mathbf{D} \otimes \mathbf{E} \rightarrow_{\perp} \mathbf{E}$ then there is a unique $h: \mathbf{D}^* \rightarrow_{\perp} \mathbf{E}$ such that the following two diagrams commute:



Give internal and external axiomatisations of $\mathbf{D}^*$ relating them to each other and the above universal property. [Hint For the internal axiomatisation you may find it helpful to introduce functions, $\text{isnil}: \mathbf{D}^* \rightarrow_{\perp} \mathbf{T}$, $\text{head}: \mathbf{D}^* \rightarrow_{\perp} \mathbf{D}$, and $\text{tail}: \mathbf{D}^* \rightarrow_{\perp} \mathbf{D}^*$.] Show how $(\cdot)^*$ can be considered as a covariant functor, $(\cdot)^*: \mathcal{CPO}_{\perp} \rightarrow \mathcal{CPO}_{\perp}$. Establish the isomorphism, $\mathbf{D}^* \cong \mathbf{0} + \mathbf{D} \otimes \mathbf{D}^*$.

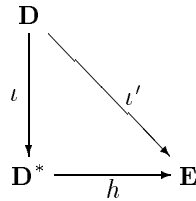
16. Define $\lambda: \mathbf{D}^*, \iota: \mathbf{D} \rightarrow_{\perp} \mathbf{D}^*, \cdot: \mathbf{D}^* \otimes \mathbf{D}^* \rightarrow_{\perp} \mathbf{D}^*$, by:

$$\begin{aligned}
 \lambda &= \text{nil}(\top), \\
 \iota &= \lambda_{\perp} d: \mathbf{D}. \text{cons}(d \otimes \lambda), \\
 \cdot &= \lambda_{\perp} x: \mathbf{D}^*, y: \mathbf{D}^*. (\text{isnil}(x) \rightarrow y \mid \text{cons}(\text{head}(x), \text{tail}(x) \cdot y)).
 \end{aligned}$$

Note the infix notation used in the recursive definition of $\cdot$; in general $e \cdot e'$ is to abbreviate $\cdot(e \otimes e')$. Show that $(\mathbf{D}^*, \cdot, \lambda)$ is a monoid in that the following equations hold:

- (1) *Identity* $\forall x: \mathbf{D}^*. \lambda \cdot x = x \cdot \lambda = x.$
- (2) *Associativity* $\forall x, y, z: \mathbf{D}^*. x \cdot (y \cdot z) = (x \cdot y) \cdot z.$

Show that $(\mathbf{D}, \mathbf{D}^*, \lambda, \iota, \cdot)$ is the free monoid over $\mathbf{D}$ in the sense that if we have a structure $(\mathbf{D}, \mathbf{E}, \lambda', \iota', \cdot')$ where $\mathbf{E}$ is a cpo, $\lambda': \mathbf{E}, \iota': \mathbf{D} \rightarrow_{\perp} \mathbf{E}, \cdot': \mathbf{E} \otimes \mathbf{E} \rightarrow_{\perp} \mathbf{E}$ and $(\mathbf{E}, \lambda', \cdot')$ is a monoid in the above sense then there is a unique $h: \mathbf{D}^* \rightarrow_{\perp} \mathbf{E}$ such that the following diagram commutes:

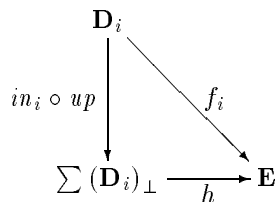


and such that h is a monoid homomorphism in that $h(\lambda) = \lambda'$ and for all x, y in $\mathbf{D}^*$, $h(x \cdot y) = h(x) \cdot' h(y)$.

17. Show that the external axioms for the lifting construction follow from the internal ones and establish the functor laws. Show that lifting can be taken as a covariant functor $(\cdot)_{\perp}: \mathcal{CPO}_{\perp} \rightarrow \mathcal{CPO}_{\perp}$.

18. Establish the isomorphism $\omega \cong \omega_{\perp}$ and, more generally, $\mathbf{X}^{\infty} \cong (\mathbf{X}_{\perp}) \otimes \mathbf{X}^{\infty}$ (see Exercise 1.17). Show that $\mathbf{Tapes} \cong \mathbf{Tapes}_{\perp} + \mathbf{Tapes}_{\perp}$, $\mathbf{D}_{\perp} \otimes \mathbf{E}_{\perp} \cong (\mathbf{D} \times \mathbf{E})_{\perp}$, $\mathbf{Btrees} \cong \mathbf{N} + (\mathbf{Btrees}_{\perp} \otimes \mathbf{Btrees}_{\perp})$ and find a cpo $\mathbf{D}$ such that $\mathbf{D} \cong \mathbf{N} \otimes \mathbf{D}_{\perp} \otimes \mathbf{D}_{\perp}$.

19. Establish the following universal property of the maps $\text{in}_i \circ \text{up}: \mathbf{D}_i \rightarrow \sum (\mathbf{D}_i)_{\perp}$: if $f_i: \mathbf{D}_i \rightarrow \mathbf{E}$ ($i < n$) are continuous maps then there is a unique $h: \sum (\mathbf{D}_i)_{\perp} \rightarrow_{\perp} \mathbf{E}$ such that, for all i , with $i < n$, the following diagram commutes:



and show, too, that $\prod_{i < n} (\mathbf{D}_i \rightarrow \mathbf{E}) \cong (\sum_{i < n} (\mathbf{D}_i)_\perp) \rightarrow_\perp \mathbf{E}$. Consider the semi-separated sum $\mathbf{D}_\perp + \mathbf{E}$ in the analogous way.

20. Investigate $\sum_{n \in \omega} (\mathbf{D}^n)_\perp (= (\mathbf{D}_\perp)^*)$ in an analogous way to $\mathbf{D}^*$ (see Exercises 15 and 16). What happens in the other two cases, namely $\sum \mathbf{D}^n$ and $\sum (\mathbf{D}^\otimes)_\perp$?

21. Suppose we want to specify *Dec* with $\mathbf{Op}_0 = \{\mathbf{0}, \mathbf{true}, \mathbf{false}\}$ and $\mathbf{Op}_1 = \{\mathbf{succ}, \mathbf{pred}, \mathbf{zero}, \mathbf{isnum}\}$ where the elements of $\mathbf{Op}_0$ and the arithmetical elements of $\mathbf{Op}_1$ will have much the same meanings as in the text. However we want to model an evaluation mechanism which first tries to calculate the type (number or truth-value) of an expression and, if that terminates, it then tries to calculate its value. So, if the type of e is number (truth-value) that of $\mathbf{pred}(e)$ is number (evaluation does not terminate). If the type of e is number (truth-value) that of $\mathbf{isnum}(e)$ is truth-value (truth-value) and the value is *tt* (*ff*). Give a denotational semantics which uses $\mathbf{V} = \mathbf{N}_\perp + \mathbf{T}_\perp$, the separated sum.

4. The Category of CPOs Considered as a CPO Itself

The problem of finding recursively specified domains (and its solution) can be considered to be one of the main achievements of the present theory. In order to solve it, we will have to develop certain technical tools and that is the concern of the present chapter.

We have already seen several examples of cpos which satisfy recursive domain equations. For example $\mathbf{N}$, the cpo of the integers, satisfies: $\mathbf{N} \cong \mathbf{0} + \mathbf{N}$. Equations for $\mathbf{T}^\omega$, $\mathbf{D}^*$, $\mathbf{X}^\infty$, $\mathbf{Btrees}$ can be found in Exercises 2.4, 3.15, 3.18. The desire to solve such equations in general arises when we consider certain difficulties in denotational semantics. They often concern explicit or implicit forms of self-application where one has to make sense of expressions of the kind “ $x(x)$ ”. Then if $\mathbf{V}$ is a cpo of possible values for x , it seems natural to require that $\mathbf{V}$ somehow contains its own function space, $\mathbf{V} \rightarrow \mathbf{V}$. This is well-known to be impossible when $\mathbf{V}$ is a set and $(\mathbf{V} \rightarrow \mathbf{V})$ is the partial order of all monotonic functions over $\mathbf{V}$ (see Exercise 1.19). However it is possible when $\mathbf{V}$ is a cpo and we restrict attention to the continuous functions over $\mathbf{V}$.

As a concrete example consider the extension of *Imp* (see Exercise 1.3) where we add a syntactic class, **Proc**, of parameterless procedure variables (ranged over by p) and use them to add procedure declaration and calling clauses to the definition of **Stat**:

$$s ::= (\text{proc } p \text{ begin } s \text{ end}) \mid (\text{call } p).$$

The idea is that the declaration **proc** p **begin** s **end** should have the effect of assigning to p the command s , which is run when p is called. Thus we will want a cpo, approximately:

$$\mathbf{C} = (\mathbf{S}_\perp \times \mathbf{PEnv}) \rightarrow (\mathbf{S}_\perp \times \mathbf{PEnv})$$

(a more accurate model would be $(\mathbf{S}_\perp \otimes \mathbf{PEnv}_\perp) \rightarrow_\perp (\mathbf{S}_\perp \otimes \mathbf{PEnv})$). But this is not a proper definition of $\mathbf{PEnv}$ and $\mathbf{C}$ as the definition is circular. What we will do is relax the equalities to isomorphisms and try to find cpos $\mathbf{C}$ and $\mathbf{PEnv}$ which satisfy the simultaneous recursive domain equations:

$$\begin{aligned} (1) \quad & \mathbf{C} \cong (\mathbf{S}_\perp \times \mathbf{PEnv}) \rightarrow (\mathbf{S}_\perp \times \mathbf{PEnv}), \\ (2) \quad & \mathbf{PEnv} \cong (\mathbf{Proc}_\perp \rightarrow_\perp \mathbf{C}). \end{aligned}$$

Equivalently we could just try to find a cpo $\mathbf{C}$ satisfying the equation:

$$(3) \quad \mathbf{C} \cong (\mathbf{S}_\perp \times (\mathbf{Proc}_\perp \rightarrow_\perp \mathbf{C})) \rightarrow (\mathbf{S}_\perp \times (\mathbf{Proc}_\perp \rightarrow_\perp \mathbf{C}))$$

and then specify $\mathbf{PEnv} \stackrel{\text{def}}{=} (\mathbf{Proc}_\perp \rightarrow_\perp \mathbf{C})$.

The implicit self-application lies in the meaning of the statement **call** p . This will be a command, γ , which takes the current store σ (state plus procedure environment and so an element of $\mathbf{S}_\perp \times \mathbf{PEnv}$) and applies the component of it corresponding to p , namely $(\sigma \downarrow 1)[p]$ to σ itself. Thus we have the implicit (and slightly indirect) self-application:

$$(\sigma \downarrow 1)[p](\sigma).$$

Later on we shall consider the untyped λ -calculus where the self-application is completely explicit.

Our idea for solving such recursive domain equations is to consider them as a kind of fixed-point equation. Instead of looking for an element of a cpo, which satisfies a fixed-point condition we look for a cpo (an object in $\mathcal{CPO}_\perp$) which satisfies the condition. We will be guided very explicitly by our earlier considerations on finding least fixed-points of continuous functions in cpos. There is a well-known analogy between quasi-orders (partial orders less the antisymmetry condition) and categories. Every quasiorder $(\mathbf{D}, \sqsubseteq)$ is a category if we take the elements of $\mathbf{D}$ to be the objects and stipulate that there is at most one morphism from an element, d , to another, e , which we write as $(d \rightarrow e)$; it exists iff $d \sqsubseteq e$. Thus we can consider order relation as analogous to morphisms, the reflexivity relation $d \sqsubseteq d$, is analogous to the identity, $id_{\mathbf{D}}: d \rightarrow d$ and the law of transitivity: $(c \sqsubseteq d \wedge d \sqsubseteq e \Rightarrow c \sqsubseteq e)$ is analogous to composition: $((f: c \rightarrow d \wedge g: d \rightarrow e) \Rightarrow g \circ f: c \rightarrow e)$. One can perhaps see a morphism $f: c \rightarrow d$ as showing *why* $c \sqsubseteq d$. It should be stressed that we are only offering an analogy as the reason for the particular mathematical development we are following. Some hints at a justification can be found in Chapter 4. More can be found in the detailed analysis of particular examples of semantics.

The analogue of a monotonic function, f , (where $d \sqsubseteq e$ implies $f(d) \sqsubseteq f(e)$) is a functor F (where $f:d \rightarrow e$ implies $Ff:Fd \rightarrow Fe$ and F preserves the identity and composition). Note that in forming a category-theoretic analogue one must add extra uniformity conditions. So we might expect to consider the category $\mathcal{CPO}_\perp$ and functors $F:\mathcal{CPO}_\perp \rightarrow \mathcal{CPO}_\perp$. However this does not work for equations such as (3) since one cannot compose covariant and contravariant functors in such a cavalier way. The natural attempt at the definition of F would be to put, for each cpo $\mathbf{D}$:

$$(4) \quad F(\mathbf{D}) \stackrel{\text{def}}{=} (\mathbf{S}_\perp \times (\mathbf{Proc}_\perp \rightarrow_\perp \mathbf{D})) \rightarrow (\mathbf{S}_\perp \times (\mathbf{Proc}_\perp \rightarrow_\perp \mathbf{D}))$$

and for each $f:\mathbf{D} \rightarrow_\perp \mathbf{E}$ to define $F(f):F(\mathbf{D}) \rightarrow_\perp F(\mathbf{E})$ by:

$$(5) \quad F(f) = (id_{\mathbf{S}_\perp} \times (id_{\mathbf{Proc}_\perp} \rightarrow_\perp f)) \rightarrow (id_{\mathbf{S}_\perp} \times (id_{\mathbf{Proc}_\perp} \rightarrow_\perp f)).$$

However this is not a good definition; the expression on the right has the wrong type. Its type is $(\mathbf{Y} \rightarrow \mathbf{X}) \rightarrow_\perp (\mathbf{X} \rightarrow \mathbf{Y})$ where $\mathbf{X}$ is $[\mathbf{S}_\perp \times (\mathbf{Proc}_\perp \rightarrow_\perp \mathbf{D})]$ and $\mathbf{Y}$ is $[\mathbf{S}_\perp \times (\mathbf{Proc}_\perp \rightarrow_\perp \mathbf{E})]$ but what is wanted is an expression of type $(\mathbf{X} \rightarrow \mathbf{X}) \rightarrow_\perp (\mathbf{Y} \rightarrow \mathbf{Y})$. This would be obtained if the first occurrence of the variable f on the right hand side were replaced by an expression of type $\mathbf{E} \rightarrow_\perp \mathbf{D}$, which should, presumably, involve f .

Luckily we can trace this to a somewhat questionable assumption in the analogy: there is no reason why *any* $f:\mathbf{D} \rightarrow \mathbf{E}$ should show why $\mathbf{D} \sqsubseteq \mathbf{E}$. Indeed we have not discussed how one cpo could possibly be considered as an approximation to another. Now in modelling a given computational situation we might be faced with a choice between $\mathbf{D}$ and $\mathbf{E}$. It may be that we can use both but in such a way that on the one hand the elements of $\mathbf{D}$ give only a degraded version of the information given by those in $\mathbf{E}$, and on the other hand for each element in $\mathbf{D}$ there is an element in $\mathbf{E}$ which contains exactly the same information. Thus $\mathbf{E}$ can be used to paint a much finer picture than $\mathbf{D}$. Of course, this does not mean $\mathbf{E}$ is better than $\mathbf{D}$: it all depends on the circumstances which is the more appropriate. We try to capture these correspondences between elements of $\mathbf{D}$ and $\mathbf{E}$ as a pair of continuous functions, $i:\mathbf{D} \rightarrow \mathbf{E}$, $j:\mathbf{E} \rightarrow \mathbf{D}$. The first function shows how for any element d , of $\mathbf{D}$ there is an element $i(d)$ in $\mathbf{E}$ with the same information as d ; the second one shows how for any element e in $\mathbf{E}$ there is an element $j(e)$ in $\mathbf{D}$ which contains less information than e and, since $\mathbf{D}$ is to be a degraded version of $\mathbf{E}$, $j(e)$ is the best approximation to e in $\mathbf{D}$. Thus we expect the following two equations to hold:

$$\begin{aligned} \forall d:\mathbf{D}. j \circ i(d) &= d, \\ \forall e:\mathbf{E}. i \circ j(e) &\sqsubseteq e. \end{aligned}$$

For since $i(d)$ contains the same information as d and $j(i(d))$ contains less than $i(d)$ we have $j(i(d)) \sqsubseteq d$. But since d is an approximation to $i(d)$ and $j(i(d))$ is the best one we have $j(i(d)) \sqsupseteq d$. This “demonstrates” the first equation. For the second we note that $i(j(e))$ contains the same information as $j(e)$ which is an approximation to e . Conversely suppose we have i, j which satisfy the equations and read the phrase “ d approximates e ” for an element d of $\mathbf{D}$ and an element e of $\mathbf{E}$ as “ $i(d) \sqsubseteq e$ ”. Then the second equation says that $j(e)$ approximates e and also if d approximates e then $d = j(i(d)) \sqsubseteq j(e)$, showing that $j(e)$ is the best approximation to e in $\mathbf{D}$. This seems to be a reasonable argument for regarding such pairs $\langle i, j \rangle$ as giving one kind of approximation relation between domains and we make a suitable definition.

Definition 1. Let $\mathbf{D}$ and $\mathbf{E}$ be cpos and let $i:\mathbf{D} \rightarrow \mathbf{E}$, $j:\mathbf{E} \rightarrow \mathbf{D}$ be continuous functions. Then $\langle i, j \rangle$ is a *projection pair* (from $\mathbf{D}$ to $\mathbf{E}$) and i is an *embedding* and j is a *projection* and $\mathbf{D}$ is a *projection* of $\mathbf{E}$ provided only that $j \circ i = id_{\mathbf{D}}$ and $i \circ j \sqsubseteq id_{\mathbf{E}}$.

Examples

1. For $n \in \omega$ put $[n] = \{m \in \omega \mid m < n\}$. Then the natural inclusion maps $\iota_{mn}:[m]_\perp \rightarrow [n]_\perp$ ($m \leq n$) are embeddings. The projection $\pi_{mn}:[n]_\perp \rightarrow [m]_\perp$ ($m \leq n$) corresponding to ι_{mn} is defined by:

$$\pi_{nm}(l) = \begin{cases} l & (l < m), \\ \perp & (l = \perp \text{ or } l \geq m). \end{cases}$$

2. The cpo **Tapes** is a projection of $\mathbf{T}^\omega$. With the aid of the evident functions $cons:\mathbf{T} \times \mathbf{T}^\omega \rightarrow \mathbf{T}^\omega$, $head:\mathbf{T}^\omega \rightarrow \mathbf{T}$, $tail:\mathbf{T}^\omega \rightarrow \mathbf{T}^\omega$ we can define the projection pair, **Tapes** $\xrightarrow{i} \mathbf{T}^\omega \xrightarrow{j}$ **Tapes** recursively by:

$$\begin{aligned} i(t) &= cons(hd(t) = 0, i(tl(t))), \\ j(v) &= (head(v) \rightarrow 0 \cdot j(tail(v)) \mid 1 \cdot j(tail(v))). \end{aligned}$$

3. For any cpos $\mathbf{D}, \mathbf{E}$ we have the projection pair, $(\mathbf{D} \rightarrow_\perp \mathbf{E}) \xrightarrow{\iota} (\mathbf{D} \rightarrow \mathbf{E}) \xrightarrow{strict} (\mathbf{D} \rightarrow_\perp \mathbf{E})$.

The analysis of information in terms of events in Chapter 1 warns us that more subtle approximation relations between cpos can exist. For example it was shown there that, in a sense, $\mathbf{P}\omega$ is a degraded version of $\mathbf{Tapes}$. But the discussion there only indicated a map $j: \mathbf{Tapes} \rightarrow \mathbf{P}\omega$ and none in the other direction: indeed it is not hard to show that $\mathbf{P}\omega$ is not a projection of $\mathbf{Tapes}$. However projections will prove perfectly satisfactory for the solution of recursive domain equations.

Lemma 1. *Let $\mathbf{D} \xrightarrow{i} \mathbf{E} \xrightarrow{j} \mathbf{D}$ be a projection pair. Then both i and j are strict. If $\mathbf{D} \xrightarrow{i'} \mathbf{E} \xrightarrow{j'} \mathbf{D}$ is another projection pair then $i \sqsubseteq i'$ iff $j \sqsupseteq j'$.*

Proof. First $i(\perp_{\mathbf{D}}) \sqsubseteq i \circ j(\perp_{\mathbf{E}}) \sqsubseteq \perp_{\mathbf{E}}$ showing i is strict. Second $j(\perp_{\mathbf{E}}) = j \circ i(\perp_{\mathbf{D}}) = \perp_{\mathbf{D}}$ showing j is strict.

If $i \sqsubseteq i'$ then $j \sqsupseteq j \circ i' \circ j' \sqsupseteq j \circ i \circ j' = j'$. Conversely, if $j \sqsupseteq j'$ then $i = i \circ j' \circ i' \sqsubseteq i \circ j \circ i' \sqsubseteq i'$. ■

The second part of this lemma shows that every embedding i has a *unique* corresponding projection, called its *right adjoint*, and written as $i^{\mathbf{R}}$; further, it shows that every projection, j , has a unique corresponding embedding, called its *left adjoint* and written as $j^{\mathbf{L}}$. It follows that $i^{\mathbf{RL}} = i$ and $j^{\mathbf{LR}} = j$. Note that we are not claiming that $i^{\mathbf{R}}$ is a continuous function of i , nor that $j^{\mathbf{L}}$ is a continuous function of j , we are just introducing some notation. We write $i: \mathbf{D} \triangleleft \mathbf{E}$ to indicate that i is an embedding and $j: \mathbf{E} \triangleright \mathbf{D}$ to indicate that j is a projection. With all this under our belts we now have a good idea what the category of cpos and approximations ought to be:

Definition 2. The category $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$ has as objects the cpos and as morphisms the embeddings and composition is the usual composition of functions.

To see that this is a good definition note that $id_{\mathbf{D}}: \mathbf{D} \triangleleft \mathbf{D}$ as $id_{\mathbf{D}}^{\mathbf{R}} = id_{\mathbf{D}}$ and that if $f: \mathbf{D} \triangleleft \mathbf{E}$ and $g: \mathbf{E} \triangleleft \mathbf{F}$ then $(g \circ f): \mathbf{D} \triangleleft \mathbf{F}$ as $(g \circ f)^{\mathbf{R}} = f^{\mathbf{R}} \circ g^{\mathbf{R}}$. The first claim is obvious; for the second calculate $(f^{\mathbf{R}} \circ g^{\mathbf{R}}) \circ (g \circ f) = f^{\mathbf{R}} \circ f = id_{\mathbf{D}}$ and $(g \circ f) \circ (f^{\mathbf{R}} \circ g^{\mathbf{R}}) \sqsubseteq (g \circ g^{\mathbf{R}}) \sqsubseteq id_{\mathbf{E}}$. We could have worked with $\mathcal{CP}\mathcal{O}^{\mathbf{P}}$, the category of projections, instead of $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$. However this would not make any real mathematical difference as the two categories are dual and so, since the embeddings “go in the same way” as the approximation relation we chose $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$. Another possibility would be to use the (isomorphic) category of projection pairs and here the choice of $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$ is on grounds of simplicity.

When an embedding is an inclusion map, $\iota: \mathbf{D} \subseteq \mathbf{E}$ the defining equations for a projection pair take on the simple form, $\forall d: \mathbf{D}. i(d) = d$ and $\forall e: \mathbf{E}. j(e) \sqsubseteq e$. Any embedding $i: \mathbf{D} \triangleleft \mathbf{E}$ is essentially of this form, for if we let the image, $i(\mathbf{D})$ be given the partial order inherited from $\mathbf{E}$ we find it in a subcpo of $\mathbf{E}$ as $\perp_{\mathbf{E}} = i(\perp_{\mathbf{D}})$ and if $\langle i(d_n) \rangle_{n \in \omega}$ is an increasing ω -chain so is $\langle j \circ i(d_n) \rangle_{n \in \omega} = \langle d_n \rangle_{n \in \omega}$ and we see, by the continuity of i that $\bigsqcup_{\mathbf{E}} i(d_n) = i(\bigsqcup_{\mathbf{D}} d_n)$. It follows that the inclusion map $\iota: i(\mathbf{D}) \subseteq \mathbf{E}$ is continuous. Next one easily checks that $\theta: \mathbf{D} \rightarrow i(\mathbf{D})$, where $\theta(d) \stackrel{\text{def}}{=} i(d)$. In an isomorphism with inverse $\theta^{-1} = j \upharpoonright (i(\mathbf{D}))$; it is clear that θ and θ^{-1} are monotonic and we have $\theta^{-1} \circ \theta(d) = j \circ i(d) = d$ and $\theta \circ \theta^{-1}(i(d)) = i \circ j \circ i(d) = i(d)$. Now we have the factorisation of i :

$$\begin{array}{ccc} \mathbf{D} & \xrightarrow{\theta} & i(\mathbf{D}) \\ & \searrow i & \downarrow \iota \\ & & \mathbf{E} \end{array}$$

and ι is an embedding as $\iota^{\mathbf{R}} = \theta \circ j$ for: $(\theta \circ j) \circ \iota = \theta \circ j \circ \iota \circ \theta \circ \theta^{-1} = \theta \circ j \circ i \circ \theta^{-1} = id$ and $\iota \circ (\theta \circ j) = i \circ j \sqsubseteq id$. Because of all this when $i: \mathbf{D} \triangleleft \mathbf{E}$ we often use an *identification convention* where such element d , of $\mathbf{D}$, is *identified* with its image $i(d)$. Then $\mathbf{D}$ is identified with $i(\mathbf{D})$ and i with the inclusion map, ι .

We now return to the pursuit of our analogy attempting to regard $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$ as a kind of cpo in which the approximation relation are replaced by embeddings, $i: \mathbf{D} \triangleleft \mathbf{E}$. Corresponding to the idea of a least element $\perp$ (for any x , $\perp \sqsubseteq x$) we have the idea of an initial element $\perp$ (for any $\mathbf{D}$, there is a *unique* $\perp \triangleleft \mathbf{D}$). The initial element must be $\mathbf{U}$, the one-point cpo, and the morphism $\mathbf{U} \triangleleft \mathbf{D}$ is just $K_{\perp_{\mathbf{D}}}$, the only strict map from $\mathbf{U}$ to $\mathbf{D}$. This is an embedding as $K_{\perp_{\mathbf{D}}}^{\mathbf{R}} = K_{\perp_{\mathbf{U}}}$.

Turning to functors we see next how all our functors over $\mathcal{CP}\mathcal{O}_{\perp}$, whether covariant or contravariant, become covariant functor over $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$! The clue lies in the above remarks on the type of the function $f: \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ when we tried to define a functor for equation (3). What was needed was an element of type $\mathbf{E} \rightarrow_{\perp} \mathbf{D}$ instead. This is now easy, given $i: \mathbf{D} \triangleleft \mathbf{E}$ we will use $j: \mathbf{E} \triangleright \mathbf{D}$.

Example 1 Products. Starting with $\prod: \mathcal{CPO}_\perp^n \rightarrow \mathcal{CPO}_\perp$ (see Exercise 2.6) we define $\prod^E: (\mathcal{CPO}^E)^n \rightarrow \mathcal{CPO}^E$ by putting for cpos $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$:

$$\prod_{i < n}^E \mathbf{D}_i = \prod_{i < n} \mathbf{D}_i$$

and for embeddings $f_i: \mathbf{D}_i \triangleleft \mathbf{E}_i$ ($i < n$):

$$\prod_{i < n}^E f_i = \prod_{i < n} f_i.$$

We must prove that the right hand side is also an embedding and so we show that $(\prod_{i < n} f_i)^R = \prod_{i < n} f_i^R$, by calculating:

$$\begin{aligned} \left(\prod_{i < n} f_i^R \right) \circ \left(\prod_{i < n} f_i \right) &= \prod_{i < n} (f_i^R \circ f_i) && \text{functor laws for } \prod \\ &= \prod_{i < n} id_{\mathbf{D}_i} && \text{property of embeddings} \\ &= id_{\prod \mathbf{D}_i} && \text{functor laws for } \prod \\ &= id_{\prod^E \mathbf{D}_i} && \text{definition of } \prod^E, \end{aligned}$$

and

$$\begin{aligned} \left(\prod_{i < n} f_i \right) \circ \left(\prod_{i < n} f_i^R \right) &= \prod_{i < n} (f_i \circ f_i^R) \\ &\sqsubseteq \prod_{i < n} id_{\mathbf{E}_i} && \prod \text{ acts monotonically on morphisms} \\ &= id_{\prod^E \mathbf{E}_i}. \end{aligned}$$

The functor laws for $\prod^E$ follow immediately from those for $\prod$.

Example 2 Exponentiation. Starting with $\rightarrow: \mathcal{CPO}_\perp^2 \rightarrow \mathcal{CPO}_\perp$ (see Exercise 2.9) we define the *covariant* functor $\rightarrow^E: (\mathcal{CPO}^E)^2 \rightarrow \mathcal{CPO}^E$ by putting for cpos $\mathbf{D}$ and $\mathbf{E}$:

$$(\mathbf{D} \rightarrow^E \mathbf{E}) = (\mathbf{D} \rightarrow \mathbf{E})$$

and for $f: \mathbf{D} \triangleleft \mathbf{D}'$, $g: \mathbf{E} \triangleleft \mathbf{E}'$:

$$(f \rightarrow^E g) = f^R \rightarrow g,$$

using the idea hinted at above. To show $f \rightarrow^E g: (\mathbf{D} \rightarrow \mathbf{E}) \triangleleft (\mathbf{D}' \rightarrow \mathbf{E}')$ we prove that $(f \rightarrow^E g)^R = (f \rightarrow g^R)$ by calculating:

$$\begin{aligned} (f \rightarrow g^R) \circ (f^R \rightarrow g) &= (f^R \circ f) \rightarrow (g^R \circ g) && \text{functor laws} \\ &= id_{\mathbf{D}} \rightarrow id_{\mathbf{E}} \\ &= id_{\mathbf{D} \rightarrow \mathbf{E}} \end{aligned}$$

and

$$\begin{aligned} (f^R \rightarrow g) \circ (f \rightarrow g^R) &= (f \circ f^R) \rightarrow (g \circ g^R) \\ &\sqsubseteq id_{\mathbf{D}'} \rightarrow id_{\mathbf{E}'} && \rightarrow \text{ acts monotonically on morphisms} \\ &= id_{\mathbf{D}' \rightarrow \mathbf{E}'}. \end{aligned}$$

We should also check the functor laws:

$$(id_{\mathbf{D}} \rightarrow^E id_{\mathbf{E}}) = id_{\mathbf{D}} \rightarrow id_{\mathbf{E}} = id_{(\mathbf{D} \rightarrow \mathbf{E})} = id_{(\mathbf{D} \rightarrow^E \mathbf{E})}$$

and for $f: \mathbf{D} \triangleleft \mathbf{D}'$, $f': \mathbf{D}' \triangleleft \mathbf{D}''$, $g: \mathbf{E} \triangleleft \mathbf{E}'$, $g': \mathbf{E}' \triangleleft \mathbf{E}''$:

$$\begin{aligned} (f' \rightarrow^E g') \circ (f \rightarrow^E g) &= (f'^R \rightarrow g') \circ (f^R \rightarrow g) \\ &= ((f'^R \circ f^R) \rightarrow (g' \circ g)) \\ &= (f' \circ f)^R \rightarrow (g' \circ g) \\ &= (f' \circ f) \rightarrow^E (g' \circ g). \end{aligned}$$

In order to avoid tediously checking the details for all our other functors we formulate a general result.

Definition 3. Let $T: \mathcal{CPO}_{\perp}^{m+n} \rightarrow \mathcal{CPO}_{\perp}$ ($m, n \geq 0$) be contravariant in its first m arguments and covariant in its following n arguments. Then T is *locally monotonic* iff whenever $f_i, f'_i: \mathbf{D}'_i \rightarrow \mathbf{D}_i$ ($i < m$) and $g_j, g'_j: \mathbf{E}_j \rightarrow \mathbf{E}'_j$ ($j < n$) are such that $f_i \sqsubseteq f'_i$ ($i < m$) and $g_j \sqsubseteq g'_j$ ($j < n$) then:

$$T(f_0, \dots, f_{m-1}, g_0, \dots, g_{n-1}) \sqsubseteq T(f'_0, \dots, f'_{m-1}, g'_0, \dots, g'_{n-1}).$$

Clearly all the functors $\prod, \rightarrow, \otimes, \rightarrow_{\perp}, \sum, (\cdot)_{\perp}, (\cdot)^*$ (see Exercise 3.15) are or can be regarded as locally monotonic functions. There are two other important examples. The covariant projection functor $P_i: \mathcal{CPO}_{\perp}^n \rightarrow \mathcal{CPO}_{\perp}$ ($n \geq 0, i < n$) is defined by:

$$\begin{aligned} P_i(\mathbf{D}_0, \dots, \mathbf{D}_{n-1}) &= \mathbf{D}_i, \\ P_i(f_0, \dots, f_{n-1}) &= f_i. \end{aligned}$$

Both the projection and constant functors are clearly locally monotonic.

Lemma 2. Every locally monotonic functor $T: \mathcal{CPO}_{\perp}^{m+n} \rightarrow \mathcal{CPO}_{\perp}$, as in Definition 3 can be turned into a covariant functor $T^E: (\mathcal{CPO}^E)^{m+n} \rightarrow \mathcal{CPO}^E$ by stipulating for cpos $\mathbf{D}_0, \dots, \mathbf{D}_{m-1}, \mathbf{E}_0, \dots, \mathbf{E}_{n-1}$ that:

$$T^E(\mathbf{D}_0, \dots, \mathbf{D}_{m-1}, \mathbf{E}_0, \dots, \mathbf{E}_{n-1}) = T(\mathbf{D}_0, \dots, \mathbf{D}_{m-1}, \mathbf{E}_0, \dots, \mathbf{E}_{n-1})$$

and for embeddings, $f_i: \mathbf{D}_i \triangleleft \mathbf{D}'_i$ ($i < m$), $g_i: \mathbf{E}_i \triangleleft \mathbf{E}'_i$ ($i < n$) that

$$T^E(f_0, \dots, f_{m-1}, g_0, \dots, g_{n-1}) = T(f_0^R, \dots, f_{m-1}^R, g_0, \dots, g_{n-1}).$$

Proof. The proof is left to the reader. ■

It is now clear how to define the functor, $F: \mathcal{CPO}^E \rightarrow \mathcal{CPO}^E$ needed for equation (3). For a cpo $\mathbf{D}$ we write, as in equation (4):

$$F(\mathbf{D}) = (\mathbf{S}_{\perp} \times (\mathbf{Proc}_{\perp} \rightarrow_{\perp} \mathbf{D})) \rightarrow (\mathbf{S}_{\perp} \times (\mathbf{Proc}_{\perp} \rightarrow_{\perp} \mathbf{D}))$$

but for an embedding $f: \mathbf{D} \triangleleft \mathbf{E}$ we write:

$$F(f) = (id_{\mathbf{S}_{\perp}} \times (id_{\mathbf{Proc}_{\perp} \rightarrow_{\perp}} f^R)) \rightarrow (id_{\mathbf{S}_{\perp}} \times (id_{\mathbf{Proc}_{\perp} \rightarrow_{\perp}} f)).$$

What this means is that F is a composition of covariant functors over $\mathcal{CPO}^E$ as:

$$\begin{aligned} F &= \rightarrow^E(G, G), \quad \text{where} \\ G &= \times^E(K_{\mathbf{S}_{\perp}}^E, \rightarrow_{\perp}^E(K_{\mathbf{Proc}_{\perp}}^E, Id^E)) \end{aligned}$$

and we leave it to the reader to check that these two definitions amount to the same thing.

When we want to solve simultaneous equations, such as (1), (2) we have to find simultaneous fixed-points of covariant functors over $\mathcal{CPO}^E$. For example in this case we define $F_0, F_1: (\mathcal{CPO}^E)^2 \rightarrow \mathcal{CPO}^E$ by:

$$\begin{aligned} F_0 &= \rightarrow^E(\times^E(K_{\mathbf{S}_{\perp}}^E, P_0^E), \times^E(K_{\mathbf{S}_{\perp}}^E, P_0^E)) \\ F_1 &= \rightarrow_{\perp}^E(K_{\mathbf{Proc}_{\perp}}^E, P_1^E) \end{aligned}$$

and then we have to find cpos $\mathbf{C}, \mathbf{PEnv}$ such that:

$$\begin{aligned} \mathbf{C} &= F_0(\mathbf{C}, \mathbf{PEnv}) \\ \mathbf{PEnv} &= F_1(\mathbf{C}, \mathbf{PEnv}). \end{aligned}$$

As may be expected, it will not prove much harder to solve such simultaneous equations than it is to solve ordinary ones.

Continuing with the analogy we now see what corresponds to lubs of increasing ω -chains, $\sqcup_{n \in \omega} x_n$. First of all the analogue of an increasing ω -chain, $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$ is a (*direct*) ω -chain $\Delta = \langle \mathbf{D}_m, f_{mn} \rangle$ of cpos $\mathbf{D}_n$ and embeddings $f_{mn}: \mathbf{D}_m \triangleleft \mathbf{D}_n$ ($m \leq n < \omega$) such that all the following triangles commute where $l \leq m \leq n$:

$$\begin{array}{ccc} \mathbf{D}_l & \xrightarrow{f_{lm}} & \mathbf{D}_m \\ & \searrow f_{ln} & \downarrow f_{mn} \\ & & \mathbf{D}_n \end{array}$$

Clearly the embeddings $f_{m(m+1)}$ determine all the others and sometimes we write (or define) an ω -chain as $\langle \mathbf{D}_m, f_m \rangle$ where $f_m: \mathbf{D}_m \triangleleft \mathbf{D}_{m+1}$ is $f_{m(m+1)}$; we can picture ω -chain as:

$$\mathbf{D}_0 \xrightarrow{f_0} \mathbf{D}_1 \xrightarrow{f_1} \mathbf{D}_2 \xrightarrow{f_2} \dots \xrightarrow{f_{n-1}} \mathbf{D}_n \xrightarrow{f_n} \dots$$

An example of an ω -chain in $\langle [m], \iota_{mn} \rangle$ which we might picture as:

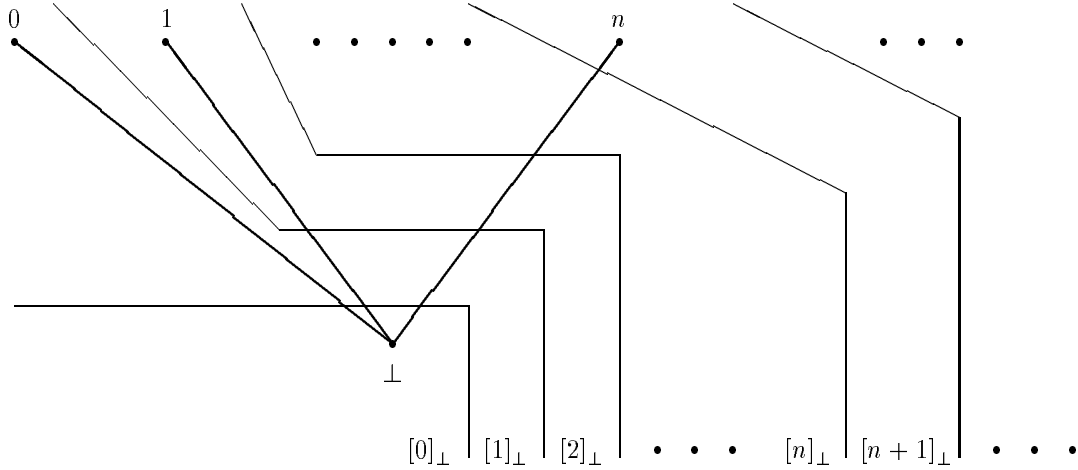


Figure XIII: A direct ω -chain.

Corresponding to an upper bound, x , of an increasing ω -chain, we clearly need for a chain $\Delta = \langle \mathbf{D}_m, f_{mn} \rangle$ not only a cpo $\mathbf{D}$ but also a whole collection $\rho = \langle \rho_m \rangle_{m \in \omega}$ of embeddings $\rho_m: \mathbf{D}_m \triangleleft \mathbf{D}$ which satisfy a suitable coherence condition. So we say that $\rho: \Delta \rightarrow \mathbf{D}$ is a *cone* from Δ to $\mathbf{D}$ if ρ is a sequence $\langle \rho_n \rangle_{n \in \omega}$ of embeddings $\rho_n: \mathbf{D}_n \triangleleft \mathbf{D}$ such that all the following triangles commute where $m \leq n < \omega$:

$$\begin{array}{ccc} \mathbf{D}_m & \xrightarrow{\rho_m} & \mathbf{D} \\ f_{mn} \downarrow & \nearrow \rho_n & \\ \mathbf{D}_n & & \end{array}$$

Note that it is only necessary to check that they commute for the cases with $n = m+1$ as the other cases then follow using the commuting triangles for chains. For the chain $\Delta = \langle [m], \iota_{mn} \rangle$ we have the cone $\rho: \Delta \rightarrow \mathbf{N}$ where $\rho_m = \iota_{m\omega}$.

Finally corresponding to the idea of a *least* upper bound of an increasing chain we have the idea of an *initial* (also called *universal*) cone. If $\rho: \Delta \rightarrow \mathbf{D}$, $\rho': \Delta \rightarrow \mathbf{D}'$ are cones a morphism, $\theta: \rho \triangleleft \rho'$ is defined to be an embedding $\theta: \mathbf{D} \triangleleft \mathbf{D}'$ such that all the following triangles commute:

$$\begin{array}{ccc} \mathbf{D}_m & \xrightarrow{\rho_m} & \mathbf{D} \\ & \searrow \rho'_m & \downarrow \theta \\ & & \mathbf{D}' \end{array}$$

This, as may easily be verified, gives a category of cones from Δ and an initial cone is one from which there is exactly one morphism to any other cone. This specifies ρ up to an isomorphism.

In other words, $\rho: \Delta \rightarrow \mathbf{D}$ is initial (= universal) iff whenever $\rho': \Delta \rightarrow \mathbf{D}'$ is a cone there is exactly one embedding $\theta: \mathbf{D} \triangleleft \mathbf{D}'$ (called the mediating morphism) which makes all the above triangles commute. Clearly this specifies $\mathbf{D}$ and also the ρ_n up to isomorphism, as we have already essentially remarked; when $\rho: \Delta \rightarrow \mathbf{D}$ is universal we sometimes write $\mathbf{D} = \lim_{\rightarrow} \Delta$ (the *direct* limit of Δ) and in the light of the above remarks on isomorphism this should cause little confusion.

For example, as the reader no doubt expected, the cone $\langle \iota_{m\omega} \rangle_{m \in \omega}: \langle [m], \iota_{mn} \rangle \rightarrow \mathbf{N}$ is universal as if $\rho: \langle [m], \iota_{mn} \rangle \rightarrow \mathbf{D}$ is a cone then we must have for the mediating morphism, θ , that $\theta(\perp) = \perp$ and for $m < \omega$, $\theta(m) = \theta(\iota_{m\omega}(m)) = \rho_m(m)$, which clearly determines θ and shows that it exists.

It is awkward to work with the category-theoretic definition directly and in order to show, following the analogy, that direct limits always exist we find an equivalent local order-theoretic condition. If $\rho: \Delta \rightarrow \mathbf{D}$ is a cone where $\Delta = \langle \mathbf{D}_m, f_m \rangle$ then the chain $\langle \rho_m \circ \rho_m^R(x) \rangle_{m \in \omega}$ of the best approximations to an element of x at level m is increasing as we have:

$$\rho_m \circ \rho_m^R = (\rho_{m+1} \circ f_m) \circ (\rho_{m+1} \circ f_m)^R = \rho_{m+1} \circ f_m \circ f_m^R \circ \rho_{m+1}^R \sqsubseteq \rho_{m+1} \circ \rho_{m+1}^R.$$

It is natural to expect that if $\mathbf{D}$ is to be the direct limit then x is the limit of the chain and indeed we have:

Lemma 3. *Let $\Delta = \langle \mathbf{D}_m, f_m \rangle$ be a direct ω -chain of embeddings and let $\rho: \Delta \rightarrow \mathbf{D}$ be a cone of embeddings. Then if $(\bigsqcup_n \rho_n \circ \rho_n^R) = id_{\mathbf{D}}$ it follows that ρ is universal.*

Proof. Suppose $\rho': \Delta \rightarrow \mathbf{D}'$ is a cone of embeddings. Then $\langle \rho'_m \circ \rho_m^R \rangle_{m \in \omega}$ is an increasing ω -chain as $\rho'_m \circ \rho_m^R = \rho'_{m+1} \circ f_m \circ f_m^R \circ \rho_{m+1}^R \sqsubseteq \rho'_{m+1} \circ \rho_{m+1}^R$ and so we may define $\theta: \mathbf{D} \rightarrow \mathbf{D}'$ by:

$$\theta = \bigsqcup_m \rho'_m \circ \rho_m^R.$$

To prove θ is an embedding we note that $\langle \rho_m \circ \rho_m^R \rangle_{m \in \omega}$ must also be an increasing ω -sequence and prove that $\theta^R = \bigsqcup_m \rho_m \circ \rho_m^R$ by calculating:

$$\begin{aligned} \left(\bigsqcup_m \rho_m \circ \rho_m^R \right) \circ \left(\bigsqcup_m \rho'_m \circ \rho_m^R \right) &= \bigsqcup_m \rho_m \circ \rho_m^R \circ \rho'_m \circ \rho_m^R && \text{composition is continuous} \\ &= \bigsqcup_m \rho_m \circ \rho_m^R \\ &= id_{\mathbf{D}} && \text{by assumption} \end{aligned}$$

and

$$\begin{aligned} \left(\bigsqcup_m \rho'_m \circ \rho_m^R \right) \circ \left(\bigsqcup_m \rho_m \circ \rho_m^R \right) &= \bigsqcup_m \rho'_m \circ \rho_m^R \circ \rho_m \circ \rho_m^R \\ &= \bigsqcup_m \rho'_m \circ \rho_m^R \\ &\sqsubseteq id_{\mathbf{D}'} . \end{aligned}$$

Next we must check θ is mediating:

$$\begin{aligned}
\theta \circ \rho_m &= \bigsqcup_{n \geq m} \rho'_n \circ \rho_n^R \circ \rho_m \\
&= \bigsqcup_{n \geq m} \rho'_n \circ \rho_n^R \circ \rho_n \circ f_{mn} \\
&= \bigsqcup_{n \geq m} \rho'_n \circ f_{mn} \\
&= \rho'_m.
\end{aligned}$$

Finally we check uniqueness. Let $\theta': \mathbf{D} \triangleleft \mathbf{D}'$ be a mediating morphism. Then,

$$\theta' = \theta' \circ id_{\mathbf{D}} = \theta' \circ \left(\bigsqcup_m \rho_m \circ \rho_m^R \right) = \bigsqcup_m (\theta' \circ \rho_m) \circ \rho_m^R = \bigsqcup_m \rho'_m \circ \rho_m^R. \blacksquare$$

It turns out that we can always construct the direct limit by using the criterion of Lemma 3.

Lemma 4. *Let $\Delta = \langle \mathbf{D}_m, f_{mn} \rangle$ be a direct ω -chain of embeddings. Then there is a cone $\rho: \Delta \rightarrow \mathbf{D}$ of embedding such that $\bigsqcup_m \rho_m \circ \rho_m^R = id_{\mathbf{D}}$.*

Proof. Let $\mathbf{D}$ be the subset of $\prod_{m \in \omega} \mathbf{D}_m$ where, $\mathbf{D} = \{d \in \prod_{m \in \omega} \mathbf{D}_m \mid \forall m. f_m^R(d_{m+1}) = d_m\}$, with the inherited coordinatewise order. Then $\mathbf{D}$ is a subcpo of the product with $\perp_{\mathbf{D}} = \langle \perp_{\mathbf{D}_m} \rangle_{m \in \omega}$ and $\bigsqcup_n d^{(n)} = \langle \bigsqcup_n d_m^{(n)} \rangle_{m \in \omega}$. Now define a cone $\rho: \Delta \rightarrow \mathbf{D}$ by: putting for any d in $\mathbf{D}_m$

$$\rho_m(d) = \langle f_{mn}(d) \rangle_{n \in \omega},$$

where if $m > n$, $f_{mn} \stackrel{\text{def}}{=} f_{nm}^R$. One easily checks that $\rho_m(d)$ is an element of $\mathbf{D}$ and that ρ_m is continuous; further ρ_m is an embedding as one can check that $\rho_m^R(d) = (d \downarrow m)$.

As $f_m^R \circ \rho_{m+1}^R(d) = f_m^R(d \downarrow (m+1)) = (d \downarrow m) = \rho_m^R(d)$ we have $\rho_m^R = f_m^R \circ \rho_{m+1}^R$ and so $\rho_m = \rho_m^{RL} = (f_m^R \circ \rho_{m+1}^R)^L = \rho_{m+1}^{RL} \circ f_m^{RL} = \rho_{m+1} \circ f_m$ showing that ρ is a cone.

Finally as $\rho_m \circ \rho_m^R \sqsubseteq id_{\mathbf{D}}$ on the one hand and on the other for any d in $\mathbf{D}$, $(\rho_m \circ \rho_m^R(d)) \downarrow m = d_m$ we have, $\bigsqcup_m \rho_m \circ \rho_m^R = id_{\mathbf{D}}$, which concludes the proof. $\blacksquare$

Theorem 1. *For any direct ω -chain, Δ , of embeddings, $\lim_{\rightarrow} \Delta$ always exists. Indeed $\rho: \Delta \rightarrow \mathbf{D}$ is universal iff $\bigsqcup_m \rho_m \circ \rho_m^R = id_{\mathbf{D}}$.*

Proof. By Lemma 4 a cone $\rho: \Delta \rightarrow \mathbf{D}$ satisfying the conditions of Lemma 3 exists and so $\lim_{\rightarrow} \Delta$ exists. Lemma 3 shows that if $\rho': \Delta \rightarrow \mathbf{D}'$ satisfies the condition that $\bigsqcup_m \rho'_m \circ \rho_m'^R = id_{\mathbf{D}'}$ then it is universal. Conversely if ρ' is universal then there is an isomorphism $\theta: \rho \rightarrow \rho'$ and so $\bigsqcup_m \rho'_m \circ \rho_m'^R = \bigsqcup_m (\theta \circ \rho_m) \circ (\theta \circ \rho_m)^R = \bigsqcup_m \theta \circ \rho_m \circ \rho_m^R \circ \theta^{-1} = \theta \circ (\bigsqcup_m \rho_m \circ \rho_m^R) \circ \theta^{-1} = \theta \circ id_{\mathbf{D}} = id_{\mathbf{D}'}$. $\blacksquare$

Suppose $\rho: \Delta \rightarrow \mathbf{D}$ is a chain where $\Delta = \langle \mathbf{D}_m, f_{mn} \rangle$. Then, as remarked earlier, we can consider that all the ρ_m are inclusions $\rho_m: \mathbf{D}_m \subseteq \mathbf{D}$. Then each f_{mn} is also an inclusion as for d in $\mathbf{D}_m$ we have $f_{mn}(x) = \rho_n^R \circ \rho_n \circ f_{mn}(x) = \rho_n^R \circ \rho_m(x) = \rho_n^R(x) = \rho_n^R \circ \rho_n(x) = x$. So we have $\mathbf{D}_0 \subseteq \mathbf{D}_1 \subseteq \dots$ and $\mathbf{D} \supseteq \bigcup_m \mathbf{D}_m$. Suppose now that ρ is universal; then each element x of $\mathbf{D}$ is the lub of its best approximants in $\mathbf{D}_n$ as: $x = \bigsqcup_m \rho_m \circ \rho_m^R(x) = \bigsqcup_m \rho_m^R(x)$, and so $x \sqsubseteq y$ iff $\forall m. \rho_m^R(x) \sqsubseteq \rho_m^R(y)$. Thus we can consider $\mathbf{D}$ as being $\bigcup_m \mathbf{D}_m$ with lubs added to turn it into a cpo.

For example in the case where the $\mathbf{D}_m$ are all flat all chains of best approximants will be trivial (eventually constant) and we will have $\mathbf{D} = \bigcup_m \mathbf{D}_m$, as is the case with the universal cone to $\mathbf{N}$ which we discussed above.

Continuing the analogy we now see what corresponds to continuous functions, which are those which preserve lubs of increasing ω -chains. Now monotonic functions preserves increasing ω -chains and upper bounds of them. Analogously, if $F: \mathcal{CPQ}^E \rightarrow \mathcal{CPQ}^E$ is a covariant functor and $\rho: \Delta \rightarrow \mathbf{D}$ is a cone where $\Delta = \langle \mathbf{D}_m, f_{mn} \rangle$ then $F(\rho): F(\Delta) \rightarrow F(\mathbf{D})$ is also a cone where $F(\Delta) \stackrel{\text{def}}{=} \langle F(\mathbf{D}_m), F(f_{mn}) \rangle$ and $F(\rho) \stackrel{\text{def}}{=} \langle F(\rho_m) \rangle_{m \in \omega}$. More generally for a covariant functor of several arguments, $F: (\mathcal{CPQ}^E)^n \rightarrow \mathcal{CPQ}^E$ if $\rho^{(i)}: \Delta^{(i)} \rightarrow \mathbf{D}^{(i)}$ are cones where $\Delta^{(i)} = \langle \mathbf{D}_m^{(i)}, f_{mn}^{(i)} \rangle$ (for $i < n$) then

$$F(\rho^{(0)}, \dots, \rho^{(n-1)}): F(\Delta^{(0)}, \dots, \Delta^{(n-1)}) \rightarrow F(\mathbf{D}^{(0)}, \dots, \mathbf{D}^{(n-1)})$$

is a cone where

$$F(\Delta^{(0)}, \dots, \Delta^{(n-1)}) \stackrel{\text{def}}{=} \langle F(\mathbf{D}_m^{(0)}, \dots, \mathbf{D}_m^{(n-1)}), F(f_{mm'}^{(0)}, \dots, f_{mm'}^{(n-1)}) \rangle$$

and

$$F(\rho^{(0)}, \dots, \rho^{(n-1)}) \stackrel{\text{def}}{=} \langle F(\rho_m^{(0)}, \dots, \rho_m^{(n-1)}) \rangle_{m \in \omega}.$$

Definition 4. A covariant functor, $F: (\mathcal{CPO}^E)^n \rightarrow \mathcal{CPO}^E$ is *continuous* iff whenever $\rho^{(i)}: \Delta^{(i)} \rightarrow \mathbf{D}^{(i)}$ are universal cones (for $i < n$) so is $F(\rho^{(0)}, \dots, \rho^{(n-1)}): F(\Delta^{(0)}, \dots, \Delta^{(n-1)}) \rightarrow F(\mathbf{D}^{(0)}, \dots, \mathbf{D}^{(n-1)})$. Note that this definition implies that $F(\lim_{\rightarrow} \Delta^{(0)}, \dots, \lim_{\rightarrow} \Delta^{(n-1)}) \cong \lim_{\rightarrow} F(\Delta^{(0)}, \dots, \Delta^{(n-1)})$; essentially it is obtained from the latter condition by adding uniformity conditions on the morphisms.

Example. We show that exponentiation is continuous. Let $\rho: \Delta \rightarrow \mathbf{D}$ and $\rho': \Delta' \rightarrow \mathbf{D}'$ be universal cones of embeddings. Then $(\rho \rightarrow^E \rho'): (\Delta \rightarrow^E \Delta') \rightarrow (\mathbf{D} \rightarrow^E \mathbf{D}')$ is universal as we calculate:

$$\begin{aligned} \bigsqcup_m (\rho \rightarrow^E \rho')_m \circ (\rho \rightarrow^E \rho')_m^R &= \bigsqcup_m (\rho_m \rightarrow^E \rho'_m) \circ (\rho_m \rightarrow^E \rho'_m)^R \\ &= \bigsqcup_m (\rho_m^R \rightarrow \rho'_m) \circ (\rho_m \rightarrow \rho'_m^R) \\ &= \bigsqcup_m (\rho_m \circ \rho_m^R) \rightarrow (\rho'_m \circ \rho'_m^R) \\ &= (\bigsqcup_m \rho_m \circ \rho_m^R) \rightarrow (\bigsqcup_m \rho'_m \circ \rho'_m^R) \\ &\quad \text{exponentiation acts continuously on morphisms} \\ &= id_{\mathbf{D}} \rightarrow id_{\mathbf{D}'} \quad \text{by Theorem 1} \\ &= id_{\mathbf{D} \rightarrow^E \mathbf{D}'} \end{aligned}$$

and by Theorem 1 that shows that $\rho \rightarrow \rho'$ is universal.

Rather than consider all our other constructions separately we formulate a general result.

Definition 5. Let $T: (\mathcal{CPO})_{\perp}^{m+n} \rightarrow \mathcal{CPO}_{\perp}$ be contravariant in its first m arguments and covariant in its following n arguments. Then T is *locally continuous* iff whenever $\langle f_l^{(i)} \rangle_{l \in \omega}$ is an increasing chain of continuous functions, $f_l^{(i)}: \mathbf{D}'_i \rightarrow_{\perp} \mathbf{D}_i$, for $i < m$ and also $\langle g_l^{(j)} \rangle_{l \in \omega}$ is an increasing chain of continuous functions $g_l^{(j)}: \mathbf{E}_j \rightarrow_{\perp} \mathbf{E}'_j$ for $j < n$ then

$$T(f_l^{(0)}, \dots, f_l^{(m-1)}, g_l^{(0)}, \dots, g_l^{(n-1)}): T(\mathbf{D}_0, \dots, \mathbf{D}_{m-1}, \mathbf{E}_0, \dots, \mathbf{E}_{n-1}) \rightarrow_{\perp} T(\mathbf{D}'_0, \dots, \mathbf{D}'_{m-1}, \mathbf{E}'_0, \dots, \mathbf{E}'_{n-1})$$

is an increasing chain of continuous functions with the lub, $T(\bigsqcup_l f_l^{(0)}, \dots, \bigsqcup_l f_l^{(m-1)}, \bigsqcup_l g_l^{(0)}, \dots, \bigsqcup_l g_l^{(n-1)})$.

Clearly any locally-continuous functor is locally-monotonic. All our functors, $\prod, \rightarrow, \otimes, \rightarrow_{\perp}, \sum, (\cdot)_{\perp}, (\cdot)^*$, $P_i, K_{\mathbf{D}}$ are locally-continuous.

Theorem 2. Let $T: \mathcal{CPO}_{\perp}^{m+n} \rightarrow \mathcal{CPO}_{\perp}$ be a locally continuous functor as in Definition 5. Then T^E is continuous.

Proof. The proof is similar to the case of exponentiation and is left to the reader. ■

We should check that continuity is preserved under composition. Suppose $F_i: (\mathcal{CPO}^E)^m \rightarrow \mathcal{CPO}^E$ ($i < n$) are n continuous functors of degree m and $F: (\mathcal{CPO}^E)^n \rightarrow \mathcal{CPO}^E$ is also continuous. Then so is the composition $F(F_0, \dots, F_{n-1}): (\mathcal{CPO}^E)^m \rightarrow \mathcal{CPO}^E$ of F with $F_0, \dots, F_{n-1}$. For suppose $\rho^{(0)}, \dots, \rho^{(m-1)}$ are universal cones. Then as the F_i are all continuous the cones $F_i(\rho^{(0)}, \dots, \rho^{(m-1)})$ are also universal. But then as F is continuous the cone $F(F_0(\rho^{(0)}, \dots, \rho^{(m-1)}), \dots, F_{n-1}(\rho^{(0)}, \dots, \rho^{(m-1)})) = F(F_0, \dots, F_{n-1})(\rho^{(0)}, \dots, \rho^{(m-1)})$ is also universal.

This concludes all the basic technical material needed to solve recursive domain equations. We recapitulate the analogy, as developed so far, in a table.

Order $x \sqsubseteq y$	Morphism $f: \mathbf{D} \triangleleft \mathbf{E}$
Quasi-order $x \sqsubseteq x$ $(x \sqsubseteq y \wedge y \sqsubseteq z) \Rightarrow x \sqsubseteq z$	Category $id_{\mathbf{D}}: \mathbf{D} \triangleleft \mathbf{D}$ $(f: \mathbf{D} \triangleleft \mathbf{E} \wedge g: \mathbf{E} \triangleleft \mathbf{F}) \Rightarrow (g \circ f): \mathbf{D} \triangleleft \mathbf{F}$
Least element $\perp \sqsubseteq x$	Initial element $\exists! f: \mathbf{U} \triangleleft \mathbf{D}$
Monotonic function $x \sqsubseteq y \Rightarrow fx \sqsubseteq fy$ (and several arguments)	Covariant functor $f: \mathbf{D} \triangleleft \mathbf{E} \Rightarrow Ff: F\mathbf{D} \triangleleft F\mathbf{E}$ $F(id_{\mathbf{D}}) = id_{F\mathbf{D}}$ $F(g \circ f) = F(g) \circ F(f)$ (and several arguments)
Lub of ω-chain $x_n \sqsubseteq \bigsqcup_n x_n$ $(\forall n. x_n \sqsubseteq y) \Rightarrow \bigsqcup_n x_n \sqsubseteq y$	Direct limit of ω-chain $\rho: \Delta \rightarrow \lim_{\rightarrow} \Delta$ $(\rho': \Delta \rightarrow \mathbf{E}) \Rightarrow (\exists! \theta: \rho \triangleleft \rho')$
Continuous function $f(\bigsqcup_n x_n) = \bigsqcup_n f(x_n)$ (and several arguments)	Continuous functor $(\rho \text{ universal}) \Rightarrow (F(\rho) \text{ universal})$ (and several arguments)

Exercises

1. Suppose we add a class **Proc** of (parameterless) procedure variables to the syntax of *Imp* and add procedure declaration of limited scope (rather than the unbounded scope considered in this chapter so far) and calling clauses to the syntax of *Stat*:

$$s ::= (\text{letproc } p \text{ be } s \text{ in } s) \mid (\text{call } p)$$

Show that it is possible to give a denotational semantics without needing any recursively specified cpos.

2. Show that $\mathbf{P}\omega \times \mathbf{P}\omega \triangleleft \mathbf{P}\omega$, $\mathbf{T}^\omega \times \mathbf{T}^\omega \triangleleft \mathbf{T}^\omega$, $\mathbf{Tapes} \triangleleft \mathbf{Btrees}$, but that $(\mathbf{P}\omega \rightarrow \mathbf{P}\omega) \not\triangleleft \mathbf{P}\omega$ (this is not easy), that $\mathbf{P}\omega \not\triangleleft \mathbf{Tapes}$, $\mathbf{Tapes} \times \mathbf{Tapes} \not\triangleleft \mathbf{Tapes}$, $\mathbf{T}^\omega \rightarrow \mathbf{T}^\omega \not\triangleleft \mathbf{T}^\omega$.

3. Show that if $\mathbf{D}$ and $\mathbf{E}$ are finite and $\mathbf{D} \triangleleft \mathbf{E} \triangleleft \mathbf{D}$ then $\mathbf{D} \cong \mathbf{E}$; show that $\mathbf{T}^\omega \triangleleft \mathbf{N}^\omega \triangleleft \mathbf{T}^\omega$ but $\mathbf{T}^\omega \not\cong \mathbf{N}^\omega$.

4. Show that $\pi_i: \prod_i \mathbf{D}_i \triangleright \mathbf{D}_i$, $(\mathbf{D} \rightarrow \mathbf{E}) \triangleright \mathbf{E}$, $\Pi_i: \mathbf{D}_i \triangleleft \sum_i \mathbf{D}_i$, $\mathbf{D} \otimes \mathbf{E} \triangleleft \mathbf{D} \times \mathbf{E}$, $\mathbf{D} \triangleleft \mathbf{D}_\perp$, $\mathbf{D} \triangleleft \mathbf{D}^*$. When do we have $\mathbf{0} \triangleleft \mathbf{D}$? Show that if $\mathbf{D}$ has a continuous glb operation, $\sqcap$, then $\sqcap: \mathbf{D} \times \mathbf{D} \triangleright \mathbf{D}$.

5. For any continuous $p: \mathbf{D} \rightarrow \mathbf{D}$ let $\mathbf{Fix}_p$ be the cpo of the fixed-points of p , under the inherited order. Sometimes a $p: \mathbf{D} \rightarrow \mathbf{D}$ is called a projection if $p = p \circ p \sqsubseteq id_{\mathbf{D}}$. Show that if p is a projection in this sense then $\iota: \mathbf{Fix}_p \subseteq \mathbf{D}$ is an embedding with $\iota^R(d) = p(d)$. Show that if $i: \mathbf{C} \rightarrow \mathbf{D}$ is an embedding then $i \circ j$ is a projection with $i(\mathbf{C}) = \mathbf{Fix}_{i \circ j}$.

6. The cpo $[0, 1]$ is the real interval with the usual ordering on the reals. Show that the Cantor middle third set is a projection of it and it is a projection of the Cantor middle third set. (The Cantor middle third set is the set of all elements of $[0, 1]$ whose ternary expansion contains no 2's.) What are the relations between $[0, 1]$ and $[-\infty, +\infty]$ and $[0, +\infty]$?

7. Let $\mathbf{Tapes}_n$ ($n \leq \omega$) be the subcpo of $\mathbf{Tapes}$ consisting of all tapes of length $\leq n$. Show that the inclusions $\mathbf{Tapes}_m \subseteq \mathbf{Tapes}_n \subseteq \mathbf{Tapes}$ ($m \leq n$) are embeddings and hence find a universal cone to $\mathbf{Tapes}$. Do the same sort of thing for $\mathbf{P}\omega$, $\mathbf{T}^\omega$, ω and $\mathbf{Btrees}$.

8. A pair $\mathbf{D} \xrightarrow{i} \mathbf{E} \xrightarrow{j} \mathbf{D}$ of continuous maps is an adjoint pair iff $j \circ i \sqsupseteq id_{\mathbf{D}}$ and $i \circ j \sqsubseteq id_{\mathbf{E}}$ and we say that i is the *left* adjoint and j is the *right* adjoint. Show that $\langle i, j \rangle$ is an adjoint pair iff $(\forall x: \mathbf{D}, y: \mathbf{E}. x \sqsubseteq jy \text{ iff } ix \sqsubseteq y)$. Show that if $\mathbf{D} \xrightarrow{i'} \mathbf{E} \xrightarrow{j'} \mathbf{D}$ is another adjoint pair then $i \sqsubseteq i'$ iff $j \sqsupseteq j'$, proving that left and right adjoints are unique (and we have i^R, j^L as before). Let $\langle i, j \rangle$ be a fixed adjoint pair from now on. Show that i is an embedding iff i is 1-1 iff j is onto. If $j \circ i \sqsupseteq id_{\mathbf{D}}$ and $i \circ j = id_{\mathbf{E}}$ we say that i (and also $j \circ i$) is a *closure*, ($f: \mathbf{D} \rightarrow \mathbf{D}$ is a closure in general iff $f = f \circ f \sqsupseteq id_{\mathbf{D}}$). Show that j is a closure iff i is onto iff j is into. Show i is an isomorphism iff it is an embedding and a closure. Show that $i = i \circ j \circ i$ and $j = j \circ i \circ j$. Show

that i is completely additive (preserves all lubs) and that j is completely multiplicative (preserves all glbs), generating strictness as $\perp = \bigsqcup_{\mathbf{D}} \emptyset = \prod_{\mathbf{D}} \mathbf{D}$. Show that i factorises as:

$$\begin{array}{ccc} \mathbf{D} & \xrightarrow{i'} & \mathbf{X} \\ & \searrow i & \downarrow i'' \\ & & \mathbf{E} \end{array}$$

where i' is a closure and i'' is an embedding. What goes wrong with the general theory if we use left adjoints instead of embeddings? Can it be repaired? (This is an interesting open question which is probably not hard.)

9. Show that $\mathbf{P}\omega \times \mathbf{P}\omega$ and $\mathbf{P}\omega \rightarrow \mathbf{P}\omega$ are both closures of $\mathbf{P}\omega$. Show that there are no non-trivial closures of **Tapes** or $\mathbf{T}^\omega$.

10. Find an increasing chain of isomorphisms, $\theta_n: \mathbf{D} \cong \mathbf{D}$ such that $\bigsqcup_n \theta_n$ is not an embedding or a projection; find a decreasing chain of isomorphisms $\theta_n: \mathbf{D} \cong \mathbf{D}$ whose glb, $\prod_n \theta_n$, exists but is not an embedding or a projection. Show that neither of $(\cdot)^L$, $(\cdot)^R$ are even monotonic.

11. Is $\prod^E \mathbf{D}_i$ the categorical product in $\mathcal{CPO}^E$? What about the other constructions.

12. Define the natural “forgetful functor” $V: \mathcal{CPO}^E \rightarrow \mathcal{CPO}_\perp$ and show that if $T: \mathcal{CPO}_\perp \rightarrow \mathcal{CPO}_\perp$ is covariant and locally monotonic then T^E is the unique functor, $F: \mathcal{CPO}^E \rightarrow \mathcal{CPO}^E$ such that the following diagram of functors commutes:

$$\begin{array}{ccc} \mathcal{CPO}^E & \xrightarrow{F} & \mathcal{CPO}^E \\ V \downarrow & & \downarrow V \\ \mathcal{CPO}_\perp & \xrightarrow{T} & \mathcal{CPO}_\perp \end{array}$$

Extend this to covariant functors of several arguments. What can you do with general functors which may be mixed covariant and contravariant? [Hint Look at $\mathcal{CPO}^D$ which has as morphisms pairs $\mathbf{D} \xrightarrow{f} \perp \mathbf{E} \xrightarrow{g} \perp \mathbf{D}$ of strict continuous functions.]

13. Show that if $\rho: \Delta \rightarrow \mathbf{D}$ is universal, where $\Delta = \langle \mathbf{D}_m, f_{mn} \rangle$, then so is $\rho^-: \Delta^- \rightarrow \mathbf{D}$ where $\Delta^- = \langle \mathbf{D}_{m+1}, f_{(m+1)(n+1)} \rangle$ and $\rho^- = \langle \rho_{m+1} \rangle_{m \in \omega}$. In other words dropping a few terms makes no difference to universality. What happens if we drop an infinite subset, but still leave an infinite subset?

14. Fill in the missing details in the proof of Lemma 3.

15. Suppose $\rho: \langle \mathbf{D}_m, f_{mn} \rangle \rightarrow \mathbf{D}$ is a universal cone of embeddings and the ρ_m and f_{mn} are all inclusions. Suppose too that all increasing ω -chains in any of the $\mathbf{D}_m$ are trivial. Let $\bigcup_m \mathbf{D}_m$ be the partial order where for x in $\mathbf{D}_m$, y in $\mathbf{D}_n$, $x \sqsubseteq y$ iff $x \sqsubseteq_{\mathbf{D}_l} y$ where $l = \max(m, n)$. Show that the monotonic inclusion map, $\iota: \bigcup_m \mathbf{D}_m \rightarrow \mathbf{D}$ is universal in the sense that if $f: \bigcup_m \mathbf{D}_m \rightarrow \mathbf{E}$ is any other monotonic map then there is a unique continuous map $h: \mathbf{D} \rightarrow \mathbf{E}$ such that the following diagram commutes:

$$\begin{array}{ccc} \bigcup_n \mathbf{D}_n & & \\ \iota \downarrow & \searrow f & \\ \mathbf{D} & \xrightarrow{h} & \mathbf{E} \end{array}$$

Show that ι need not be continuous in general. What goes wrong if you drop the assumption on the $\mathbf{D}_m$?

16. Try to axiomatise direct limits in $\mathcal{CPO}^E$ in something of the same style we axiomatised the other constructions such as $\prod$ and $\rightarrow$.

17 Coincidence of Direct and Inverse Limits. A direct ω -chain in $\mathcal{CPO}_\perp$ is a structure $\Delta = \langle \mathbf{D}_m, f_{mn} \rangle$ where $f_{mn}: \mathbf{D}_m \rightarrow_\perp \mathbf{D}_n$ ($m \leq n$) and then a cone $\rho: \Delta \rightarrow \mathbf{D}$ in $\mathcal{CPO}_\perp$ is a system of $\rho_m: \mathbf{D}_m \rightarrow_\perp \mathbf{D}$ making the appropriate triangles commute. Define a universal cone in $\mathcal{CPO}_\perp$ and show that if Δ is a direct ω -chain in $\mathcal{CPO}^E$ and $\rho: \Delta \rightarrow \mathbf{D}$ is a universal cone in $\mathcal{CPO}^E$ then it is universal in $\mathcal{CPO}_\perp$ too. An *inverse* ω -chain in $\mathcal{CPO}_\perp$ is a structure $\Delta = \langle \mathbf{D}_m, f_{mn} \rangle$ where $f_{mn}: \mathbf{D}_n \rightarrow_\perp \mathbf{D}_m$ ($m \leq n$). Define cones $\rho: \mathbf{D} \rightarrow \Delta$ and universal cones $\rho: \mathbf{D} \rightarrow \Delta$ in $\mathcal{CPO}_\perp$ (and here we write $\mathbf{D} = \lim_\omega \Delta$, the inverse limit). Show that if $\rho: \langle \mathbf{D}_m, f_{mn} \rangle \rightarrow \mathbf{D}$ is a universal cone in $\mathcal{CPO}^E$ then $\langle \rho_m^R \rangle_{m \in \omega}: \mathbf{D} \rightarrow \langle \mathbf{D}_m, f_{mn}^R \rangle$ is a universal cone in $\mathcal{CPO}_\perp$.

18 Inverse Limits. Show that both $\mathcal{CPO}$ and $\mathcal{CPO}_\perp$ have inverse limits of inverse ω -chains.

19 Equalisers. If $f, g: \mathbf{D} \rightarrow \mathbf{E}$ are morphisms (in any category) an equaliser is a morphism $h: \mathbf{C} \rightarrow \mathbf{D}$ such that $f \circ h = g \circ h$ and (universal property) such that if $h': \mathbf{C}' \rightarrow \mathbf{D}$ is any morphism with that property then there is a unique $\theta: \mathbf{C}' \rightarrow \mathbf{C}$ such that $h' = h \circ \theta$. Show that $\mathcal{CPO}$, $\mathcal{CPO}_\perp$ but not $\mathcal{CPO}^E$ always have equalisers. Show that equalisers in $\mathcal{CPO}_\perp$ are order-monics. Are order-monics always equalisers? Show that embeddings are order-monics too, but the converse fails in general.

20 Direct Limits. Show that $\mathcal{CPO}$ does not have direct limits of direct ω -chains, but that $\mathcal{CPO}_\perp$ does. [Warning This is rather hard.]

21 Co-Equalisers. If $f, g: \mathbf{D} \rightarrow \mathbf{E}$ are morphisms (in any category) a co-equaliser is a morphism $h: \mathbf{E} \rightarrow \mathbf{F}$ such that $h \circ f = h \circ g$ (universal property) such that if $h': \mathbf{E} \rightarrow \mathbf{F}'$ is any other morphism with that property then there is a unique $\theta: \mathbf{F} \rightarrow \mathbf{F}'$ such that $h' = \theta \circ h$. Which of $\mathcal{CPO}$, $\mathcal{CPO}_\perp$, $\mathcal{CPO}^E$ always have co-equalisers? [Warning This is rather hard. The same difficulty as in Exercise 5 arises.]

22. We can generalise from sums of cpos over sets $\sum_{i \in S} \mathbf{D}_i$ to sums over cpos. Let $\mathbf{D}$ be a cpo, which can as remarked in this chapter be considered as a category. A functor $F: \mathbf{D} \rightarrow \mathcal{CPO}_\perp$ is continuous iff whenever $\langle d_n \rangle_{n \in \omega}$ is an increasing ω -chain in $\mathbf{D}$ with lub d then $\rho: \Delta \rightarrow F(d)$ is universal where $\Delta = \langle F(d_n), F(d_n \rightarrow d_{n+1}) \rangle$ and $\rho_n = F(d_n \rightarrow d)$. [Everything works just as well if we use functors $F: \mathbf{D} \rightarrow \mathcal{CPO}^E$ instead.]

Now if $\mathbf{D}$ is a cpo and $F: \mathbf{D} \rightarrow \mathcal{CPO}_\perp$ is an ω -continuous functor, then $\sum_{\mathbf{D}} F$ is defined as the set $\{ \langle d, e \rangle \mid e \in F(d) \}$ ordered by:

$$\langle d, e \rangle \sqsubseteq \langle d', e' \rangle \quad \text{iff} \quad d \sqsubseteq d' \quad \text{and} \quad F(d \rightarrow d')(e) \sqsubseteq_{F(d')} e'.$$

Show that $\sum_{\mathbf{D}} F$ is a cpo. Show that by suitable choice of $\mathbf{D}$ and F , the sum $\sum_{\mathbf{D}} F$ can be any one of: $\mathbf{A}_\perp$, $\mathbf{A}_\perp + \mathbf{B}_\perp$, $\mathbf{A} \times \mathbf{B}$, $\sum_{i \in S} (\mathbf{A}_i)_\perp$.

23. We can generalise from products of cpos over sets to products over cpos too. Let $\mathbf{D}$ be a cpo and let $F: \mathbf{D} \rightarrow \mathcal{CPO}_\perp$ be a continuous functor as in Exercise 22. A function $f: \mathbf{D} \rightarrow \bigcup_{d \in \mathbf{D}} F(d)$ is *well-typed* iff $\forall d \in \mathbf{D}. f(d) \in F(d)$; further it is *monotonic* iff $\forall d, d' \in \mathbf{D}. d \sqsubseteq d' \rightarrow (F(d \rightarrow d')(f(d))) \sqsubseteq f(d')$; further it is *continuous* iff whenever $\langle d_n \rangle_{n \in \omega}$ is an increasing ω -chain with lub d then $f(d) = \bigsqcup_n F(d_n \rightarrow d)(f(d_n))$.

We define $\prod_{\mathbf{D}} F$ to be the set $\{ f: \mathbf{D} \rightarrow \bigcup_{d \in \mathbf{D}} F(d) \mid f \text{ is well-typed, monotonic and continuous} \}$ with the pointwise ordering:

$$f \sqsubseteq g \quad \text{iff} \quad \forall d \in \mathbf{D}. f(d) \sqsubseteq_{F(d)} g(d).$$

Show that $\prod_{\mathbf{D}} F$ is a cpo. Show that by a suitable choice of $\mathbf{D}$ and F the product $\prod_{\mathbf{D}} F$ can be any one of: $\mathbf{A} \times \mathbf{B}$, $\mathbf{A} \rightarrow \mathbf{B}$, $\prod_{i \in S} \mathbf{A}_i$.

24. Let $\mathbf{D}$ be a cpo and $F: \mathbf{D} \rightarrow \mathcal{CPO}_\perp$ be a continuous functor as used in Exercises 22 and 23. Using an informal notation which we will leave informal here, define the natural function, $in: \prod_{\mathbf{D}} \lambda d: \mathbf{D}. (F(d) \rightarrow_\perp \sum_{\mathbf{D}} F)$ by:

$$in(d) = \lambda e: F(d). \langle d, e \rangle.$$

Show in is well-defined. Show that if $\mathbf{E}$ is any cpo and $f: \prod_{\mathbf{D}} \lambda d: \mathbf{D}. (F(d) \rightarrow_\perp \mathbf{E})$ then there is a unique continuous $[f]: \sum_{\mathbf{D}} F \rightarrow_\perp \mathbf{E}$ such that for all d in $\mathbf{D}$ the following diagram commutes:

$$\begin{array}{ccc}
 F(d) & \xrightarrow{in(d)} & \sum_{\mathbf{D}} F \\
 & \searrow f(d) & \downarrow [f] \\
 & & \mathbf{E}
 \end{array}$$

Show that $[\cdot]: [\prod_{\mathbf{D}} \lambda d: \mathbf{D}. (F(d) \rightarrow_{\perp} \mathbf{E})] \cong (\sum_{\mathbf{D}} F \rightarrow_{\perp} \mathbf{E})$. Give internal and external axiomatisations for $\sum_{\mathbf{D}} F$ (perhaps without worrying too much about the details of the metalanguage). Investigate $\prod_{\mathbf{D}} F$ along similar lines.

5. Solving Recursive Domain Equations

With the aid of our grand analogy we can now solve recursive domain equations by following the idea of the fixed-point theorem. Let $F: \mathcal{CPO}^E \rightarrow \mathcal{CPO}^E$ be a continuous functor, which is analogous to a continuous function, $f: \mathbf{D} \rightarrow \mathbf{D}$. One fixed-point of f is the lub of the increasing ω -chain $\langle f^m(\perp) \rangle_{m \in \omega}$. As $\mathbf{U}$ is the analogue of $\perp$, we should find a direct ω -chain $\Delta = \langle F^m(\mathbf{U}), f_{mn} \rangle$ and as direct limits are the analogue of lubs of ω -chains we should then consider a universal cone, $\rho: \Delta \rightarrow (\lim_{\rightarrow} \Delta)$. It remains to specify the f_m ($= f_{m(m+1)}$). Now f_0 must be $\perp: \mathbf{U} \rightarrow F(\mathbf{U})$ and it seems natural to take $f_m = T^m(\perp)$. With this choice we can calculate:

$$\begin{aligned} F(\lim_{\rightarrow} \Delta) &\cong \lim_{\rightarrow} (F(F^m(\mathbf{U})), F(F^m(\perp))) && F \text{ is continuous} \\ &= \lim_{\rightarrow} \Delta && \text{dropping a term does not affect direct limits — see Exercise 4.13.} \end{aligned}$$

and we have indeed the equation $\mathbf{D} \cong F(\mathbf{D})$. Note that the f_m are determined by the requirement that the above proof works. A little analysis shows what the isomorphism actually is. As $\rho: \Delta \rightarrow \lim_{\rightarrow} \Delta$ is universal we have $F\rho: F\Delta \rightarrow F(\lim_{\rightarrow} \Delta)$ is universal as F is continuous and on the other hand $\rho^-: \Delta^- \rightarrow \lim_{\rightarrow} \Delta$ is universal too where $\Delta^- = \langle F^{m+1}(\mathbf{U}), F^{m+1}(\perp) \rangle$ and $\rho^- = \langle \rho_{m+1} \rangle_{m \in \omega}$ by the considerations of Exercise 4.13. But $F\Delta = \Delta^-$ and so there is an isomorphism $\theta: F\rho \cong \rho^-$, which by the considerations of Theorem 4 is $\bigsqcup_m \rho_{m+1} \circ (F\rho_m)^R$, and θ^{-1} is $\bigsqcup_m (F\rho_m) \circ \rho_{m+1}^R$. We usually write $\theta: F(\lim_{\rightarrow} \Delta) \cong \lim_{\rightarrow} \Delta$ as η_F .

It is also quite easy to solve simultaneous equations. Suppose for example we have two continuous functors, $F_i: (\mathcal{CPO}^E)^2 \rightarrow \mathcal{CPO}^E$ ($i = 0, 1$). Define $\Gamma = \langle \mathbf{C}_n, f_n \rangle$, $\Delta = \langle \mathbf{D}_n, g_n \rangle$ by:

$$\begin{aligned} \mathbf{C}_0 &= \mathbf{D}_0 = \mathbf{U}; \\ \mathbf{C}_{n+1} &= F_0(\mathbf{C}_n, \mathbf{D}_n); \quad \mathbf{D}_{n+1} = F_1(\mathbf{C}_n, \mathbf{D}_n); \\ f_0 &= \perp; \quad g_0 = \perp; \\ f_{n+1} &= F_0(f_n, g_n); \quad g_{n+1} = F_1(f_n, g_n). \end{aligned}$$

Now we can calculate:

$$F_0(\lim_{\rightarrow} \Gamma, \lim_{\rightarrow} \Delta) \cong \lim_{\rightarrow} F_0(\Gamma, \Delta) = \lim_{\rightarrow} \Gamma^- = \lim_{\rightarrow} \Gamma$$

and similarly

$$F_1(\lim_{\rightarrow} \Gamma, \lim_{\rightarrow} \Delta) \cong \lim_{\rightarrow} \Delta.$$

All in all we have essentially proved that:

Theorem 1. *Let $F_i: (\mathcal{CPO}^E)^n \rightarrow \mathcal{CPO}$ ($i < n$) be n continuous functors then there are cpos $\mathbf{D}_0, \dots, \mathbf{D}_{n-1}$ such that:*

$$\mathbf{D}_i \cong F_i(\mathbf{D}_0, \dots, \mathbf{D}_{n-1}) \quad (i < n).$$

Proof. A slight generalisation of the above consideration and left to the reader. ■

In view of our analogy it is now natural to ask if our solutions to recursive domain equations are initial (analogue of least) in some sense. However we postpone this discussion for a while as we can now solve many problems in denotational semantics by using the solution. We now present a series of typical examples of the use of recursive domain equations.

Example 1 The Integers. We have already seen that $\mathbf{N}$ satisfies the equation $\mathbf{N} \cong \mathbf{0} + \mathbf{N}$ and so $\mathbf{N} \cong T(\mathbf{N})$ where $T = +^E(K_{\mathbf{0}}^E, Id^E)$. Then we calculate $T(\mathbf{U}), T^2(\mathbf{U}), \dots, T^n(\mathbf{U}), \dots$ are:

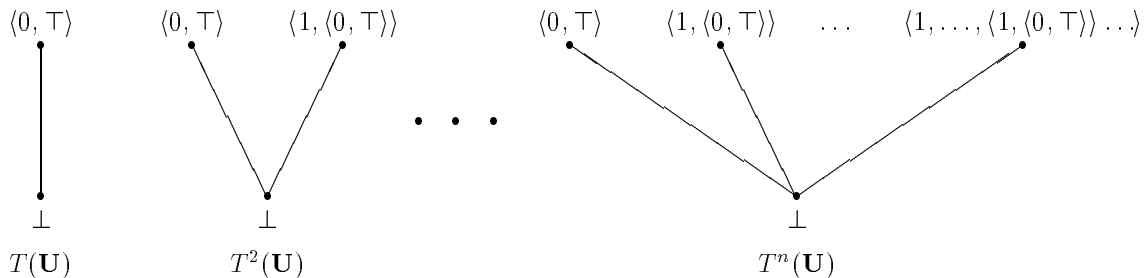


Figure XIV.

So $\mathbf{U} \subseteq T(\mathbf{U}) \subseteq T^2(\mathbf{U}) \subseteq \dots \subseteq T^n(\mathbf{U}) \subseteq \dots$. Further the embeddings $T^m(\perp)$ are the inclusions as $\perp$ is and T preserves inclusions. Since the $T^m(\mathbf{U})$ are all discrete it follows that $\mathbf{D} = \lim_{\rightarrow} \langle T^m(\mathbf{U}), T^m(\perp) \rangle$ is $\bigcup_m T^m(\mathbf{U})$ with the evident (flat) ordering (following the considerations of Chapter 4). Thus the solution $\mathbf{D}$ is isomorphic to $\mathbf{N}$.

Example 2 Procedures in *Imp*. We consider the extension of *Imp* discussed in Chapter 4. By Theorem 1 there are solutions to the simultaneous recursive domain equations say:

$$\begin{aligned}\alpha: \mathbf{C} &\cong (\mathbf{S} \times \mathbf{PEnv}) \rightarrow (\mathbf{S} \times \mathbf{PEnv}), \\ \beta: \mathbf{PEnv} &\cong (\mathbf{Proc}_{\perp} \rightarrow_{\perp} \mathbf{C}).\end{aligned}$$

The semantic functions $\mathcal{B}: \mathbf{BExp} \rightarrow (\mathbf{S} \rightarrow \mathbf{T})$, $\mathcal{A}: \mathbf{Act} \rightarrow (\mathbf{S} \rightarrow \mathbf{S})$ are assumed given, as before, and $\mathcal{C}: \mathbf{Stat} \rightarrow \mathbf{C}$ is defined by structural induction:

- (i) $\mathcal{C}[a] = \alpha^{-1}(\lambda\sigma: \mathbf{S}, \pi: \mathbf{PEnv}. \langle \mathcal{A}[a](\sigma), \pi \rangle);$
- (ii) $\mathcal{C}[\text{dummy}] = \alpha^{-1}(\lambda\sigma: \mathbf{S}, \pi: \mathbf{PEnv}. \langle \sigma, \pi \rangle);$
- (iii) $\mathcal{C}[s_0; s_i] = \alpha^{-1}(\alpha(\mathcal{C}[s_1]) \circ \alpha(\mathcal{C}[s_0]));$
- (iv) $\mathcal{C}[\text{if } b \text{ then } e_0 \text{ else } e_1] = \alpha^{-1}(\lambda\sigma: \mathbf{S}, \pi: \mathbf{PEnv}. \mathcal{B}[b](\sigma) \rightarrow \alpha(\mathcal{C}[e_0])(\sigma, \pi) \mid \alpha(\mathcal{C}[e_1])(\sigma, \pi));$
- (v) $\mathcal{C}[\text{while } b \text{ do } s] = \mu\gamma: \mathbf{C}. \alpha^{-1}(\lambda\sigma: \mathbf{S}, \pi: \mathbf{PEnv}. \mathcal{B}[b](\sigma) \rightarrow \alpha(\gamma)(\alpha(\mathcal{C}[s])(\sigma, \pi)) \mid \langle \sigma, \pi \rangle);$
- (vi) $\mathcal{C}[\text{proc } p \text{ begin } s \text{ end}] = \alpha^{-1}(\lambda\sigma: \mathbf{S}, \pi: \mathbf{PEnv}. \langle \sigma, \pi[\mathcal{C}[s]/p] \rangle);$
- (vii) $\mathcal{C}[\text{call } p] = \alpha^{-1}(\lambda\sigma: \mathbf{S}, \pi: \mathbf{PEnv}. \alpha(\beta(\pi)[p])(\sigma, \pi)).$

Here $[\cdot/\cdot]: \mathbf{PEnv} \times \mathbf{C} \times \mathbf{Proc} \rightarrow \mathbf{PEnv}$ is defined by:

$$\pi[\gamma/p] = \beta^{-1}(\lambda p': \mathbf{Proc}. p' = p \rightarrow \gamma \mid \beta(\pi)[p']).$$

It is annoying to continually write all the isomorphisms such as α , α^{-1} , β , β^{-1} above and we usually omit them.

Example 3 An Applied λ -calculus.

Syntax.

1. **Id** — a set of *identifiers*, ranged over by x .
2. **Exp** — the set of *expressions* ranged over by e and given in terms of **Id** and **Exp** by the syntax:

$$e ::= x \mid \mathbf{0} \mid \text{true} \mid \text{false} \mid \text{succ} \mid \text{pred} \mid \text{zero} \mid \delta \mid (\text{if } e \text{ then } e \text{ else } e) \mid e(e) \mid \lambda x. e.$$

Semantic Domains. Expressions will have values in a domain of values, $\mathbf{V}$, which should contain $\mathbf{T}$ and $\mathbf{N}$ and since explicit self-application is allowed its function space $(\mathbf{V} \rightarrow \mathbf{V})$ is also included and we define $\mathbf{V}$ recursively by:

$$\mathbf{V} \cong \mathbf{T} + \mathbf{N} + (\mathbf{V} \rightarrow \mathbf{V})_{\perp}.$$

The use of the semi-separated sum is to differentiate the least function from $\perp$; the idea is that δ which is to discriminate between functions and basic values $(\mathbf{T} + \mathbf{N})$ should be, in some sense, sequential.

Denotations. The denotation function is $\mathcal{E}: \mathbf{Exp} \rightarrow (\mathbf{Env} \rightarrow \mathbf{V})$ where $\mathbf{Env} = \mathbf{Id} \rightarrow_{\perp} \mathbf{V}$. It is defined by structural induction on expressions:

- (i) $\mathcal{E}[x] = \lambda\rho: \mathbf{Env}. \rho[x];$
- (ii) $\mathcal{E}[\mathbf{0}] = \lambda\rho: \mathbf{Env}. in_1(0);$
- (iii) $\mathcal{E}[\text{true}] = \lambda\rho: \mathbf{Env}. in_0(tt);$
- (iv) $\mathcal{E}[\text{false}] = \lambda\rho: \mathbf{Env}. in_0(ff);$
- (v) $\mathcal{E}[\text{succ}] = \lambda\rho: \mathbf{Env}. in_2 \circ up(\lambda v: \mathbf{V}. in_1((v \mid \mathbf{N}) + 1));$
- (vi) $\mathcal{E}[\text{pred}] = \lambda\rho: \mathbf{Env}. in_2 \circ up(\lambda v: \mathbf{V}. in_1((v \mid \mathbf{N}) - 1));$
- (vii) $\mathcal{E}[\text{zero}] = \lambda\rho: \mathbf{Env}. in_2 \circ up(\lambda v: \mathbf{V}. in_0 \circ Z(v \mid \mathbf{N}));$
- (viii) $\mathcal{E}[\delta] = \lambda\rho: \mathbf{Env}. in_2 \circ up(\lambda v: \mathbf{V}. in_0 \circ is_2(v));$
- (ix) $\mathcal{E}[\text{if } e_0 \text{ then } e_1 \text{ else } e_2] = \lambda\rho: \mathbf{Env}. ((\mathcal{E}[e_0](\rho) \mid \mathbf{T}) \rightarrow \mathcal{E}[e_1](\rho) \mid \mathcal{E}[e_2](\rho));$
- (x) $\mathcal{E}[e_0(e_1)] = \lambda\rho: \mathbf{Env}. down(out_2(\mathcal{E}[e_0](\rho)))(\mathcal{E}[e_1](\rho));$
- (xi) $\mathcal{E}[\lambda x. e] = \lambda\rho: \mathbf{Env}. in_2(up(\lambda v: \mathbf{V}. \mathcal{E}[e](\rho[v/x])));$

where $[\cdot/\cdot]: \mathbf{Env} \times \mathbf{V} \times \mathbf{Id} \rightarrow \mathbf{Env}$ is defined much as usual.

Example 4 Input/Output.

Syntax. We extend *Imp* with a command for inputting from an input channel and outputting to an output channel (we assume one of each). So the syntactic sets are **BExp**, **Act**, as before and **Stat** is given by:

$$s ::= a \mid \mathbf{dummy} \mid \mathbf{read} \mid \mathbf{write} \mid (s; s) \mid (\mathbf{if} \ b \ \mathbf{then} \ s \ \mathbf{else} \ s) \mid (\mathbf{while} \ b \ \mathbf{do} \ s).$$

Semantic Domains. As well as a flat cpo of states, **S**, there will be an input cpo **I** and one output cpo **O** (e.g. **I** = **N** and **O** = **T**). There are also two basic functions: $read: \mathbf{I} \otimes \mathbf{S} \rightarrow_{\perp} \mathbf{S}$, $write: \mathbf{S} \rightarrow_{\perp} \mathbf{O}$ (for example read might put the input in a standard location and write might take the output from some standard location). Commands are now elements of a cpo $\mathbf{C} = \mathbf{S} \rightarrow_{\perp} \mathbf{R}$ where **R** is a domain of results (or resumptions). The cpo **R** contains **S**, for final states, $\mathbf{I} \rightarrow_{\perp} \mathbf{R}$, for reading and $\mathbf{O} \otimes \mathbf{R}_{\perp}$ for outputting. So it is recursively defined by:

$$\mathbf{R} \cong \mathbf{S} + (\mathbf{I} \rightarrow_{\perp} \mathbf{R})_{\perp} + (\mathbf{O} \otimes \mathbf{R}_{\perp}).$$

Note that in this case there is not implicit or explicit self-application as can be seen from the fact that on the right hand side **R** never occurs in the contravariant position of an exponentiation functor.

Denotations. We have a given $\mathcal{B}: \mathbf{BExp} \rightarrow (\mathbf{S} \rightarrow \mathbf{T})$ and a given $\mathcal{A}: \mathbf{Act} \rightarrow (\mathbf{S} \rightarrow \mathbf{S})$ and $\mathcal{C}: \mathbf{Stat} \rightarrow \mathbf{C}$ is defined by structural induction on statements. We only give a few of the equations here.

- (iii) $\mathcal{C}[\mathbf{read}] = \lambda_{\perp} \sigma: \mathbf{S}. in_1(up(\lambda_{\perp} i: \mathbf{I}. in_0(read(i \otimes \sigma))));$
- (iv) $\mathcal{C}[\mathbf{write}] = \lambda_{\perp} \sigma: \mathbf{S}. in_2(write(\sigma) \otimes up(in_0(\sigma)));\mathcal{C}[s_0; s_1] = \lambda_{\perp} \sigma: \mathbf{S}. \mathcal{C}[s_0](\sigma) * \mathcal{C}[s_1].$
- (v)

Here $*$: $\mathbf{R} \times \mathbf{C} \rightarrow \mathbf{R}$ is defined recursively by:

$$\begin{aligned} r * \gamma &= (is_0(r) \rightarrow \gamma(out_0(r)) \\ &\mid (is_1(r) \rightarrow in_1(up(\lambda_{\perp} i: \mathbf{I}. (down(out_1(r)))(i) * \gamma)) \\ &\mid (is_2(r) \rightarrow in_2((out_2(r) \downarrow 0) \otimes up(down(out_2(r) \downarrow 1) * \gamma)) \\ &\mid \perp)). \end{aligned}$$

Example 5 Dynamic Binding. This is a version of *Dec* in which, as well as the usual static binding mechanism in which, in an expression of the form (**let** x **be** e_0 **in** e_1), the variable x is assigned the value of e_0 calculated in the “definition-time” environment, we also include a more “dynamic” version in which the calculation of e_0 is postponed until the value of x is required for the evaluation of e_1 and then the “use-time” environment is used.

Syntax.

1. **Id** — a class of identifiers ranged over by x .
2. **Exp** — a class of expressions ranged over by e and given by:

$$e ::= x \mid 0 \mid 1 \mid (e + e) \mid (\mathbf{let} \ x \ \mathbf{bestat} \ e \ \mathbf{in} \ e) \mid (\mathbf{let} \ x \ \mathbf{bedyn} \ e \ \mathbf{in} \ e).$$

For example **let** x **bestat** **2** **in** (**let** y **bedyn** $x + 1$ **in** (**let** x **bedyn** **10** **in** $x + y$)) where **2** abbreviates **1 + 1** and so on, will have value 21 whereas if static assignment had been used throughout the expression would have had value 13.

Semantic Domains. The cpo of environments, **Env**, rather than being $\mathbf{Id} \rightarrow_{\perp} \mathbf{N}$ must give an identifier a function of **Env** itself in order that the value of the identifier may depend on the call-time environment. So we define **Env** recursively by:

$$\mathbf{Env} \cong \mathbf{Id}_{\perp} \rightarrow_{\perp} (\mathbf{Env} \rightarrow \mathbf{N}).$$

Denotations. The denotation function is $\mathcal{E}: \mathbf{Exp} \rightarrow (\mathbf{Env} \rightarrow \mathbf{N})$ and it is defined by structural induction on expressions; we only give a few of the equations.

- (i) $\mathcal{E}[x] = \lambda \rho: \mathbf{Env}. \rho[x](\rho);$
- (v) $\mathcal{E}[\mathbf{let} \ x \ \mathbf{bestat} \ e_0 \ \mathbf{in} \ e_1] = \lambda \rho: \mathbf{Env}. \mathcal{E}[e_1](\rho[\lambda \rho': \mathbf{Env}. \mathcal{E}[e_0](\rho)/x]);$
- (vi) $\mathcal{E}[\mathbf{let} \ x \ \mathbf{bedyn} \ e_0 \ \mathbf{in} \ e_1] = \lambda \rho: \mathbf{Env}. \mathcal{E}[e_1](\rho[\lambda \rho': \mathbf{Env}. \mathcal{E}[e_0](\rho')/x]).$

Here $\cdot[\cdot/\cdot]: \mathbf{Env} \times (\mathbf{Env} \rightarrow \mathbf{N}) \times \mathbf{Id} \rightarrow \mathbf{Id}$ is defined as usual.

Example 6 Syntax Considered as a CPO. As a matter of fact we could have considered syntax as a cpo much earlier. For example, in the case of *Imp*, we would consider the flat cpo $\mathbf{Stat}_\perp$. However we would have had to define various functions over $\mathbf{Stat}_\perp$ and it turns out that if we define $\mathbf{Stat}_\perp$ by a recursive domain equation we get them for free, so to speak. Further as the recursive domain equation follows the grammar closely, we have a reasonably systematic method. (The caveat is inserted as it is not so clear how to accommodate non-context-free grammars, although we will always expect to have some sort of phrase structure.)

Syntax. There are three *cpo*s of syntactic items.

1. **BExp** — a given flat cpo of *boolean expressions* ranged over by b .
2. **Act** — a given flat cpo of *primitive actions* ranged over by a .
3. **Stat** — a cpo of statements specified by the recursive domain equation:

$$\mathbf{Stat} \cong \mathbf{Act} + \mathbf{0} + \mathbf{Stat} \otimes \mathbf{Stat} + \mathbf{BExp} \otimes \mathbf{Stat} \otimes \mathbf{Stat}.$$

Note how the definition follows the syntactical specification in Chapter 1. Each alternative is modelled by a (strict) sum and the various components of a clause are put together by the strict product. This gives a flat cpo, which is a kind of abstract syntax in that there is no commitment as to how particular statements are to be written.

It is particularly tedious when using such syntactical domains to continually make type conversions using *in*, *out*, $\langle \cdot, \cdot \rangle$, $(\cdot \downarrow i)$, *up*, *down*, etc. We therefore define some helpful extra functions:

$$\begin{aligned} isact &= is_0; \\ actof &= out_0; \\ mkact &= in_0; \\ isdum &= is_1; \\ dumof &= out_1; \\ mkdum &= in_1; \\ isseq &= is_2; \\ seqof_0 &= \pi_0 \circ out_2; \\ seqof_1 &= \pi_1 \circ out_2; \\ mkseq &= in_2 \circ \otimes; \\ iscond &= is_3; \\ condof_0 &= \pi_0 \circ out_3; \\ condof_1 &= \pi_1 \circ out_3; \\ condof_2 &= \pi_2 \circ out_3; \\ mkcond &= in_3 \circ \otimes. \end{aligned}$$

Semantic Domains. There are the flat cpos, $\mathbf{T}$, of truth-values and $\mathbf{N}$, of integers and also the cpo $\mathbf{C} = (\mathbf{S} \rightarrow_\perp \mathbf{S})$ of commands.

Denotations. The functions $\mathcal{B}: \mathbf{BExp} \rightarrow_\perp (\mathbf{S} \rightarrow_\perp \mathbf{T})$ and $\mathcal{A}: \mathbf{Act} \rightarrow_\perp \mathbf{C}$ are assumed given and are now elements of cpos. The functions $\mathcal{C}: \mathbf{Stat} \rightarrow_\perp \mathbf{C}$ is defined by *recursion* as:

$$\begin{aligned} \mathcal{C} = \lambda_\perp s: \mathbf{Stat}. & (isact(s) \rightarrow \mathcal{A}[\![actof(s)]\!] \mid \\ & (isdum(s) \rightarrow (\lambda_\perp \sigma: \mathbf{S}. \sigma) \mid \\ & (isseq(s) \rightarrow \mathcal{C}[\![seqof_1(s)]\!] \circ \mathcal{C}[\![seqof_0(s)]\!] \mid \\ & (iscond(s) \rightarrow \lambda_\perp \sigma: \mathbf{S}. (\mathcal{B}[\![condof_0(s)]\!](\sigma) \rightarrow \mathcal{C}[\![condof_1(s)]\!](\sigma) \mid \mathcal{C}[\![condof_2(s)]\!](\sigma)) \mid \perp))) \end{aligned}$$

Once we have the idea of specifying syntax in this way we can explore other kinds of equations for **Stat**. For example we might want to allow partially specified programs such as $(\perp; (a; \perp))$ (where $s; s'$ is $mkseq(s, s')$ and so on) other than $\perp$. This in turn will lead to infinite programs as the limit of increasing chains of the partial ones. There is a standard choice available: replace every occurrence of a recursively

specified domain, $\mathbf{D}$, on the right hand side by $(\mathbf{D})_\perp$ to allow non-strict combination with $\perp$. So we would now specify **Stat** by:

$$\mathbf{Stat} \cong \mathbf{Act} + \mathbf{0} + \mathbf{Stat}_\perp \otimes \mathbf{Stat}_\perp + \mathbf{BExp} \otimes \mathbf{Stat}_\perp \otimes \mathbf{Stat}_\perp$$

which is the same as (see Exercise 3.18):

$$\mathbf{Stat} \cong \mathbf{Act} + \mathbf{0} + (\mathbf{Stat} \times \mathbf{Stat})_\perp + \mathbf{BExp} \otimes (\mathbf{Stat} \times \mathbf{Stat})_\perp.$$

It would then be necessary to make a few changes in the definitions of the auxiliary functions. For example we would put:

$$\begin{aligned} isseq &= is_2; \\ seqof_0 &= down \circ \pi_0 \circ out_2; \\ seqof_1 &= down \circ \pi_1 \circ out_2; \\ mkseq &= in_2 \circ \otimes \circ \langle up, up \rangle. \end{aligned}$$

But then with these changes made the denotational function would have exactly the same definition as before. Note that we now have some identifications such as: $\perp_{\mathbf{Stat}}; \perp_{\mathbf{Stat}} = \perp_{\mathbf{Stat}}$ and $(\mathbf{if} \perp_{\mathbf{BExp}} \mathbf{then} e_0 \mathbf{else} e_1) = \perp_{\mathbf{Stat}}$, but not others such as $(\perp_{\mathbf{Stat}}; s) = \perp_{\mathbf{Stat}}$ which we might want. All this is a matter of the choice of domain equation.

We now see how our solution is initial and concentrate on a fixed functor $F: \mathcal{CP}\mathcal{O}^E \rightarrow \mathcal{CP}\mathcal{O}^E$ of one argument. Recall that the solution we constructed was obtained from a universal cone $\rho: \Delta \rightarrow \lim_{\rightarrow} \Delta$ where $\Delta = \langle F^m(\mathbf{U}), F^m(\perp) \rangle$. Let us choose a fixed ρ for F and call $\lim_{\rightarrow} \Delta$, $\mathbf{Fix}_F$. The isomorphism $\eta_F: F(\mathbf{Fix}_F) \rightarrow \mathbf{Fix}_F$ is $\eta_F = \bigsqcup_m \rho_{m+1} \circ F(\rho_m)^R$ and its inverse is $\bigsqcup_m F(\rho_m) \circ \rho_{m+1}^R$, as we saw above. Our solution of the domain equation, $\mathbf{D} \cong F(\mathbf{D})$, clearly consists of the pair $\langle \mathbf{Fix}_F, \eta_F \rangle$.

Definition 1. A *fixed-point* of F is a pair $\langle \mathbf{D}, \alpha \rangle$ where $\alpha: F\mathbf{D} \cong \mathbf{D}$. A *prefixed-point* of F is a pair $\langle \mathbf{D}, \alpha \rangle$ where $\alpha: F\mathbf{D} \triangleleft \mathbf{D}$.

Sometimes prefixed-points are called *F-algebras* for reasons to be seen below.

Definition 2. Let $\langle \mathbf{D}, \alpha \rangle$ and $\langle \mathbf{D}', \alpha' \rangle$ be prefixed-points of F . A morphism $h: \langle \mathbf{D}, \alpha \rangle \rightarrow \langle \mathbf{D}', \alpha' \rangle$ (sometimes called an *F-homomorphism*) is an embedding $h: \mathbf{D} \triangleleft \mathbf{D}'$ such that the following diagram commutes:

$$\begin{array}{ccc} F\mathbf{D} & \xrightarrow{\alpha} & \mathbf{D} \\ Fh \downarrow & & \downarrow h \\ F\mathbf{D}' & \xrightarrow{\alpha'} & \mathbf{D}' \end{array}$$

We already know that $\langle \mathbf{Fix}_F, \eta_F \rangle$ is a fixed-point of F ; we now find the analogue of the fixed-point theorem:

Theorem 2 The Initial Fixed-point Theorem. *The fixed-point $\langle \mathbf{Fix}_F, \eta_F \rangle$ is the initial prefixed-point of F .*

Proof. Suppose $\alpha: F\mathbf{D} \triangleleft \mathbf{D}$ is a prefixed-point of F . Put $\mathbf{F}_m = F^m(\mathbf{U})$, $f_m = F^m(\perp)$, $\Delta = \langle \mathbf{F}_m, f_m \rangle$ and then as $\rho: \Delta \rightarrow \mathbf{Fix}_F$ is the universal cone, each triangle:

$$\begin{array}{ccc} F(\mathbf{F}_m) & & \\ \downarrow F(\rho_m) & \searrow \rho_{m+1} & \\ F(\mathbf{Fix}_F) & \xrightarrow{\eta_F} & \mathbf{Fix}_F \end{array}$$

commutes.

For uniqueness, suppose $h: \mathbf{Fix}_F \triangleleft \mathbf{D}$ is a morphism of F -algebras so that this square commutes:

$$\begin{array}{ccc} F(\mathbf{Fix}_F) & \xrightarrow{\eta_F} & \mathbf{Fix}_F \\ Fh \downarrow & & \downarrow h \\ F(\mathbf{D}) & \xrightarrow{\alpha} & \mathbf{D} \end{array}$$

Now $h \circ \rho_0 = \perp$ and $h \circ \rho_{m+1} = h \circ \eta_F \circ F(\rho_m) = \alpha \circ F(h) \circ F(\rho_m) = \alpha \circ F(h \circ \rho_m)$. So each $h \circ \rho_m$ is determined and so too therefore is h as $h = h \circ id_{\mathbf{Fix}_F} = \bigsqcup_m (h \circ \rho_m) \circ \rho_m^R$.

For existence define $\rho'_m: \mathbf{F}_m \triangleleft \mathbf{D}$ by: $\rho'_0 = \perp$, $\rho'_{m+1} = \alpha \circ F(\rho'_m)$. Then $\rho' \stackrel{\text{def}}{=} \langle \rho'_m \rangle_{m \in \omega}$ is a cone from Δ to $\mathbf{D}$, i.e. each of these triangles commutes:

$$\begin{array}{ccc} \mathbf{F}_m & \xrightarrow{f_m} & \mathbf{F}_{m+1} \\ & \searrow \rho'_m & \downarrow \rho'_{m+1} \\ & & \mathbf{D} \end{array}$$

For $\rho'_1 \circ f_0 = \rho'_1 \circ \perp = \perp = \rho'_0$ and then, by induction, $\rho'_{m+2} \circ f_{m+1} = \alpha \circ F(\rho'_{m+1}) \circ F(f_m) = \alpha \circ F(\rho'_m) = \rho'_{m+1}$.

Hence there is a mediating $h: \mathbf{Fix}_F \triangleleft \mathbf{D}$ from ρ to ρ' so that all of these triangles commute:

$$\begin{array}{ccc} \mathbf{F}_m & \xrightarrow{\rho_m} & \mathbf{Fix}_F \\ & \searrow \rho'_m & \downarrow h \\ & & \mathbf{D} \end{array}$$

But this makes the above homomorphism square commute. For both $h \circ \eta_F$ and $\alpha \circ Fh$ mediate between the universal cone $F\rho$ and $(\rho')^-: F\Delta \rightarrow \mathbf{D}$ as can be seen from the following two diagrams:

$$\begin{array}{ccc} F(\mathbf{F}_m) & \xrightarrow{F\rho_m} & F(\mathbf{Fix}_F) \\ \rho'_{m+1} \downarrow & \searrow \rho_{m+1} & \downarrow \eta_F \\ \mathbf{D} & \xleftarrow{h} & \mathbf{Fix}_F \end{array} \qquad \begin{array}{ccc} F(\mathbf{F}_m) & \xrightarrow{F\rho_m} & F(\mathbf{Fix}_F) \\ \rho'_{m+1} \downarrow & \searrow F\rho'_m & \downarrow Fh \\ \mathbf{D} & \xleftarrow{\alpha} & F(\mathbf{D}) \end{array}$$

This concludes the proof. $\blacksquare$

What we would like next would be a proof rule, an axiomatisation, for recursively defined domains. However this is not available at present and constitutes an interesting open problem. There are techniques available for particular cones. Investigations of typeless λ -calculi have looked at the $F^m(\mathbf{U})$ in detail by labelling expressions and subexpressions by integers, for example $e^{(m)}$, and then investigating $\mathcal{E}[\![e^{(m)}]\!] \stackrel{\text{def}}{=} \rho_m^R(\mathcal{E}[\![e]\!])$. Other investigations have used recursively defined inductive relations to establish the particular properties in question. The difficulty arises from contravariance as we are able to give a satisfactory axiomatisation for covariant functors $F: \mathcal{CP}\mathcal{O}_\perp \rightarrow \mathcal{CP}\mathcal{O}_\perp$ where $F \stackrel{\text{def}}{=} T^E$. That is we have a satisfactory solution when domain equations do not contain any occurrences of the recursively specified cpos in contravariant positions (such as the first argument of an exponentiation). Our solution will be a general form of structural induction.

We begin by looking at T -algebra in $\mathcal{CPO}_\perp$, where the terminology will make good sense. We consider a fixed covariant locally continuous functor, $T: \mathcal{CPO}_\perp \rightarrow \mathcal{CPO}_\perp$.

Definition 3. A T -algebra is a pair $\langle \mathbf{D}, \alpha \rangle$ where $\alpha: T\mathbf{D} \rightarrow_\perp \mathbf{D}$ is a strict continuous function. A T -homomorphism, $h: \langle \mathbf{D}, \alpha \rangle \rightarrow \langle \mathbf{D}', \alpha' \rangle$ between T -algebras is a strict continuous function, $h: \mathbf{D} \rightarrow_\perp \mathbf{D}'$ such that the following diagram commutes:

$$\begin{array}{ccc} T\mathbf{D} & \xrightarrow{\alpha} & \mathbf{D} \\ Th \downarrow & & \downarrow h \\ T\mathbf{D}' & \xrightarrow{\alpha'} & \mathbf{D}' \end{array}$$

Example 1 The Integers. Consider the continuous algebras $A = \langle \mathbf{A}, 0, succ \rangle$ where $\mathbf{A}$ is a cpo, 0 is an element of $\mathbf{A}$, and $succ: \mathbf{A} \rightarrow_\perp \mathbf{A}$ is a strict continuous function. A morphism $h: \langle \mathbf{A}, 0, succ \rangle \rightarrow \langle \mathbf{A}', 0', succ' \rangle$ is a continuous function $h: \mathbf{A} \rightarrow_\perp \mathbf{A}'$ such that $h(0) = 0'$ and $h(succ(x)) = succ'(h(x))$, for all elements x of $\mathbf{A}$.

Now suppose for the purpose of this example that $T: \mathcal{CPO}_\perp \rightarrow \mathcal{CPO}_\perp$ is given by $T(\mathbf{D}) = \mathbf{0} + \mathbf{D}$ (more accurately, by $T = +(K_{\mathbf{0}}, Id)$). Any algebra $\langle \mathbf{A}, 0, succ \rangle$ induces the T -algebra $\alpha_{\mathbf{A}}: \mathbf{0} + \mathbf{A} \rightarrow \mathbf{A}$ where $\alpha_{\mathbf{A}} = [\lambda_\perp x.0, succ]$; conversely any T -algebra $\alpha: \mathbf{0} + \mathbf{D} \rightarrow \mathbf{D}$ induces the algebra $\mathbf{A}_\alpha = \langle \mathbf{D}, \alpha \circ inl(\top), \alpha \circ inr \rangle$. The correspondences are functorial as if $h: \mathbf{A} \rightarrow_\perp \mathbf{A}'$ is a homomorphism then $h: \langle \mathbf{A}, \alpha_{\mathbf{A}} \rangle \rightarrow \langle \mathbf{A}', \alpha_{\mathbf{A}'} \rangle$ is a T -homomorphism and conversely. Then we see that the functors are mutual inverses — that is the categories are isomorphic.

Note that we have already shown how the integers form a T -algebra $\alpha: \mathbf{0} + \mathbf{N} \rightarrow \mathbf{N}$ where $\alpha = [\lambda_\perp x.0, strict((+1))]$, and indeed α is an isomorphism. One easily proves that $\langle \mathbf{N}, \alpha \rangle$ is the initial T -algebra by induction. For suppose $\langle \mathbf{D}, \alpha' \rangle$ is another T -algebra and $h: \langle \mathbf{N}, \alpha \rangle \rightarrow_\perp \langle \mathbf{D}, \alpha' \rangle$ is a T -homomorphism. Then $h(\perp) = \perp$, $h(0) = h \circ \alpha(inl(\top)) = \alpha' \circ (id_{\mathbf{0}} + h)(inl(\top)) = \alpha'(inl(\top))$ and $h(n+1) = h \circ \alpha(inr(n)) = \alpha' \circ (id_{\mathbf{0}} + h)(inr(n)) = \alpha'(h(n))$. So it follows by ordinary induction that h is determined. One easily shows the converse: that if h is defined by the above equation (namely $h(0) = h \circ \alpha(inl(\top))$ and $h(n+1) = \alpha'(h(n))$) then h is a T -homomorphism. We saw above that $\mathbf{Fix}_{T^E}$ was isomorphic to the integers. We now see that (among many other things), $\langle \mathbf{Fix}_{T^E}, \eta_{T^E} \rangle$ is the initial T -algebra and so we see that $\langle \mathbf{N}, \alpha \rangle$ and $\langle \mathbf{Fix}_{T^E}, \eta_{T^E} \rangle$ are isomorphic as *algebras*.

Theorem 3 The Initial Algebra Theorem. The T^E -algebra, $\langle \mathbf{Fix}_{T^E}, \eta_{T^E} \rangle$ is the initial T -algebra.

Proof. Define $\mathbf{F}_m, f_m, \Delta, \rho$ as in the proof of Theorem 2. Let $\langle \mathbf{D}, \alpha \rangle$ be a T -algebra. Uniqueness of a possible T -homomorphism, $h: \langle \mathbf{Fix}_{T^E}, \eta_{T^E} \rangle \rightarrow_\perp \langle \mathbf{D}, \alpha \rangle$ is proved just as in Theorem 2.

For existence define $\rho'_m: \mathbf{F}_m \rightarrow_\perp \mathbf{D}$ by $\rho'_0 = \perp$, $\rho'_{m+1} = \alpha \circ T(\rho'_m)$ then, as in the proof of Theorem 2, $\rho' \stackrel{\text{def}}{=} \langle \rho'_m \rangle_{m \in \omega}$ is a cone, $\rho': \Delta \rightarrow \mathbf{D}$. By Exercise 4.17, ρ is universal in $\mathcal{CPO}_\perp$ and so there is a unique mediating $h: \rho \rightarrow \rho'$. The proof then finishes as in Theorem 2 using the fact that $T\rho$ is universal in $\mathcal{CPO}_\perp$ as it is universal in $\mathcal{CPO}^E$ (being $T^E\rho$). ■

One application of this theorem is when T is chosen for the specification of a recursively defined syntactical cpo. For instance, continuing Example 6 above we can consider:

$$T = +(K_{\mathbf{Act}}, K_{\mathbf{0}}, \otimes(Id, Id), \otimes(K_{\mathbf{BExp}}, Id, Id)).$$

Then if one can take the semantic domain, here $\mathbf{C} = (\mathbf{S} \rightarrow_\perp \mathbf{S})$, and turn it into a T -algebra, $\langle \mathbf{C}, \alpha \rangle$ it follows that there is a unique homomorphism $h: \langle \mathbf{Stat}, \eta_{T^E} \rangle \rightarrow \langle \mathbf{C}, \alpha \rangle$ which we can take as the semantics. This view of semantics is called *initial-algebra semantics*. To see that it is reasonable, we note that in this case following the ideas in Example 1 above, a T -algebra amounts to a structure $\langle \mathbf{C}, mkact', mkdum', mkseq', mkcond' \rangle$ where $mkact': \mathbf{Act} \rightarrow_\perp \mathbf{C}$ is strict, $mkdum': \mathbf{0} \rightarrow_\perp \mathbf{C}$ is strict, $mkseq': \mathbf{C} \times \mathbf{C} \rightarrow_\perp \mathbf{C}$ is bistrict and $mkcond': \mathbf{BExp} \times \mathbf{C} \times \mathbf{C} \rightarrow_\perp \mathbf{C}$ is tristrict and we have already seen that for $\mathbf{Stat}$ we obtain the algebra $\langle \mathbf{Stat}, mkact, mkdum, mkseq, mkcond \rangle$. Then a homomorphism $h: \langle \mathbf{Stat}, \dots \rangle \rightarrow \langle \mathbf{C}, \dots \rangle$ is a strict continuous function $h: \mathbf{Stat} \rightarrow_\perp \mathbf{C}$ such that:

$$\begin{aligned} h(mkact(a)) &= mkact'(a) & (a: \mathbf{Act}), \\ h(mkdum(\top)) &= mkdum'(\top), \\ h(mkseq(s_0, s_1)) &= mkseq'(h(s_0), h(s_1)) & (s_0, s_1: \mathbf{Stat}), \\ h(mkcond(b, s_0, s_1)) &= mkcond'(b, h(s_0), h(s_1)) & (b: \mathbf{BExp}, s_0, s_1: \mathbf{Stat}) \end{aligned}$$

and this is just an abstract form of the usual semantics if we define:

$$\begin{aligned} mkact' &= \mathcal{A}, \\ mkdum' &= \lambda_{\perp} x: \mathbf{0}. (\lambda_{\perp} \sigma: \mathbf{S}. \sigma), \\ mkseq' &= \lambda \gamma: \mathbf{C}, \gamma': \mathbf{C}. \gamma' \circ \gamma, \\ mkcond' &= \lambda b: \mathbf{BExp}, \gamma: \mathbf{C}, \gamma': \mathbf{C}. \lambda_{\perp} \sigma: \mathbf{S}. (\mathcal{B}[\![b]\!](\sigma) \rightarrow \gamma(\sigma) \mid \gamma'(\sigma)) \end{aligned}$$

and indeed with this definition it turns out that $h = \mathcal{C}$. Of course it is equally easy to give an initial algebra semantics treatment of the other kinds of syntactic cpos considered above.

From now on, although it is not really essential, we assume that T preserves inclusions (that is if $\iota: \mathbf{D} \subseteq \mathbf{E}$ then $T(\iota): T(\mathbf{D}) \subseteq T(\mathbf{E})$). This is the case for all our covariant functors and also for each of the functors: $\rightarrow(K_{\mathbf{D}}, Id)$, $\rightarrow_{\perp}(K_{\mathbf{D}}, Id)$ (i.e. those obtained by fixing the contravariant argument of the exponentiation functor); further this property is preserved under the composition of functors.

Theorem 4 The Structural Induction Theorem. *Let $\alpha: T\mathbf{D} \rightarrow_{\perp} \mathbf{D}$ be a T -algebra. Then the following assertions are equivalent.*

- (1) $\langle \mathbf{D}, \alpha \rangle$ is the initial T -algebra.
- (2) The morphism, α , is a strict order-monic and further for every strict order-monic $m: \mathbf{P} \rightarrow_{\perp} \mathbf{D}$ if $\mathbf{P}$ can be extended to a T -algebra $\langle \mathbf{P}, \beta \rangle$ such that $m: \langle \mathbf{P}, \beta \rangle \rightarrow \langle \mathbf{D}, \alpha \rangle$ is a T -homomorphism then m is an isomorphism.
- (3) The morphism m is an order-monic and further for every ω -inductive $\mathbf{P} \subseteq \mathbf{D}$ the following principle of structural induction holds:

$$(\mathbf{P}(\perp) \wedge (\forall x: T(\mathbf{D}). T(\mathbf{P})(x) \Rightarrow \mathbf{P}(\alpha x))) \Rightarrow \forall x: \mathbf{D}. \mathbf{P}(x).$$

- (4) The morphism α is an isomorphism and $id_{\mathbf{D}} = \mu h. \alpha \circ T(h) \circ \alpha^{-1}$.

Proof. (1) $\Rightarrow$ (2). From Theorem 1 we know that in the initial T -algebra $(\mathbf{Fix}_{T^E}, \eta_{T^E})$ η_{T^E} is an isomorphism; so α is as well. Now suppose $m: \mathbf{P} \rightarrow_{\perp} \mathbf{D}$ is a strict order-monic and also $m: \langle \mathbf{P}, \beta \rangle \rightarrow \langle \mathbf{D}, \alpha \rangle$ is a T -homomorphism. As $\langle \mathbf{D}, \alpha \rangle$ is initial there is a T -homomorphism $h: \langle \mathbf{D}, \alpha \rangle \rightarrow \langle \mathbf{P}, \beta \rangle$. Then $m \circ h: \langle \mathbf{D}, \alpha \rangle \rightarrow \langle \mathbf{D}, \alpha \rangle$ is a T -homomorphism which must be $id_{\mathbf{D}}$ as $\langle \mathbf{D}, \alpha \rangle$ is initial. Now we have $m \circ (h \circ m) = m \circ id$ which shows that $h \circ m = id$ as m is a monic. Thus m is an isomorphism with inverse h .

(2) $\Rightarrow$ (1). First we show α is an isomorphism. Considering the diagram:

$$\begin{array}{ccc} T(T\mathbf{D}) & \xrightarrow{T\alpha} & T\mathbf{D} \\ T\alpha \downarrow & & \downarrow \alpha \\ T\mathbf{D} & \xrightarrow{\alpha} & \mathbf{D} \end{array}$$

we can apply (2) with $\mathbf{P} = T\mathbf{D}$ and $m = \alpha$. So α is an isomorphism.

Next let $\langle \mathbf{P}, \beta \rangle$ be the initial T^E -algebra, viz. $(\mathbf{Fix}_{T^E}, \eta_{T^E})$. Then as we now know that $\langle \mathbf{D}, \alpha \rangle$ is a T^E -algebra, as α is an embedding being an isomorphism, there must be an embedding $m: \mathbf{P} \rightarrow \mathbf{D}$ which is a homomorphism. As every embedding is a strict order-monic, we can apply (2) to see that m is an isomorphism and hence an isomorphism of the T -algebra. So $\langle \mathbf{D}, \alpha \rangle$ is isomorphic to $\langle \mathbf{P}, \beta \rangle$ which is the initial T^E -algebra and so, by Theorem 3, is the initial T -algebra.

(2) $\Rightarrow$ (3). Let $\mathbf{P} \subseteq \mathbf{D}$ be ω -inductive and suppose that $\mathbf{P}(\perp)$ and $\forall x: T(\mathbf{D}). T(\mathbf{P})(x) \Rightarrow \mathbf{P}(\alpha x)$. Then $\iota: \mathbf{P} \subseteq \mathbf{D}$ is a strict order-monic. Define $\beta: T\mathbf{P} \rightarrow \mathbf{P}$:

$$\beta(x) = \alpha \circ T\iota(x).$$

This is well-defined by the assumption and since $T\iota$ is an inclusion as T preserves inclusions. Next β is strict as $\beta(\perp_{T\mathbf{P}}) = \alpha(T\iota(\perp_{T\mathbf{P}})) = \alpha(\perp_{T\mathbf{D}}) = \perp_{\mathbf{D}} = \perp_{\mathbf{P}}$; also β is continuous as $\beta(\bigsqcup_{T\mathbf{P}} x_n) = \alpha(T\iota(\bigsqcup_{T\mathbf{P}} x_n)) = \alpha(\bigsqcup_{T\mathbf{D}} (T\iota(x_n))) = \bigsqcup_{\mathbf{D}} \alpha \circ T\iota(x_n) = \bigsqcup_{\mathbf{P}} \beta(x_n)$. Next the following diagram commutes:

$$\begin{array}{ccc}
T\mathbf{P} & \xrightarrow{\beta} & \mathbf{P} \\
T\iota \downarrow & & \downarrow \iota \\
T\mathbf{D} & \xrightarrow{\alpha} & \mathbf{D}
\end{array}$$

For any x in $T(\mathbf{P})$, $\iota \circ \beta(x) = \iota \circ \alpha \circ T\iota(x) = \alpha \circ T\iota(x)$. So we can apply (2) to find that ι is an isomorphism. But the ι is onto and we have that $\mathbf{P} = \mathbf{D}$.

(3) $\Rightarrow$ (2). Suppose $m: \mathbf{P} \rightarrow_{\perp} \mathbf{D}$ is a strict order-monic and there is a $\beta: T\mathbf{P} \rightarrow \mathbf{P}$ such that $m: \langle \mathbf{P}, \beta \rangle \rightarrow \langle \mathbf{D}, \alpha \rangle$ is a T -homomorphism. Then m factorises as $\mathbf{P} \xrightarrow{\theta} \mathbf{Q} \xrightarrow{\iota} \mathbf{D}$ where θ is an isomorphism and ι is an inclusion and $\mathbf{Q}(\perp_{\mathbf{D}})$ holds and $\mathbf{Q}$ is ω -inductive and we have the following commuting diagram.

$$\begin{array}{ccc}
T\mathbf{P} & \xrightarrow{\beta} & \mathbf{P} \\
T\theta \downarrow & & \downarrow \theta \\
T\mathbf{Q} & \xrightarrow{T\theta^{-1} \circ \beta \circ \theta} & \mathbf{Q} \\
T\iota \downarrow & & \downarrow \iota \\
T\mathbf{D} & \xrightarrow{\alpha} & \mathbf{D}
\end{array}$$

Now suppose $x: T(\mathbf{D})$ and $T(\mathbf{Q})(x)$ then $\alpha x = \alpha \circ T\iota(x)$ ($T\iota$ is an inclusion) $= \iota \circ (T\theta^{-1} \circ \beta \circ \theta)(x) = T\theta^{-1} \circ \beta \circ \theta(x) \in \mathbf{Q}$. So we may apply (3) to see that $\mathbf{D} \subseteq \mathbf{Q}$ and so $\iota = id_{\mathbf{D}}$ and $m = \theta$ is an isomorphism.

(1) $\Rightarrow$ (4). We already know (1) implies α is an isomorphism. Let $h = \mu h. \alpha \circ Th \circ \alpha^{-1}$. Then $h = \alpha \circ Th \circ \alpha^{-1}$ and so $h \circ \alpha = \alpha \circ Th$ which shows that $h: \langle \mathbf{D}, \alpha \rangle \rightarrow \langle \mathbf{D}, \alpha \rangle$ is a T -isomorphism. But then we must have $h = id_{\mathbf{D}}$ as $\langle \mathbf{D}, \alpha \rangle$ is initial.

(4) $\Rightarrow$ (2). Suppose $m: \mathbf{P} \rightarrow_{\perp} \mathbf{D}$ is a strict order-monic and $m: \langle \mathbf{P}, \beta \rangle \rightarrow \langle \mathbf{D}, \alpha \rangle$ is a T -homomorphism. Let α^{-1} be the inverse of α . Let $g = \mu g. \beta \circ Tg \circ \alpha^{-1}$. Then $g = \beta \circ Tg \circ \alpha^{-1}$ and so $g \circ \alpha = \beta \circ Tg$ and we see that the following diagram commutes.

$$\begin{array}{ccc}
T\mathbf{P} & \xrightarrow{\beta} & \mathbf{P} \\
Tm \downarrow & & \downarrow m \\
T\mathbf{D} & \xrightarrow{\alpha} & \mathbf{D} \\
Tg \downarrow & & \downarrow g \\
T\mathbf{P} & \xrightarrow{\beta} & \mathbf{P}
\end{array}$$

We now show that $m \circ g = id$. It will then follow that m is an isomorphism with inverse g from the fact that m is a monic, as in the first part of the proof.

First we show that $m \circ g \subseteq id$ by fixed point induction on the definition of g . Clearly $m \circ \perp = \perp \subseteq id$, as m is strict. Now suppose, for the sake of the induction that $m \circ g \subseteq id$. Then: $m \circ (\beta \circ Tg \circ \alpha^{-1}) = (\alpha \circ Tm) \circ (Tg \circ \alpha^{-1}) = \alpha \circ T(m \circ g) \circ \alpha^{-1} \subseteq id$ (by the induction hypothesis).

Finally we use the hypothesis that $id = \mu h. \alpha \circ Th \circ \alpha^{-1}$ to prove that $id \sqsubseteq m \circ g$ by fixed-point induction on the definition of $\mu h. \alpha \circ Th \circ \alpha^{-1}$. Clearly $\perp \sqsubseteq m \circ g$. Suppose $h \sqsubseteq m \circ g$. Then: $\alpha \circ Th \circ \alpha^{-1} \sqsubseteq \alpha \circ T(m \circ g) \circ \alpha^{-1} = \alpha \circ (Tm \circ Tg) \circ \alpha^{-1} = (m \circ \beta) \circ Tg \circ \alpha^{-1} = m \circ g$ (as $g = \beta \circ Tg \circ \alpha^{-1}$). This concludes the proof. ■

Note that the only place the assumption that T preserves inclusions was used was in the proof of the equivalence of (2) and (3). If this were to fail we could just use (2) in place of (3). For information on strict order-monics consult Exercise 2.28. Part (4) is sometimes used in axiomatisations, but it is invariably used to derive a form of structural induction and we only include it for the sake of completeness.

So as our internal axiomatisation we can now just take Theorem 3.3. In this form the axioms are a little unfamiliar and we now see how they work out with a few examples.

Example 1 The Integers. Here we have the algebra $\alpha: \mathbf{0} + \mathbf{D} \cong \mathbf{D}$ where $T = +(K_{\mathbf{0}}, Id)$, $\alpha = \eta_T$ and $\mathbf{D} = \mathbf{Fix}_T$. Now we can define $0 = \alpha(inl(\top))$, $succ = \alpha \circ inr$ and $iszero = isl \circ \alpha^{-1}$, $pred = outr \circ \alpha^{-1}$. Conversely α and α^{-1} can be defined in terms of them by: $\alpha = [\lambda \perp x: \mathbf{0}. 0, succ]$, $\alpha^{-1} = (\lambda \perp n: \mathbf{D}. iszero(n) \rightarrow inl(\top) \mid inr(pred(n)))$ as is easily verified.

The following axioms are equivalent to $\alpha^{-1} \circ \alpha = id$, as is also easily verified (and $\alpha^{-1} \circ \alpha = id$ implies α is strict order-monic):

- (1) $pred(0) = \perp$,
- (2) $pred(succ(x)) = x$,
- (3) $iszero(0) = tt$,
- (4) $iszero(succ(x)) = (\partial x \rightarrow ff \mid \perp)$.

We obtain essentially the usual form of induction by rewriting the main clause of the structural induction principle:

$$\begin{aligned} (\forall x: T(\mathbf{D}). T(\mathbf{P})(x) \Rightarrow \mathbf{P}(\alpha x)) &\equiv \forall x: \mathbf{0} + \mathbf{D}. (\mathbf{0} + \mathbf{P})(x) \Rightarrow \mathbf{P}(\alpha x) \\ &\equiv [\forall x: \mathbf{0}. \mathbf{P}(\alpha \circ inl(x))] \wedge [\forall x: \mathbf{P}. \mathbf{P}(\alpha \circ inr(x))] \\ &\equiv \mathbf{P}(\perp) \wedge \mathbf{P}(0) \wedge \forall x: \mathbf{P}. \mathbf{P}(succ(x)). \end{aligned}$$

Thus we obtain the following principle for all ω -inductive $\mathbf{P} \subseteq \mathbf{D}$ (which is to say, all $\mathbf{P} \subseteq \mathbf{D}$):

$$(5) [\mathbf{P}(\perp) \wedge \mathbf{P}(0) \wedge (\forall x: \mathbf{P}. \mathbf{P}(succ(x)))] \Rightarrow \forall x. \mathbf{P}(x).$$

This together with (1)–(4) can be considered as a form of the Peano axioms adapted from the category of sets to $\mathcal{CPO}_{\perp}$. Instead of proving that $(\mathbf{N}, 0, +1)$ was also the initial T -algebra we could just observe that it satisfies the above axioms (with (-1) and Z). Then the isomorphism with $(\mathbf{D}, \eta_T)$ follows from Theorem 4.

Example 2 Tapes. Here we consider $T(\mathbf{D}) = \mathbf{T} \otimes \mathbf{D}_{\perp}$ and the initial algebra $\alpha: \mathbf{T} \otimes \mathbf{D}_{\perp} \rightarrow_{\perp} \mathbf{D}$. The appropriate auxiliary functions are $cons = \alpha \circ \otimes(id_{\mathbf{T}} \times up): \mathbf{T} \times \mathbf{D} \rightarrow \mathbf{D}$; $head = \pi_0 \circ \alpha^{-1}: \mathbf{D} \rightarrow_{\perp} \mathbf{T}$; $tail = down \circ \pi_1 \circ \alpha^{-1}: \mathbf{D} \rightarrow_{\perp} \mathbf{D}$. Then the equivalent axioms to $\alpha^{-1} \circ \alpha = id$ are:

- (1) $cons(\perp, x) = x$,
- (2) $head(cons(t, x)) = t$,
- (3) $tail(cons(t, x)) = (\partial t \rightarrow x \mid \perp)$.

For structural induction we look at:

$$\begin{aligned} \forall x: \mathbf{T} \otimes \mathbf{D}_{\perp}. (\mathbf{T} \otimes \mathbf{P}_{\perp})(x) \Rightarrow \mathbf{P}(\alpha x) &\equiv \forall t: \mathbf{T}. \forall x: \mathbf{P}_{\perp}. \mathbf{P}_{\perp}(x) \Rightarrow \mathbf{P}(\alpha(t \otimes x)) \\ &\equiv [\forall t: \mathbf{T}. \forall x: \mathbf{P}. \mathbf{P}(x) \Rightarrow \mathbf{P}(\alpha(t \otimes up(x)))] \wedge \forall t: \mathbf{T}. \mathbf{P}(\alpha(t \otimes \perp)) \\ &\equiv [\forall t: \mathbf{T}. \forall x: \mathbf{P}. \mathbf{P}(x) \Rightarrow \mathbf{P}(cons(t, x))] \wedge \mathbf{P}(\perp) \end{aligned}$$

and we have the following structural induction principle for ω -inductive subsets of $\mathbf{D}$:

$$[\mathbf{P}(\perp) \wedge (\forall t: \mathbf{T}. \forall x: \mathbf{P}. \mathbf{P}(x) \Rightarrow \mathbf{P}(cons(t, x)))] \Rightarrow \forall x. \mathbf{P}(x).$$

Again we can use this axiomatisation to show that $\langle \mathbf{D}, \alpha \rangle$ is isomorphic to an algebra with **Tapes**.

Example 3 Results as in Example 4. Here $T = +(K_{\mathbf{S}}, (\rightarrow_{\perp}(K_{\mathbf{I}}, Id))_{\perp}, \otimes(K_{\mathbf{O}}, (Id)_{\perp}))$ and the initial algebra is, say, $\alpha: \mathbf{S} + (\mathbf{I} \rightarrow_{\perp} \mathbf{R})_{\perp} + (\mathbf{O} \otimes \mathbf{R}_{\perp}) \rightarrow_{\perp} \mathbf{R}$. An auxiliary functions we define:

$$\begin{aligned}
mkend &= \alpha \circ in_0: \mathbf{S} \rightarrow_{\perp} \mathbf{R}; \\
isend &= is_0 \circ \alpha^{-1}: \mathbf{R} \rightarrow_{\perp} \mathbf{T}; \\
endof &= out_0 \circ \alpha^{-1}: \mathbf{R} \rightarrow_{\perp} \mathbf{S}; \\
mkinp &= \alpha \circ in_1 \circ up: (\mathbf{I} \rightarrow_{\perp} \mathbf{R}) \rightarrow \mathbf{R}; \\
isinp &= is_1 \circ \alpha^{-1}: \mathbf{R} \rightarrow_{\perp} \mathbf{T}; \\
inpoof &= down \circ out_1 \circ \alpha^{-1}: \mathbf{R} \rightarrow (\mathbf{I} \rightarrow \mathbf{R}); \\
mkout &= \alpha \circ \otimes \circ (id \times up): \mathbf{O} \times \mathbf{R} \rightarrow \mathbf{R}; \\
outof &= \pi_0 \circ out_2 \circ \alpha^{-1}: \mathbf{R} \rightarrow_{\perp} \mathbf{O}; \\
restof &= down \circ \pi_1 \circ out_2 \circ \alpha^{-1}: \mathbf{R} \rightarrow_{\perp} \mathbf{R};
\end{aligned}$$

The equation for $\alpha^{-1} \circ \alpha = id$ are:

$$\begin{aligned}
(1) \quad endof(mkend(\sigma)) &= \sigma & (\sigma: \mathbf{S}); \\
(2) \quad inpoof(mkinp(f)) &= f & (f: \mathbf{I} \rightarrow_{\perp} \mathbf{R}); \\
(3) \quad mkout(\perp, r) &= \perp & (r: \mathbf{R}); \\
(4) \quad outof(mkout(o, r)) &= o & (o: \mathbf{O}, r: \mathbf{R}); \\
(5) \quad restof(mkout(o, r)) &= (\partial o \rightarrow r \mid \perp) & (o: \mathbf{O}, r: \mathbf{R}); \\
(6) \quad isend(mkend(\sigma)) &= (\partial \sigma \rightarrow tt \mid \perp) & (\sigma: \mathbf{S}); \\
(7) \quad isend(mkinp(f)) &= ff & (f: \mathbf{I} \rightarrow_{\perp} \mathbf{R}); \\
(8) \quad isend(mkout(o, r)) &= (\partial o \rightarrow ff \mid \perp) & (o: \mathbf{O}, r: \mathbf{R}); \\
(9) \quad isinp(mkend(\sigma)) &= (\partial \sigma \rightarrow ff \mid \perp) & (\sigma: \mathbf{S}); \\
(10) \quad isinp(mkinp(f)) &= tt & (f: \mathbf{I} \rightarrow_{\perp} \mathbf{R}); \\
(11) \quad isinp(mkout(o, r)) &= (\partial o \rightarrow ff \mid \perp) & (o: \mathbf{O}, r: \mathbf{R}).
\end{aligned}$$

For structural induction we look at: $\forall x: \mathbf{S} + (\mathbf{I} \rightarrow_{\perp} \mathbf{P})_{\perp} + (\mathbf{O} \otimes \mathbf{P}_{\perp}). \mathbf{P}(\alpha x) \equiv A \wedge B \wedge C$ where:

$$\begin{aligned}
A &= \forall \sigma: \mathbf{S}. \mathbf{P}(\alpha \circ in_0(\sigma)) \\
&\equiv \forall \sigma: \mathbf{S}. \mathbf{P}(mkend(\sigma)) \\
B &= \forall x: (\mathbf{I} \rightarrow_{\perp} \mathbf{P})_{\perp}. \mathbf{P}(\alpha \circ in_1(x)) \\
&\equiv \mathbf{P}(\perp) \wedge \forall f: \mathbf{I} \rightarrow_{\perp} \mathbf{P}. \mathbf{P}(\alpha \circ in_1 \circ up(f)) \\
&\equiv \mathbf{P}(\perp) \wedge \forall f: \mathbf{I} \rightarrow_{\perp} \mathbf{R}. [(\forall i: \mathbf{I}. \mathbf{P}(fi)) \Rightarrow \mathbf{P}(mkinp(f))] \\
C &= \forall w: \mathbf{O} \otimes \mathbf{P}_{\perp}. \mathbf{P}(\alpha \circ in_2(w)) \\
&\equiv \forall o: \mathbf{O}. \forall x: \mathbf{P}_{\perp}. \mathbf{P}(\alpha \circ in_2(o \otimes x)) \\
&\equiv \mathbf{P}(\perp) \wedge \forall o: \mathbf{O}. \forall r: \mathbf{P}. \mathbf{P}(\alpha \circ in_2(o \otimes up(r))) \\
&\equiv \mathbf{P}(\perp) \wedge \forall o: \mathbf{O}. \forall r: \mathbf{R}. (\mathbf{P}(r) \Rightarrow \mathbf{P}(mkout(o, r)))
\end{aligned}$$

and this gives us the following induction principle for ω -inductive subsets, $\mathbf{P}$, of $\mathbf{R}$:

$$\begin{aligned}
&[\mathbf{P}(\perp) \wedge (\forall \sigma: \mathbf{S}. \mathbf{P}(mkend(\sigma))) \\
&\wedge (\forall f: \mathbf{I} \rightarrow_{\perp} \mathbf{R}. [(\forall i: \mathbf{I}. \mathbf{P}(fi)) \Rightarrow \mathbf{P}(mkinp(f))]) \\
&\wedge (\forall o: \mathbf{O}. \forall r: \mathbf{R}. (\mathbf{P}(r) \Rightarrow \mathbf{P}(mkout(o, r)))] \Rightarrow \forall r: \mathbf{R}. \mathbf{P}(r).
\end{aligned}$$

6. Algebraic Domains

Algebraicity is an important idea which formalises some intuitive ideas in CW/2 and Chapter 1 pp. 5–8 of *finiteness* (finite amount of information) and objects as *limits* of their finite approximations. All of the previous constructions easily accommodate algebraicity except for exponentiation (which will need yet another axiom). Algebraicity allows definitions of *computability* to provide links with recursive function theory and allow results on *definability*. It allows easy consideration of constructions such as *powerdomains* and enables us to visualise domains as *completions* of structures of finite information.

Definition. Let $\mathbf{D}$ be a cpo. An element d is *finite* ($=$ *isolated* $=$ *compact*) iff for all increasing chains, $\langle x_n \rangle$ of elements of $\mathbf{D}$, $d \sqsubseteq \bigsqcup_n x_n$ implies that there is an n such that $d \sqsubseteq x_n$.

Idea: finite — finite amount of information.

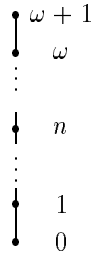
Examples

0. $\perp$ is finite.
1. Every element in a discrete or a finite cpo is finite.
2. The finite elements in $\mathbf{V}_\omega$, $\mathbf{P}\omega$, $\mathbf{T}^\omega$, **Tapes**, **Trees**, ... are the evident ones.
3. For cpos $\mathbf{D}$, $\mathbf{E}$ and finite elements d, e of $\mathbf{D}$, $\mathbf{E}$ respectively. The *step-functions* $(d \Rightarrow e)$ (or, sometimes, written $\vec{e}[d, e]$) defined by:

$$(d \Rightarrow e)(x) = \begin{cases} e & (x \sqsupseteq d), \\ \perp & (\text{otherwise}) \end{cases}$$

is continuous and finite.

4. If d, e are finite elements of $\mathbf{D}$, so is $d \sqcup e$ if it exists, but $d \sqcap e$ need not be when it exists.
5. *Finite $\neq$ Finite Height.* e.g. $\mathbf{V}_{\omega+1}$:



or $(K\top)$ ($= \perp \Rightarrow \top$) in $\mathbf{N} \rightarrow \mathbf{O}$. But it is often the case as in $\mathbf{P}\omega$, $\mathbf{T}^\omega$, **Tapes**, **Trees**, ...

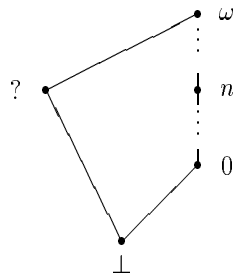
Definition. A cpo is ω -*algebraic* ($=$ *countably algebraic*) iff for every point x in $\mathbf{D}$ there is an increasing chain $\langle d_n \rangle$ of finite elements such that $x = \bigsqcup_n d_n$ and, furthermore, there are denumerably many finite elements altogether.

Idea: Points are “real” ($=$ finite) or “ideal” ($=$ limits).

Examples

1. Discrete and finite cpos; $\mathbf{V}_\omega$, $\mathbf{P}\omega$, $\mathbf{T}^\omega$, **Tapes**, **Trees**, ...
2. *Origin of Names.* Let $A = \langle A, + \rangle$ be an algebra with one binary operation $+: A^2 \rightarrow A$ (say). The *subalgebras* are (essentially) the sets $X \subseteq A$ such that $X + X \subseteq X$ (i.e. closed under $+$). Under inclusion the subalgebras form a complete lattice which is ω -algebraic.
3. The interval $[0, 1]$ under $\leq$ is a cpo which is not algebraic, similarly $\mathbf{R}$ (CW/3 p. 9) is not ω -algebraic. Can you add more points to $[0, 1]$ (intervals to $\mathbf{R}$) to make it ω -algebraic? These kinds of cpos can be dealt with by an idea of ω -*continuity*.

4. The example here is not ω -algebraic:



Note. On the whole ω -continuous domains are not much used although there is hope [Nasser Saheb-Djahromi: Probabilistic Non-Determinism, CSR-7-77, Dept. of Computer Science, University of Edinburgh]. So we do not consider them in this part of the course.

Exercises

1 Alternative Definitions. One often sees the following alternative series of definitions. Let $\mathbf{D}$ be a partial order. A subset X of $\mathbf{D}$ is *directed* iff $X \neq \emptyset$ and if $d, e \in X$ then there is an x in X such that $d \sqsubseteq x$ and $e \sqsubseteq x$. Then $\mathbf{D}$ is a *dcpo* iff it has a least element, $\perp$, and every directed subset, X , has a lub, $\bigsqcup_{\mathbf{D}} X$. An element, d , of a dcpo $\mathbf{D}$ is *finite* iff for every directed set X , $d \sqsubseteq \bigsqcup X$ implies $d \sqsubseteq x$ for some member, x , of X . The dcpo $\mathbf{D}$ is *algebraic* iff for every element x , the set $B_x = \{d \in \mathbf{D} \mid d \text{ finite} \wedge d \sqsubseteq x\}$ is directed and has lub x ; it is ω -*algebraic* if it is algebraic and has denumerably many finite elements. (If we want to talk about both sets of concepts at once we can write ω -cpo/dcpo ω -finite/dfinite etc.: only in the exercises will this be needed.)

Show that these two definitions of ω -algebraic coincide.

2 Topology. The *order-topology* on any cpo $\mathbf{D}$ has as open sets those sets O s.t.:

- (1) $x \in O \wedge x \sqsubseteq y \Rightarrow y \in O$,
- (2) $\bigsqcup_n x_n \in O \wedge \langle x_n \rangle \uparrow \Rightarrow \exists n. x_n \in O$.

Show that if $\mathbf{D}$ is ω -algebraic, the sets $O_d = \{x \in D \mid x \sqsupseteq d\}$ (d finite) form a denumerable subbasis of compact open sets for the order-topology.

3 Notions of Finiteness. Call the property of being ω -finite I_0 . Define for a given ω -cpo $\mathbf{D}$:

$$I_1(d) \equiv \forall \text{ increasing } \langle d_n \rangle. d = \bigsqcup_n d_n \Rightarrow \exists n. d = d_n.$$

For some notions of finite height, first in any partial order, $\mathbf{P}$ is a $a \sqsubseteq b$ are elements of $\mathbf{P}$ the interval $[a, b] = \{x \mid a \sqsubseteq x \sqsubseteq b\}$ and a *chain* in $\mathbf{P}$ is just a totally ordered subset of $\mathbf{P}$ (i.e. $X \subseteq \mathbf{P}$ is a chain iff $\forall x, y \in X. x \sqsubseteq y \vee y \sqsubseteq x$). So far in these notes only ω -chains were considered. Now define in $\mathbf{D}$:

- $H_0(d) \equiv [\perp, d]$ is finite,
- $H_1(d) \equiv$ there is an upper bound on the cardinality of any chain in $[\perp, d]$,
- $H_2(d) \equiv$ every chain in $[\perp, d]$ is finite,
- $H_3(d) \equiv$ there is no increasing ω -chain $\langle d_n \rangle$ with $d_n \sqsubseteq d$ for all n ,
- $H_4(d) \equiv$ there is no decreasing countably infinite chain $d = d_0 \sqsupseteq d_1 \sqsupseteq d_2 \dots$

Show that following implications and that these are all that hold for an arbitrary cpo.

$$I_0 \rightarrow I_1 \qquad H_0 \rightarrow H_1 \begin{matrix} \nearrow H_2 \\ \searrow H_3 \end{matrix}$$

What happens if $\mathbf{D}$ is given to be ω -algebraic?

Continuous Functions

We formalise CW/2 pp. 13, 14 and Chapter 1 p. 6. Idea (2) on p. 13 gives as “ ε - δ ” version of continuity. Note parts (1), (3) are *true* for ω -algebraic cpos.

Theorem 1. *Let $\mathbf{D}$ and $\mathbf{E}$ be ω -algebraic cpos. Then $f: \mathbf{D} \rightarrow \mathbf{E}$ is continuous iff for all x in $\mathbf{D}$ and finite c in $\mathbf{E}$:*

$$c \sqsubseteq f(x) \equiv \exists \text{ finite } b \text{ in } \mathbf{D}. b \sqsubseteq x \wedge c \sqsubseteq f(b).$$

Proof. $\Rightarrow$) Suppose f is continuous.

Suppose $c \sqsubseteq f(x)$ for x in $\mathbf{D}$, c finite in $\mathbf{E}$. Then $x = \bigsqcup_n b_n$ where $\langle b_n \rangle$ is an increasing ω -chain of finite elements in $\mathbf{E}$. So $c \sqsubseteq f(x) = \bigsqcup_n f(b_n)$ by continuity. So as c is finite, for some n , $c \sqsubseteq f(b_n)$.

Suppose $c \sqsubseteq f(b) \wedge b \sqsubseteq x$ for a finite b in $\mathbf{D}$. Then, by monotonicity $c \sqsubseteq f(b) \sqsubseteq f(x)$.

$\Leftarrow$) Suppose the equivalence holds. For monotonicity suppose $x \sqsubseteq y$ holds in $\mathbf{D}$, and $c \sqsubseteq f(x)$ for a finite c . Then for a finite b in $\mathbf{D}$, $b \sqsubseteq x \wedge c \sqsubseteq f(b)$. So $b \sqsubseteq y$ (as $x \sqsubseteq y$) $\wedge c \sqsubseteq f(b)$. So $c \sqsubseteq f(y)$. So $\forall$ finite c in $\mathbf{E}. c \sqsubseteq f(x) \Rightarrow c \sqsubseteq f(y)$. So $f(x) \sqsubseteq f(y)$.

For continuity suppose $\langle x_n \rangle$ is an increasing ω -chain in $\mathbf{D}$. By monotonicity, $f(\bigsqcup_n x_n) \supseteq \bigsqcup_n f(x_n)$. Suppose $c \sqsubseteq f(\bigsqcup_n x_n)$ for a finite c . Then for a finite $b \sqsubseteq \bigsqcup_n x_n$, $c \sqsubseteq f(b)$. Then for some n , $b \sqsubseteq x_n$ and so $c \sqsubseteq f(b) \sqsubseteq f(x_n)$, by monotonicity, showing that for any finite c , $c \sqsubseteq f(\bigsqcup_n x_n) \Rightarrow c \sqsubseteq \bigsqcup_n f(x_n)$. Thus $f(\bigsqcup_n x_n) \sqsubseteq f(x_n)$. ■

Exercises

1. Reformulate the ε - δ condition as the conjunction of:

- (1) f is monotonic,
- (2) $\forall x \in \mathbf{D}. \forall \text{ finite } c \in \mathbf{E}. (c \sqsubseteq f(x) \Rightarrow \exists \text{ finite } b \in \mathbf{D}. b \sqsubseteq x \wedge c \sqsubseteq f(b))$.

2. Use ε - δ to show that the identity is continuous, that the composition of continuous functions is continuous and that $\sqcap$ if it always exists is continuous. Is the latter true in a general cpo?

3. Show that for a continuous $f: \mathbf{D} \rightarrow \mathbf{E}$ between ω -algebraic cpos, $f: \mathbf{D} \rightarrow \mathbf{E}$ is continuous iff:

$$f = \bigsqcup \{b \Rightarrow c \mid b, c \text{ finite elements of } \mathbf{D}, \mathbf{E} \text{ respectively}\}.$$

Completion by Ideals

Every ω -algebraic cpo is determined by its basis (its set of finite elements). We investigate this determination.

Definition. A *countable pointed quasiorder* (ω -pq) is a structure $\langle \mathbf{P}, \sqsubseteq, \perp \rangle$ with $\mathbf{P}$ countable, $\sqsubseteq$ a quasiorder and $\perp$ the least element.

If $\mathbf{D}$ is ω -algebraic $B_{\mathbf{D}} \stackrel{\text{def}}{=} \langle \{b \in \mathbf{D} \mid b \text{ finite}\}, \sqsubseteq, \perp \rangle$ is a countable pointed po. (all inherited from $\mathbf{D}$); $B_{\mathbf{D}}$ is called the *basis* of $\mathbf{D}$. (It is convenient to let the above equation define $B_{\mathbf{D}}$ for *all* cpos $\mathbf{D}$.)

Ideas: Let $\mathbf{P}$ be a countable pointed quasiorder. A subset X of $\mathbf{P}$ is an ideal iff:

- (1) *Pointed* $\perp \in X$,
- (2) *Left Closed* $\forall y \in X. \forall x \in \mathbf{P}. x \sqsubseteq y \Rightarrow x \in X$.

An ideal is *directed* iff any two elements of the ideal have an upper bound in the ideal.

Exercise

1. Show that for any subset X of $\mathbf{P}$ there is a smallest ideal which contains it (called the *generated* ideal). Show that the directed ideals are just the ideals generated by (= least ideals containing) increasing ω -chains.

Completion

Let $\mathbf{P}$ be an ω -pq. Its *completion by ideals* is $\overline{\mathbf{P}} \stackrel{\text{def}}{=} \{X \subseteq \mathbf{P} \mid X \text{ is a directed ideal of } \mathbf{P}\}$ ordered by inclusion: $X \sqsubseteq Y$ iff $X \subseteq Y$. Let $[\cdot]: \mathbf{P} \rightarrow \overline{\mathbf{P}}$ be the monotonic map where $[x] \stackrel{\text{def}}{=} \{y \in \mathbf{P} \mid y \sqsubseteq x\}$.

Theorem 2.

- (1) For any countable pointed quasiorder $\mathbf{P}$, $\overline{\mathbf{P}}$ is ω -algebraic. Further for any other ω -algebraic cpo $\mathbf{D}$ and (strict) monotonic map $f: \mathbf{P} \rightarrow \mathbf{D}$ there is a unique continuous map $f^\sharp: \overline{\mathbf{P}} \rightarrow \mathbf{D}$ such that the following (strict) diagram commutes:

$$\begin{array}{ccc} \mathbf{P} & & \\ \downarrow [\cdot] & \searrow f & \\ \overline{\mathbf{P}} & \xrightarrow{f^\sharp} & \mathbf{D} \end{array}$$

- (2) For any ω -algebraic $\mathbf{D}$, $\mathbf{D} \cong \overline{B_{\mathbf{D}}}$. (Further for any countable pointed quasiorder, $B_{\overline{\mathbf{P}}} = \{[x] \mid x \in \mathbf{P}\} = [\mathbf{P}]$, i.e. $\mathbf{P} \equiv$ where $\equiv$ is the equivalence associated with the quasiorder.)

Proof. (1) The least element of $\overline{\mathbf{P}}$ is $\{\perp_{\mathbf{P}}\}$. $\sqcup$ is $\bigcup$ on increasing ω -chains, as is all obvious. For $b \in \mathbf{P}$, $[b]$ is finite as $[b] \subseteq \bigcup X_n \Rightarrow b \in X_n$ for some X_n . For any directed ideal X there is a cofinal increasing chain x_n in X (i.e. $\forall x \in X. \exists n. x \subseteq x_n$). Then $X = \bigcup [x_n]$ expresses the ideal as a lub of an increasing chain of finite elements of the form $[x]$. So every finite element has that form and $\overline{\mathbf{P}}$ is ω -algebraic.

Suppose the diagram commutes. Then:

$$\begin{aligned} f^\sharp(X) &= f^\sharp\left(\bigsqcup_n [x_n]\right) & x_n \text{ a cofinal chain in } X \\ &= \bigsqcup_n f^\sharp([x_n]) \\ &= \bigsqcup_n f(x_n). \end{aligned}$$

So $f^\sharp$ is determined. Conversely use this equation to determine $f^\sharp$, as the definition is independent of the choice of cofinal set and check the triangle ($f^\sharp$ is strict iff f is).

- (2) The isomorphism $\theta: \mathbf{D} \rightarrow \overline{B_{\mathbf{D}}}$ is: $\theta(x) = \{b \in B_{\mathbf{D}} \mid b \subseteq x\}$. It has inverse $\theta^{-1}(X) = \bigsqcup_{\mathbf{D}} x_n$. The last assertion is part of the proof of (1). ■

Examples

1. If, as with discrete cpos or finite ones, all points are finite then $\mathbf{D} \cong B_{\mathbf{D}}$ and $\overline{B_{\mathbf{D}}} \cong B_{\mathbf{D}}$. (So $\mathbf{D} \cong \overline{\mathbf{D}}$. Any other examples?)
2. $\mathbf{V} = \overline{\omega}$ where $\omega = \langle \{n \in \omega\}, \leq \rangle$. What is $\overline{\omega \times \omega}$? What is $\overline{\mathbf{V}}_\omega$?
3. $\mathbf{P}\omega = \overline{\text{Finite Subsets of } \omega}$, $\mathbf{Tapes} = \overline{\text{Finite Tapes}}$ etc, etc.

Exercises

1 Cantor Space. What is $\overline{\mathbf{Rat}}$ where $\mathbf{Rat}$ is the rationals in $0, 1$ with the usual order? It is not $[0, 1]$ as that is not ω -algebraic. Show $[0, 1] \not\triangleleft \overline{\mathbf{Rat}}$ showing projections need not preserve ω -algebraicity.

2 Infinite Words. Let Σ be a finite alphabet. Order $(\Sigma \cup \{\perp\})^*$ by

- (1) $\perp \subseteq x$,
- (2) If $x \subseteq x', y \subseteq y'$ then $xy \subseteq x'y'$.

Let $\mathbf{Words} = (\Sigma \cup \{\perp\})^*$. It is a domain of finite and infinite words. (Say $\Sigma = \{0, 1\}$.) What are the solutions to the equations $x = 0x$, $x = x0$, $x = 0x0$? Are they the same? Does $\mathbf{Words}$ have lubs/glbs of pairs of elements?

- 3.** If $f: \mathbf{P} \rightarrow \mathbf{P}'$ is a monotonic function of countable pointed quasiorders then $\overline{f}: \overline{\mathbf{P}} \rightarrow \overline{\mathbf{P}'}$ is defined by:

$$\overline{f}(X) = \{y \in \mathbf{P}' \mid \exists x \in X. y \subseteq f(x)\}.$$

Show this makes $\overline{f}$ well-defined and continuous and indeed that $\overline{f} = ([\cdot] \circ f)^\sharp$, the unique continuous $g: \overline{\mathbf{P}} \rightarrow \overline{\mathbf{P}'}$ s.t. the following diagram commutes:

$$\begin{array}{ccc}
\mathbf{P} & \xrightarrow{f} & \mathbf{P}' \\
\downarrow [\cdot] & & \downarrow [\cdot]' \\
\overline{\mathbf{P}} & \xrightarrow{g} & \overline{\mathbf{P}'}
\end{array}$$

Conclude that the correspondence $f \mapsto \overline{f}$ is functorial i.e. that $\overline{g \circ f} = \overline{g} \circ \overline{f}$ and $\overline{id_{\mathbf{P}}} = id_{\overline{\mathbf{P}}}$. Show that the correspondence is an *isomorphism* of $\mathbf{P} \rightarrow \mathbf{P}'$, the monotonic functions from $\mathbf{P}$ to $\mathbf{P}'$, and $\overline{\mathbf{P}} \rightarrow_{\text{fin}} \overline{\mathbf{P}'}$ the continuous functions from $\overline{\mathbf{P}}$ to $\overline{\mathbf{P}'}$ which preserves finiteness.

4. Show that the correspondence f to $f^\sharp$ of the theorem is an isomorphism as is shown by the following axioms (external):

$$\begin{aligned}
\forall f: \mathbf{P} \rightarrow \mathbf{D}. f &= f^\sharp \circ [\cdot], \\
\forall g: \overline{\mathbf{P}} \rightarrow \mathbf{D}. g &= (g \circ [\cdot])^\sharp.
\end{aligned}$$

Note that in the case $\mathbf{P} = B_{\mathbf{C}}$ for $\mathbf{C}$ ω -algebraic we have $(\mathbf{C} \rightarrow \mathbf{D}) \cong (B_{\mathbf{C}} \rightarrow \mathbf{D})$ showing that the continuous functions are just given by monotonic functions on their base.

5. A clumsier way to define completions is to use cofinal chains. If $\langle a_n \rangle, \langle b_m \rangle$ are increasing chains in an ω -pq $\mathbf{P}$ then they are cofinal, written $\langle a_n \rangle \sim \langle b_m \rangle$ iff:

$$(\forall n. \exists m. a_n \subseteq b_m) \wedge (\forall m. \exists n. b_m \subseteq a_n).$$

Show that $\overline{\mathbf{P}} \cong$ the equivalence classes of cofinal chains in $\mathbf{P}$ ordered by:

$$[\langle a_n \rangle]_{\sim} \subseteq [\langle b_m \rangle]_{\sim} \quad \text{iff} \quad \forall n. \exists m. a_n \subseteq b_m.$$

6. The definition of $\overline{\mathbf{P}}$ does not use the countability of $\mathbf{P}$. Develop a version of Theorem 2, for arbitrary quasiorders, $\mathbf{P}$, and algebraic dcpos $\mathbf{D}$. Show that the map $\sqcup: \overline{\mathbf{D}} \rightarrow \mathbf{D}$ (where $\mathbf{D}$ is any algebraic dcpo) defined by:

$$\sqcup(X) = \bigsqcup_{\mathbf{D}} x$$

is continuous and universal in the sense that if $f: \overline{\mathbf{P}} \rightarrow \mathbf{D}$ is any other continuous map then there is a unique monotonic map $f^\flat: \mathbf{P} \rightarrow \mathbf{D}$ such that the following diagram commutes:

$$\begin{array}{ccc}
\overline{\mathbf{D}} & \xleftarrow{\overline{f^\flat}} & \overline{\mathbf{P}} \\
\downarrow \sqcup & \nearrow f & \\
\mathbf{D} & &
\end{array}$$

7. Show that there is a completion $\tilde{\mathbf{P}}$, which *retains existing ω -lubs*. More precisely, a function $f: \mathbf{P} \rightarrow \mathbf{Q}$ of quasiorders is continuous if for every increasing chain $\langle x_n \rangle$ in $\mathbf{P}$ s.t. $\bigsqcup_{\mathbf{P}} x_n$ exists, so does $\bigsqcup_{\mathbf{Q}} f(x_n)$ and $f(\bigsqcup_{\mathbf{P}} x_n) = \bigsqcup_{\mathbf{Q}} f(x_n)$.

We want a cpo $\tilde{\mathbf{P}}$ and a continuous map $[\cdot]: \mathbf{P} \rightarrow \tilde{\mathbf{P}}$ such that if $f: \mathbf{P} \rightarrow \mathbf{D}$ is any other continuous map with $\mathbf{D}$ a cpo then there is a unique continuous map $f^\sharp: \tilde{\mathbf{P}} \rightarrow \mathbf{D}$ s.t. the following diagram commutes:

$$\begin{array}{ccc}
\mathbf{P} & & \\
\downarrow [\cdot] & \searrow f & \\
\tilde{\mathbf{P}} & \xrightarrow{f^\sharp} & \mathbf{D}
\end{array}$$

(Warning This is quite hard. If you solve it you should then be able to solve Exercises 20, 21 of Chapter 4.)

The following little lemma will be handy.

Lemma 1. *For any cpo $\mathbf{D}$, if $B \subseteq B_{\mathbf{D}}$ has the property that every element in $\mathbf{D}$ is a lub of an increasing ω -chain of elements in B then $B_{\mathbf{D}} = B$ and $\mathbf{D}$ is ω -algebraic if B is denumerable.*

Proof. Easy exercise. ■

I: Products. The product $\mathbf{P} \times \mathbf{Q}$ of two countable pointed quasiorders is defined as the Cartesian product with the coordinatewise ordering. As such it is also a countable pointed quasiordering.

Fact. *For any ω -algebraic cpos $\mathbf{D}$ and $\mathbf{E}$, $\mathbf{D} \times \mathbf{E}$ is ω -algebraic and, indeed, $B_{\mathbf{D} \times \mathbf{E}} = B_{\mathbf{D}} \times B_{\mathbf{E}}$.*

Proof. If $\langle d, e \rangle \in B_{\mathbf{D}} \times B_{\mathbf{E}}$ and $\langle x_n, y_n \rangle$ is an increasing chain in $\mathbf{D} \times \mathbf{E}$ s.t. $\langle d, e \rangle \sqsubseteq \bigsqcup_n \langle x_n, y_n \rangle$ then $d \sqsubseteq \bigsqcup_n x_n$, $e \sqsubseteq \bigsqcup_n y_n$ and so for some n_1, n_2 $d \sqsubseteq x_{n_1}$, $e \sqsubseteq y_{n_2}$ and so for $n = \max(n_1, n_2)$, $\langle d, e \rangle \sqsubseteq \langle x_n, y_n \rangle$. Thus $B_{\mathbf{D}} \times B_{\mathbf{E}} \subseteq B_{\mathbf{D} \times \mathbf{E}}$.

If $\langle x, y \rangle \in \mathbf{D} \times \mathbf{E}$ then $x = \bigsqcup_n d_n$ for an increasing ω -chain of $\mathbf{D}$ -finite elements and similarly $y = \bigsqcup_n e_n$. Then $\langle x, y \rangle = \bigsqcup_n \langle d_n, e_n \rangle$ expresses $\langle x, y \rangle$ as the lub of an increasing chain of elements of $B_{\mathbf{D}} \times B_{\mathbf{E}}$. So by Lemma 1, $B_{\mathbf{D}} \times B_{\mathbf{E}} = B_{\mathbf{D} \times \mathbf{E}}$ and $\mathbf{D} \times \mathbf{E}$ is ω -algebraic. ■

Corollary. *For any two countable pointed quasiorders, $\mathbf{P}, \mathbf{Q}$:*

$$\overline{\mathbf{P} \times \mathbf{Q}} \cong \overline{\mathbf{P}} \times \overline{\mathbf{Q}}.$$

Proof.

$$\begin{aligned} B_{\overline{\mathbf{P} \times \mathbf{Q}}} &= B_{\overline{\mathbf{P}}} \times B_{\overline{\mathbf{Q}}} && \text{by the Fact} \\ &\cong (\mathbf{P}/\equiv) \times (\mathbf{Q}/\equiv) && \text{Theorem 2} \\ &\cong (\mathbf{P} \times \mathbf{Q})/\equiv \\ &\cong B_{\overline{\mathbf{P} \times \mathbf{Q}}} && \text{Theorem 2. } \blacksquare \end{aligned}$$

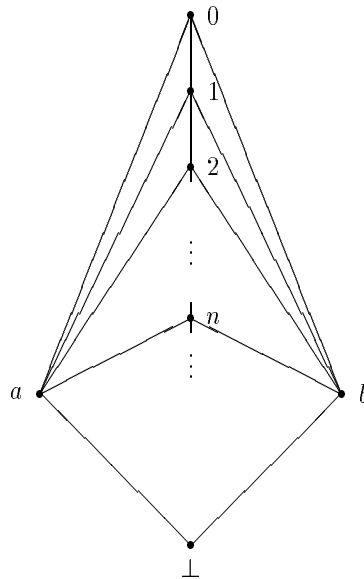
Exercises

1. Show that $\overline{\mathbf{P}} \xleftarrow{\pi_0} \overline{\mathbf{P} \times \mathbf{Q}} \xrightarrow{\pi_1} \overline{\mathbf{Q}}$ form the universal pair of projection maps.
2. Show ω -algebraicity is preserved by finite and ω -products.

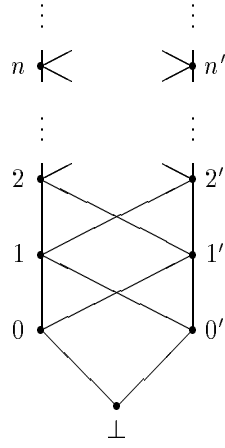
II: Function Space. The function space construction does not preserve ω -algebraicity.

Counterexamples

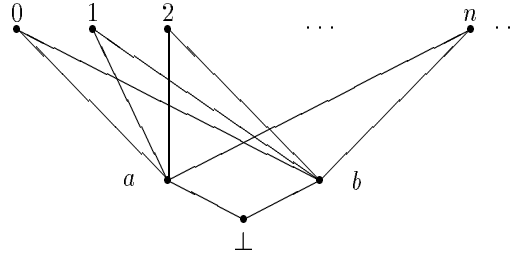
- 1.



2.



3.



Open Problem. Is there a weaker condition than ω -algebraicity which will remove this blemish but keep all the other good points of ω -algebraicity? (One possibility is that the complete lattice of open sets (under inclusion) of the cpo is ω -algebraic.)

We will deal with function spaces by adding *completeness* axioms (lattice, etc.).

Exercise 1. Show $\overline{\mathbf{P}}^{\text{monotonic}} \mathbf{Q} \neq \overline{\mathbf{P}}^{\text{continuous}} \overline{\mathbf{Q}}$.

This means that there is no evident way to turn $B_{\mathbf{D}}$ into a functor. However from the ε - δ Theorem 1, any continuous $f: \mathbf{D} \rightarrow \mathbf{E}$ is given by $\{\langle a, b \rangle \in B_{\mathbf{D}} \times B_{\mathbf{E}} \mid b \sqsubseteq f(a)\}$ and we can use this to turn the countable pointed partial orders into a suitable category. Define a morphism $R: \mathbf{P} \rightarrow \mathbf{Q}$ of countable pointed partial orders as any $R \subset \mathbf{P} \times \mathbf{Q}$ such that:

- (1) $\forall a \in \mathbf{P}. \exists b \in \mathbf{Q}. a R b$.
- (2) $\forall a, a' \in \mathbf{P}. \forall b, b' \in \mathbf{Q}. a' \sqsupseteq a R b \sqsupseteq b' \Rightarrow a' R b'$.
- (3) $\forall a, a', a'' \in \mathbf{P}. \forall b', b'' \in \mathbf{Q}. (a \sqsupseteq a' \wedge a \sqsupseteq a'' \wedge a' R b' \wedge a'' R b'' \Rightarrow \exists b \sqsupseteq b', b''. a R b)$.

Provide a definition of identity and composition to obtain a category $\omega\text{-}\mathcal{PP}$ and define the evident functor $B: \omega\text{-}\mathcal{ALG} \rightarrow \omega\text{-}\mathcal{PP}$ where $\omega\text{-}\mathcal{ALG}$ is the category of ω -algebraic cpos and continuous functions. Define, too, a functor $(\cdot): \omega\text{-}\mathcal{PP} \rightarrow \omega\text{-}\mathcal{ALG}$ and show that the two functors are a natural isomorphism of categories (see MacLane for a definition of this concept).

III: Disjoint Sum. For two given countable pointed quasiorders their disjoint sum is defined just as we did for cpos.

Fact. For any ω -algebraic cpos $\mathbf{D}$ and $\mathbf{E}$, $\mathbf{D} + \mathbf{E}$ is ω -algebraic with $B_{\mathbf{D} + \mathbf{E}} = \text{inl}(B_{\mathbf{D}}) \cup \text{inr}(B_{\mathbf{E}}) = B_{\mathbf{D}} + B_{\mathbf{E}}$.

Proof. Left to the reader (the last part is by definition). ■

Corollary. For any two countable pointed quasiorders $\mathbf{P}$ and $\mathbf{Q}$:

$$\overline{\mathbf{P} + \mathbf{Q}} \cong \overline{\mathbf{P}} + \overline{\mathbf{Q}}.$$

IV: Smash Product. For two given countable pointed quasiorders their smash product is defined just as we did for cpos.

Fact. For any two ω -algebraic cpos $\mathbf{D}$ and $\mathbf{E}$, $\mathbf{D} \otimes \mathbf{E}$ is ω -algebraic with $B_{\mathbf{D} \otimes \mathbf{E}} = B_{\mathbf{D}} \otimes B_{\mathbf{E}}$.

Corollary. For any two countable pointed quasiorders $\mathbf{P}, \mathbf{Q}$:

$$\overline{\mathbf{P} \otimes \mathbf{Q}} \cong \overline{\mathbf{P}} \otimes \overline{\mathbf{Q}}.$$

V: Lifting. For one given countable pointed quasiorder $\mathbf{P}$, its lifting $(\mathbf{P}_{\perp})$ is defined just as we did for cpos.

Fact. For any one ω -algebraic cpo, $\mathbf{D}$, $\mathbf{D}_{\perp}$ is ω -algebraic with $B_{\mathbf{D}_{\perp}} = (B_{\mathbf{D}})_{\perp}$.

Corollary. For any one countable pointed quasiorder $\mathbf{P}$: $(\overline{\mathbf{P}})_{\perp} \cong \overline{(\mathbf{P}_{\perp})}$.

Exercise 1. Show the following for two countable pointed quasiorders $\mathbf{P}, \mathbf{Q}$:

- (1) $\overline{\mathbf{P}} \xrightarrow{\text{inl}} \overline{\mathbf{P} + \mathbf{Q}} \xleftarrow{\text{inr}} \overline{\mathbf{Q}}$ is a universal pair.
- (2) $\overline{\mathbf{P} \times \mathbf{Q}} \xrightarrow{\otimes} \overline{\mathbf{P}} \otimes \overline{\mathbf{Q}}$ is a universal bistrict map.
- (3) $\overline{\mathbf{P}} \xrightarrow{\text{up}} \overline{\mathbf{P}_{\perp}}$ is a universal continuous map.

Here $\text{inl}, \dots, \text{up}$ are defined by analogy with Chapter 3.

VI: Direct Limit of Embeddings.

Lemma 2. If $f: \mathbf{D} \triangleleft \mathbf{E}$ then $f(B_{\mathbf{D}}) \subseteq B_{\mathbf{E}}$.

Proof. Take $d \in B_{\mathbf{D}}$ and let $\langle x_n \rangle$ be an increasing ω -chain in $\mathbf{E}$ such that $f(d) \sqsubseteq \bigsqcup_n x_n$, then $d = f^R(f(d)) \sqsubseteq \bigsqcup_n f^R(x_n)$ and so for some n , $d \sqsubseteq f^R(x_n)$. Then $f(d) \sqsubseteq f \circ f^R(x_n) \sqsubseteq x_n$ showing $f(d)$ is finite. ■

Exercise 1. Show $f^R(B_{\mathbf{E}}) \subseteq B_{\mathbf{D}}$ fails, in general.

Fact. Let $\Delta: \mathbf{D}_0 \xrightarrow{f_0} \mathbf{D}_1 \xrightarrow{f_1} \mathbf{D}_2 \xrightarrow{f_2} \dots$ be a direct chain in $\mathcal{CPO}^E$ where the $\mathbf{D}_i$ are all ω -algebraic, with $B_{\mathbf{D}} = \bigcup_n \rho_n(B_{\mathbf{D}_n})$.

Proof. By the lemma $\rho_n(B_{\mathbf{D}_n}) \subseteq B_{\mathbf{D}}$. Take x in $\mathbf{D}$. For each $\rho_n^R(x)$ let $\langle d_m^{(n)} \rangle_{m \in \omega}$ be an increasing chain of elements of $B_{\mathbf{D}}$ s.t. $\rho_n^R(x) = \bigsqcup_m d_m^{(n)}$. Then $B' = \{\rho_n(d_m^{(n)}) \mid n, m \in \omega\}$ is a countable directed set of elements of $\bigcup_n \rho_n(B_{\mathbf{D}_n})$ with lub x as:

$$\begin{aligned} \bigsqcup B' &= \bigsqcup_{n \in \omega} \bigsqcup_{m \in \omega} \rho_n(d_m^{(n)}) \\ &= \bigsqcup_{n \in \omega} \rho_n(\bigsqcup_{m \in \omega} d_m^{(n)}) \\ &= \bigsqcup_{n \in \omega} \rho_n(\rho_n^R(x)) \\ &= x \end{aligned}$$

(the arguments for existence of the lub go “from x to $\bigsqcup B'$ ”).

Finally as B' is countable and directed it has a cofinal increasing ω -chain. Using Lemma 2 the proof concludes. ■

Corollary. If $F: \mathcal{CPO}^E \rightarrow \mathcal{CPO}^E$ is continuous and preserves ω -algebraicity then $\mathbf{Fix}_F$ is ω -algebraic (and similarly for multiple fixed points).

Proof. $\mathbf{U}$ is ω -algebraic so the $F^n(\mathbf{U})$ are too as F preserves ω -algebraicity. Now apply VI to the definition of $\mathbf{Fix}_F$. ■

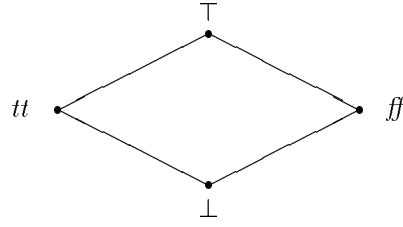
Thus all the domain equations we solve have ω -algebraic solutions (except, perhaps, for those involving exponentiation).

Note. If the functor F is defined using $+$, $\times$, $\otimes$, $(\cdot)_{\perp}$, projections and constant functors $K_{\mathbf{D}}$ where every point in $\mathbf{D}$ is finite then F preserves the property of being finite and we see that $B_{\mathbf{Fix}_F} = \bigcup_n \rho_n(F^n(\mathbf{U}))$. So e.g. the domain of trees is the limit of the domains of *finite* trees.

Exercise 2. Suppose $\mathbf{D}$ is ω -algebraic and $f: \mathbf{D} \rightarrow \mathbf{D}$ is a closure. Show that the cpo of fixed-points of $\mathbf{D}$ is also ω -algebraic and that: $B_{\mathbf{Fix}_f} = f(B_{\mathbf{D}})$. Find a condition on projections, f , on $\mathbf{D}$ so that $\mathbf{Fix}_f$ is ω -algebraic if $\mathbf{D}$ is. Find a necessary and sufficient condition for $\mathbf{Fix}_f$ to be ω -algebraic when f is a retraction.

Completeness

The most obvious axiom is: *every* subset has a lub. This gives us the complete lattices, $\mathbf{D}$, which have the top element $\top_{\mathbf{D}} \stackrel{\text{def}}{=} \bigsqcup \mathbf{D}$. This can be interpreted as “inconsistent” or “overloaded” and causes problems. For example the truth-values, $\mathbf{T}$, not being a lattice, would have to be made into one by adding a top element giving:



It is now unclear how the conditional function $(\cdot \rightarrow \cdot \mid \cdot): \mathbf{T} \times \mathbf{D} \times \mathbf{D} \rightarrow \mathbf{D}$ is to be defined at the top element:

- (1) $(\top \rightarrow d \mid e) = \top,$
- (2) $(\top \rightarrow d \mid e) = d \sqcup e.$

Both result in the failure of natural identities which hold for the three-valued cpo of truth-values:

- (1) $(p \rightarrow (p \rightarrow a \mid b) \mid c) = (p \rightarrow a \mid c),$
- (2) $(p \rightarrow (q \rightarrow a \mid b) \mid (q \rightarrow c \mid d)) = (q \rightarrow (p \rightarrow a \mid c) \mid (p \rightarrow b \mid d)),$

respectively.

Instead we adopt a compromise axiom.

Definition. Let $\mathbf{D}$ be a cpo. A subset, X , of $\mathbf{D}$ is *consistent* iff it has an upper bound in $\mathbf{D}$, and then we write $\uparrow S$ (and otherwise $\n# S$). (A subset X is ω -consistent if every finite subset of X has an upper bound in $\mathbf{D}$.) Then $\mathbf{D}$ is *consistently complete* iff every consistent subset has a lub. We want to consider the consistently complete ω -algebraic cpos and there are several equivalent axiomatisations.

Exercises

1. Consider the four statements about a cpo $\mathbf{D}$:

- (1) Every ω -consistent subset has a lub.
- (2) (*Consistent completeness*) Every consistent subset has a lub.
- (3) Every consistent pair of elements has a lub.
- (4) Every nonempty subset has a glb.

Show $(2) \Leftrightarrow (4)$, $(1) \Leftrightarrow (2) + \mathbf{D}$ is a dcpo $\Leftrightarrow (3) + \mathbf{D}$ is a dcpo. Conclude that if $\mathbf{D}$ is an ω -algebraic cpo then $(1) \Leftrightarrow (2) \Leftrightarrow (3) \Leftrightarrow (4)$.

2 **Complete Lattices.** Consider the following three statements about a cpo $\mathbf{D}$:

- (1) Every subset has a lub.
- (2) Every subset has a glb.
- (3) Every pair of elements has a lub.

Show $(1) \Leftrightarrow (2) \Leftrightarrow (3) + \mathbf{D}$ is a dcpo. Conclude that if $\mathbf{D}$ is an ω -algebraic cpo then $(1) \Leftrightarrow (2) \Leftrightarrow (3)$.

The evidence for consistent completeness is rather empirical: all our basic examples are consistently complete and all the constructions $+$, $\times$, $\otimes$, $(\cdot)_{\perp}$, $\rightarrow$, $\rightarrow_{\perp}$ preserves it.

Notice that the basis of a consistently complete ω -algebraic cpo is closed under lubs of consistent pairs. Conversely: if a countable pointed quasiorder, $\mathbf{P}$ has lubs of consistent pairs of elements, then $\overline{\mathbf{P}}$ is consistently complete.

Exercise 3. Prove this.

There is also an idea of a *subbasis*.

Definition. Let $\mathbf{D}$ be a cpo. A set $X \subseteq \mathbf{D}$ is a *subbasis* iff for any element x of $\mathbf{D}$, $x = \bigsqcup \{u \in X \mid u \sqsubseteq x\}$.

Lemma 3. If a consistently complete cpo $\mathbf{D}$ has a denumerable subbasis of finite elements it is ω -algebraic.

Proof. Apply Lemma 1 to the set, B , of lubs of consistent finite subsets of the subbasis. ■

Function Spaces

First consistent-completeness is preserved. Let $\mathbf{E}$ be consistently complete and suppose $F \subseteq (\mathbf{D} \rightarrow \mathbf{E})$ has a lub, g . Then:

$$(\bigsqcup F)(x) = \bigsqcup_{f \in F} f(x).$$

For, the lub on the RHS exists as $F(x)$ has $g(x)$ as an upper bound. For continuity: $(\bigsqcup F)(\bigsqcup_n x_n) = \bigsqcup_{f \in F} f(\bigsqcup_n x_n) = \bigsqcup_{f \in F} \bigsqcup_n f(x_n)$ and also, $\bigsqcup_n (\bigsqcup F)(x_n) = \bigsqcup_n \bigsqcup_{f \in F} f(x_n)$ and we can interchange the order of the lubs.

Now if $\mathbf{D}, \mathbf{E}$ are ω -algebraic and consistently complete so is $\mathbf{D} \rightarrow \mathbf{E}$. We have seen it is consistently complete. We have seen that $\{d \Rightarrow e \mid d \in B_{\mathbf{D}} \wedge e \in B_{\mathbf{E}}\}$ is a subbasis of finite elements (Example 3, p. 59, Exercise 3, p. 61). So by Lemma 3, it is ω -algebraic too: its basis is the lubs of consistent finite subsets of elements of the form $d \Rightarrow e$.

Exercises

1. Let $d_1, \dots, d_n$ be finite elements of $\mathbf{D}$ and $e_1, \dots, e_n$ be finite elements of $\mathbf{E}$. Show

$$\uparrow\{d_i \Rightarrow e_i \mid 1 \leq i \leq n\} \text{ iff } \forall I \subseteq \{1, \dots, n\}. (\uparrow\{d_i \mid i \in I\} \Rightarrow \uparrow\{e_i \mid i \in I\}).$$

2. Show $(d \Rightarrow e) \sqsubseteq \bigsqcup \{d_i \Rightarrow e_i \mid i = 1, \dots, n\}$ iff

$$\exists I \subseteq \{1, \dots, n\}. \uparrow\{d_i \mid i \in I\} \wedge d \sqsubseteq \bigsqcup \{d_i \mid i \in I\} \wedge e \sqsubseteq \bigsqcup \{e_i \mid i \in I\}.$$

3 **Product.** If $X \subseteq \mathbf{D} \times \mathbf{E}$ is consistent where $\mathbf{D}, \mathbf{E}$ are consistently complete then

$$\bigsqcup_{\mathbf{D} \times \mathbf{E}} X = \langle \bigsqcup_{x \in X} x_0, \bigsqcup_{x \in X} x_1 \rangle.$$

So product preserves consistent completeness.

Show $\uparrow X$ iff $\uparrow\pi_0(X) \wedge \uparrow\pi_1(X)$.

4 **Sum.** Show sum preserves consistent completeness. If $X \subseteq \mathbf{D} + \mathbf{E}$ where $\mathbf{D}, \mathbf{E}$ are consistently complete then $\uparrow X$ iff $(X \subseteq \text{inl}(\mathbf{D}) \wedge \uparrow \text{outl}(X)) \vee (X \subseteq \text{inr}(\mathbf{E}) \wedge \uparrow \text{outr}(X))$, and then $\bigsqcup X = \text{inl}(\bigsqcup_{\mathbf{D}} \text{outl}(X))$ if $X \subseteq \text{inl}(\mathbf{D})$ and $\bigsqcup X = \text{inr}(\bigsqcup_{\mathbf{E}} \text{outr}(X))$ if $X \subseteq \text{inr}(\mathbf{E})$.

5 **Smash Product.** Show smash product preserves consistent completeness: If $X \subseteq \mathbf{D} \otimes \mathbf{E}$ where $\mathbf{D}$ and $\mathbf{E}$ are consistently complete then $\uparrow X$ iff $\uparrow\pi_0(X) \wedge \uparrow\pi_1(X)$ and if $\uparrow X$ then $\bigsqcup X = (\bigsqcup_{\mathbf{D}} \pi_0(X)) \otimes (\bigsqcup_{\mathbf{E}} \pi_1(X))$.

6 **Lifting.** Show lifting preserves consistent completeness: If $X \subseteq \mathbf{D}$ where $\mathbf{D}$ is consistently complete then $\uparrow X$ iff $\uparrow \text{down}(X)$ and if $\uparrow X$ then $\bigsqcup_{\mathbf{D}_{\perp}} X = \perp$ if $X \subseteq \{\perp\}$ and $\bigsqcup_{\mathbf{D}_{\perp}} X = \text{up}(\bigsqcup_{\mathbf{D}} \text{down}(X))$ otherwise.

7 **Strict Function Space.** Show strict function space preserve consistent completeness and consistent completeness + ω -algebraicity. Treat this construction as we just did the function space. As subbasis we can take $\{d \Rightarrow e \mid d \in B_{\mathbf{D}} \wedge e \in B_{\mathbf{E}} \wedge d \neq \perp\}$ or, more precisely, $\{d \Rightarrow_{\perp} e \mid d \in B_{\mathbf{D}} \wedge e \in B_{\mathbf{E}}\}$ where:

$$(d \Rightarrow_{\perp} e)(x) = \begin{cases} e & (x \sqsupseteq d \wedge x \neq \perp), \\ \perp & (\text{otherwise}). \end{cases}$$

8 **Direct Limits.** Show that consistent completeness is preserved by direct limits of ω -chains in $\mathcal{CPO}^{\mathbf{E}}$. In particular if $\Delta = \langle \mathbf{D}_n, f_n \rangle$ is such a chain and $\rho: \Delta \rightarrow \mathbf{D}$ is a universal cone then for $X \subseteq \mathbf{D}$, $\uparrow X$ iff $\forall n. \uparrow_{\mathbf{D}_n} \rho_n^{\mathbf{R}}(X)$ and $\bigsqcup_{\mathbf{D}} X = \bigsqcup_n \rho_n(\bigsqcup_{\mathbf{D}_n} \rho_n^{\mathbf{R}}(X))$ where $\langle \rho_n(\bigsqcup_{\mathbf{D}_n} \rho_n^{\mathbf{R}}(X)) \rangle_{n \in \omega}$ is an increasing ω -chain.

If $X \subseteq \bigcup_{n \leq m} \rho_n(\mathbf{D}_n)$ we have a more constructive version:

$$\uparrow X \text{ iff } \uparrow_{\mathbf{D}_m} \rho_m^{\mathbf{R}}(X) \text{ and } \bigsqcup_{\mathbf{D}} X = \rho_m(\bigsqcup_{\mathbf{D}_m} \rho_m^{\mathbf{R}}(X)).$$

9. Characterise $\bigsqcap$ (glb) of nonempty sets in all the above constructions.

10. Show that if $r: \mathbf{D} \rightarrow \mathbf{D}$ is a retraction where $\mathbf{D}$ is consistently complete then $\mathbf{Fix}_r$ is also consistently complete.

We easily conclude that if $F: \mathcal{CPO}^E \rightarrow \mathcal{CPO}^E$ is continuous and preserves the combination, consistently complete + ω -algebraic then $\mathbf{Fix}_F$ is also consistently complete and ω -algebraic and similarly for multiple fixed-points. So all the domain equations we solve, as above, give consistently-complete ω -algebraic cpos.

Exercises

1 Stable Functions. Let $\mathbf{D}, \mathbf{E}$ be consistently complete cpos and let $f: \mathbf{D} \rightarrow \mathbf{E}$ be a continuous function. Show the following two conditions are equivalent:

- (1) **Stability.** $\forall x \in \mathbf{D}. \forall y \in \mathbf{E}. y \sqsubseteq f(x) \Rightarrow [\exists t \sqsubseteq x.y \sqsubseteq f(t) \wedge \forall u \sqsubseteq x.y \sqsubseteq f(u) \Rightarrow u \sqsupseteq t]$. Here the t whose existence is postulated is clearly unique and is written as $m(f, x, y)$ and has the properties that, if $y \sqsubseteq f(x)$:

- (a) $y \sqsubseteq f(m(f, x, y))$,
- (b) $y \sqsubseteq f(u) \wedge u \sqsubseteq x \Rightarrow u \sqsupseteq m(f, x, y)$.

Thus $m(f, x, y)$ is the part of x required for y .

- (2) $\forall X \subseteq \mathbf{D}. (\uparrow X \wedge X \neq \emptyset) \Rightarrow f(\sqcap X) = \sqcap f(X)$.

Suppose further that $\mathbf{D}, \mathbf{E}$ are ω -algebraic and that $\mathbf{D}$ obeys:

$$\text{Axiom I: } \forall d \in B_{\mathbf{D}}. \{x \in \mathbf{D} \mid x \sqsubseteq d\} \text{ is finite.}$$

Then show stability is also equivalent to:

- (3) $\forall x, y \in \mathbf{D}. x \uparrow y \Rightarrow f(x \sqcap y) = f(x) \sqcap f(y)$.

Find examples of stable and nonstable functions. The *stable order* is defined by, for stable $f, g: \mathbf{D} \rightarrow \mathbf{E}$ ($\mathbf{D}, \mathbf{E}$ consistently complete):

$$f \leq g \equiv (f \sqsubseteq g) \wedge \forall x \in \mathbf{D}. \forall y \in \mathbf{E}. [y \sqsubseteq f(x) \Rightarrow m(f, x, y) = m(g, x, y)].$$

Show it can also be defined by: $f \leq g \equiv (f \sqsubseteq g) \wedge [\forall x, y \in \mathbf{D}. x \uparrow y \Rightarrow f(x) \sqcap g(y) = f(y) \sqcap g(x)]$. Show that the set of stable functions, ordered by $\leq$, is a cpo, given that $\sqcap$ is continuous.

2 The Category $\mathcal{SFP}$. There are weaker completeness conditions than consistent completeness which work. A *strongly algebraic* cpo $\mathbf{D}$ ($\mathcal{SFP}$ object) is one such that $\mathbf{D} \cong \lim_{\mathcal{CPO}^E} \langle \mathbf{D}_n, f_n \rangle$ where each $\mathbf{D}_n$ is finite. Show that the strongly algebraic cpos are closed under $+$, $\times$, $\otimes$, $\vec{\rightarrow}$, $\rightarrow_{\perp}$ and $\varinjlim_{\mathcal{CPO}^E}$.

Conjecture. If $\mathbf{D}$ and $(\mathbf{D} \rightarrow \mathbf{D})$ are ω -algebraic then $\mathbf{D}$ is strongly algebraic.

If true this would imply that $\mathcal{SFP}$ is the largest subcategory of the ω -algebraic cpos closed under exponentiation. Let $\mathbf{D}$ be a cpo. A *minimal upper bound*, u of a subset X of $\mathbf{D}$ is an upper bound which dominates no other upper bound; i.e. $(\forall x \in X. v \sqsupseteq x) \wedge (u \sqsupseteq v) \Rightarrow u = v$. The set $U(X)$ is the set of minimal upper bounds of X ; $U^*(X)$ is the smallest set, Y , such that $X \subseteq Y$ and if $X \subseteq Y$ then $U(X) \subseteq Y$; i.e. $U^*(X)$ is the least set closed under U^* and containing X . Show that the strongly algebraic cpos are just the ω -algebraic cpos $\mathbf{D}$ such that for every finite subset X of $B_{\mathbf{D}}$:

- (1) If v is an upper bound of X then for some $u \in U(X)$, $v \sqsupseteq u$.
- (2) The set $U^*(X)$ is finite.

3 Other Completeness Conditions. There are completeness conditions stronger than consistent completeness but weaker than being a complete lattice. Let $\mathbf{D}$ be a cpo. For $\kappa \leq \omega$, we say a subset X of $\mathbf{D}$ is κ -consistent iff for every $Y \subseteq \mathbf{D}$ s.t. $\|Y\| < \kappa$, Y has an upper bound in $\mathbf{D}$. We say $\mathbf{D}$ is κ -consistently complete iff every κ -consistent subset has a lub. Show $\mathbf{D}$ κ -consistently complete implies $\mathbf{D}$ λ -consistently complete for $\kappa \leq \lambda \leq \omega$. Show that if $2 \leq \kappa < \lambda$ then there are κ -consistently complete cpos which are not λ -consistently complete (and $0- = 1- = 2- =$ complete lattice). Show that the κ -consistently complete ω -algebraic cpos are closed under $+$, $\times$, $\otimes$, $\vec{\rightarrow}$, $\rightarrow_{\perp}$, $\varinjlim_{\mathcal{CPO}^E}$.

7. Computability

We give a theory of computability for consistently complete ω -algebraic cpos — which from now on we will just call *domains*. The idea is to postulate a suitable recursive structure on the finite elements and define the computable elements to be the limits of effective chains of finite elements. We will use some standard ideas and notation from classical recursive function theory. Thus we assume n -tupling functions $\langle m_0, \dots, m_{n-1} \rangle$ and corresponding projections $\pi_i = (\cdot)_i$ ($i < n$) and $\varphi_m = \{m\}$ is the m th pr. function in the standard enumeration, W_m is the m th re set, F_m is the m th finite set (and $\{m_0, \dots, m_{n-1}\}$ is the code of $\{m_0, \dots, m_{n-1}\}$).

Definition 1. Let $\mathbf{D}$ be a domain. An *enumeration* $\langle \mathbf{D}, \varepsilon \rangle$ of its basis is any surjection, $\varepsilon: \mathbf{N} \rightarrow B_{\mathbf{D}}$ (and we write ε_m for $\varepsilon(m)$).

Further it is *effectively given*, w.r.t. ε , (by an *index* $\langle c, b, u \rangle$) if

- (1) The relation $\varepsilon_m \uparrow \varepsilon_n$ on m and n is recursive (and has index c).
- (2) $\varepsilon_b = \perp$.
- (3) The relation $\varepsilon_n = \varepsilon_m \sqcup \varepsilon_{m'}$ on m, m', n is recursive (and has index u).

Examples

1. *Truthvalues.* $\varepsilon_0, \varepsilon_1, \varepsilon_2, \varepsilon_3, \dots$ is $\perp, tt, ff, ff, ff, \dots$ with $\varepsilon_m \sqsubseteq \varepsilon_n \equiv m = 0 \vee n = m \vee (m \geq 2 \wedge n \geq 2)$, $\varepsilon_m \uparrow \varepsilon_n \equiv (\varepsilon_m \sqsubseteq \varepsilon_n \vee \varepsilon_n \sqsubseteq \varepsilon_m)$, $\varepsilon_0 = \perp$, $\varepsilon_n = \varepsilon_m \sqcup \varepsilon_{m'} \equiv (\varepsilon_n = \varepsilon_{\max(m, m')}) \wedge \varepsilon_m \uparrow \varepsilon_{m'}$.
2. *Integers.* $\varepsilon_0, \varepsilon_1, \varepsilon_2, \dots$ is $\perp, 0, 1, \dots$

We always use these “natural” enumerations except where otherwise stated (and similarly in other cases).

Note that we can always insist that $\varepsilon_0 = \perp$ for starting with ε we change to ε' where $\varepsilon'_0 = \perp$, $\varepsilon'_{n+1} = \varepsilon_n$ and perform the evident (recursive) change in the index. Note too that $\varepsilon_m \sqsubseteq \varepsilon_n$ is recursive in m and n as it is equivalent to $\varepsilon_n = \varepsilon_m \sqcup \varepsilon_n$.

Exercises

1. Show that any finite domain can be effectively given as can **Tapes**, **V ω** , **P ω** , **T ω** , **Trees**, ...
2. Let $A = \langle \omega, + \rangle$ be a recursive algebras (i.e. $+: \omega^2 \rightarrow \omega$ is recursive). Show that the lattice of subalgebras is *effectively given*. What about partial algebras and pr. functions?
3. Show that the partial order of consistent theories of propositional logic (over given letters $P_0, P_1, \dots$) is an effectively given domain. What happens with theories in the predicate calculus? What happens with equational theories? (They are sets of equations $t = u$ closed under deduction; they are consistent if not every equation can be proved.)

There is a useful alternative definition in terms of subbases.

Lemma 1. Let $\mathbf{D}$ be a domain. Then it is *effectively given* iff there is an enumeration $\beta_0, \beta_1, \dots$ of a subbasis of $\mathbf{D}$ such that

- (1) $\uparrow\{\beta_{m_0}, \dots, \beta_{m_{n-1}}\}$ is a recursive predicate of $\{m_0, \dots, m_{n-1}\}$,
- (2) The relation $\beta_m \sqsubseteq \sqcup\{\beta_{m_0}, \dots, \beta_{m_{n-1}}\}$ is recursive in m and $\{m_0, \dots, m_{n-1}\}$.

Proof. $\Rightarrow$) Clear, taking $\beta = \varepsilon$.

$\Leftarrow$) Define

$$\varepsilon_{\{m_0, \dots, m_{n-1}\}} = \begin{cases} \sqcup_{i < n} \beta_{m_i} & (\uparrow\{\beta_{m_0}, \dots, \beta_{m_{n-1}}\}), \\ \perp & (\text{otherwise}). \end{cases}$$

The rest of the proof is left to the reader. ■

With the evident definition of an *index* of such a subbasis one notes from the above proof that there are recursive functions translating each type of index to the other (note that given the subbasis one finds a code for $\perp$ by searching for an m s.t. $\beta_m \sqsubseteq \sqcup \emptyset$).

Exercises

1. Show that one can replace condition (3) of the definition of an effectively given domain (making appropriate changes in the definition of index) by:

- (3') The relation $\varepsilon_m = \varepsilon_n$ is recursive.
- (4') There is a recursive function, f , such that $\varepsilon_{f(m, n)} = \varepsilon_m \sqcup \varepsilon_n$ wherever $\varepsilon_m \uparrow \varepsilon_n$.

2. Show that one can develop a theory of *weakly effectively given domains* with the conditions (1), (4'), and:

- (3'') (a) The relation $\varepsilon_m = \varepsilon_n$ is re.
 (b) The predicate $\varepsilon_m = \perp$ is recursive.

3. What can you get in the way of a theory of effectively given ω -algebraic cpos? Try as axioms

- (3''') The relation $\varepsilon_n \sqsubseteq \varepsilon_m$ is recursive and there could also be a weaker version à la (3'') (a), (b) with $=$ replaced by $\sqsubseteq$.

4. Show there are uncountably many domains which are not effectively given (and try to give at least one example!) Find domains which are weakly effectively given but not effectively given as ω -algebraic cpos.

5 **SFP**. Develop a theory of effectively given **SFP** objects with the definition that **D** is eff. given w.r.t. ε with index $\langle a, b \rangle$:

- (1) The relation $\varepsilon_m = \varepsilon_n$ is recursive in m and n (and has index a).
 (2) There is a recursive function u such that for any m :

$$u(\varepsilon(F_m)) = \varepsilon(F_{u(m)})$$

(and u has index b).

Include a characterisation of eff. given **SFP** objects in terms of effective colimits of finite objects in $\mathbf{SFP}^E$ (see the later development of effective colimits in $\mathbf{CPO}^E$ for some hints).

6. Let $\langle \mathbf{P}, \sqsubseteq, \perp \rangle$ be a countable pointed quasi-order with lubs of pairs of consistent elements. It is *effectively given* iff there is an enumeration (= surjection) $\varepsilon: \mathbf{N} \rightarrow \mathbf{P}$ and an index satisfying (1), (2), (3) of Definition 1. Show **P** effectively given iff $\overline{\mathbf{P}}$ is (and so **D** is effectively given iff $B_{\mathbf{D}}$ is).

The following definitions will prove useful on occasion.

Definition 1. Consider an enumeration $\langle \mathbf{D}, \varepsilon \rangle$. The relations $\lesssim, \simeq$ on $\mathbf{N}$ are defined by:

$$\begin{aligned} m \lesssim n &\equiv \varepsilon(m) \sqsubseteq \varepsilon(n), \\ m \simeq n &\equiv \varepsilon(m) = \varepsilon(n). \end{aligned}$$

A *morphism* of enumerations is a pair $\langle r, f \rangle: \langle \mathbf{D}, \varepsilon \rangle \rightarrow \langle \mathbf{D}', \varepsilon' \rangle$ where $r: \mathbf{N} \rightarrow \mathbf{N}$ is recursive, $f: \mathbf{D} \rightarrow \mathbf{D}'$ is continuous and the following diagram commutes:

$$\begin{array}{ccc} \mathbf{N} & \xrightarrow{r} & \mathbf{N} \\ \varepsilon \downarrow & & \downarrow \varepsilon' \\ \mathbf{D} & \xrightarrow{f} & \mathbf{D}' \end{array}$$

Exercises

1. Show the enumerations and their morphisms form a category. If $\langle r, f \rangle: \langle \mathbf{D}, \varepsilon \rangle \rightarrow \langle \mathbf{D}', \varepsilon' \rangle$ is a morphism then r clearly determines f . Show that any recursive $r: \mathbf{N} \rightarrow \mathbf{N}$ can be extended to a morphism iff it is monotonic w.r.t. $\lesssim$; show too that it is not enough that r preserves $\simeq$.

2. Show that if **D** is finite then there is always a morphism from $\langle \mathbf{D}, \varepsilon \rangle$ to $\langle \mathbf{D}', \varepsilon' \rangle$ (thus *reducing* ε to ε'); show this need not hold in general — e.g. when $\mathbf{D} = \mathbf{N}$.

3. Show that for any enumeration $\langle \mathbf{D}, \varepsilon \rangle$ we have $B_{\mathbf{D}} \cong \langle \mathbf{N}, \lesssim \rangle / \simeq$ (and deduce there are only countably many eff. given domains, to within isomorphism).

Computability

We use the idea that a computable element is just a limit of an effective chain of finite elements, imagining a machine generating more and more information about the element, but at any finite point in its computation, only having generated a finite amount. This is sometimes called the *enumeration* model of computation following the case where, for example, one enumerates all the elements of a set: at finite time one has generated a finite subset.

Definition. Let $\mathbf{D}$ be effectively given w.r.t. ε . An *effective chain* in $\mathbf{D}$ with index g is any increasing chain $\langle \varepsilon_{\{g\}_n} \rangle_{n \in \omega}$. An element d is computable if there is an effective chain with lub d . We write $C_{(\mathbf{D}, \varepsilon)}$ for the set of computable elements of $\mathbf{D}$ (and $C_{\mathbf{D}}$ when ε is understood).

Exercises

1. Every finite element is computable.
2. The computable elements of **Tapes** are just the finite ones and the recursive infinite sequence (of 0's and 1's); the computable elements of **P ω** are the re. sets, of **T $^\omega$** are the pr. predicates. What about **V $_\omega$** , **Trees**, ...
3. Show how to use the typed λ -calculus to define all computable elements of **Tapes**, **P ω** , **T $^\omega$** , ...
[Hint Using the idea of generation just define appropriate functions $f: \mathbf{N} \rightarrow \mathbf{Tapes}$ so that for a given tape, $t = t_0 t_1 t_2 \dots$, we have:

$$f(n) = t_n t_{n+1} \dots$$

and use similar ideas in the other cases.]

4. Show that by varying ε one can vary $C_{(\mathbf{P}\omega, \varepsilon)}$. Kanda and Park have shown (When are two effectively given domains identical? Proc. 4th AI Conference.) that for some $\mathbf{D}$, ε , ε' we even have: $C_{(\mathbf{D}, \varepsilon)} \not\cong C_{(\mathbf{D}, \varepsilon')}$. Can we take $\mathbf{D} = \mathbf{P}\omega$?

Now there is a little technical difficulty if we want to *enumerate* all the computable elements as not all integers, g , are indexes as $\{g\}$ may not be recursive and $\langle \varepsilon_{\{g\}_n} \rangle$ may not be increasing. To avoid dealing with partial surjections (*Exercise* try working with them) we note that for any g there is an effectively obtainable g' with $\text{range}(\{g'\}) = \text{range}(\{g\}) \cup \{\text{a code of } \perp\}$. Now define $f: \mathbf{N} \rightarrow \mathbf{N}$ by:

$$\begin{aligned} f(0) &= \text{a code of } \perp, \\ f(n+1) &= \begin{cases} \text{a code of } \varepsilon_{f(n)} \sqcup \varepsilon_{g'(n+1)} & (\text{if } \varepsilon_{f(n)} \uparrow \varepsilon_{g'(n+1)}), \\ f(n) & (\text{otherwise}). \end{cases} \end{aligned}$$

This f is clearly recursive and an index for it is effectively obtainable (say via d) from m (via n); clearly for $d(m)$ is an index of a computable element and:

$$\bigsqcup_n \varepsilon_{\{d(g)\}_n} = \bigsqcup_n \varepsilon_{\{g\}_n}$$

if g is an index. We can now define our (total) enumeration:

Definition 1. Let $\langle \mathbf{D}, \varepsilon \rangle$ be an enumerated domain. Then the enumeration $\zeta: \mathbf{N} \rightarrow C_{\mathbf{D}}$ is defined by:

$$\zeta_g = \bigsqcup_n \varepsilon_{\{d(g)\}_n}.$$

Exercises

1. Let $\mathbf{D}$ be effectively given w.r.t. ε . Show that the following conditions on an element d are all effectively equivalent.
 - (a) d is computable.
 - (b) $\{n \mid \varepsilon_n \sqsubseteq d\}$ is re.
 - (c) For some k , $\bigsqcup \varepsilon(W_k) = d$.
 - (d) For some k , $\varepsilon(W_k)$ is directed and has lub d .
2. Use 1(c) to obtain another total enumeration ζ^S of $C_{\mathbf{D}}$; this is the *single-valued* enumeration of Egli-Constable; ours, ζ , is the *directed* enumeration of Kanda-Park.
3. Show that as every finite element is computable there is a recursive function k such that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{N} & \xrightarrow{k} & \mathbf{N} \\ \varepsilon \downarrow & & \downarrow \zeta \\ B_{\mathbf{D}} & \xrightarrow{\iota} & C_{\mathbf{D}} \end{array}$$

This will prove useful later.

4. (1) Show that the relation $\varepsilon_m \sqsubseteq \zeta_n$, is re. in m and n . Is it ever recursive (ignoring here — and below — the case $\mathbf{D} = \perp$)?
 (2) Show that the relation $\zeta_m \not\sqsubseteq \zeta_n$ is re. in m and n . Is it ever recursive?
 (3) Show that there is a recursive function, $\sqcup$,

$$\zeta_{m \sqcup n} = \zeta_m \sqcup \zeta_n \quad (\text{when } \zeta_m \uparrow \zeta_n).$$

- (4) Show there is a recursive function, $\sqcap$, of two arguments such that:

$$\zeta_{m \sqcap n} = \zeta_m \sqcap \zeta_n.$$

- (5) Are either of the relations $\zeta_m \sqsubseteq \zeta_n$, $\zeta_m = \zeta_n$ (or their negations) ever re?

5 Effective Limits. Show the lubs of increasing effective chains are computable and effectively obtainable. That is there is a recursive function, $\sqcup$, of one argument such that if c is the index of an *effective chain of computable elements* (i.e. if $\{c\}$ is recursive and $\langle \zeta_{\{c\}(n)} \rangle$ is increasing) then:

$$\zeta_{\sqcup c} = \bigsqcup_n \zeta_{\{c\}(n)}.$$

6 Computable Open Sets. An open set O of an effectively given $\mathbf{D}$, is *computable* iff given any effective chain of computable elements with index c with $\zeta_{\sqcup c}$ in O one can effectively obtain from c an n s.t. $\zeta_{\{c\}(n)} \in O$. Show that an open set O is computable iff

$$O = \bigcup_{n \in W_m} O_{\varepsilon_n}$$

for some m . Show the computable open sets are closed under intersection and unions of re. collections of open sets. Note all proofs and closure properties are effective.

Computable Functions

To obtain a definition of computable function we effectivise the ε - δ version of continuity: given a finite amount of information about the input one can enumerate all the finite pieces of information about the output.

Definition. Let $\mathbf{D}$ and $\mathbf{E}$ be domains effectively given with respect to enumerations $\varepsilon, \varepsilon'$. Then a continuous function $f: \mathbf{D} \rightarrow \mathbf{E}$ is *computable* iff

$$\text{Graph}(f) = \{\langle m, n \rangle \mid \varepsilon'_n \sqsubseteq f(\varepsilon_m)\}$$

is re. (and g is an index for f if $\text{Graph}(f) = W_g$). We delay consideration of enumerating the computable functions until the discussion of function spaces.

Examples

1. The computable $f: \mathbf{N} \rightarrow \mathbf{N}$ are the constant functions and the (functions corresponding to) the partial recursive functions.
2. The identity is computable as:

$$\text{Graph}(id) = \{\langle m, n \rangle \mid \varepsilon_n \sqsubseteq \varepsilon_m\}.$$

3. If $\mathbf{D} \xrightarrow{f} \mathbf{E} \xrightarrow{g} \mathbf{F}$ are computable w.r.t. $\varepsilon, \varepsilon'$ and $\varepsilon', \varepsilon''$ then $g \circ f$ is too w.r.t. $\varepsilon, \varepsilon''$ as:

$$\text{Graph}(g \circ f) = \text{Graph}(f) \circ \text{Graph}(g)$$

(and an idea for $g \circ f$ is clearly effectively obtainable from those of g and f). Thus we have a *category* of enumerated effectively given domains and computable functions.

Exercises

1. What are the computable $f: \mathbf{N} \rightarrow \mathbf{T}$? What about $f: \mathbf{P}\omega \rightarrow \mathbf{P}\omega$? [Hint Read about enumeration operators in Rogers.] What about $f: \mathbf{T}^\omega \rightarrow \mathbf{T}^\omega$, $f: \mathbf{Tapes} \rightarrow \mathbf{Tapes}$? Are they all definable in the typed λ -calculus? [Hint Try using *parcond* if needed.]

2. Show that a continuous $f: \mathbf{D} \rightarrow \mathbf{E}$ can be extended to a morphism of enumerations $\langle \mathbf{D}, \varepsilon \rangle, \langle \mathbf{E}, \varepsilon' \rangle$ of effectively given domains iff f preserves finiteness and $\text{Graph}(f)$ is recursive.

Another natural idea for a definition of the computability of a function f is that given a method of generating an argument x one can effectively obtain a method of generating the result $f(x)$. But it turns out that this is a theorem:

Theorem 1. *Let $\langle \mathbf{D}, \varepsilon \rangle, \langle \mathbf{E}, \varepsilon' \rangle$ be enumerations of effectively given domain. Then a continuous function $f: \mathbf{D} \rightarrow \mathbf{E}$ is computable w.r.t. $\varepsilon, \varepsilon'$ iff there is a recursive $r: \mathbf{N} \rightarrow \mathbf{N}$ such that the following diagram commutes:*

$$\begin{array}{ccc} \mathbf{N} & \xrightarrow{r} & \mathbf{N} \\ \zeta \downarrow & & \downarrow \zeta' \\ \mathbf{D} & \xrightarrow{f} & \mathbf{E} \end{array}$$

Proof. $\Rightarrow$) If f is computable then for any computable x with $\zeta(g) = x$:

$$\begin{aligned} X &= \{n \mid \varepsilon'(n) \subseteq f(x)\} \\ &= \{n \mid \exists \langle m, n \rangle \in \text{Graph}(f). \varepsilon(m) \subseteq \zeta(g)\}. \end{aligned}$$

and clearly X is re. (see Exercise 4, p. 73) and an index for it is effectively obtainable from one for $\text{Graph}(f)$. Then one effectively obtains an index from X for $f(x)$ (see Exercise 1, p. 72).

$\Leftarrow$) With k as in Exercise 3, p. 72, we have: $\langle m, n \rangle \in \text{Graph}(f)$ iff $\varepsilon'_m \subseteq f(\varepsilon_n)$ iff $\varepsilon'_m \subseteq f(\zeta_{k(n)})$ iff $\varepsilon'_m \subseteq \zeta(r(k(n)))$ and by Exercise 4, p. 73, this is clearly a partial recursive relation. ■

Of course the theorem says, among other things, that computable functions preserve computability. The theorem suggests a natural definition of morphisms $\langle r, f \rangle: \langle \mathbf{D}, \zeta \rangle \rightarrow \langle \mathbf{D}', \zeta' \rangle$ of enumerations of computable elements as pairs $r: \mathbf{N} \rightarrow \mathbf{N}$, $f: \mathbf{D} \rightarrow \mathbf{D}'$ of recursive functions and continuous functions which make the diagram of the theorem commute. The theorem says that if $\langle r, f \rangle$ is a morphism then f is computable and if f is computable there is a recursive r making $\langle r, f \rangle$ a morphism; note the equivalence is effective. We shall have occasion in the exercises to greatly strengthen this result.

Exercises

1. Show that a continuous $f: \mathbf{D} \rightarrow \mathbf{D}'$ of domains effectively given w.r.t. $\varepsilon, \varepsilon'$ is computable iff $O' \mapsto f^{-1}(O')$ effectively sends computable open sets in $\mathbf{D}'$ to computable open sets in $\mathbf{D}$.

2 (Rice-Shapiro). Show that for any $X \subseteq C_{\mathbf{D}}$ we have $\zeta^{-1}(X)$ re. iff there is a (necessarily unique) computable open set O with $X = O \cap C_{\mathbf{D}}$. This means that any partial recursive predicate on codes (= texts) for computable elements which respects extensional equality ($m \simeq_{\zeta} n \Leftrightarrow \zeta(m) = \zeta(n)$) must be continuous on $\mathbf{D}$. [Hint Apply the idea of the proof of the classical Rice-Shapiro theorem.]

3 (Myhill-Shepherdson). Let $\mathbf{D}, \mathbf{D}'$ be eff. given w.r.t. $\varepsilon, \varepsilon'$. Show that if $r: \mathbf{N} \rightarrow \mathbf{N}$ is a recursive function which respects extensional equality (i.e. $m \simeq_{\zeta} n \Rightarrow r(m) \simeq_{\zeta'} r(n)$) then there is a (necessarily unique) computable function $f: \mathbf{D} \rightarrow \mathbf{D}'$ which makes $\langle r, f \rangle$ a morphism of enumerations of computable elements. [Hint By Theorem 1 the only problem is to prove continuity. This can be done by applying Exercise 2, or the Myhill-Shepherdson.] Again this shows how an intensional operator which respects extensional equivalence is (essentially) a computable function — providing more evidence for continuity.

Least Fixed-Points

Of course, we can also effectivise the least fixed-point construction. Let $f: \mathbf{D} \rightarrow \mathbf{D}$ be computable. Then with r as in Theorem 1 we have:

$$f^n(\perp) = \zeta_{r^n(k(b))}.$$

Hence by Exercise 5, p. 73 we obtain an index of Yf and this index is effectively obtainable from an index for f .

Exercises

1. Let $\mathbf{D}$ be eff. given w.r.t. ε . Show that for any partial recursive function f there is a total recursive g , such that for any m :

- (1) If $f(n) \downarrow$ then $g(n) \simeq_{\zeta} f(n)$,
- (2) If $f(n) \uparrow$ then $g(n) \simeq_{\zeta} k(b)$.

2 Ershov-Kleene Fixed-Point Theorem. Show that for any partial recursive r there is an n (effectively obtainable from any index for r) such that if $r(n) \downarrow$ then $r(n_r) \simeq_{\zeta} n_r$.

Open Question. In case $\langle r, f \rangle: \langle \mathbf{D}, \zeta \rangle \rightarrow \langle \mathbf{D}, \zeta \rangle$ is a morphism do we have $\zeta_n = Yf$? That is do intensional and extensional fixed-points coincide — the answer may depend on exactly how n_r is defined.

Remark. In “Gödel numbering of domain theoretic computable functions — a generalisation of Rogers Indexing Theory, Report 138, University of Leeds”, Kanda considers general systems of indexings, τ , where if $\mathbf{D}$ is eff. given w.r.t. ε then $\tau_{\varepsilon}: \mathbf{N} \rightarrow C_{\mathbf{D}}$ is an enumeration of the computable elements of $\mathbf{D}$. He gave axioms for such a system to be *admissible* (our ζ 's form an admissible system) and showed that if ζ, ζ' are admissible then for any $\langle \mathbf{D}, \varepsilon \rangle$ there is a recursive isomorphism

$$\langle r, id \rangle: \tau_{\varepsilon} \rightarrow \tau'_{\varepsilon}$$

effectively obtainable from the index of $\mathbf{D}$.

We now turn to effectivising the concerns of Chapter 2, 3, 4, 5.

Products

Let $\mathbf{D}, \mathbf{E}$ be eff. given domains w.r.t. $\varepsilon, \varepsilon'$ with indices $\langle c, b, u \rangle$ and $\langle c', b', u' \rangle$. Then we can define an enumeration $\varepsilon'' = \varepsilon \times \varepsilon'$ of $B_{\mathbf{D} \times \mathbf{E}} = B_{\mathbf{D}} \times B_{\mathbf{E}}$ by Chapter 6:

$$\varepsilon \times \varepsilon'(\langle m, n \rangle) = \langle \varepsilon_m, \varepsilon'_n \rangle.$$

This makes $\mathbf{D} \times \mathbf{E}$ effectively given as we can see from:

- (1) $\varepsilon''_{\langle m, m' \rangle} \uparrow \varepsilon''_{\langle n, n' \rangle} \equiv (\varepsilon_m \uparrow \varepsilon_n) \wedge (\varepsilon'_{m'} \uparrow \varepsilon'_{n'})$.
- (2) $\varepsilon''_{\langle b, b' \rangle} = \perp_{\mathbf{D} \times \mathbf{E}}$.
- (3) $\varepsilon''_{\langle n, n' \rangle} = \varepsilon''_{\langle l, l' \rangle} \sqcup \varepsilon''_{\langle m, m' \rangle} \equiv (\varepsilon_n = \varepsilon_l \sqcup \varepsilon_m \wedge \varepsilon'_{n'} = \varepsilon'_{l'} \sqcup \varepsilon'_{m'})$.

Exercise

1. Show that $\zeta'': \mathbf{N} \rightarrow C_{\mathbf{D} \times \mathbf{E}}$ is recursively equivalent to $\langle \zeta, \zeta' \rangle: \mathbf{N} \rightarrow (C_{\mathbf{D}} \times C_{\mathbf{E}}) (= C_{\mathbf{D} \times \mathbf{E}})$ in that there are morphisms $\langle r, id \rangle: \zeta'' \rightarrow \langle \zeta, \zeta' \rangle$ and $\langle \bar{r}, id \rangle: \langle \zeta, \zeta' \rangle \rightarrow \zeta''$.

Of course all this is effective in the indices of $\mathbf{D}$ and $\mathbf{E}$ and so there is a recursive x so that if i, j are indices of $\mathbf{D}$ and $\mathbf{E}$ then $i \times j$ is one of $\mathbf{D} \times \mathbf{E}$.

Note that the projections $\mathbf{D} \xrightarrow{\pi_0} \mathbf{D} \times \mathbf{E} \xrightarrow{\pi_1} \mathbf{E}$ are computable as:

$$\begin{aligned} \text{Graph}(\pi_0) &= \{ \langle m, n \rangle \mid \varepsilon_n \sqsubseteq \varepsilon_{\pi_0(m)} \}, \\ \text{Graph}(\pi_1) &= \{ \langle m, n \rangle \mid \varepsilon_n \sqsubseteq \varepsilon_{\pi_1(m)} \}. \end{aligned}$$

Let $\mathbf{C}$ also be an effectively given domain, w.r.t. ε'' and suppose $f: \mathbf{C} \rightarrow \mathbf{D}, g: \mathbf{C} \rightarrow \mathbf{E}$ are computable w.r.t. $\varepsilon'', \varepsilon \times \varepsilon'$ as:

$$\text{Graph}(\langle f, g \rangle) = \{ \langle m, n \rangle \mid \langle m, n_0 \rangle \in \text{Graph}(f) \wedge \langle m, n_1 \rangle \in \text{Graph}(g) \}$$

and this is clearly an effective construction. As a consequence if $f: \mathbf{D}' \rightarrow \mathbf{D}, g: \mathbf{E}' \rightarrow \mathbf{E}$ are computable so is $f \times g: \mathbf{D}' \times \mathbf{E}' \rightarrow \mathbf{D} \times \mathbf{E}$ and the product construction is effective on the graphs.

Exercises

2. Carry out the same details for finite and effective denumerable products.

3. Show $\sqcap: \mathbf{D} \times \mathbf{D} \rightarrow \mathbf{D}$ is computable (see Exercise 4) for any effectively given $\mathbf{D}$ as is $B_{\sqcup}: \mathbf{D}^3 \rightarrow \mathbf{D}$ where $B_{\sqcup}(x, y, z) = (x \sqcap z) \sqcup (y \sqcap z)$. If $\mathbf{D}$ is a complete lattice show $\top$ and $\sqcup$ are computable.

4. For any eff. given $\mathbf{D}$ w.r.t. ε show $\#_n: \mathbf{D} \rightarrow \mathbf{T}$ is computable where:

$$\#_n(x) = \begin{cases} tt & (x \# \varepsilon_n), \\ ff & (x \sqsupseteq \varepsilon_n), \\ \perp & (\text{otherwise}). \end{cases}$$

Show also that $\text{cond}, \text{parcond}: \mathbf{T} \times \mathbf{D} \times \mathbf{D} \rightarrow \mathbf{D}, =: \mathbf{D} \times \mathbf{D} \rightarrow \mathbf{T}$ ($\mathbf{D}$ discrete), $\partial: \mathbf{D} \rightarrow \mathbf{T}$ are all computable.

Function Spaces

Let $\mathbf{D}, \mathbf{E}$ be effectively given domains w.r.t. $\varepsilon, \varepsilon'$ and indices i, j . Then by p. 68, Chapter 6 we can define an enumeration, β , of a subbasis of $\mathbf{D} \rightarrow \mathbf{E}$ by:

$$\beta_{\langle m, n \rangle} = (\varepsilon_m \Rightarrow \varepsilon'_n).$$

By Exercise 2, p. 68, Chapter 6 we see that the conditions of Lemma 1 hold and we obtain an enumeration $(\varepsilon \rightarrow \varepsilon')$ of $(\mathbf{D} \rightarrow \mathbf{D}')$ as an effectively given domain, where:

$$(\varepsilon \rightarrow \varepsilon')_{\llbracket m_0, \dots, m_{n-1} \rrbracket} = \begin{cases} \bigsqcup_{a \in \{m_0, \dots, m_{n-1}\}} \varepsilon_{(a)_0} \Rightarrow \varepsilon'_{(a)_1} & (\uparrow \{\beta_{m_i} \mid i < n\}), \\ \perp & (\text{otherwise}). \end{cases}$$

This is all effective and so we have a recursive function $\rightarrow$ s.t. if i, j are indices of $\mathbf{D}, \mathbf{E}$ w.r.t. $\varepsilon, \varepsilon'$ then $i \rightarrow j$ is one of $\mathbf{D} \rightarrow \mathbf{E}$ w.r.t. $\varepsilon \rightarrow \varepsilon'$.

Now we have an enumeration $\zeta: \mathbf{N} \rightarrow C_{(\mathbf{D} \rightarrow \mathbf{E})}$ of the computable elements of $(\mathbf{D} \rightarrow \mathbf{E})$ and see further that

$$\begin{aligned} f: \mathbf{D} \rightarrow \mathbf{E} \text{ is in } C_{\mathbf{D} \rightarrow \mathbf{E}} & \text{ iff } \{m \mid (\varepsilon \rightarrow \varepsilon')_m \sqsubseteq f\} \text{ is re.} \\ & \text{ iff } \{\llbracket m_0, \dots, m_{n-1} \rrbracket \mid \uparrow \{\beta_{m_0}, \dots, \beta_{m_{n-1}}\} \wedge \forall i < n. \beta_{m_i} \sqsubseteq f\} \text{ is re.} \\ & \text{ iff } \{\llbracket m_0, \dots, m_{n-1} \rrbracket \mid \uparrow \{\beta_{m_0}, \dots, \beta_{m_{n-1}}\} \wedge \forall i < n. m_i \in \text{Graph}(f)\} \text{ is re.} \\ & \text{ iff } \text{Graph}(f) \text{ is re.} \\ & \text{ iff } f \text{ is computable.} \end{aligned}$$

All this is effective and one recursively translates between indices m with $f = \zeta(m)$ and indices of f as a computable function. This gives us the enumeration of computable functions we shall use.

Exercises

1. Show there is a recursive function id so that if $\mathbf{D}$ has index i then $id(i)$ is an index of $id_{\mathbf{D}}$.
2. Let $\mathbf{D}, \mathbf{E}, \mathbf{F}$ be eff. given w.r.t. $\varepsilon, \varepsilon', \varepsilon''$ with indices i, j, k . Show there is a recursive function, $\circ$, such that if $\zeta: \mathbf{N} \rightarrow C_{(\mathbf{D} \rightarrow \mathbf{E})}$, $\zeta': \mathbf{N} \rightarrow C_{(\mathbf{E} \rightarrow \mathbf{F})}$, $\zeta'': \mathbf{N} \rightarrow C_{(\mathbf{D} \rightarrow \mathbf{F})}$ are the appropriate enumerations of $C_{(\mathbf{D} \rightarrow \mathbf{E})}$ etc. w.r.t. $(\varepsilon \rightarrow \varepsilon')$ etc. then for all m, n :

$$\zeta_m \circ \zeta_n = \zeta_{m \circ_{i,j,k} n}$$

(where $m \circ_{i,j,k} n$ means $\circ(m, i, j, k, n)$).

3. Let $\mathbf{D}$ be eff. given w.r.t. ε . Show that $Y_{\mathbf{D}}: (\mathbf{D} \rightarrow \mathbf{D}) \rightarrow \mathbf{D}$ is computable w.r.t. $(\varepsilon \rightarrow \varepsilon) \rightarrow \varepsilon$ and there is a recursive function Y s.t. if i is an index of $\mathbf{D}$ then $Y_{\mathbf{D}}$ has index $Y(i)$.

4. Show that there are recursive functions $\pi_0, \pi_1, \langle \cdot, \cdot \rangle, \times$ such that if $\mathbf{D}, \mathbf{E}, \mathbf{C}, \mathbf{D}', \mathbf{E}'$ are eff. given domains w.r.t. $\varepsilon_0, \varepsilon_1, \varepsilon, \varepsilon'_0, \varepsilon'_1$ and indices i, j, k, l, m then:

$$\begin{aligned} \pi_0: \mathbf{D} \times \mathbf{E} \rightarrow \mathbf{D} &= \zeta_{\pi_0(\langle i, j \rangle)}, \\ \pi_1: \mathbf{D} \times \mathbf{E} \rightarrow \mathbf{E} &= \zeta_{\pi_1(\langle i, j \rangle)} \end{aligned}$$

and if $f: \mathbf{C} \rightarrow \mathbf{D}, g: \mathbf{C} \rightarrow \mathbf{E}$ have indices a, b then

$$\langle f, g \rangle = \zeta_{\langle a, b \rangle_{i,j,k}}$$

and if $f: \mathbf{D} \rightarrow \mathbf{D}', g: \mathbf{E} \rightarrow \mathbf{E}'$ have indices a, b then

$$f \times g = \zeta_{a \times_{i,j,l,m} b}.$$

In general we will drop the indices of the spaces when they can be understood from the context. Continuing the study of function space we now see that $eval: (\mathbf{D} \rightarrow \mathbf{E}) \times \mathbf{D} \rightarrow \mathbf{E}$ is computable w.r.t. $(\varepsilon \rightarrow \varepsilon') \times \varepsilon, \varepsilon'$. One way is to note that

$$\begin{aligned} eval &= \bigsqcup \{ \langle \varepsilon_m \Rightarrow \varepsilon'_n, \varepsilon_m \rangle \Rightarrow \varepsilon'_n \} \\ &= \bigsqcup \{ \langle \beta_{\langle m, n \rangle}, \varepsilon_m \rangle \Rightarrow \varepsilon'_n \} \\ &= \bigsqcup \{ \langle (\varepsilon \rightarrow \varepsilon')_{\llbracket m, n \rrbracket}, \varepsilon_m \rangle \Rightarrow \varepsilon'_n \} \\ &= \bigsqcup \{ \langle ((\varepsilon \rightarrow \varepsilon') \times \varepsilon)_k \Rightarrow \varepsilon'_n \mid k = \langle \llbracket m, n \rrbracket, m \rangle \} \\ &= \bigsqcup \{ \langle (((\varepsilon \rightarrow \varepsilon') \times \varepsilon) \rightarrow \varepsilon')_k \mid k = \langle \langle \llbracket m, n \rrbracket, m \rangle, n \rangle \} \} \end{aligned}$$

and use Exercise 1, p. 72. Another way is to use the effective nature of Theorem 1. Of course this is all effective and we have a recursive *eval* with

$$eval = \zeta_{eval(i,j)}.$$

Now let F be eff. given w.r.t. ε'' and with index k . We check that if $f: (\mathbf{F} \times \mathbf{D}) \rightarrow \mathbf{E}$ is computable w.r.t. $\varepsilon'' \times \varepsilon, \varepsilon'$ then $Curry(f): \mathbf{F} \rightarrow (\mathbf{D} \rightarrow \mathbf{E})$ is computable w.r.t. $\varepsilon'', (\varepsilon \Rightarrow \varepsilon')$. For $Curry(f) = \bigsqcup \{\varepsilon_k'' \Rightarrow (\varepsilon_l \Rightarrow \varepsilon_m') \mid \varepsilon_m' \sqsubseteq f(\varepsilon_k'', \varepsilon_l)\}$. This is all effective in $Graph(f)$ and we see that $Curry$ itself is computable (e.g. use Theorem 1). Finally we see there is a recursive curry with

$$Curry = \zeta_{Curry(i,j,k)}.$$

Now we also see there is a recursive $\rightarrow$ s.t. if $\mathbf{D}', \mathbf{D}, \mathbf{E}, \mathbf{E}'$ are eff. given w.r.t. $\varepsilon'_0, \varepsilon_0, \varepsilon_1, \varepsilon'_1$ and indices i, j, k, l then for any computable $f: \mathbf{D}' \rightarrow \mathbf{D}, g: \mathbf{E} \rightarrow \mathbf{E}'$ w.r.t. $\varepsilon'_0, \varepsilon_0$ and $\varepsilon_1, \varepsilon'_1$ and indices a, b we have:

$$f \rightarrow g = \zeta_{a \rightarrow_{i,j,k,l} b}.$$

Exercises

1 Combinatory Logic. Given $\mathbf{C}, \mathbf{D}, \mathbf{E}$, eff. given w.r.t. $\varepsilon, \varepsilon', \varepsilon''$ show $K: \mathbf{D} \rightarrow (\mathbf{E} \rightarrow \mathbf{D}), S: (\mathbf{C} \rightarrow \mathbf{D} \rightarrow \mathbf{E}) \rightarrow (\mathbf{C} \rightarrow \mathbf{D}) \rightarrow (\mathbf{C} \rightarrow \mathbf{E})$ by demonstrating the formulae:

$$\begin{aligned} K &= \bigsqcup \{\varepsilon_m' \Rightarrow (\varepsilon_n'' \Rightarrow \varepsilon_m')\}, \\ S &= \bigsqcup \{(\varepsilon_l \Rightarrow \varepsilon_m' \Rightarrow \varepsilon_n'') \Rightarrow (\varepsilon_l \Rightarrow \varepsilon_m') \Rightarrow (\varepsilon_l \Rightarrow \varepsilon_n'')\}. \end{aligned}$$

What is the formula for $Y: (\mathbf{D} \rightarrow \mathbf{D}) \rightarrow \mathbf{D}$?

2 λ -calculus. Show that all typed λ -calculus expressions denote computable functions of their free variables.

3 Smash Product and Strict Function Space. Treat the smash product $\mathbf{D} \otimes \mathbf{E}$ of effectively given domains $\mathbf{D}, \mathbf{E}$ w.r.t. $\varepsilon, \varepsilon'$ using the enumeration:

$$\varepsilon \otimes \varepsilon'_{(m,n)} = \varepsilon_m \otimes \varepsilon'_n.$$

showing that $\mathbf{D} \otimes \mathbf{E}$ is eff. given w.r.t. $\varepsilon \otimes \varepsilon'$ and $\otimes: \mathbf{D} \times \mathbf{E} \rightarrow \mathbf{D} \otimes \mathbf{E}, \iota: \mathbf{D} \otimes \mathbf{E} \rightarrow \mathbf{D} \times \mathbf{E}$ being computable (and all indices are effectively obtainable from indices for $\mathbf{D}$ and $\mathbf{E}$).

Treat the strict function space $\mathbf{D} \rightarrow_{\perp} \mathbf{E}$ w.r.t. the enumeration

$$\beta_{\perp(m,n)} = \varepsilon_m \Rightarrow \varepsilon'_n.$$

Show that by considering appropriate formulae that the following functions are all computable: $\otimes: [(\mathbf{D} \rightarrow_{\perp} \mathbf{E}) \otimes (\mathbf{D}' \rightarrow_{\perp} \mathbf{E}')] \rightarrow (\mathbf{D} \otimes \mathbf{D}' \rightarrow_{\perp} \mathbf{E} \otimes \mathbf{E}')$ and $eval: (\mathbf{D} \rightarrow_{\perp} \mathbf{E}) \otimes \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ and $Curry: (\mathbf{F} \otimes \mathbf{D} \rightarrow_{\perp} \mathbf{E}) \rightarrow_{\perp} (\mathbf{F} \rightarrow_{\perp} \mathbf{D} \rightarrow_{\perp} \mathbf{E})$ and $\rightarrow_{\perp}: (\mathbf{D}' \rightarrow_{\perp} \mathbf{D}) \times (\mathbf{E} \rightarrow_{\perp} \mathbf{E}') \rightarrow (\mathbf{D} \rightarrow_{\perp} \mathbf{E}) \rightarrow_{\perp} (\mathbf{D}' \rightarrow_{\perp} \mathbf{E}')$ and that all indices are effectively obtainable from indices for $\mathbf{D}, \mathbf{E}, \mathbf{D}', \mathbf{E}', \mathbf{F}$.

4. Let $\mathbf{D}, \mathbf{E}$ be eff. given and show that each element of the projection pair $(\mathbf{D} \rightarrow_{\perp} \mathbf{E}) \rightarrow (\mathbf{D} \rightarrow \mathbf{E}) \rightarrow (\mathbf{D} \rightarrow_{\perp} \mathbf{E})$ given in Chapter 4 is computable. Conclude $f: \mathbf{D} \rightarrow_{\perp} \mathbf{E}$ is computable iff it is computable as an element of $\mathbf{D} \rightarrow \mathbf{E}$.

5. Show that if we turn $\times, \rightarrow$ into functors on $\mathcal{CPO}_{\perp}$ then they still act computably on morphisms.

6 Disjoint Sum. Let $\mathbf{D}, \mathbf{E}$ be eff. given domain w.r.t. $\varepsilon, \varepsilon'$. Show that $\mathbf{D} + \mathbf{E}$ is eff. given w.r.t. $\varepsilon + \varepsilon'$ where:

$$(\varepsilon + \varepsilon')_m = \begin{cases} inl(\varepsilon_n) & \text{(if } m = 2n), \\ inr(\varepsilon'_n) & \text{(if } m = 2n + 1). \end{cases}$$

Show that $inl, inr, outl, outr, isl, isr, [\cdot, \cdot], +, \nabla$ are all computable by demonstrating such appropriate formulae as:

$$\nabla = \bigsqcup (\{inl(\varepsilon_m) \Rightarrow_{\perp} \varepsilon_m\} \cup \{inr(\varepsilon'_n) \Rightarrow_{\perp} \varepsilon'_n\})$$

and note everything is recursive in the indices of $\mathbf{D}$ and $\mathbf{E}$.

7 Lifting. Treat lifting.

8. Show that $x \in \mathbf{D} \otimes \mathbf{E}$ is computable iff $(x)_0$ and $(x)_1$ are; that $x \in \mathbf{D} + \mathbf{E}$ is computable iff it is $\perp$ or $\text{inl}(d)$ for a computable d in $\mathbf{D}$ or $\text{inr}(e)$ for a computable e in $\mathbf{E}$; that $x \in \mathbf{D}_\perp$ is computable iff it is $\perp$ or $\text{up}(d)$ for a computable d in $\mathbf{D}$. What about $\mathbf{D}^*$?

9. Treat iterated finite sums and smash products and effective denumerable sums and smash products.

10. Treat the $(\cdot)^*$ functor.

11. Let $\mathbf{D}$ be effectively given w.r.t. ε and let $r: \mathbf{D} \rightarrow \mathbf{D}$ be a closure operation with recursive graph. Show $\mathbf{Fix}_r$ is effectively given w.r.t. $\varepsilon_m^R = r(\varepsilon_m)$. What happens if r is only computable?

12. Show that the computable $f: \mathbf{N}^{\otimes} \rightarrow \perp \mathbf{N}$ correspond exactly to the pr. functions of m arguments.

13. Show the computable $f: \mathbf{A} \rightarrow \mathbf{B}$ where $\mathbf{A}, \mathbf{B}$ are $\mathbf{N}^k, (\mathbf{T}^\omega)^k, (\mathbf{Tapes})^k, \dots$ etc. are definable in the typed λ -calculus using *parcond* (or, equally, parallel *or*). What difficulties do you find if you try to define the computable functions $f: \mathbf{N}^k \rightarrow \mathbf{N}$ which are Milner-Vuillemin sequential? Find a notion of effective sequentiality and show all of the effectively sequential $f: \mathbf{N}^k \rightarrow \mathbf{N}$ can be defined with parallel *or* (or equally *parcond*).

Easy Open Question. What basic stable functions (e.g. no *parcond*) are needed to define all computable stable $f: \mathbf{N}^k \rightarrow \mathbf{N}$? Perhaps you will need a notion of effective stability.

14. Define *type symbols*, σ , recursively by saying: o, ι are symbols and $(\sigma \times \tau)$ and $(\sigma \rightarrow \tau)$ are if σ, τ are. Let $\mathbf{D}_o = \mathbf{T}$, $\mathbf{D}_\iota = \mathbf{N}$ and $\mathbf{D}_{\sigma \times \tau} = \mathbf{D}_\sigma \times \mathbf{D}_\tau$ and $\mathbf{D}_{\sigma \rightarrow \tau} = \mathbf{D}_\sigma \rightarrow \mathbf{D}_\tau$. Show that every computable element of $\mathbf{D}_\sigma$ is definable by the typed λ -calculus using *parcond* and the (computable!) function $\exists: \mathbf{D}_{(\iota \rightarrow \iota) \rightarrow o}$ where:

$$\exists f = \begin{cases} tt & (\exists n. f(n) = tt), \\ ff & (f(\perp) = ff). \end{cases}$$

[Warning This is rather hard. See “Plotkin, LCF considered as a programming language, TCS, 5, pp. 223–255” for ideas.]

What we have done so far is show that all our constructions $\times, \rightarrow, \otimes, \rightarrow_\perp, +, (\cdot)_\perp$ are *computable*.

Definition. Let $T: \mathcal{CPO}_\perp^m \times \mathcal{CPO}_\perp^n \rightarrow \mathcal{CPO}_\perp$ be a locally continuous functor contravariant in its first m arguments and covariant in its last n arguments. It is *computable* if there is a function ε_T on enumerations and two recursive functions both called T , such that:

- (1) If $\mathbf{D}_0, \dots, \mathbf{D}_{m-1}, \mathbf{E}_0, \dots, \mathbf{E}_{n-1}$ are eff. given w.r.t. $\varepsilon_0, \dots, \varepsilon_{m-1}, \varepsilon'_0, \dots, \varepsilon'_{n-1}$ and have indices $i_0, \dots, i_{m-1}, j_0, \dots, j_{n-1}$ then $T(\mathbf{D}_0, \dots, \mathbf{D}_{m-1}, \mathbf{E}_0, \dots, \mathbf{E}_{n-1})$ is eff. given w.r.t. $\varepsilon_T(\varepsilon_0, \dots, \varepsilon_{m-1}, \varepsilon'_0, \dots, \varepsilon'_{n-1})$ with index $T(i_0, \dots, i_{m-1}, j_0, \dots, j_{n-1})$.
- (2) If $\mathbf{D}'_k, \mathbf{E}_l, \mathbf{D}_k, \mathbf{E}'_l$ are eff. given w.r.t. $\varepsilon'_k, \bar{\varepsilon}_l, \varepsilon_k, \bar{\varepsilon}'_l$ and indices $i'_k, \bar{i}_l, i_k, \bar{i}'_l$ then if $f_k: \mathbf{D}'_k \rightarrow \mathbf{D}_k, g_l: \mathbf{E}_l \rightarrow \mathbf{E}'_l$ are computable with indices a_k, b_k we have that $T(\dots f_k \dots, \dots g_l \dots)$ is computable w.r.t. $\varepsilon_T(\dots \varepsilon_k \dots, \dots \bar{\varepsilon}_l \dots), \varepsilon_T(\dots \varepsilon'_k \dots, \dots \bar{\varepsilon}'_l \dots)$ with index $T_{\dots i'_k \dots, \dots \bar{i}_l \dots, \dots i_k \dots, \dots \bar{i}'_l \dots}(\dots a_k \dots, \dots b_l \dots)$.

Clearly too the projection functors and constant functors $K_{\mathbf{D}}$, with $\mathbf{D}$ eff. given, are computable in this sense.

Embeddings

Unfortunately it is not enough to take the computable embeddings to be the embeddings which are computable as their right adjoints need not be computable (see Exercise 1 below).

Definition. Let $\mathbf{D}, \mathbf{E}$ be eff. given w.r.t. $\varepsilon, \varepsilon'$. An embedding $i: \mathbf{D} \triangleleft \mathbf{E}$ is *computable* w.r.t. $\varepsilon, \varepsilon'$ and with index $\langle a, b \rangle$ iff a is an index of i w.r.t. $\varepsilon, \varepsilon'$ (as a computable function) and b is an index of i^R w.r.t. $\varepsilon', \varepsilon$.

Exercises

1 Category.

- (1) For any eff. given $\mathbf{D}$ the embedding $\text{id}: \mathbf{D} \triangleleft \mathbf{D}$ is computable, an index being eff. obtainable from one for $\mathbf{D}$.
- (2) For any eff. given $\mathbf{D}, \mathbf{E}, \mathbf{F}$ w.r.t. $\varepsilon, \varepsilon', \varepsilon''$ if $f: \mathbf{D} \triangleleft \mathbf{E}, g: \mathbf{E} \triangleleft \mathbf{F}$ are computable embedding so is $g \circ f$; indeed there is a recursive function, $\circ$, such that if a is an index for f , w.r.t. $\varepsilon, \varepsilon'$ and b is an index for g w.r.t. $\varepsilon', \varepsilon''$ then $b \circ a$ is one for $g \circ f$ w.r.t. $\varepsilon, \varepsilon''$ and further an index for $\circ$ is effectively obtainable from ones for $\mathbf{D}, \mathbf{E}, \mathbf{F}$.

2 Initial Object. For any eff. given $\langle \mathbf{D}, \varepsilon \rangle$ the embedding $\perp: \mathbf{U} \triangleleft \mathbf{D}$ is computable, an index being effectively obtainable from one of $\mathbf{D}$.

3. All the examples of embeddings in Chapter 4 are computable.

Theorem 2. Let $i: \mathbf{D} \triangleleft \mathbf{E}$ be an embedding of eff. given domains $\mathbf{D}, \mathbf{E}$ w.r.t. $\varepsilon, \varepsilon'$. Then i is computable iff for some recursive r we have that $\langle r, i \rangle: \langle \mathbf{D}, \varepsilon \rangle \rightarrow \langle \mathbf{D}, \varepsilon' \rangle$ i.e. that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{N} & \xrightarrow{r} & \mathbf{N} \\ \varepsilon \downarrow & & \downarrow \varepsilon' \\ \mathbf{D} & \xrightarrow{i} & \mathbf{D}' \end{array}$$

Proof. $\Rightarrow$) We know that for any finite a in $\mathbf{D}$ we have $i(a)$ finite. Now:

$$\begin{aligned} i(a) = b &\equiv (b \subseteq i(a)) \wedge (i(a) \subseteq b) \\ &\equiv (b \subseteq i(a)) \wedge (a \subseteq i^R(b)). \end{aligned}$$

So as i and i^R are computable, the relation $i(\varepsilon_m) = \varepsilon'_n$ is re. in m and n . But then it must be recursive as given m and n one searches for (and finds) a k with $i(\varepsilon_m) = \varepsilon'_k$ and checks if $\varepsilon_k = \varepsilon'_n$.

$\Leftarrow$) We have $\varepsilon'_n \subseteq i(\varepsilon_m) \equiv \varepsilon'_n \subseteq \varepsilon'_{r(m)}$ which is re. and also $\varepsilon_n \subseteq i^R(\varepsilon'_m)$ iff $i(\varepsilon_n) \subseteq \varepsilon'_m \equiv \varepsilon'_{r(n)} \subseteq \varepsilon'_m$ which is re. ■

Note the proof of the equivalence is effective.

Exercise

1. Find an embedding $i: \mathbf{0}^\omega \rightarrow (\mathbf{0}_\perp)^\omega$ which is computable as a function but not as an embedding. [Hint Use the two embeddings of $\mathbf{0}$ in $\mathbf{0}_\perp$.] Is it the case the i^R computable implies i computable, in general? If not is the condition $\text{Graph}(i^R)$ recursive strong enough?

Definition. Let $F: (\mathcal{CPO}^E)^n \rightarrow \mathcal{CPO}^E$ be a covariant functor. It is *computable* if there is a function ε_F on enumerations and two recursive functions both called F such that

- (1) If $\dots \mathbf{D}_k \dots$ are eff. given w.r.t. $\dots \varepsilon_k \dots$ and indices $\dots i_k \dots$ then $F(\dots \mathbf{D}_k \dots)$ is eff. given w.r.t. $\varepsilon_F(\dots \varepsilon_k \dots)$ and index $F(\dots i_k \dots)$.
- (2) If $\dots \mathbf{D}_k \dots, \dots \mathbf{E}_k \dots$ are eff. given w.r.t. $\dots \varepsilon_k \dots, \dots \varepsilon'_k \dots$ and indices $\dots i_k \dots, \dots j_k \dots$ then if $\dots f_k: \mathbf{D}_k \triangleleft \mathbf{E}_k \dots$ are computable embeddings w.r.t. $\dots \varepsilon_k \dots, \dots \varepsilon'_k \dots$ with indices $\dots a_k \dots$ then $F(\dots f_k \dots)$ is computable w.r.t. $\varepsilon_F(\dots \varepsilon_k \dots), \varepsilon_F(\dots \varepsilon'_k \dots)$ and has index $F(\dots i_k \dots, \dots j_k \dots)(\dots a_k \dots)$.

Clearly if $T: \mathcal{CPO}_\perp^m \times \mathcal{CPO}_\perp^n \rightarrow \mathcal{CPO}_\perp$ is computable so is $T^E: (\mathcal{CPO}^E)^{m+n} \rightarrow \mathcal{CPO}^E$; further any composition of computable functors over $\mathcal{CPO}^E$ is also computable. We leave the proof of this assertion to the reader (for simplicity he should just consider the case $m = n = 1$).

Direct Limits

Definition. Let $\Delta = \langle \mathbf{D}_m, \varepsilon_m, f_m \rangle$ be a chain of eff. given domains $\mathbf{D}_m$ w.r.t. ε_m , and with $f_m: \mathbf{D}_m \triangleleft \mathbf{D}_{m+1}$ being computable embeddings w.r.t. $\varepsilon_m, \varepsilon_{m+1}$. Then Δ is *effective*, with index δ , if $g = \{\delta\}$ is a recursive function such that

- (1) $\pi_0 \circ g(m)$ is an index of $\mathbf{D}_m$ w.r.t. ε_m .
- (2) $\pi_1 \circ g(m)$ is an index of f_m w.r.t. $\varepsilon_m, \varepsilon_{m+1}$.

Definition. Let $\Delta = \langle \mathbf{D}_m, \varepsilon_m, f_m \rangle$ be an effective chain. Then an effective cone $\rho: \Delta \rightarrow \langle \mathbf{D}, \varepsilon \rangle$ from Δ to an effectively given $\mathbf{D}$ w.r.t. ε is a cone $\rho: \langle \mathbf{D}_m, f_m \rangle \rightarrow \mathbf{D}$ with $\rho_m: \mathbf{D}_m \triangleleft \mathbf{D}$ computable w.r.t. $\varepsilon_m, \varepsilon$ and such that:

- (1) d is an index $\mathbf{D}$ w.r.t. ε .
- (2) For any m , $\{\rho\}(m)$ is an index of $\mathbf{D}_m$ w.r.t. ε_m .

Definition. Let $\Delta = \langle \mathbf{D}_m, \varepsilon_m, f_m \rangle$ be an effective chain, and let $\rho: \Delta \rightarrow \langle \mathbf{D}, \varepsilon \rangle$ be an effective cone. Then ρ is *effectively universal* iff $\rho: \langle \mathbf{D}_m, f_m \rangle \rightarrow \mathbf{D}$ is universal and if $\rho': \Delta \rightarrow \langle \mathbf{D}', \varepsilon' \rangle$ is any effective cone there is a mediating computable $\theta: \mathbf{D} \triangleleft \mathbf{D}'$ w.r.t. $\varepsilon, \varepsilon'$ and an index for θ is effectively obtainable from one for ρ' (and an index for ρ is $\langle a, b \rangle$ where a is an index for ρ as an effective cone and b is an index s.t. if m is an index for ρ' then $\{a\}(m)$ is one for θ w.r.t. $\varepsilon, \varepsilon'$).

Theorem 3. Let $\Delta = \langle \mathbf{D}_m, \varepsilon_m, f_m \rangle$ be an effective chain and let $\rho: \langle \mathbf{D}_m, f_m \rangle \rightarrow \mathbf{D}$ be universal (see Chapter 4). Define $\varepsilon: \mathbf{N} \rightarrow B_{\mathbf{D}}$ by:

$$\varepsilon_{\langle m, n \rangle} = \rho_m(\varepsilon_m(n))$$

(see Chapter 5). Then $\rho: \Delta \rightarrow \langle \mathbf{D}, \varepsilon \rangle$ is effectively universal and an index for ρ is effectively obtainable from one for Δ .

Proof. First we check that $\mathbf{D}$ is effectively given w.r.t. ε by the formulae:

$$\begin{aligned} 1. \quad \varepsilon_{\langle k, l \rangle} \uparrow \varepsilon_{\langle m, n \rangle} &\equiv \rho_k(\varepsilon_k(l)) \uparrow \rho_m(\varepsilon_m(n)) \\ &\equiv \rho_r[f_{kr}(\varepsilon_k(l))] \uparrow \rho_r[f_{mr}(\varepsilon_m(n))] && \text{where } r = \max(k, m) \\ &\equiv \rho_r[\varepsilon_r(g_{kr}(l))] \uparrow \rho_r[\varepsilon_r(g_{mr}(n))] \\ &= \varepsilon_r(g_{kr}(l)) \uparrow \varepsilon_r(g_{mr}(n)) \end{aligned}$$

(where $g_m: \mathbf{N} \rightarrow \mathbf{N}$ is the recursive function for $f_m: \mathbf{D}_m \triangleleft \mathbf{D}_{m+1}$ guaranteed effectively obtainable from any index of f_m by Theorem 2, and g_{ln} is an evident product of g_m 's).

2. If b is a code of $\perp_{\mathbf{D}_0}$ then $\langle 0, b \rangle$ is one of $\perp_{\mathbf{D}}$.

$$\begin{aligned} 3. \quad \varepsilon_{\langle i, j \rangle} &= \varepsilon_{\langle k, l \rangle} \sqcup \varepsilon_{\langle m, n \rangle} \\ &\equiv \rho_i(\varepsilon_i(j)) \\ &= \rho_k(\varepsilon_k(l)) \sqcup \rho_m(\varepsilon_m(n)) \\ &\equiv \rho_r(\varepsilon_r(g_{ir}(j))) \\ &= \rho_r(\varepsilon_r(g_{kr}(l))) \sqcup \rho_r(\varepsilon_r(g_{mr}(n))) && \text{where } r = \max(i, k, m) \\ &\equiv \varepsilon_r(g_{ir}(j)) \\ &= \varepsilon_r(g_{kr}(l)) \sqcup \varepsilon_r(g_{mr}(n)) \end{aligned}$$

and so, clearly, an index for $\mathbf{D}$ w.r.t. ε is effectively obtainable from one for Δ .

To see that $\rho_m: \mathbf{D}_m \rightarrow \mathbf{D}$ is computable w.r.t. $\varepsilon_m, \varepsilon$ just note that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{N} & \xrightarrow{\langle m, - \rangle} & \mathbf{N} \\ \varepsilon_m \downarrow & & \downarrow \varepsilon \\ \mathbf{D}_m & \xrightarrow{\rho_m} & \mathbf{D} \end{array}$$

and apply Theorem 2. We have now essentially seen that ρ is an effective cone and an index for it is effectively obtainable from one for Δ .

Now let $\rho': \Delta \rightarrow \langle \mathbf{D}', \varepsilon' \rangle$ be any other effective cone. The mediating $\theta: \mathbf{D} \triangleleft \mathbf{D}'$ is given by the formula:

$$\theta = \bigsqcup \rho'_n \circ \rho_n^R$$

and by Exercises 5, p. 73, 1, p. 78 and since ρ' and ρ are effective the function θ is computable. Equally the formula $\theta^R = \bigsqcup \rho_n \circ \rho_n'^R$ shows θ^R to be computable and we see that θ is a computable embedding with index effectively obtainable from ρ' . Thus ρ is effectively universal with an index effectively obtainable from any one for Δ . ■

What this theorem says is that the inverse limit construction, $\lim_{\leftarrow}$ is effective when viewed as a function from effective chains to universal cones.

Theorem 4 Continuity of Functors. Let $T: \mathcal{CPQ}^E \rightarrow \mathcal{CPQ}^E$ be a computable functor. Then it is *effectively continuous* in that if $\rho: \langle \mathbf{D}, \varepsilon_m, f_m \rangle \rightarrow \langle \mathbf{D}, \varepsilon \rangle$ is effectively universal so is $T(\rho): \langle T(\mathbf{D}), \varepsilon_T(\varepsilon_m), T(f_m) \rangle \rightarrow \langle T(\mathbf{D}), \varepsilon_T(\varepsilon) \rangle$ and an index for $T(\rho)$ is effectively obtainable from one for ρ .

Proof. This is an effective version of Theorem 2, Chapter 4 and is left to the reader. ■

Algebras

The end of our long haul is now in sight! In what follows we let $T: \mathcal{CPO}^E \rightarrow \mathcal{CPO}^E$ be a computable functor.

Definition.

- (1) A *computable* T -algebra with index $\langle a, b \rangle$ is $\langle \mathbf{D}, \varepsilon, \alpha \rangle$ where $\langle \mathbf{D}, \alpha \rangle$ is a T -algebra, $\mathbf{D}$ is eff. given w.r.t. ε with index a and α is computable w.r.t. $\varepsilon_T(\varepsilon)$, ε with index b .
- (2) An *effectively-initial* T -algebra with index $\langle a, b \rangle$ is a computable T -algebra $\langle \mathbf{D}, \varepsilon, \alpha \rangle$ with index a and with $\langle \mathbf{D}, \alpha \rangle$ initial such that for any computable T -algebra $\langle \mathbf{E}, \varepsilon', \beta \rangle$ with index a' the unique T -homomorphism, $\theta: \langle \mathbf{D}, \beta' \rangle \rightarrow \langle \mathbf{E}, \beta \rangle$ is computable w.r.t. $\varepsilon, \varepsilon'$ and has index $\{b\}(a')$.

Theorem 5 Effective Initial Solution of Recursive Domain Equations. *The initial algebra $\langle \mathbf{Fix}_T, \eta_T \rangle$ is effectively initial (for a suitable ε).*

Proof. As $\mathbf{U}$ is eff. given w.r.t. the evident ε^u and by Exercise 2, p. 79 and as T is computable, the chain $\Delta = \langle T^n(\mathbf{U}), \varepsilon_T(\varepsilon^u), T^n(\perp) \rangle$ is effective. Let $\rho: \langle T^n(\mathbf{U}), T^n(\perp) \rangle \rightarrow \mathbf{D}$ be universal and take ε as in Theorem 3. Then $\rho: \Delta \rightarrow \langle \mathbf{D}, \varepsilon \rangle$ is effectively universal. Then $T\rho: T\Delta \rightarrow \langle T\mathbf{D}, \varepsilon_T(\varepsilon) \rangle$ is also effectively universal, by Theorem 4. Further $\eta_T = \bigsqcup_n \rho_{n+1} \circ (T\rho_n)^R$ and so η_T is computable w.r.t. $\varepsilon_T(\varepsilon)$, ε . Thus $\langle \mathbf{Fix}_T, \varepsilon, \eta_T \rangle$ is a computable T -algebra. Let $\langle \mathbf{D}, \varepsilon, \alpha' \rangle$ be any other one. Then following the proof of Theorem 1, Chapter 5 define $\rho'_m: T^n(\mathbf{U}) \triangleleft \mathbf{D}$ by $\rho'_0 = \perp$ and $\rho'_{m+1} = \alpha \circ T(\rho'_m)$. Then as in Theorem 1 we see ρ' is a cone from $\langle T^n(\mathbf{U}), T^n(\perp) \rangle$ to $\mathbf{D}$ and it is easily seen (by induction on n) to be an effective cone from Δ to $\langle \mathbf{D}, \varepsilon \rangle$. But $\rho: \Delta \rightarrow \langle \mathbf{D}, \varepsilon \rangle$ is effectively universal and so we can obtain an index for the mediating computable $\theta: \mathbf{Fix}_T \rightarrow \mathbf{D}$ effectively from one for ρ and hence from one for $\langle \mathbf{D}, \varepsilon \rangle$. But, as in the proof of Theorem 1, this θ is the required mediating T -homomorphism and we have seen that $\langle \mathbf{Fix}_T, \varepsilon, \eta_T \rangle$ is indeed effectively initial. ■

In fact this proof is also effective and, with suitable definitions, one obtains an index for the effectively-initial T -algebra from one for T . So: taking effectively-initial fixed-points of computable functors is itself an effective operation.

The above theorem easily extends to multiple fixed-points and we see that all the domains and functions we have considered in the notes are computable and so whatever the programming language for any program, P , and semantic function, M , defined by our methods we have that $M(P)$ is computable. This gives the *possibility* of an operational semantics. For example if $\mathbf{S} = \mathbf{Id} \rightarrow_\perp \mathbf{N}$ and $M(P): \mathbf{S} \rightarrow_\perp \mathbf{S}$ we have that given an index for σ we get one for $M(P)(\sigma)(x)$ for any variable x and the required integer (if it exists) can be found by running a Turing Machine on this index.

However this is all only of theoretical interest. It would be more useful to patch up the various spotty results on definability mentioned in the exercises by showing that some (formally defined) metalanguage can define exactly the computable elements and that the language has a good operational semantics (as in, for example, the LCF paper) and then one can apply Mosses' ideas to write a *complete* compiler-compiler.

Exercises

1. Give an effective version of Theorem 3, p. 54, Chapter 5.
2. Is there anything more one can say along the lines of Theorem 4, p. 55, Chapter 5 by considering effectiveness?
3. It is boring to continually mention indices. Can you formulate a metatheoretical assertion to the effect that if a proof is effective (in a suitable intuitionistic logic) then one obtains, via a Kleene realizability, a suitable statement on indices?

8. Nondeterminism and Parallelism

We show how to extend our theory to deal with problems of nondeterminism and parallelism. The interest of the latter is clear; that of the former arises when we try to reduce parallelism to nondeterminism and the idea is to explain a parallel construct $c \parallel c'$ in terms of the possible *atomic actions* of c and c' and the set of their *interleavings*. It is surprising how many difficulties arise and we do not regard the theory as settled.

Discrete Powerdomains

Suppose we enrich the usual simple imperative language with a nondeterministic choice construct, $c \text{ or } c'$. The idea is that an execution of $c \text{ or } c'$ consists of a choice of c and an execution of c or a choice of c' and an execution of c' . (This choice may be made by an implementation or forced by some external circumstance.) We regard $c \text{ or } c'$ as correct, terminating (etc.). Now consider the following three programs (where $\text{loop} \stackrel{\text{def}}{=} \text{while true do sleep}$)

- (1) $x := 1$,
- (2) $x := 1 \text{ or loop}$,
- (3) loop .

Which should be considered equivalent? One view is that none are since (1) always terminates unlike (2) & (3) and (2) can terminate unlike (3). Another is that (1) = (2) $\neq$ (3) since (1) and (2) give the same set of results, if any. Another is that (1) $\neq$ (2) = (3) since (2) and (3) can both (unlike (1)) fail to terminate and so nothing can be guaranteed of either of them. All three views have their place: the first is “what actually happens”, the second for partial correctness properties (Hoare), the third for total correctness (Dijkstra). We shall give semantics for all three.

The Relational View

To this end let $\mathbf{S}$ be the countable set of states and suppose we want a semantics for the second view (the simplest case). Then for a given initial state σ a program c should give us a *set* $X \subseteq \mathbf{S}$ of final states (since we do not wish to record nontermination). Thus we could expect

$$\mathcal{C}[[c]]: \mathbf{S} \rightarrow \mathcal{P}(\mathbf{S})$$

(where, of course, $\mathcal{P}(\mathbf{S})$ is the set of all subsets of $\mathbf{S}$). That is we use a *nondeterministic function* for our semantics. And it will also be natural to order $\mathcal{P}(\mathbf{S})$ by subset and use the induced pointwise order on $\mathcal{C}[[c]]$.

Exercise 1. Show that

$$\mathbf{S} \rightarrow \mathcal{P}(\mathbf{S}) \cong \mathcal{P}(\mathbf{S} \times \mathbf{S})$$

The set $\mathcal{P}(\mathbf{S} \times \mathbf{S})$ is the set of all *relations* over $\mathbf{S}$; try rephrasing what we do below in terms of these relations.

In order to compare nondeterministic functions we note a very important fact which will prove a reliable guide in more general situations.

Fact 1. *Let X and Y be countable sets. Then for any $f: X \rightarrow \mathcal{P}(Y)$ there is a unique strict continuous, additive $f^\dagger: \mathcal{P}(X) \rightarrow_{\text{SA}} \mathcal{P}(Y)$ (called the extension of f) such that the following diagram commutes.*

$$\begin{array}{ccc} X & & \\ \downarrow \{\cdot\} & \searrow f & \\ \mathcal{P}(X) & \xrightarrow{f^\dagger} & \mathcal{P}(Y) \end{array}$$

Further $(\cdot)^\dagger: (X \rightarrow \mathcal{P}(Y)) \cong \mathcal{P}(X) \rightarrow_{\text{SA}} \mathcal{P}(Y)$ is an isomorphism of partial orders.

Proof. Suppose $g: \mathcal{P}(X) \rightarrow_{\text{SA}} \mathcal{P}(Y)$ makes the diagram commute.

$\Leftarrow$) Then for any $A \subseteq X$ we have

$$\begin{aligned} g(A) &= g\left(\bigcup_{a \in A} \{a\}\right) \\ &= \bigcup_{a \in A} g(\{a\}) && g \text{ is strict, additive and continuous and } A \subseteq X \text{ is countable} \\ &= \bigcup_{a \in A} f(a) \end{aligned}$$

showing uniqueness. Conversely using this formula to define $f^\dagger$ we see that $f^\dagger$ is clearly continuous and additive and makes the diagram commute as:

$$f^\dagger(\{a\}) = \bigcup_{a' \in \{a\}} f(a') = f(a).$$

It follows at once that $\dagger$ is a bijection with inverse $\lambda g. g \circ \{\cdot\}$ and as both $\dagger$ and its inverse are evidently monotonic they are therefore isomorphisms (of partial orders). ■

Exercise 2. Where was the countability of X and Y used in the above (if at all)? Can you extend the definitions of nondeterministic functions so that Fact 1 holds for arbitrary X ?

Exercise 3. Noting that $X \rightarrow \mathcal{P}(Y) \cong X_\perp \rightarrow_\perp \mathcal{P}(Y)$ can you provide a variant of Fact 1 to apply to nondeterministic functions of type $X_\perp \rightarrow \mathcal{P}(Y)$?

Exercise 4. Show using Exercise 1 and Fact 1 that there is an isomorphism $\Lambda: (X \rightarrow \mathcal{P}(Y)) \cong \mathcal{P}(Y) \rightarrow_{\text{SA}} \mathcal{P}(X)$ given by:

$$\Lambda(m)(B) = \{a \in X \mid m(a) \cap B \neq \emptyset\}.$$

Definition 1. For any function $f: \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$, its *dual* $\tilde{f}: \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$ is defined by:

$$\tilde{f}(B) = X \setminus f(Y \setminus B).$$

Exercise 5. Show that $\tilde{\tilde{f}} = f$, that $f \subseteq g$ iff $\tilde{f} \supseteq \tilde{g}$, that f is monotonic, continuous, additive, ... iff $\tilde{f}$ is monotonic, dual-continuous, multiplicative, ... Conclude that $(\cdot): [\mathcal{P}(Y) \rightarrow \mathcal{P}(X)] \rightarrow [\mathcal{P}(Y) \rightarrow \mathcal{P}(X)]$ is an isomorphism and cuts down to an isomorphism of continuous and dual continuous functions, etc., etc.

Exercise 6. For any $m: X \rightarrow \mathcal{P}(Y)$ and $B \subseteq Y$ we define the *weakest liberal precondition* $wlp(m, B)$ by

$$wlp(m, B) = \{a \in X \mid m(a) \subseteq B\}.$$

Show that $\lambda B. wlp(m, B)$ is the dual of $\Lambda(m)$.

Composition. Now we can compose two nondeterministic primitives $X \xrightarrow{f} \mathcal{P}(Y)$ and $Y \xrightarrow{g} \mathcal{P}(Z)$ by defining $f; g: X \rightarrow \mathcal{P}(Z)$ as

$$f; g = g^\dagger \circ f.$$

Exercise 7. Show that composition is associative with unit the singleton function. [Hint This is easy using Fact 1.]

Now consider the following little nondeterministic imperative language

$$c ::= a \mid \text{skip} \mid c; c \mid \text{if } b \text{ then } c \text{ else } c \mid \text{while } b \text{ do } c \mid c \text{ or } c$$

where we are given semantics $\mathcal{A}: \mathbf{ACom} \rightarrow (\mathbf{S} \rightarrow \mathbf{S})$ and $\mathcal{B}: \mathbf{BExp} \rightarrow (\mathbf{S} \rightarrow \mathbf{T})$ for atomic commands and Boolean expressions. Here is a semantics

$$\mathcal{C}: \mathbf{Com} \rightarrow (\mathbf{S} \rightarrow \mathcal{P}(\mathbf{S}))$$

for the language:

$$\begin{aligned} \mathcal{C}[a] &= \lambda \sigma \in \mathbf{S}. \{\mathcal{A}[a]\}; \\ \mathcal{C}[\text{skip}] &= \{\cdot\}; \\ \mathcal{C}[c_1; c_2] &= \mathcal{C}[c_1]; \mathcal{C}[c_2]; \\ \mathcal{C}[\text{if } b \text{ then } c_1 \text{ else } c_2] &= \lambda \sigma \in \mathbf{S}. \text{if } \mathcal{B}[b](\sigma) \text{ then } \mathcal{C}[c_1](\sigma) \text{ else } \mathcal{C}[c_2](\sigma); \\ \mathcal{C}[\text{while } b \text{ do } c] &= \mu m \in \mathbf{S} \rightarrow \mathcal{P}(\mathbf{S}). \lambda \sigma \in \mathbf{S}. \text{if } \mathcal{B}[b](\sigma) \text{ then } \mathcal{C}[c]; m \text{ else } \{\sigma\}; \\ \mathcal{C}[c_1 \text{ or } c_2] &= \lambda \sigma \in \mathbf{S}. \mathcal{C}[c_1](\sigma) \cup \mathcal{C}[c_2](\sigma). \end{aligned}$$

So all we need is singleton, extension (for composition) and binary union.

Exercise 8. Show that Cartesian Product, $\mathcal{P}(A) \times \mathcal{P}(B) \xrightarrow{\times} \mathcal{P}(A \times B)$ is strict continuous and additive in each argument. Show that for any $f: A \times B \rightarrow \mathcal{P}(C)$ there is a unique $f^{\dagger_2}: \mathcal{P}(A) \times \mathcal{P}(B) \rightarrow \mathcal{P}(C)$ which is strict, continuous and additive in each argument and which makes the following diagram commute

$$\begin{array}{ccc} A \times B & & \\ \downarrow \{\cdot\} \times \{\cdot\} & \searrow f & \\ \mathcal{P}(A) \times \mathcal{P}(B) & \xrightarrow{f^{\dagger_2}} & \mathcal{P}(C) \end{array}$$

Show too that $\dagger_2$ is given by: $f^{\dagger_2} = f^{\dagger} \circ \times$ (and so $\times = \{\cdot\}^{\dagger_2}$). What is the natural function from $\mathcal{P}(A \times B)$ to $\mathcal{P}(A) \times \mathcal{P}(B)$? What relationship do you thereby find between $\mathcal{P}(A \times B)$ and $\mathcal{P}(A) \times \mathcal{P}(B)$? Extend all this to products of several powersets.

Exercise 9. Let Σ be a finite alphabet. Consider the following little applicative language for denoting subsets of Σ^* :

$$e ::= a \mid \lambda \mid e.e \mid e \text{ or } e \mid \mu x.e \mid x$$

where a ranges over Σ , λ is the empty word, $e.e'$ is language product (the extension considered in the previous exercise of semigroup multiplication in Σ^*) and the rest is obvious. Show that the closed expressions give exactly the context-free languages. How would you alter the notation to obtain exactly the regular languages?

Exercise 10. Nondeterministic phenomena arise even in deterministic languages when we perform *dataflow analysis*. This exercise gives a *very* rough picture. Consider the simple language

$$c ::= a \mid \text{skip} \mid c; c \mid \text{if } b \text{ then } c \text{ else } c \mid \text{while } b \text{ do } c$$

where we are given $\mathcal{A}: \mathbf{ACom} \rightarrow (\mathbf{S} \rightarrow \mathbf{S})$ and $\mathcal{B}: \mathbf{BExp} \rightarrow (\mathbf{S} \rightarrow \mathbf{T})$ and define $\mathcal{C}: \mathbf{Com} \rightarrow (\mathbf{S} \rightarrow \mathbf{S}_{\perp})$ is usual. Suppose we try to represent $\mathbf{S}$ by a set $\mathbf{R}$ via a surjective *representation* function $r: \mathbf{S} \rightarrow \mathbf{R}$ (example $\mathbf{S} = \mathbf{N}$ and $\mathbf{R} = \{\text{"odd"}, \text{"even"}\}$). We regard the inverse of r as being of type $r^{-1}: \mathbf{R} \rightarrow \mathcal{P}(\mathbf{S})$. Now define abstractions $\mathcal{A}_R: \mathbf{ACom} \rightarrow (\mathbf{R} \rightarrow \mathcal{P}(\mathbf{R}))$ and $\mathcal{B}_R: \mathbf{BExp} \rightarrow (\mathbf{S} \rightarrow \mathcal{P}(\mathbf{T}))$ of the semantics of atomic commands and Boolean expressions by

$$\begin{aligned} \mathcal{A}_R[a] &= r^{-1}; \{\cdot\} \circ r \circ \mathcal{A}[a], \\ \mathcal{B}_R[b] &= r^{-1}; \{\cdot\} \circ \mathcal{B}[b]. \end{aligned}$$

The exercise is to define an abstraction $\mathcal{C}_R: \mathbf{Com} \rightarrow (\mathbf{R} \rightarrow \mathcal{P}(\mathbf{R}))$ of the semantics of commands and prove that

$$\mathcal{C}_R[c] \supseteq r^{-1}; (\{\cdot\} \circ \mathcal{C}[c]).$$

Reference. P. Cousot and R. Cousot. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Program by Construction or Approximation or Fixpoints. 4th POPL.

The Egli-Milner Order

Now we turn to the first possibility where we want to record “what actually happens”. It is natural here to consider subsets of $\mathbf{S}_{\perp}$ but we must answer two questions.

- A. What subsets should we consider?
- B. How should they be ordered?

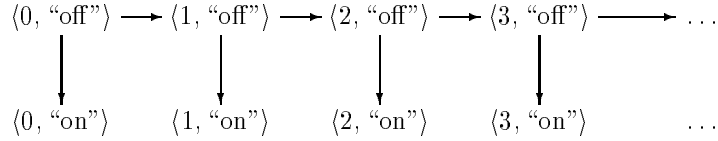
This situation is similar to that faced before when we had to decide which functions between cpo’s to consider and how to order them.

A. *What Subsets?* First since every program in our language must produce something (even if it is just $\perp$) we consider only nonempty subset. (Variants with an empty subset are considered in the exercises — they provide inessential, but useful alternatives.) More importantly consider the following attempt to write a program to output an arbitrary integer:

```

 $x := 0; \text{ alarm} := \text{"off"};$ 
while  $\text{alarm} = \text{"off"}$ 
do  $x := x + 1$  or  $\text{alarm} := \text{"on"}.$ 
```

Clearly this can either produce an arbitrary integer or fail to terminate. This is not accidental as the following informal argument shows. Consider the tree of all possible execution paths. In the case of the above program it looks like this



and here and in general it will be a *finitely-branching tree*, with the branching corresponding to the possible executions of nondeterministic constructs. Therefore if there are infinitely many different possible final states (e.g. all pairs $\langle n, \text{"on"} \rangle$) then there are infinitely many nodes in the tree and so by *König's Lemma* there must be an infinite branch in the tree, that is, nontermination is possible. This is a major phenomenon when considering *finite nondeterminism* where at most finitely many choices can be made at any one point.

Summing up we take, for countable X , the *powerdomain* $\mathcal{P}(X_\perp)$ to consist of all nonempty subsets of $X_\perp$ which are either finite or contain $\perp$ and consider next:

B. *What Order?* With products, the *coordinatewise* ordering was natural where each coordinate is increased to increase the order. Now the *elementwise* ordering is natural where each element is increased perhaps to a set of greater elements. So we expect

$$A \sqsubseteq_{\text{EM}} B$$

just when for each x in A there is a nonempty set Y_x such that

- (1) $\forall y \in Y_x. x \sqsubseteq y$,
- (2) $B = \bigcup_{x \in A} Y_x$.

Here is another way (due to Egli & Milner) of saying the same thing

$$\begin{aligned}
 A \sqsubseteq_{\text{EM}} B \quad \text{iff} \quad & (\downarrow) \forall x \in A. \exists y \in B. x \sqsubseteq y, \\
 & \wedge (\uparrow) \forall y \in B. \exists x \in A. x \sqsubseteq y.
 \end{aligned}$$

Note that, putting $Y_x = \{y \in B \mid x \sqsubseteq y\}$ condition, $(\downarrow)$ says Y_x is nonempty and (1) above on Y_x is automatic and (2) follows from the new $(\uparrow)$. You can prove that the nonemptiness requirement, (1) and (2) imply $(\downarrow)$ and $(\uparrow)$. This definition makes sense for any preorder and always defines a preorder on subsets. In the case at hand where we want $A, B \subseteq X_\perp$ then

$$\begin{aligned}
 A \sqsubseteq_{\text{EM}} B \quad \text{iff} \quad & \text{either } (1) \perp \notin A \wedge A = B, \\
 & \text{or } (2) \perp \in A \wedge A \setminus \{\perp\} \subseteq B \setminus \{\perp\}.
 \end{aligned}$$

So $\mathcal{P}(X_\perp)$ equipped with $\sqsubseteq_{\text{EM}}$ is even a cpo. (For it is clearly a partial order with least elements $\{\perp\}$. And if $A_0 \sqsubseteq A_1 \sqsubseteq \dots$ is an increasing ω -chain then either $\perp \notin A_n$ for some A_n , when $\bigsqcup_i A_i = A_n$ or else $\perp \in A_n$ for all n when $\bigsqcup_i A_i = \bigcup_n A_n$.)

Union. The binary union function $\cup: \mathcal{P}(X_\perp)^2 \rightarrow \mathcal{P}(X_\perp)$ is continuous. (Easy exercise)

Definition 2. A function $f: \mathcal{P}(X_\perp) \rightarrow \mathcal{P}(Y_\perp)$ is *linear* iff $f(x \cup y) = f(x) \cup f(y)$.

Fact 2. Let X and Y be countable sets. Then for any $f: X \rightarrow \mathcal{P}(Y_\perp)$ there is a unique strict continuous linear $f^\dagger: \mathcal{P}(X_\perp) \rightarrow \mathcal{P}(Y_\perp)$ such that the following diagram commutes

$$\begin{array}{ccc} X & & \\ \downarrow \{\cdot\} & \searrow f & \\ \mathcal{P}(X_\perp) & \xrightarrow{f^\dagger} & \mathcal{P}(Y_\perp) \end{array}$$

further $\dagger$ is an isomorphism of $X \rightarrow \mathcal{P}(Y_\perp)$ and the partial order of strict continuous linear function from $\mathcal{P}(X_\perp)$ to $\mathcal{P}(Y_\perp)$.

Proof. For uniqueness we note that for *finite* $A \subseteq Y_\perp$ we have

$$f^\dagger(A) = \bigcup_{a \in A} f(a)$$

as before and for *infinite* $A = \{\perp, a_1, a_2, \dots\}$ we have

$$\begin{aligned} f^\dagger(A) &= f^\dagger\left(\bigsqcup_n \{\perp, a_1, \dots, a_n\}\right) \\ &= \bigsqcup_n f^\dagger(\{\perp, a_1, \dots, a_n\}) \\ &= \bigsqcup_n [f^\dagger(\{\perp\}) \cup \bigcup_{i < n} f^\dagger(\{a_i\})] \\ &= \bigsqcup_n [\{\perp\} \cup \bigcup_{i < n} f(a_i)] \\ &= \bigcup_{a \in A} f(a) \end{aligned}$$

and the rest of the proof is an exercise for the reader. ■

Now the composition $f; g: X \rightarrow \mathcal{P}(Z_\perp)$ of $X \xrightarrow{f} \mathcal{P}(Y_\perp)$ and $Y \xrightarrow{g} \mathcal{P}(Z_\perp)$ is defined as before by: $f; g = g^\dagger \circ f$ and again it is associative with unit the singleton function. And, again as before we can define a semantics

$$\mathcal{C}: \mathbf{Com} \rightarrow (\mathbf{S} \rightarrow \mathcal{P}(\mathbf{S}_\perp))$$

which looks just like it did before.

Exercise 11. Draw $\mathcal{P}(\mathbf{U})$, $\mathcal{P}(\mathbf{0})$, $\mathcal{P}(\mathbf{T})$, $\mathcal{P}(\mathbf{N}_\perp)$.

Exercise 12. Show that $\mathcal{P}(\mathbf{N}_\perp)$ is ω -algebraic and its finite elements are just those of finite cardinality. Is it also consistently complete?

Exercise 13. Try to find an *analogue* of $\mathbf{S} \rightarrow \mathcal{P}(\mathbf{S}) \cong \mathcal{P}(\mathbf{S} \times \mathbf{S})$. [Hint Consider certain pairs $\langle T, R \rangle$ with $T \subseteq \mathbf{S}$, $R \subseteq \mathbf{S}^2$, the idea being that R is the relation and T is its “termination domain”.]

Exercise 14. Investigate analogue of Exercises 2, 3, 8 above.

Exercise 15. Show that $\mathcal{P}(\mathbf{S}) \cong \mathcal{P}(\mathbf{S}_\perp) / \sqsubseteq_R$ where

$$X \sqsubseteq_R Y \quad \text{iff} \quad (\downarrow) \forall x \in X. \exists y \in Y. x \subseteq y.$$

(If $\langle \mathbf{P}, \sqsubseteq \rangle$ is a preorder and $\leq$ is a preorder on $\mathbf{P}$ including $\sqsubseteq$ then $\mathbf{P} / \leq$ is the partial order $\langle \{a \downarrow \mid a \in \mathbf{P}\}, \subseteq \rangle$ where $a \downarrow \stackrel{\text{def}}{=} \{b \in \mathbf{P} \mid b \leq a\}$. Note that natural monotonic map $\downarrow: \mathbf{P} \rightarrow \mathbf{P} / \leq$.) So there is a natural map $\mathcal{P}(\mathbf{S}_\perp) \rightarrow \mathcal{P}(\mathbf{S})$. What is it?

It is possible to consider applicative programming languages for various classes of functions: $f: \mathcal{P}(X_\perp^{(1)}) \times \dots \times \mathcal{P}(X_\perp^{(n)}) \rightarrow \mathcal{P}(Y_\perp)$ which are continuous and perhaps n -strict and perhaps $\sqsubseteq$ -monotonic or linear in each argument. For information see M. Hennessy “Powerdomain and Nondeterministic Recursive Definitions” (to appear).

The Smyth Order

Finally we consider the third possibility that $(1) \neq (2) = (3)$ (due to M. Smyth). Here we want to *identify* all sets containing $\perp$ since nothing can be guaranteed of any of them. However we want to go even further; for example we want the program

$$(4) \quad x := 1 \text{ or } x := 2.$$

to have a *smaller* meaning than (1) above, the idea being that less can be guaranteed of it. So if in some context (1) fails to terminate so must (4), but the converse does not hold (consider the context $C[] = []$; **if** $x = 2$ **then loop else skip**). Thus we want: $\{1, 2\} \sqsubseteq \{1\}$.

For a formal definition we identify all sets containing $\perp$ with a standard one which for convenience we take to be $\mathbf{N}_\perp$. The remaining sets are the finite nonempty ones and so we take for any countable set X :

$$\mathcal{P}_S(X_\perp) = \langle \{A \subseteq X_\perp \mid A = X_\perp \text{ or } A \text{ is nonempty and finite, and } A \subseteq X\}, \supseteq \rangle.$$

Exercise 16. Show that $\mathcal{P}_S(X_\perp)$ is an ω -algebraic, consistently complete cpo and draw a few examples.

Clearly $\cup: \mathcal{P}_S(X_\perp)^2 \rightarrow \mathcal{P}_S(X_\perp)$ is again well-defined and continuous. It is left to the reader as of **Exercise 17** to work out an analogue to Fact 2 thereby obtaining a third semantics for the simple imperative language.

Predicate Transformers

The Smyth order is strongly linked to Dijkstra's 'weakest precondition' predicate transformers.

Definition 3. Let X and Y be countable sets. A *predicate transformer* is any $p: \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$ which is continuous, strict and multiplicative. (The last condition means that for any $A, B \subseteq Y$ we have $p(A \cap B) = p(A) \cap p(B)$.)

We develop the theory by means of some exercises.

Exercise 18. Suppose $m: X \rightarrow \mathcal{P}_S(Y_\perp)$. Define the *weakest precondition* function by putting for any $B \subseteq Y$

$$wp(m, B) = \{a \in X \mid m(a) \subseteq B\}.$$

Show that $\Omega(m) \stackrel{\text{def}}{=} \lambda B. wp(m, B)$ is a predicate transformer. Show too that Ω is monotonic and that the following properties hold:

$$\begin{aligned} \Omega(\{\cdot\}_X) &= id_{\mathcal{P}(X)}, \\ \Omega(m; m') &= \Omega(m) \circ \Omega(m'), \\ \Omega(m \cup m') &= \Omega(m) \cap \Omega(m') \end{aligned}$$

(where $m \cup m' \stackrel{\text{def}}{=} \lambda a \in X. m(a) \cup m'(a)$ and for any $p, q: \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$, $p \cap q \stackrel{\text{def}}{=} \lambda B \subseteq Y. p(B) \cap q(B)$).

Exercise 19. Let $p: \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$ be a predicate transformer. Show that p is a *stable* function in the sense of Chapter 6 Exercise 1 p. 69. That is show that if $a \in p(Y)$ then there is a finite set $M(p, a)$ such that

$$\forall B \subseteq Y. a \in p(B) \quad \text{iff} \quad M(p, a) \subseteq B.$$

Show too that p is *completely multiplicative* in that if $\langle B_\lambda \rangle_{\lambda \in \Lambda}$ is a collection of subsets of Y then

$$p\left(\bigcap_{\lambda} B_\lambda\right) = \bigcap_{\lambda} p(B_\lambda).$$

Exercise 20. Let $ST(X, Y)$ be $X \rightarrow \mathcal{P}_S(Y)$ with the pointwise ordering and let $PT(X, Y)$ be the set of predicate transformers $p: \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$ with the pointwise ordering ($p \sqsubseteq q$ iff $\forall B \subseteq Y. p(B) \subseteq q(B)$). Show that

$$\Omega: ST(X, Y) \rightarrow PT(X, Y)$$

is even an *isomorphism* of partial orders. [Hint Use the previous exercise to find a definition of Ω^{-1} .]

Exercise 21. Define a predicate transformer semantics

$$\mathcal{W}: \mathbf{Com} \rightarrow PT(\mathbf{S}, \mathbf{S})$$

(remember to prove that $\mathcal{W}[[c]]$ is always a predicate transformer!). Let $\mathcal{C}_S: \mathbf{Com} \rightarrow ST(\mathbf{S}, \mathbf{S})$ be the evident semantics based on the Smyth Powerdomain. Show that the following diagram commutes.

$$\begin{array}{ccc} \mathbf{Com} & \xrightarrow{\mathcal{C}_S} & ST(\mathbf{S}, \mathbf{S}) \\ & \searrow \mathcal{W} & \downarrow \Omega \\ & & PT(\mathbf{S}, \mathbf{S}) \end{array}$$

These exercises demonstrate complete equivalence between the predicate transformer approach and the nondeterministic position approach using the Smyth powerdomain.

Exercise 22. Let $DPT(X, Y)$ be the additive predicate transformers from Y to X (i.e. those ones p such that for all $A, B \subseteq Y$ we have $p(A \cup B) = p(A) \cup p(B)$).

Show that they correspond to the deterministic state transformation functions in the sense that Ω cuts down to an isomorphism

$$(X \rightarrow_P Y) \cong DPT(X, Y).$$

Exercise 23. Find a relational view of predicate transformers and Smyth nondeterministic state transformation functions.

References on Predicate Transformers

E. W. Dijkstra. A Discipline of Programming. Prentice-Hall, 1976.

G. D. Plotkin. Dijkstra's Predicate Transformers and Smyth's Powerdomains. Abstract Software Specifications (ed. D. Bjørner), LNCS Vol. 86, 1980.

Note the reversal of direction in the above between state transformation functions and predicate transformers. What we have is a *duality* between the category $\mathcal{ST}$ of state transformation functions and the category $\mathcal{PT}$ of predicate transformers. (A *duality* of categories $\mathcal{K}$ and $\mathcal{L}$ is an isomorphism $\mathcal{K}^{\text{op}} = \mathcal{L}$.)

Note that Exercises 4, 5, 6 give a duality theory for the relational case, there seems not to be such a theory for the Egli-Milner ordering.

General Powerdomains

We begin by generalising the relational powerdomain to an arbitrary ω -algebraic cpo $\mathbf{D}$. The easy mathematics involved will make that needed for the other two cases more comprehensible. More importantly we will quickly be in a position to demonstrate the techniques made available by powerdomain for the denotational semantics of parallelism. As in the case of a simple nondeterministic language the form of the semantics remains unchanged for the other semantics; only the mathematical properties of the resulting models will differ.

How to Order Subsets of $\mathbf{D}$

Let $\sqsubseteq$ be the desired preorder (with associated equivalence $\simeq$); the intuition is that $X \sqsubseteq Y$ means that anything X can do Y can do better, and we look for various formalisations of this idea. We will restrict attention to *nonempty* X and Y . First we generalise a definition of Exercise 15.

Definition 4. For X, Y nonempty subsets of $\mathbf{D}$,

$$X \sqsubseteq_{\mathbf{R}} Y \quad \text{iff} \quad \forall x \in X. \exists y \in Y. x \sqsubseteq y$$

and let $=_{\mathbf{R}}$ be the associated equivalence.

Note that $X \subseteq Y$ implies $X \sqsubseteq_{\mathbf{R}} Y$. From the above discussion we would expect that

$$X \sqsubseteq_{\mathbf{R}} Y \text{ implies } X \sqsubseteq Y.$$

Exercise 24. Show that if $f: \mathbf{D} \rightarrow \mathbf{E}$ is continuous and if for two nonempty subsets X and Y of $\mathbf{D}$ we have $X \sqsubseteq_{\mathbf{R}} Y$ then $f(X) \sqsubseteq_{\mathbf{R}} f(Y)$.

It may now seem natural that $\sqsubseteq$ should be preserved by continuous functions and since it should be $\sqsubseteq_{\mathbf{R}}$ on discrete cpos we define for all nonempty $X, Y \subseteq \mathbf{D}$

$$X \sqsubseteq_{\mathbf{R}} Y \text{ iff } \forall f: \mathbf{D} \rightarrow \mathbf{O}. f(X) \sqsubseteq_{\mathbf{R}} f(Y)$$

(and let $\simeq_{\mathbf{R}}$ be the associated equivalence) and expect that

$$X \sqsubseteq Y \text{ implies } X \sqsubseteq_{\mathbf{R}} Y.$$

Thinking of $f: \mathbf{D} \rightarrow \mathbf{O}$ as an *experiment* or *detectable property* of $\mathbf{D}$, one can understand $X \sqsubseteq_{\mathbf{R}} Y$ as meaning that any experiment which can succeed on X can also succeed on Y .

Exercise 25. Show that for any continuous $f: \mathbf{D} \rightarrow \mathcal{P}(Z)$ with Z countable we have: $X \sqsubseteq_{\mathbf{R}} Y$ implies $\bigcup f(X) \subseteq \bigcup f(Y)$.

Thus there is no loss in considering only $\mathbf{O}$.

Now $\sqsubseteq$ cannot be $\sqsubseteq_{\mathbf{R}}$ if union is to work as a continuous function; for let X be a set containing an increasing sequence $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$ but not containing any upper bound of $x = \bigsqcup_n x_n$. Then $X \not\sqsubseteq_{\mathbf{R}} X \cup \{x\}$ but $X \cup \{x\} = X \cup \{\bigsqcup_n x_n\} \simeq \bigsqcup_n (X \cup \{x_n\})$ (by assumption) $= X$. We therefore take $\sqsubseteq$ equal to $\sqsubseteq_{\mathbf{R}}$. (The reader can adapt the argument given below to show that $\sqsubseteq$ should be $\sqsubseteq_{\mathbf{R}}$ for finitely generable sets.) The next theorem gives several characterisations of $\sqsubseteq_{\mathbf{R}}$ to increase the reader's confidence in the naturality of the relation.

Theorem 1. Consider the following conditions on nonempty subsets X and Y of $\mathbf{D}$.

- (1) $X \sqsubseteq_{\mathbf{R}} Y$,
- (2) $X \sqsubseteq_{\mathbf{R}} Y$,
- (3) $\forall x \in X. \forall b \in B_{\mathbf{D}}. (b \sqsubseteq x \Rightarrow \exists y \in Y. b \sqsubseteq y)$,
- (4) $\forall A \subseteq \mathbf{D}. (A \text{ finite} \wedge A \sqsubseteq_{\mathbf{R}} X) \Rightarrow A \sqsubseteq_{\mathbf{R}} Y$.

Then $(1) \Rightarrow (2) \Leftrightarrow (3) \Leftrightarrow (4)$ and if $X \subseteq B_{\mathbf{D}}$ or Y is finite then $(1) \Leftrightarrow (2)$.

Proof. (1) $\Rightarrow$ (2). Already known.

(2) $\Rightarrow$ (3). Suppose $b \sqsubseteq x \in X$. Then $\top \in (b \Rightarrow \top)(X) \sqsubseteq_{\mathbf{R}} (b \Rightarrow \top)(Y)$ and so $\top \in (b \Rightarrow \top)(Y)$ and so $b \sqsubseteq$ some y in Y .

(3) $\Rightarrow$ (4). Trivial.

(4) $\Rightarrow$ (2). Left as an exercise.

(1) $\Leftrightarrow$ (2) when $X \subseteq B_{\mathbf{D}}$ or Y is finite. Immediate from (3). ■

So for finite nonempty sets we have $X \sqsubseteq_{\mathbf{R}} Y \equiv X \sqsubseteq Y$ and condition (3) show how close $\sqsubseteq_{\mathbf{R}}$ is to being $\sqsubseteq_{\mathbf{R}}$ in general and (4) says any finite information about X 's possibilities is also true of Y .

There is no need in the relational case to consider any restriction on the sets allowed into the powerdomain; all are "computationally feasible".

Definition 5. For any nonempty $X \subseteq \mathbf{D}$ put $X^\circ \stackrel{\text{def}}{=} \{b \in B_{\mathbf{D}} \mid \exists x \in X. b \sqsubseteq x\}$. (Note that X° is an *ideal* in $B_{\mathbf{D}}$.)

Now Theorem 1(3) shows that $X \simeq_{\mathbf{R}} X^\circ$. Let $X^\circ = \{b_{(1)}, b_{(2)}, \dots\}$. Let $P_{(n)}$ be a program whose behaviour is $b_{(n)}$, if possible. (For example when $\mathbf{D} = \mathbf{Tapes}$ $P_{(n)}$ could be **print** $b_{(n)}$.) The set of computations of

$$\mathbf{letrec} \ f(n) = P_{(n)} \ \mathbf{or} \ f(n+1) \ \mathbf{in} \ f(0)$$

is X° since the possible nontermination of $f(0)$, $\perp$ is already accounted for in X° .

So we can take the powerdomain to be $\mathcal{P}_{\mathbf{R}}(\mathbf{D}) \stackrel{\text{def}}{=} \langle \mathcal{P}(\mathbf{D}), \sqsubseteq_{\mathbf{R}} \rangle / \sqsubseteq_{\mathbf{R}}$ where $\mathcal{P}(\mathbf{D})$ is the set of all nonempty subsets of $\mathbf{D}$. But a clearer picture is easily obtained.

Definition 6. For any $X \subseteq \mathbf{D}$ put $Cl_R(X) \stackrel{\text{def}}{=} \{x \mid \forall b \sqsubseteq x. \exists y \in X. b \sqsubseteq y\}$ ($= \{x \mid B_x \subseteq X^\circ\}$).

Exercise 26. Show that Cl_R is a *topological closure operator*. That is

- (1) $Cl_R(X) \supseteq X$,
- (2) $Cl_R(Cl_R(X)) = Cl_R(X)$,
- (3) $Cl_R(X \sqcup Y) = Cl_R(X) \cup Cl_R(Y)$.

In fact $Cl_R(X) = X$ iff X is closed in the Scott topology of $\mathbf{D}$.

Lemma 1. For nonempty subsets X and Y of $\mathbf{D}$,

- (1) $X \simeq Cl_R(X)$,
- (2) $X \simeq Y$ iff $Cl_R(X) = Cl_R(Y)$,
- (3) $X \sqsubset_R Y$ iff $Cl_R(X) \subseteq Cl_R(Y)$.

Proof. An easy **Exercise 27**. ■

Lemma 2. The relational powerdomain $\mathcal{P}_R(\mathbf{D})$ is $\langle \text{Closed nonempty subsets of } \mathbf{D}, \subseteq \rangle$. Further $\mathcal{P}(\mathbf{D})$ is an ω -algebraic complete lattice: $\perp = (\perp_{\mathbf{D}})$ and $\bigsqcup \chi = Cl_R(\bigsqcup_{X \in \chi} X)$ (and $X \sqcup Y = X \cup Y$) and the finite elements are those of the form $Cl_R(A) = A^\circ$ where A is a finite nonempty subsets of $B_{\mathbf{D}}$.

Proof. First the maps $\mathcal{P}(\mathbf{D}) \xrightarrow[\delta]{\theta} \langle \text{Closed nonempty subsets of } \mathbf{D}, \subseteq \rangle$ defined by $\theta(X \downarrow) = Cl_R(X)$, $\delta(X) = X \downarrow$ are well-defined using Theorem 1(2) and monotonic using 1(3). Therefore using 1(1) they are mutual inverses, showing they are isomorphisms. The rest is easy **Exercise 28** (use $X \simeq X^\circ$). ■

Clearly $\cup$ is continuous (since it is $\sqcup$); also we have an “empty set”: $\emptyset_R \stackrel{\text{def}}{=} \{\perp\}$ as $\forall X \in \mathcal{P}_R(\mathbf{D}). \{\perp\} \subseteq X$; also we can define a singleton function $\{\cdot\}_R: \mathbf{D} \rightarrow \mathcal{P}_R(\mathbf{D})$ by

$$\{d\}_R \stackrel{\text{def}}{=} \{d\} \downarrow \quad (\text{corresponding to } \{x \in \mathbf{D} \mid x \sqsubseteq d\} = Cl_R(\{d\}))$$

and will often drop the suffix below.

Exercise 29. This exercise develops $\mathcal{P}_R(\mathbf{D})$ in terms of the basis. Show that for any nonempty sets X, Y :

- (1) $X \simeq_R X^\circ$,
- (2) $X \simeq_R Y$ iff $X^\circ = Y^\circ$,
- (3) $X \sqsubseteq_R Y$ iff $X^\circ \subseteq Y^\circ$.

Now show that $\mathcal{P}_R(\mathbf{D})$ is (isomorphic to) $\langle \text{Nonempty ideals of } B_{\mathbf{D}}, \subseteq \rangle$ with $\perp = \{\perp_{\mathbf{D}}\}$, $\bigsqcup \chi = \bigcup \chi$ and finite elements A° with A a finite nonempty subset of $B_{\mathbf{D}}$.

Now we want to develop a suitable notion of function extension. In fact we can characterise $\mathcal{P}_R(\mathbf{D})$ as the *free ω -complete cpo over $\mathbf{D}$* (ω -complete cpo being one with lubs of all countable sets).

Theorem 2. For any ω -complete cpo $\mathbf{L}$ and (strict) continuous $f: \mathbf{D} \rightarrow_{(\perp)} \mathbf{L}$ there is a unique (strict) additive and continuous $f^\dagger: \mathcal{P}_R(\mathbf{D}) \rightarrow_{(S)\mathbf{A}} \mathbf{L}$ such that the following diagram commutes:

$$\begin{array}{ccc} \mathbf{D} & & \\ \downarrow \{\cdot\} & \searrow f & \\ \mathcal{P}_R(\mathbf{D}) & \xrightarrow{f^\dagger} & \mathbf{L} \end{array}$$

Further $(\cdot)^\dagger: \mathbf{D} \rightarrow_{(\perp)} \mathbf{L} \cong \mathcal{P}_R(\mathbf{D}) \rightarrow_{(S)\mathbf{A}} \mathbf{L}$ is an isomorphism of partial orders.

Proof. Let $f: \mathbf{D} \rightarrow \mathbf{L}$ be continuous and supposing $g: \mathcal{P}_R(\mathbf{D}) \rightarrow \mathbf{L}$ is continuous, additive and makes the diagram commute. Then using the ideal characterisation of $\mathcal{P}_R(\mathbf{D})$ we have

$$\begin{aligned} g(X) &= g\left(\bigcup_{b \in X} \{b\}\right) \\ &= \bigsqcup_{b \in X} g(\{b\}) \quad \text{as } X \text{ is nonempty and countable and } g \text{ is continuous and additive} \\ &= \bigsqcup_{b \in X} f(b). \end{aligned}$$

So g is unique and is strict if f is. Thus defined g is clearly additive and continuous. To see the diagram commutes, we calculate:

$$\begin{aligned} g(\{x\}) &= g(\{b \in B_{\mathbf{D}} \mid b \sqsubseteq x\}) \\ &= \bigsqcup_{b \sqsubseteq x} f(b) \\ &= f(x) \end{aligned} \quad \text{as } f \text{ is continuous.}$$

Clearly $(\cdot)^\dagger$ has inverse $\lambda g. g \circ \{\cdot\}$ and both are monotonic showing they are isomorphisms. ■

For ω -algebraic $\mathbf{D}$, $\mathbf{E}$ and $\mathbf{F}$ we define the composition of non-deterministic functions as usual: if $f: \mathbf{D} \rightarrow \mathcal{P}_R(\mathbf{E})$ and $g: \mathbf{E} \rightarrow \mathcal{P}_R(\mathbf{F})$ then $f; g = g^\dagger \circ f$. Of course composition is associative continuous and the unit is the singleton function.

We can now note that $\mathcal{P}_R$ is (easily considered as) a locally ω -continuous functor over $\omega\text{-}\mathcal{ALG}$. For any $f: \mathbf{D} \rightarrow \mathbf{E}$ let $\mathcal{P}_R(f): \mathcal{P}_R(\mathbf{D}) \rightarrow \mathcal{P}_R(\mathbf{E})$ be the unique additive and continuous function such that

$$\begin{array}{ccc} \mathbf{D} & \xrightarrow{f} & \mathbf{E} \\ \{\cdot\} \downarrow & & \downarrow \{\cdot\} \\ \mathcal{P}_R(\mathbf{D}) & \xrightarrow{\mathcal{P}_R(f)} & \mathcal{P}_R(\mathbf{E}) \end{array}$$

commutes (by Theorem 2).

Exercise 30. Prove that $\mathcal{P}_R$ is a locally continuous functor which preserves strictness. [Naturally this is all immediate from Theorem 2.]

Hence we can solve domain equations in $\omega\text{-}\mathcal{ALG}$ provided we do not have any function spaces; if we use the consistently complete ω -algebraic cpos we can also allow ourselves function-spaces as well, for since $\mathcal{P}_R(\mathbf{D})$ is always a complete lattice it cuts down to a functor on the latter category. This concludes the mathematics we need. A few exercises follow.

Exercise 31. Show that $\mathbf{D}$ is a *closure* of $\mathcal{P}_R(\mathbf{D})$ when $\mathbf{D}$ is itself an ω -complete cpo.

Exercise 32. Let $\mathbf{D}$ and $\mathbf{E}$ be ω -algebraic cpos. A *relation* between $\mathbf{D}$ and $\mathbf{E}$ is any $R \subseteq B_{\mathbf{D}} \times B_{\mathbf{E}}$ such that

- (1) $\forall a \in B_{\mathbf{D}}. a R \perp$,
- (2) $\forall a, a' \in B_{\mathbf{D}}. \forall b, b' \in B_{\mathbf{E}}. a \sqsupseteq a' R b' \sqsupseteq b \Rightarrow a R b$.

The collection of such relations is written as $Rel(\mathbf{D}, \mathbf{E})$. Show that $\mathbf{D} \rightarrow \mathcal{P}_R(\mathbf{E}) \cong \langle Rel(\mathbf{D}, \mathbf{E}), \sqsubseteq \rangle$. How should the composition of relations be defined? What is the identity relation for this composition? Reformulate this exercise in terms of relations $R \subseteq \mathbf{D} \times \mathbf{E}$.

Exercise 33 Duality. Show that $(\mathbf{D} \rightarrow \mathcal{P}_R(\mathbf{E})) \cong \mathcal{O}(\mathbf{E}) \rightarrow_{\text{STA}} \mathcal{O}(\mathbf{D})$ the partial order of strict, top-preserving, additive, continuous functions from $\mathcal{O}(\mathbf{E})$ to $\mathcal{O}(\mathbf{D})$ (here $\mathcal{O}(\mathbf{D}), \mathcal{O}(\mathbf{E})$ are the ω -algebraic complete lattices of the open subsets of $\mathbf{D}$, resp. $\mathbf{E}$ ordered by subset (recall $\mathcal{O}(\mathbf{D}) \cong \mathbf{D} \rightarrow \mathbf{0}$ etc.). The isomorphism is Λ where

$$\Lambda(m)(V) = \{d \in \mathbf{D} \mid m(d) \cap V \neq \emptyset\}.$$

Show that the *strict* functions from $\mathbf{D}$ to $\mathcal{P}_R(\mathbf{E})$ correspond under Λ to those g which *reflect* $\top$ (i.e. for all open V , $g(V) = \mathbf{D}$ implies $V = \mathbf{E}$).

Establish the following equalities (the first two showing that Λ is a duality of categories):

- (1) $\Lambda(\{\cdot\}) = id$,
- (2) $\Lambda(f; g) = \Lambda(f) \circ \Lambda(g)$,
- (3) $\Lambda(f \cup g) = \Lambda(f) \cup \Lambda(g)$ (with the evident pointwise definitions).

For any $m: \mathbf{D} \rightarrow \mathcal{P}_R(\mathbf{E})$ and Scott-closed $B \subseteq \mathbf{E}$ define the *weakest liberal precondition* $wlp(m, B)$ by:

$$wlp(m, B) = \{d \in \mathbf{D} \mid m(d) \subseteq B\}.$$

How does wlp relate to Λ ? Conclude that $\mathbf{D} \rightarrow \mathcal{P}_R(\mathbf{E})$ is isomorphic to the collection of strict $\perp$ -reflecting, $\top$ -preserving, multiplicative, dual continuous maps from $\mathcal{C}(\mathbf{E})$ to $\mathcal{C}(\mathbf{D})$ (where $\mathcal{C}(\mathbf{D})$, $\mathcal{C}(\mathbf{E})$ are the partial orders of the Scott-closed subsets of $\mathbf{D}$ resp. $\mathbf{E}$ partially ordered by subset). Specialise this to a correspondence between $\mathbf{D} \rightarrow_{\perp} \mathcal{P}_R(\mathbf{E})$ and certain maps from $\mathcal{P}_R(\mathbf{E})$ to $\mathcal{P}_R(\mathbf{D})$.

Exercise 34. Extend Exercise 8 to general relational powerdomains.

Exercise 35. Let $\mathbf{P} \subseteq \mathcal{P}_R(\mathbf{D})$ be ω -inductive. Show the following rule is valid:

$$\frac{\forall d \in \mathbf{D}. \mathbf{P}(\{d\}) \quad \forall X, Y \in \mathcal{P}_R(\mathbf{D}). \mathbf{P}(X) \wedge \mathbf{P}(Y) \supset \mathbf{P}(X \cup Y)}{\forall X. \mathbf{P}(X)}.$$

Exercise 36. Let $\mathbf{P} \subseteq \mathbf{D}$ be ω -inductive. Show that an ω -inductive $\mathbf{Q} \subseteq \mathcal{P}_R(\mathbf{D})$ can be defined by: $\mathbf{Q}(X)$ iff $X \subseteq \mathbf{P}$. Show that $\mathbf{Q}(X)$ iff $\exists d. X = Cl_R(\{d\})$ is ω -inductive. What about $\exists d, d'. X = Cl_R(\{d, d'\})$?

Exercise 37. Show that if $\mathbf{D}$ is effectively given so is $\mathcal{P}_R(\mathbf{D})$ and the index of $\mathcal{P}_R(\mathbf{D})$ is effectively obtainable from that of $\mathbf{D}$. Show that $\cup: \mathcal{P}_R(\mathbf{D})^2 \rightarrow \mathcal{P}_R(\mathbf{D})$ is computable. Show that $\{\cdot\}: \mathbf{D} \rightarrow \mathcal{P}_R(\mathbf{D})$ is computable. Show that if $f: \mathbf{D} \rightarrow \mathcal{P}_R(\mathbf{E})$ is computable (where $\mathbf{E}$ is also effectively given) then $f^\dagger: \mathcal{P}_R(\mathbf{D}) \rightarrow \mathcal{P}_R(\mathbf{E})$ is computable, an index for it being effectively obtainable from one for f . Conclude that $\mathcal{P}_R$ is computable.

Denotational Semantics

We consider languages where there are several processes sharing a common store. Later in the chapter we shall have a look at languages where processes communicate with each other along logical channels (lines, through ports, by broadcasting, etc.) and each has its own local store. A basic language is supplied by the grammar:

$$c ::= a \mid \mathbf{skip} \mid c; c \mid \mathbf{if } b \mathbf{ then } c \mathbf{ else } c \mid \mathbf{while } b \mathbf{ do } c \mid c \parallel c$$

where atomic commands and Boolean expressions are as usual and we shall feel free to give specific examples without formal definition. The important construction $c_1 \parallel c_2$ conceived of as the concurrent execution of c_1 and c_2 . So $c_1 \parallel \dots \parallel c_n$ corresponds to Dijkstra's **parbegin** $c_1; \dots; c_n$ **parend**.

When there is no interference between concurrent processes (perhaps achieved by additional syntactic constraints) there is a legitimate semantics of type $\mathcal{C}: \mathbf{Com} \rightarrow (\mathbf{S} \rightarrow \mathcal{P}_R(\mathbf{S}_{\perp}))$ with crucial clause:

$$\mathcal{C}[c_1 \parallel c_2] = (\mathcal{C}[c_1]; \mathcal{C}[c_2]) \cup (\mathcal{C}[c_2]; \mathcal{C}[c_1]).$$

But this is not valid when there is interference. For example take $\mathbf{S} = \mathbf{N}$, $c_1 = \mathbf{skip}$, $c_2 = (x := 1)$, $c_3 = (c := x + 1; x := x - 1)$. Then $\mathcal{C}[c_1 \parallel c_2](2)$ should be $\{1\}$ whereas (assuming programs interruptible between but not during assignments) $\mathcal{C}[c_3 \parallel c_2](2)$ should be $\{0, 1\}$. But these are different, $\mathcal{C}[c_1]$ and $\mathcal{C}[c_3]$ should be the same and so there can be *no* semantics of the form

$$\mathcal{C}[c_1 \parallel c_2] = \varphi(\mathcal{C}[c_1], \mathcal{C}[c_2]).$$

To get over this problem we have to account for the phenomenon of interruption. Stating c in σ we can reach in interruption point with an intermediate state σ' and some more program to execute when control returns after the interruption. Of course we can also terminate with a final state and since c itself could be of the form $c_1 \parallel c_2$ several of each of these possibilities could occur. Let r be the meaning of c . Then $r(\sigma)$ should be a set of things either states σ' of pairs $\langle \sigma', r' \rangle$ where σ' is the interruption state and r' is the meaning of the program remaining to execute. Thus we introduce $\mathbf{R}$ as a cpo of *resumption* given by the domain equations

$$\mathbf{R} \cong \mathbf{S}_{\perp} \rightarrow_{\perp} \mathcal{P}_R(\mathbf{S}_{\perp} + (\mathbf{S}_{\perp} \otimes \mathbf{R}_{\perp})).$$

In fact this idea (and variations) is all that we need for the semantics of concurrency. It was introduced in a complicated form by Milner in

A. J. R. G. Milner. Processes: A Mathematical Model of Computing Agents, Logic Colloquium '73. North Holland.

and independently by Bekic in

H. Bekic. Towards a Mathematical Theory of Processes. Technical Report TR25.125, IBM Laboratory Vienna, 1971.

Powerdomains were not available in that paper (as a substitute for $\mathcal{P}_R(\mathbf{D})$ one used $(\mathbf{O} \rightarrow \mathbf{D})$ where $\mathbf{O}$ is a suitable cpo of *oracles* e.g. **Tapes** or $\mathbf{N}^{\omega}$). In

G. D. Plotkin. A Powerdomain Construction, SIAM J. Comput. Vol. 5, No. 3, 1976

the present form was introduced and it was applied to a language for communicating processes in

G. Milne and A. J. R. G. Milner. Concurrent Processes and Their Syntax. JACM 1977.

Every resumption can be *flattened* to produce its corresponding state transformation. To define this we introduce a very handy notation which makes this and other definitions quite transparent. Let e_1 be an expression of type $\mathcal{P}_{\mathbf{R}}(\mathbf{S}_{\perp} + (\mathbf{S}_{\perp} \otimes \mathbf{R}_{\perp}))$ and let $e_2[\sigma']$ be an expression of type $\mathbf{D}$ (possibly with an occurrence of σ' , a variable of type $\mathbf{S}_{\perp}$) and let $e_3[\sigma', r']$ be another expression of type $\mathbf{D}$ (possibly with free occurrences of variables σ', r' of types $\mathbf{S}_{\perp}$ and $\mathbf{R}$ respectively). Then

$$\begin{aligned} &\text{cases } e_1 \\ &\quad \text{first } \sigma'.e_2[\sigma'] \\ &\quad \text{second } \sigma', r'.e_3[\sigma', r'] \end{aligned}$$

is an expression of type $\mathbf{D}$ which means the same as:

$$[\lambda_{\perp} \sigma'.e_2[\sigma'], \lambda_{\perp} \sigma', x.lift(\lambda r'.e_3[\sigma', r'])(x)]^{\dagger}(e_1).$$

Now we may recursively define a flattening combinator $|\cdot|: \mathbf{R} \rightarrow (\mathbf{S}_{\perp} \rightarrow \mathcal{P}(\mathbf{S}))$ by

$$\begin{aligned} |r|(\sigma) = &\text{cases } r(\sigma) \\ &\text{first } \sigma'.\{\sigma'\} \\ &\text{second } \sigma', r'.|r'|(\sigma'). \end{aligned}$$

Exercise 38. Note that the cases construction is strict and linear in its first argument. Establish these two identities (where we have omitted some injection functions) where $\sigma' \in \mathbf{S}$, $r' \in \mathbf{R}$,

- (1) $\text{cases } \{\text{inl}(\sigma')\} \text{ first } \sigma'.e_2 \text{ second } \sigma, r'.e_3 = e_2,$
- (2) $\text{cases } \{\text{inr}(\langle \sigma', \text{up}(r') \rangle)\} \text{ first } \sigma'.e_2 \text{ second } \sigma', r'.e_3 = e_3.$

Now only one or two more combinators are needed.

Composition. We define $*$: $\mathbf{R}^2 \rightarrow \mathbf{R}$ by

$$\begin{aligned} r_1 * r_2(\sigma) = &\text{cases } r_1(\sigma) \\ &\text{first } \sigma'.\{\text{inr}(\sigma' \otimes \text{up}(r_2))\} \\ &\text{second } \sigma', r'_1.\{\text{inr}(\sigma' \otimes \text{up}(r'_1 * r_2))\}. \end{aligned}$$

Parallelism. We define $\|$: $\mathbf{R}^2 \rightarrow \mathbf{R}$ by

$$\begin{aligned} r_1 \| r_2(\sigma) = &\text{cases } r_1(\sigma) \\ &\text{first } \sigma'.\{\text{inr}(\sigma' \otimes \text{up}(r_2))\} \\ &\text{second } \sigma', r'_1.\{\text{inr}(\sigma' \otimes \text{up}(r'_1 \| r_2))\} \\ &\cup \\ &\text{cases } r_2(\sigma) \\ &\text{first } \sigma'.\{\text{inr}(\sigma' \otimes \text{up}(r_1))\} \\ &\text{second } \sigma', r'_2.\{\text{inr}(\sigma' \otimes \text{up}(r_1 \| r'_2))\}. \end{aligned}$$

Exercise 39. This parallel operator is one where “ $r_1 \| r_2$ terminates when r_1 and r_2 do”; write one which terminates when one does.

Exercise 40. Establish whether $*$ is associative with identity $\lambda_{\perp} \sigma.\{\text{inl}(\sigma)\}$ and whether $\|$ is associative and commutative.

Semantics. $\mathcal{R}: \mathbf{Com} \rightarrow \mathbf{R}$ is defined by

$$\begin{aligned} \mathcal{R}[\![a]\!] &= \lambda_{\perp} \sigma.\{\text{inl}(\mathcal{A}[\![a]\!](\sigma))\}; \\ \mathcal{R}[\![\text{skip}]\!] &= \lambda_{\perp} \sigma.\{\text{inl}(\sigma)\}; \\ \mathcal{R}[\![c_1; c_2]\!] &= \mathcal{R}[\![c_1]\!] * \mathcal{R}[\![c_2]\!]; \\ \mathcal{R}[\![\text{if } b \text{ then } c_1 \text{ else } c_2]\!] &= \lambda_{\perp} \sigma.\text{if } \mathcal{B}[\![b]\!](\sigma) \text{ then } \{\text{inr}(\sigma \otimes \text{up}(\mathcal{R}[\![c_1]\!]))\} \text{ else } \{\text{inr}(\sigma \otimes \text{up}(\mathcal{R}[\![c_2]\!]))\}; \\ \mathcal{R}[\![\text{while } b \text{ do } c]\!] &= \mu r.\lambda_{\perp} \sigma.\text{if } \mathcal{B}[\![b]\!](\sigma) \text{ then } \{\text{inr}(\sigma \otimes \text{up}(\mathcal{R}[\![c]\!] * r))\} \text{ else } \{\text{inr}(\sigma \otimes \text{up}(\lambda_{\perp} \sigma.\{\text{inl}(\sigma)\}))\}; \\ \mathcal{R}[\![c_1 \| c_2]\!] &= \mathcal{R}[\![c_1]\!] \| \mathcal{R}[\![c_2]\!]. \end{aligned}$$

Exercise 41. In the semantics of conditional and while-loops there is an interruption point between the evaluation of the Boolean expression and the body. Give a semantics where no such point exists (essentially the so-called *test and set* idea).

Exercise 42. Give a semantics of the construct **critical-section**(c) (alternative syntax: $[c]$) which is c but with *no* interruption points.

Exercise 43. Give a semantics of assignment statements where there is an interruption point between the evaluation of the right-hand-side of an assignment and the updating of the variable.

Suppose atomic commands are assignments $x := e$ where the class of expressions is, say, $e ::= 0 \mid 1 \mid e + e \mid e * e \mid \dots$. Give a semantics where expressions can be interrupted at any point during their evaluation. You may find it useful to use *expression resumptions* defined by

$$\mathbf{E} \cong \mathbf{S}_\perp \rightarrow_\perp \mathcal{P}_R((\mathbf{N}_\perp \otimes \mathbf{S}_\perp) + (\mathbf{S}_\perp \otimes \mathbf{E}_\perp)).$$

This exercise can be taken quite far by allowing recursive functions to be defined as in: **letrec** $f(x) = e$ **in** e or commands in expressions as in **begin** c **result** e .

Synchronisation Methods

We consider these via some exercises.

Exercise 44. The *when construct* is $b \Rightarrow c$ (alternative concrete syntax is: **when** b **do** c or **await** $b; c$). The idea is that $b \Rightarrow c$ cannot execute unless b is true. Give this a semantics using as resumptions:

$$\mathbf{R} \cong \mathbf{S}_\perp \rightarrow_\perp \mathcal{P}_R(\mathbf{S}_\perp + \mathbf{S}_\perp \otimes \mathbf{R}_\perp)$$

the idea being that $r(\sigma) = \emptyset_R$ corresponds to r 's “not being able to execute” i.e. *deadlock*. Your semantics should be such that the first interruption point of $b \Rightarrow c$ is the first interruption point of c . (If it were instead just after b or just after c that would then corresponding to the semantics of $b \Rightarrow \mathbf{skip}; c$ and $b \Rightarrow [c]$ (equivalently $[b \Rightarrow c]$; give alternative semantics for these two properties and establish the correspondence).

Add the operator $c_1 \Box c_2$ which can execute just when c_1 **or** c_2 can.

Consider allowing interruption points during the execution of the conditional.

Exercise 45 Semaphores. Let **SId** be a finite set of semaphore identifiers ranged over by v ; they will be integer semaphores. We introduce the commands $P(v)$ (alternate syntax; *Wait*(v), *Grab*(v)) and $V(v)$ (alternate syntax; *Signal*(v), *Release*(v)). In terms of the **when** construct — see previous exercise — $P(v)$ means $v > 0 \Rightarrow v := v - 1$ and $V(v)$ means $v := v + 1$ where the assignments are uninterruptible.

Give a resumption semantics, using resumptions:

$$\mathbf{R} \cong \mathbf{States} \rightarrow_\perp \mathcal{P}_R(\mathbf{States} + \mathbf{States} \otimes \mathbf{R}_\perp)$$

with the same idea as in the previous exercise but where

$$\mathbf{States} = \mathbf{S}_\perp \times (\mathbf{Sid}_\perp \rightarrow_\perp \mathbf{N}_\perp \otimes \mathbf{T}).$$

The point of the truth-values is to show whether the semaphore is busy or not. As the separation of identifiers is becoming clumsy, you might want to consider allowing only one set of identifiers but introduce a suitable declaration mechanism for variables and semaphores.

Exercise 46 I/O Devices. Add the construct **print** e where e is an arithmetic expression. As a resumption model you could take:

$$\mathbf{R} \cong \mathbf{S}_\perp \rightarrow_\perp \mathcal{P}_R(\mathbf{S}_\perp + \mathbf{N}_\perp \otimes \mathbf{R}_\perp + \mathbf{S}_\perp \otimes \mathbf{R}_\perp).$$

Define a suitable flattening function $|\cdot|: \mathbf{R} \rightarrow \mathbf{NS}$ where $\mathbf{NS}$ is the domain of finite sequences of integers, given by the domain equation:

$$\mathbf{NS} \cong \mathbf{N}_\perp + \mathbf{N}_\perp \otimes \mathbf{NS}_\perp.$$

This clearly continues the ideas of Chapter 5, Example 4 allowing infinitely proceeding output and showing the connection between the present resumptions and those considered there. Adapt the ideas of that example to allow a read command. Extend all this to more sophisticated I/O devices. You may find points of contact with the semantics of communicating processes.

Exercise 47 Parallel For-Loops. Invent such a concept and give its semantics.

Exercise 48 Critical Regions. The idea here is to have “structured semaphores”. Each variable has attached to it a binary semaphore and the new syntactic construct is **with** x **do** c which means $P(v); c; V(v)$ where v is the binary semaphore, and so there are no explicit semaphore. Give this a semantics. Strictly speaking there should also be a static semantics so that:

- (1) There is no use of variables except inside **with** statements. The point of this is to ensure that processes obey the semaphores discipline of not accessing resources until they are owned by the process.
- (2) No two parallel processes have common variables with left-hand-side occurrences. This ensures resources are uniquely allocated to processes at any one time (though the allocation can change as **with** statements are entered and exited).

Give such a static semantics (see Mike Gordon’s book for hints on how to do that).

Add **when** statements to your language and semantics. This allows the explanation of some well-known constructs. For example

$$\begin{array}{lll}
 \text{with } r \text{ when } b \text{ do } c & \text{means} & \text{with } r \text{ do } (b \Rightarrow c) \\
 \text{region } r \text{ do} & \text{means} & \text{with } r \text{ do} \\
 \text{begin } \dots \text{ await } b; c \dots \text{ end} & & \dots (b \Rightarrow c) \dots \\
 & & \text{with } r \text{ do } (b_0 \Rightarrow c_0 \square \dots \square b_n \Rightarrow c_n)
 \end{array}$$

Exercise 49. This is a fairly substantial exercise, perhaps more of a mini-project. We think of a *monitor* as a class treated as a resource and you should begin by ignoring the parallel features and just find a semantics of classes.

Syntax.

Ground Types. These are $gt \in \{int, bool\}$.

Types. These are $t \in \mathbf{Types} = \{int, bool, \mathbf{proc } int, \mathbf{proc } bool\}$ for integer or boolean variables and call-by-value procedures with one argument being either integers or booleans.

Monitor Types. These are $mt \in \mathbf{MTypes} = \mathbf{Id} \rightarrow_{\text{fin}} \mathbf{Types} \stackrel{\text{def}}{=} \sum_{X: \text{a finite subset of } \mathbf{Id}} X \rightarrow \mathbf{Types}$.

Declarations. These are $d \in \mathbf{Dec}$ where

$$d ::= \mathbf{var } x: gt \mid \mathbf{proc } x: gt; c \mid d \mathbf{within } d \mid d; d \mid d \mathbf{init } c \mid \mathbf{monitor } m: mt \ d$$

For $d \mathbf{within } d$ and $d; d$ see Mike Gordon’s book pp. 80, 135. The initialisation construct $d \mathbf{init } c$ is elaborated by elaborating d and then executing c (for initialisation purposes). Private declarations and initialisation are useful within monitor declarations.

Commands.

$$c ::= (\text{usual stuff}) \mid c \parallel c \mid b \Rightarrow c \mid c \square c \mid d; c \mid p(e) \mid \mathbf{with } m \text{ do } c.$$

In the static semantics monitors are treated as resources so that any call of a monitor procedure or reference to a variable must be inside an appropriate **with** statement and also no concurrent access to such monitor resources is permitted.

Communicating Processes

Consider the following where α, β, γ range over a finite set **Lab** of *communication channels*

$$c ::= a \mid \mathbf{skip} \mid \alpha?x \mid \alpha!e \mid c; c \mid \mathbf{if } b \text{ then } c \text{ else } c \mid \mathbf{while } b \text{ do } c \mid c \parallel c \mid c \setminus \alpha \mid c[\alpha/\beta] \mid c \square c$$

(there should be some static semantics to ensure concurrent processes do not use common variables). Here $\alpha?x$ means: input a value off α and update x to it and $\alpha!e$ means evaluate e and send the result along α . Let us look for a semantics where we intend *synchronised communication* so that the only communication actions are an uninterruptible sending and then receiving; $c \setminus \alpha$ is c less the capability of communicating on α and $c[\alpha/\beta]$ is c but with the capabilities of communication over α transferred to β . See, for example

C. A. R. Hoare. Communicating Sequential Processes. CACM Vol. 21(8), pp. 666–677, 1978.

A. J. R. G. Milner. A Calculus of Communicating Systems, LNCS Vol. 92, 1980.

H. Ledgard. ADA: An Introduction. Springer-Verlag.

We take

$$\mathbf{R} \cong \mathbf{S}_\perp \rightarrow_\perp \mathcal{P}_R(\mathbf{S}_\perp + \mathbf{Lab}_\perp \otimes (\mathbf{N}_\perp \rightarrow_\perp \mathbf{S}_\perp \otimes \mathbf{R}_\perp)_\perp + \mathbf{Lab}_\perp \otimes \mathbf{N}_\perp \otimes \mathbf{R}_\perp + \mathbf{S}_\perp \otimes \mathbf{R}_\perp).$$

The Parallel Combinator

$$\begin{aligned}
r_1 \| r_2(\sigma) = & \text{cases } r_1(\sigma) \\
& \text{first } \sigma'. \{in_3(\sigma' \otimes up(r_2))\} \\
& \text{second } \alpha, f. \{in_1(\alpha \otimes up(\lambda_\perp n. f(n)_1 \otimes up(f(n)_2 \| r_2)))\} \\
& \text{third } \alpha, n, r'_1. \{in_2(\alpha \otimes n \otimes (up(r'_1 \| r_2)))\} \\
& \text{fourth } \sigma', r'_1. \{in_3(\sigma' \otimes up(r'_1 \| r_2))\} \\
& \cup \\
& \text{cases } r_2(\sigma) \\
& \text{first } \sigma'. \emptyset \\
& \text{second } \alpha, f. \\
& \quad \text{cases } r_2(\sigma) \\
& \quad \text{first } \sigma'. \emptyset \\
& \quad \text{second } \beta, g. \emptyset \\
& \quad \text{third } \beta, n', r'_2. \\
& \quad \quad \text{if } \alpha = \beta \\
& \quad \quad \quad \text{then } \{in_3(f(n)_1 \otimes up(f(n)_2 \| r'_2))\} \\
& \quad \quad \quad \text{else } \emptyset \\
& \quad \text{fourth } \sigma', r'_2. \emptyset \\
& \text{third } \alpha, n, r'_1. \\
& \quad \text{cases } r_2(\sigma) \\
& \quad \text{first } \sigma'. \emptyset \\
& \quad \text{second } \beta, f. \\
& \quad \quad \text{if } \alpha = \beta \\
& \quad \quad \quad \text{then } \{in_3(f(n)_1 \otimes up(r'_1 \| f(n)_2))\} \\
& \quad \quad \quad \text{else } \emptyset \\
& \quad \text{third } \beta, n', r'_2. \emptyset \\
& \quad \text{forth } \sigma', r'_2. \emptyset \\
& \text{fourth } \sigma', r'_1. \emptyset \\
& \cup \\
& \text{cases } r_2(\sigma) \\
& \text{first } \sigma'. \{in_3(\sigma' \otimes up(r_1))\} \\
& \text{second } \alpha, f. \{in_1(\alpha \otimes up(\lambda_\perp n. f(n)_1 \otimes up(r_1 \| f(n)_2)))\} \\
& \text{third } \alpha, n, r'_2. \{in_2(\alpha \otimes n \otimes up(r_1 \| r'_2))\} \\
& \text{fourth } \sigma', r'_2. \{in_3(\sigma' \otimes up(r_1 \| r'_2))\}.
\end{aligned}$$

The notation is now become barbaric!

Exercise 50. Rewrite the above definition, without using the empty set.

The Elision Combinator

$$\begin{aligned}
r \setminus \alpha(\sigma) = & \text{cases } r(\sigma) \\
& \text{first } \sigma'. \{in_0(\sigma')\} \\
& \text{second } \beta, f. \text{ if } \alpha = \beta \text{ then } \emptyset \text{ else } \{in_1(\beta \otimes up(\lambda_\perp n. f(n)_1 \otimes up(f(n)_2 \setminus \alpha)))\} \\
& \text{third } \beta, n, r'. \text{ if } \alpha = \beta \text{ then } \emptyset \text{ else } \{in_2(\beta \otimes n \otimes up(r' \setminus \alpha))\} \\
& \text{fourth } \sigma', r'. \{in_3(\sigma' \otimes up(r' \setminus \alpha))\}.
\end{aligned}$$

It is here that the empty set is really needed.

$$\begin{aligned}
 r[\alpha/\beta](\sigma) = & \text{cases } r(\sigma) \\
 & \text{first } \sigma.\{in_0(\sigma')\} \\
 & \text{second } \gamma, f. \\
 & \text{if } \gamma = \beta \\
 & \quad \text{then } \{in_1(\alpha \otimes up(\lambda_{\perp} n.f(n)_1 \otimes up(f(n)_2[\alpha/\beta])))\} \\
 & \quad \text{else } \{in_1(\gamma \otimes up(f))\} \\
 & \text{third } \gamma, n, r'. \\
 & \text{if } \gamma = \beta \\
 & \quad \text{then } \{in_2(\alpha \otimes n \otimes up(r'[\alpha/\beta]))\} \\
 & \quad \text{else } \{in_2(\gamma \otimes n \otimes up(r'[\alpha/\beta]))\} \\
 & \text{fourth } \sigma, r'.\{in_3(\sigma' \otimes up(r'[\alpha/\beta]))\}.
 \end{aligned}$$

All the semantics $\mathcal{R}: \mathbf{Com} \rightarrow \mathbf{R}$ is now obvious but for these two clauses (perhaps):

$$\begin{aligned}
 \mathcal{R}[\alpha?x](\sigma) &= \{in_1(\alpha \otimes up(\lambda_{\perp} n.\sigma[x = n] \otimes up(\lambda_{\perp} \sigma.\{in_0(\sigma)\})))\}; \\
 \mathcal{R}[\alpha!e](\sigma) &= \{in_2(\alpha \otimes \mathcal{E}[e](\sigma) \otimes up(\lambda_{\perp} \sigma.\{in_0(\sigma)\})))\}.
 \end{aligned}$$

Exercise 51. The same language can be used for other communication disciplines. For example the acts of inputting and outputting can be completely independent. Give this idea a semantics using as resumptions

$$\mathbf{R} \cong \mathbf{States}_{\perp} \rightarrow_{\perp} \mathcal{P}_{\mathbf{R}}(\mathbf{States}_{\perp} + \mathbf{States}_{\perp} \otimes \mathbf{R}_{\perp})$$

where $\mathbf{States} = \mathbf{S}_{\perp} \times (\mathbf{Lab}_{\perp} \rightarrow_{\perp} \mathbf{N}_{\perp})$.

Adapt this idea to the case where the channels are actually buffers holding sequences of integers.

Exercise 52. The above language is a mixture of those employed by Hoare and Milner. Give a semantics to Hoare's CSP as in the CACM paper, but (at least at first) without his termination conventions. You may find it helpful to look for a denotational version of

G. D. Plotkin. An Operational Semantics for CSP. IFIP WG 2.2 Working Conference, Garmisch, 1982.

Exercise 53. Give a semantics to Milner's CCS which is an applicative version of the above concurrent language:

$$\begin{aligned}
 t ::= & \mathbf{nil} \mid t + t \mid \alpha?x.t \mid \alpha!e.t \mid \mathbf{if } b \mathbf{ then } t \mathbf{ else } t \mid \mathbf{let } x \mathbf{ be } e \mathbf{ in } t \mid \mathbf{let } \dots \mid \\
 & P \mid \mathbf{let } P \mathbf{ be } t \mathbf{ in } t \mid \mathbf{letrec } P \mathbf{ be } t \mathbf{ in } t \mid \dots \mid t|t \mid t \backslash \alpha \mid t[\alpha/\beta].
 \end{aligned}$$

The meaning of something is its set of communication capabilities, e.g. $\mathbf{nil}$ means $\emptyset$, having none. Give a semantics using:

$$\mathbf{R} \cong \mathcal{P}_{\mathbf{R}}(\mathbf{Lab}_{\perp} \otimes \mathbf{N}_{\perp} \otimes \mathbf{R}_{\perp} + \mathbf{Lab}_{\perp} \otimes (\mathbf{N}_{\perp} \rightarrow_{\perp} \mathbf{R})_{\perp}).$$

Exercise 54. This is a mini-project. Formulate a version of Hewitt's actors language and find a resumption-based semantics.

As an exercise along these lines, take the language of Exercise 51 and think of α a "port in the ether". Then $\alpha!e$ means send the value of e to α through the ether and $\alpha?e$ means take a value from the port α . One way to do this is to use

$$\mathbf{States} = \mathbf{S}_{\perp} \times (\mathbf{Lab}_{\perp} \rightarrow_{\perp} \mathcal{M}(\mathbf{N})_{\perp})$$

where $\mathcal{M}(\mathbf{N})$ is the set of finite *multisets* of integers.

General Powerdomains: The Plotkin Powerdomain

We now try to construct a powerdomain of an arbitrary ω -algebraic cpo $\mathbf{D}$ generalising the Egli-Milner approach in the discrete case. As a typical example we can take $\mathbf{D} = \mathbf{Tapes}$, the set of finite strings of 0's and 1's under the subsequence ordering. To see this in a computational setting we add a print instruction to our language:

$c ::= \mathbf{print} \ t$

(where t ranges over $\{0, 1\}$) and desire a semantics such that $\mathcal{C}[[c]](\sigma)$ is the set of tapes that might be output. Here are some example programs:

- (5) $(\mathbf{print} \ 0; \mathbf{loop}) \text{ or } (\mathbf{print} \ 000; \mathbf{loop}),$
- (6) $(\mathbf{print} \ 0; \mathbf{loop}) \text{ or } (\mathbf{print} \ 00; \mathbf{loop}) \text{ or } (\mathbf{print} \ 000; \mathbf{loop}),$
- (7) $\mathit{alarm} := \text{"off"};$
 $(\mathbf{while} \ \mathit{alarm} = \text{"off"} \ \mathbf{do} \ x := x + 1 \ \mathbf{or} \ \mathit{alarm} := \text{"on"});$
 $(\mathbf{while} \ x > 0 \ \mathbf{do} \ \mathbf{print} \ 0; \ x := x - 1);$
 $(\mathbf{while} \ \mathit{true} \ \mathbf{do} \ \mathbf{print} \ 1),$
- (8) $(\mathbf{while} \ \mathit{true} \ \mathbf{do} \ \mathbf{print} \ 0) \text{ or } (7).$

Clearly (from an operational point of view) we expect (5), (6), (7), (8) to output the following sets

$$\begin{aligned} A_5 &= \{0, 000\} = \{0\} \cup \{000\}, \\ A_6 &= \{0, 00, 000\} = \{0\} \cup \{00\} \cup \{000\}, \\ A_7 &= \{\perp\} \cup \{0^n 1^\omega \mid n \geq 0\}, \\ A_8 &= \{\perp\} \cup \{0^n 1^\omega \mid n \geq 0\} \cup \{0^\omega\}. \end{aligned}$$

So we may expect that (5) $\neq$ (6) and (7) $\neq$ (8) in the anticipated semantics. *But this is not so*: we will have both (5) = (6) (and $A_5 = A_6$) and (7) = (8) (and $A_7 = A_8$) and we can show: *these identifications cannot be avoided in our present framework of cpos and continuous functions* (under a few reasonable assumptions). The latter identification is particularly upsetting: A_7 is a set of sequences which are guaranteed to contain a 1 if they contain a 0 but A_8 is not such a set.

Investigating these points will lead us to an algebraic characterisation of powerdomains, generalising Facts 1, 2 above. It will be very easy using the completion ideas of Chapter 6 to show that such powerdomains exist. Unfortunately this does not give us much insight since under this presentation the powerdomains are *not* given as collections of sets. In order to do that we have to proceed in analogy with development of the Egli-Milner order considering what subsets we expect and how to order them.

Now what a powerdomain $\mathbf{P}$ of an ω -algebraic cpo $\mathbf{D}$ should supply us with is a binary union function: $\cup: \mathbf{P}^2 \rightarrow \mathbf{P}$ and a singleton function $\{\cdot\}: \mathbf{D} \rightarrow \mathbf{P}$; what is more this data should be the universal such data in order to get an analogue of Fact 2 and Theorem 2 allowing us (for one thing!) to compose nondeterministic functions $f: \mathbf{D} \rightarrow \mathbf{P}$. Let us turn these ideas into definitions.

Definition 7. A *semilattice* is a structure $\langle \mathbf{P}, \cup \rangle$ where $\cup: \mathbf{P}^2 \rightarrow \mathbf{P}$ is a binary function satisfying the following three axioms:

$$\begin{aligned} \text{Associativity} & \quad \forall a, b, c \in \mathbf{P}. a \cup (b \cup c) = (a \cup b) \cup c. \\ \text{Commutativity} & \quad \forall a, b \in \mathbf{P}. a \cup b = b \cup a. \\ \text{Absorption} & \quad \forall a \in \mathbf{P}. a \cup a = a. \end{aligned}$$

A function $f: \mathbf{P} \rightarrow \mathbf{Q}$ between semilattice is *linear* iff for all a, b in $\mathbf{P}$ we have $f(a \cup b) = f(a) \cup f(b)$. A structure $\langle \mathbf{P}, \sqsubseteq, \cup \rangle$ is a *continuous semilattice* iff $\langle \mathbf{P}, \sqsubseteq \rangle$ is a cpo, $\langle \mathbf{P}, \cup \rangle$ is a semilattice and $\cup$ is continuous (w.r.t. the ordering $\sqsubseteq$).

Exercise 55. Let $\mathbf{P}$ be a semilattice and define a relation by $x \sqsubseteq y$ iff $x \cup y = y$. Show $\sqsubseteq$ is a partial order with lubs of pairs given by $\cup$; this gives 1-1 correspondence between semilattices and partial orders with lubs of pairs. Assuming $\mathbf{P}$ is a continuous semilattice show that $\sqsubseteq$ is ω -inductive.

Exercise 56. Show that the ω -complete cpos are exactly the continuous semilattices such that for all x, y

$$x \cup y = x \sqcup y$$

and these are the continuous semilattices such that for all x, y

$$x \sqsubseteq x \cup y$$

and these are the same as the continuous semilattices such that for all x, y

$$x \subseteq y \Rightarrow x \sqsubseteq y$$

and these are the same as the continuous semilattices such that for all x, y equivalence: $x \subseteq y \Leftrightarrow x \sqsubseteq y$.

Also the additive, continuous $f: \mathbf{D} \rightarrow \mathbf{E}$ between ω -complete cpos are just linear, continuous $f: \mathbf{D} \rightarrow \mathbf{E}$ when $\mathbf{D}$ and $\mathbf{E}$ are viewed as these kinds of semilattice. One can then reinterpret Theorem 2 as saying that $\mathcal{P}_R(\mathbf{D})$ is just the free continuous semilattice over $\mathbf{D}$ where subset and approximation agree.

So a powerdomain of $\mathbf{D}$ should give us a map $\mathbf{D} \xrightarrow{\{\cdot\}} \mathbf{P}$ with $\mathbf{P}$ a continuous semilattice and we will look for the universal one. Before doing so let us see why the above identifications of sets can be expected to hold. Let $\mathbf{D} \xrightarrow{\{\cdot\}} \langle \mathbf{P}, \cup \rangle$ be as expected with $\mathbf{D} = \mathbf{Tapes}$. Then we expect that the elements in $\mathbf{P}$ corresponding to A_5 and A_6 are $\{0\} \cup \{000\}$ and $\{0\} \cup \{00\} \cup \{000\}$ respectively and calculate:

$$\begin{aligned} \{0\} \cup \{000\} &= (\{0\} \cup \{0\}) \cup \{000\} && \text{(by absorption)} \\ &\sqsubseteq (\{0\} \cup \{00\}) \cup \{000\} && \text{since } 0 \sqsubseteq 00 \text{ and } \{\cdot\} \text{ is monotonic} \\ &\sqsubseteq (\{0\} \cup \{000\}) \cup \{000\} && \text{same reason} \\ &= \{0\} \cup \{000\} && \text{by associativity and absorption.} \end{aligned}$$

Thus $\{0\} \cup \{000\} = \{0\} \cup \{00\} \cup \{000\}$ in any reasonable powerdomain (this is what was meant by $A_5 = A_6$).

Turning to the second pair of sets, let A and B be the elements of $\mathbf{P}$ corresponding to A_7 and A_8 . We expect that $B = A \cup \{0^\omega\}$ and $A = A \cup \{\perp\} \cup \{0^n 1^\omega\}$ (any n). It follows from the second of these that $A = A \cup \{0^n\}$ (any n) — by a similar argument to the above that $A_5 = A_6$. But then

$$\begin{aligned} B &= A \cup \{0^\omega\} \\ &= A \cup \bigsqcup_n \{0^n\} && \{\cdot\} \text{ is continuous} \\ &= \bigsqcup_n (A \cup \{0^n\}) && \text{as union is continuous} \\ &= A. \end{aligned}$$

Thus it seems that these identifications cannot be avoided in the framework of cpos and continuous functions; we hope later to sweeten this bitter pill. In his thesis Lehman suggested replacing partial orders by categories; how best to do this remains an interesting open problem.

Now we give a quick construction of the powerdomain. For any $\mathbf{D}$ put

$$\mathcal{F}(\mathbf{D}) \stackrel{\text{def}}{=} \langle \{A \subseteq B_{\mathbf{D}} \mid A \text{ is finite and nonempty}\}, \sqsubseteq_{\text{EM}} \rangle.$$

That is $\mathcal{F}(\mathbf{D})$ is the collection of finite nonempty sets of finite elements of $\mathbf{D}$ under the Egli-Milner ordering. As such it is a preorder (not, in general, a partial order) with a least element $\{\perp\}$. Following Chapter 6 we put

$$\mathcal{P}(\mathbf{D}) \stackrel{\text{def}}{=} \overline{\mathcal{F}(\mathbf{D})}$$

and define $\mathbf{D} \xrightarrow{\{\cdot\}} \mathcal{P}(\mathbf{D})$ by

$$\{d\}_{\mathbf{P}} = \{\{b_1, \dots, b_n\} \mid n > 0 \text{ and } b_i \in B_{\mathbf{D}} \wedge b_i \sqsubseteq d, \text{ for } i = 1, n\}$$

and define the union function by

$$I \cup_{\mathbf{P}} J = \{A \cup B \mid A \in I, B \in J\}$$

Exercise 57. Verify that $\{\cdot\}_{\mathbf{P}}$ and $\cup_{\mathbf{P}}$ are well-defined and continuous; check that $\{b\}_{\mathbf{P}} = [\{b\}]$ for b in $B_{\mathbf{D}}$ and $[A] \cup_{\mathbf{P}} [B] = [A \cup B]$ for A, B in $\mathcal{F}(\mathbf{D})$.

The simple definition of union makes it clear that $\langle \mathcal{P}(\mathbf{D}), \cup_P \rangle$ is a semilattice; we shall drop the subscripts when clear from the context.

Theorem 3. $\mathcal{P}(\mathbf{D})$ is an ω -algebraic cpo which with $\cup$ is a continuous semilattice. Further $\{\cdot\}_P: \mathbf{D} \rightarrow \mathcal{P}(\mathbf{D})$ is a (strict) continuous function which is universal in that if $f: \mathbf{D} \rightarrow \mathbf{P}$ is any (strict) continuous function to a continuous semilattice there is a unique linear (strict) continuous function $f^\dagger: \mathcal{P}(\mathbf{D}) \rightarrow \mathbf{P}$ such that the following diagram commutes

$$\begin{array}{ccc} \mathbf{D} & & \\ \downarrow \{\cdot\}_P & \searrow f & \\ \mathcal{P}(\mathbf{D}) & \xrightarrow{f^\dagger} & \mathbf{P} \end{array}$$

Further $\dagger$ is an isomorphism between the continuous semilattice of the (strict) continuous functions from $\mathbf{D}$ to $\mathbf{P}$ and the continuous semilattice of the linear (strict) continuous function from $\mathcal{P}(\mathbf{D})$ to $\mathbf{P}$ (where the unions of the functions are defined pointwise).

Proof. By construction $\mathcal{P}(\mathbf{D}) = \overline{\mathcal{F}(\mathbf{D})}$ is ω -algebraic and we know it is a continuous semilattice. We know the singleton function is continuous; to see it is strict note that $\{\perp_{\mathbf{D}}\} = [\{\perp\}] = [\perp_{\mathcal{F}(\mathbf{D})}] = \perp_{\mathcal{P}(\mathbf{D})}$.

Next suppose $f: \mathbf{D} \rightarrow \mathbf{P}$ is a (strict) continuous linear function from $\mathbf{D}$ to a continuous semilattice $\mathbf{P}$. Since $f^\dagger$ is continuous it is enough to show it is determined on the finite elements of $\mathcal{P}(\mathbf{D})$ in order to demonstrate uniqueness. So let us calculate:

$$\begin{aligned} f^\dagger([\{b_1, \dots, b_n\}]) &= f^\dagger([\{b_1\}] \cup_P \dots \cup_P [\{b_n\}]) \\ &= f^\dagger(\{b_1\}_P \cup_P \dots \cup_P \{b_n\}_P) \\ &= f^\dagger(\{b_1\}_P) \cup \dots \cup f^\dagger(\{b_n\}_P) && \text{as } f^\dagger \text{ is linear} \\ &= f(b_1) \cup \dots \cup f(b_n) && \text{as the diagram commutes.} \end{aligned}$$

Conversely define $g: \mathcal{F}(\mathbf{D}) \rightarrow \mathbf{P}$ by:

$$g(\{b_1, \dots, b_n\}) = f(b_1) \cup \dots \cup f(b_n).$$

This is monotonic, for if $A \sqsubseteq_{\text{EM}} B$ then we can write $A = \{a_1, \dots, a_n\}$ and $B = \{b_1, \dots, b_n\}$ with $a_i \sqsubseteq b_i$ ($i = 1, n$) making it evident that $g(A) \sqsubseteq g(B)$. Now let $f^\dagger$ be the unique continuous extension of g to $\mathcal{P}(\mathbf{D})$. Since it is continuous we can check the above diagram commutes by looking only at the finite elements of $\mathbf{D}$:

$$f^\dagger(\{b\}_P) = f^\dagger([\{b\}]) = g(\{b\}) = f(b).$$

(If f is strict this shows that $f^\dagger(\perp) = f^\dagger(\{\perp\}_P) = f(\perp) = \perp$ and so $f^\dagger$ is too.) It only remains to check that $f^\dagger$ is linear and by continuity it is enough to check that for the finite elements of $\mathcal{P}(\mathbf{D})$ and we note:

$$\begin{aligned} f^\dagger([A] \cup_P [B]) &= f^\dagger([A \cup B]) \\ &= \bigcup_{a \in A} g(a) \cup \bigcup_{b \in B} g(b) \\ &= g(A) \cup g(B) \\ &= f^\dagger([A]) \cup f^\dagger([B]). \end{aligned}$$

To complete the proof we note that $(\cdot)^\dagger$ is monotonic, as if $f \sqsubseteq f'$ then $g \sqsubseteq g'$ (using an evident notation and so $f^\dagger \sqsubseteq f'^\dagger$). It therefore follows as per Fact 1 that $(\cdot)^\dagger$ is an isomorphism of partial orders with inverse $H = \lambda h. h \circ \{\cdot\}_P$. Note that H is linear:

$$\begin{aligned} H(h \cup h') &= (h \cup h') \circ \{\cdot\}_P \\ &= \lambda d \in \mathbf{D}. h \cup h'(\{d\}_P) \\ &= \lambda d. h(\{d\}_P) \cup h'(\{d\}_P) \\ &= H(h) \cup H(h'). \end{aligned}$$

It follows that $\dagger$ is linear:

$$\begin{aligned}(f \cup f')^\dagger &= (H(f^\dagger) \cup H(f'^\dagger))^\dagger \\ &= H(f^\dagger \cup f'^\dagger)^\dagger \\ &= f^\dagger \cup f'^\dagger\end{aligned}$$

(and it is clear that strictness is also preserved). ■

The import of this theorem is that powerdomains conceived of as free continuous algebras always exist. They are built by making a free monotonic algebra (here $\mathcal{F}(\mathbf{D})/\sqsubseteq_{\text{EM}}$ which is $\mathcal{F}(\mathbf{D})$, modulo $\sqsubseteq_{\text{EM}}$) and completing it to get the free continuous one. This is a very simple procedure and can be applied to any collection of equations and even inequations. Later on we shall take advantage of the idea to construct various other flavours of powerdomains. This by no means makes use of all the possibilities available and we do not really know what scope they have.

Exercise 58. Show that for $f: \mathbf{D} \rightarrow \mathcal{P}(\mathbf{E})$ the extension $f^\dagger: \mathcal{P}(\mathbf{D}) \rightarrow \mathcal{P}(\mathbf{E})$ is given by the formula:

$$f^\dagger(I) = \bigcup_{A \in I} \bigcup_{a \in A} f(a).$$

It is easy to see that $\mathcal{P}$ is a locally continuous covariant functor on $\omega\text{-}\mathcal{ALG}$. It is already defined on objects and for morphisms $f: \mathbf{D} \rightarrow \mathbf{E}$ we define

$$\mathcal{P}(f) = (\{\cdot\}_{\mathbf{E}} \circ f)^\dagger$$

as with the relational powerdomain.

Exercise 59. Show this does define a locally continuous functor. Find a formula for $\mathcal{P}(f)$.

It follows that we can solve domain equations in $\omega\text{-}\mathcal{ALG}$ using $+$, $\times$, $(\cdot)_\perp$, $\otimes$, $\mathcal{P}$ (just repeat the developments of Chapter 4 and 5 but in $\omega\text{-}\mathcal{ALG}$ using the information given in Chapter 6). And with a little effort (using $\mathcal{SFP}$) in $\omega\text{-}\mathcal{ALG}$ we can add in $\rightarrow$ and $\rightarrow_\perp$. For we know that $\mathcal{SFP}$ is closed under $+$, $\times$, $(\cdot)_\perp$, $\otimes$, $\rightarrow$ and $\rightarrow_\perp$. To see that it is closed under $\mathcal{P}$ we note that the above construction shows $\mathcal{P}(\mathbf{D})$ finite when $\mathbf{D}$ is. Therefore if $\mathbf{D}$ is an $\mathcal{SFP}$ object we can write $\mathbf{D} = \lim_{\leftarrow} \langle \mathbf{D}_n, f_n \rangle$ in $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$ and so in $\omega\text{-}\mathcal{ALG}^{\mathbf{E}}$ with the $\mathbf{D}_n$ finite and so $\mathcal{P}(\mathbf{D}) = \lim_{\leftarrow} \langle \mathcal{P}(\mathbf{D}_n), \mathcal{P}(f_n) \rangle$ in $\omega\text{-}\mathcal{ALG}^{\mathbf{E}}$ (and hence $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$) with the $\mathcal{P}(\mathbf{D}_n)$ finite, using the local continuity of $\mathcal{P}$, by analogy with Chapter 4. This shows $\mathcal{SFP}$ closed under $\mathcal{P}$ and we can solve all our domain equations in $\mathcal{SFP}$ (again repeating Chapter 4/5 in $\mathcal{SFP}$ and using the information on $\mathcal{SFP}$ given in Chapter 6).

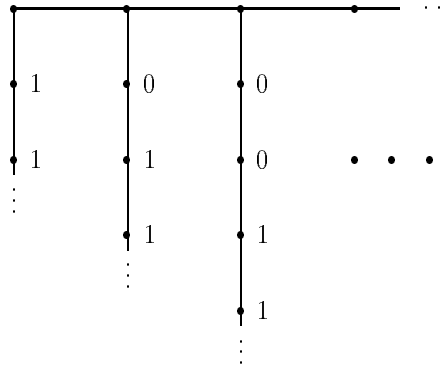
Exercise 60. The above discussion uses the fact that if all the $\mathbf{D}_n$ are ω -algebraic then $\mathbf{D} = \lim_{\leftarrow} \langle \mathbf{D}_n, f_n \rangle$ in $\mathcal{CP}\mathcal{O}^{\mathbf{E}}$ iff $\mathbf{D} = \lim_{\leftarrow} \langle \mathbf{D}_n, f_n \rangle$ in $\omega\text{-}\mathcal{ALG}^{\mathbf{E}}$. Prove this.

Exercise 61. Prove directly using the minimal upper bound characterisation of $\mathcal{SFP}$ objects given in Chapter 6 p. 69 that $\mathcal{P}(\mathbf{D})$ is in $\mathcal{SFP}$ object if $\mathbf{D}$ is.

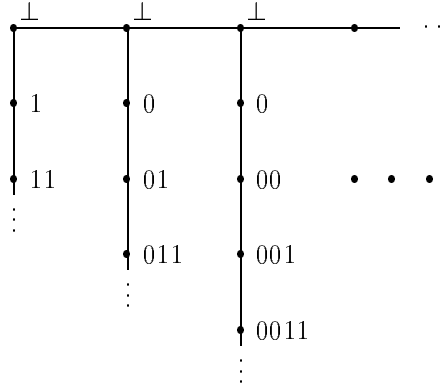
We now turn to the question of what $\mathcal{P}(\mathbf{D})$ looks like considered as a family of subsets of $\mathbf{D}$. As before we need to see what sets we want and how to order them. We will then have to check that the resulting structure is isomorphic to $\mathcal{P}(\mathbf{D})$ as defined above.

Finitely Generable Sets

If we write down the execution tree of program (7) above we arrive at the following



where we have only indicated the effects of the print instructions. Thus A_7 is obtained as the sequence of what is printed out along the branches. As a slight reformulation we might instead write what has been printed so far at a node



and then A_7 is obtained as the lubs along the branches. We have here to deal with *finitely branching trees*. A *generating finitarily branching tree* is a finitarily branching tree with nodes labelled by elements of $\mathbf{D}$ so that the closer a node is to the root the smaller is its label. Every complete branch *generates* the lub of the labels of its nodes and the tree generates the set of elements of $\mathbf{D}$ generated by its nodes. What we do is take (the finite elements) of **Tapes** as our canonical finitarily branching tree.

Definition 8. Let $\mathbf{D}$ be an ω -algebraic cpo. A subset X of $\mathbf{D}$ is *finitely generable* (fg) iff there is a continuous function $f: \mathbf{Tapes} \rightarrow \mathbf{D}$ such that $X = Bd(f)$ where the *boundary* of f is given by

$$Bd(f) = \{f(w) \mid |w| = \omega\}$$

(that is it is the set of values of f on infinite tapes). We write $\mathcal{F}_g(\mathbf{D})$ for the collection of fg subsets of $\mathbf{D}$.

Exercise 62. Show every finite nonempty set is finitely generable. Show that union of two fg sets is fg. Show that if $f: \mathbf{D} \rightarrow \mathbf{E}$ is continuous and $X \subseteq \mathbf{D}$ is fg then so is $f(X)$. Show that $\{0^\omega\} \cup \{0^n 1^\omega\}$ is fg.; show that $\{0^n 1^\omega\}$ is *not* fg (and neither is $\mathbf{N}$ as a subset of $\mathbf{N}_\perp$). How many fg subsets of $\mathbf{D}$ are there? Show that fg sets are *not* closed under intersection, complementation, infinite union or inverse functions (the last meaning that if $Y \subseteq \mathbf{E}$ is fg then $f^{-1}(Y)$ need not be with f continuous).

Exercise 63. Show that a set is fg iff it is generated by a finitely branching generating tree.

Lemma 3. If $X \subseteq \mathbf{D}$ is finitely generable then it is $Bd(f)$ for some f such that $f(w)$ is finite when w is.

Proof. Let $X = Bd(g)$. Let $b_0, b_1, \dots$ enumerate $B_{\mathbf{D}}$. Define f by putting $f(\perp) = \perp$ and for $w \in B_{\mathbf{Tapes}}$, $t \in \{0, 1\}$

$$f(w \smallfrown t) = \text{the first } b_n \text{ greater than } f(w) \text{ and } b_n \sqsubseteq g(w \smallfrown t) \text{ and } b_j \sqsubseteq b_n \text{ (s.t. } \forall j \leq |w|. b_j \sqsubseteq g(w \smallfrown t)).$$

Note that $f(w)$ is well-defined and less than $g(w)$ (for finite w). Then f is monotonic on finite tapes and so is uniquely extended to a continuous function on **Tapes**. Clearly, then, $f \sqsubseteq g$. The converse is left as

Exercise 64. ■

How to Order Our Sets

Let us denote by $\sqsubseteq$ the order we are looking for on subsets of any given ω -algebraic $\mathbf{D}$. We expect that this contains $\sqsubseteq_{\text{EM}}$ but know from the above that since $A_8 \sqsubseteq_{\text{EM}} A_7$ it cannot be identical with it.

Exercise 65. It would have been natural to take the collection of the fg sets of $\mathbf{D}$ with the natural singleton and union functions and preordered by $\sqsubseteq_{\text{EM}}$ and then divide out by the induced equivalence to get a candidate powerdomain. But we know from the above investigation that something must go wrong even with $\mathbf{D} = \mathbf{Tapes}$: either we don't get a cpo or singleton or union is not continuous. Which of these happens?

Exercise 66. Show that if $f: \mathbf{D} \rightarrow \mathbf{E}$ is continuous and for two subsets X and Y of $\mathbf{D}$ we have $X \sqsubseteq_{\text{EM}} Y$ then $f(X) \sqsubseteq_{\text{EM}} f(Y)$.

Following the last exercise it seems natural that $\sqsubseteq$ should be preserved by continuous functions and since it should be $\sqsubseteq_{\text{EM}}$ on discrete cpos we expect that if we define for all subsets of $\mathbf{D}$

$$X \sqsubseteq_{\text{EM}} Y \quad \text{iff} \quad \forall f: \mathbf{D} \rightarrow \mathbf{O}. f(X) \sqsubseteq_{\text{EM}} f(Y)$$

(and let $\simeq_{\text{EM}}$ be the associated equivalence) then $X \sqsubseteq Y$ implies $X \sqsubseteq_{\text{EM}} Y$. Thus we have: $\sqsubseteq_{\text{EM}} \subseteq \sqsubseteq \subseteq \sqsubseteq_{\text{EM}}$. One can think of $f^*: \mathbf{D} \rightarrow \mathbf{O}$ as an *experiment* on $\mathbf{D}$.

Exercise 67. Show that for any continuous $f: \mathbf{D} \rightarrow \mathcal{P}(X_{\perp})$ with X countable we have: $X \sqsubseteq_{\text{EM}} Y$ implies $\bigcup f(X) \sqsubseteq_{\text{EM}} \bigcup f(Y)$.

We now study $\sqsubseteq_{\text{EM}}$ with a view to arguing that it should be our choice of $\sqsubseteq$.

Theorem 4. Let $\mathbf{D}$ be an ω -algebraic cpo. Let X and Y be subsets of $\mathbf{D}$ and consider the following conditions.

- (1) $X \sqsubseteq_{\text{EM}} Y$,
- (2) $X \sqsubseteq_{\text{EM}} Y$,
- (3) $\forall x \in X. \forall b \in B_{\mathbf{D}}. (b \sqsubseteq x \Rightarrow \exists y \in Y. b \sqsubseteq y) \wedge \forall y \in Y. \exists x \in X. x \sqsubseteq y$,
- (4) $\forall A \subseteq B_{\mathbf{D}}. (A \text{ finite} \wedge A \sqsubseteq_{\text{EM}} X) \Rightarrow A \sqsubseteq_{\text{EM}} Y$.

Then we have (1) $\Rightarrow$ (2) $\Leftrightarrow$ (3) $\Rightarrow$ (4) with (3) $\Leftrightarrow$ (4) if X is finitely generable and (1) $\Leftrightarrow$ (2) if Y is finite or if X is a subset of $B_{\mathbf{D}}$.

Proof. (1) $\Rightarrow$ (2). Already known.

(2) $\Rightarrow$ (3). That (2) implies the first conjunct is known from Theorem 1. For the converse take $y \in Y$. Define $f: \mathbf{D} \rightarrow \mathbf{O}$ by $f(d) = \top$ iff $d \sqsubseteq y$. Then $\perp \in f(Y)$ and so $\perp \in f(X)$; then for some x in X , $f(x) = \perp$, which is to say $x \sqsubseteq y$.

(3) $\Rightarrow$ (2). Left as an exercise.

(3) $\Rightarrow$ (4). Left as an exercise.

(4) $\Rightarrow$ (3) when X is fg. Take $b \sqsubseteq x$ with $b \in B_{\mathbf{D}}$ and $x \in X$. Then $\{\perp, b\} \sqsubseteq_{\text{EM}} X$ and so $\{\perp, b\} \sqsubseteq_{\text{EM}} Y$ and so $b \sqsubseteq y$ for some y in Y . Next take $y \in Y$ and $f: \mathbf{Tapes} \rightarrow \mathbf{D}$ generating X . Suppose for the sake of contradiction that for all $x \in X$ we have $x \not\sqsubseteq y$. Then for any infinite w in $\mathbf{Tapes}$, $f(w) \not\sqsubseteq y$ and so there is a finite $a \sqsubseteq w$ with $f(a) \not\sqsubseteq y$. Now as $\mathbf{Tapes}$ is a finitarily branching tree it follows by König's Lemma that there is a finite cross section A of $\mathbf{Tapes}$ with $A \subseteq B_{\mathbf{Tapes}}$ and for all a in A , $f(a) \not\sqsubseteq y$. Then $f(A) \sqsubseteq_{\text{EM}} X$ and so $f(A) \sqsubseteq_{\text{EM}} Y$ but then there must be an $f(a)$ with $f(a) \sqsubseteq y$ giving us the required contradiction.

(2) $\Rightarrow$ (1) when X is a subset of $B_{\mathbf{D}}$. Immediate from (2) $\Rightarrow$ (4).

(2) $\Rightarrow$ (1) when Y is finite. Left as **Exercise 68** [Hint Use (3)]. ■

So for finite sets we have $\sqsubseteq_{\text{EM}} = \sqsubseteq = \sqsubseteq_{\text{EM}}$ and indeed we know that for finite $\mathbf{D}$ we have $\mathcal{P}(\mathbf{D}) = \mathcal{F}(\mathbf{D}) / \sqsubseteq_{\text{EM}}$.

Condition (3) shows how close $\sqsubseteq_{\text{EM}}$ is to being $\sqsubseteq_{\text{EM}}$ and condition (4) shows that for fg sets $X \sqsubseteq_{\text{EM}} Y$ iff any finite Egli-Milner information about X (= picture of a cross-section of the development of X) is also about Y . In fact it is natural to expect any fg set is the limit of its cross-sections and we now show this is true w.r.t. $\sqsubseteq_{\text{EM}}$.

Definition 9. For $f: \mathbf{Tapes} \rightarrow \mathbf{D}$ define $Bd_n(f) = \{f(w) \mid |w| = n\}$.

Clearly for $m \leq n \leq \omega$ we have $Bd_m(f) \sqsubseteq_{\text{EM}} Bd_n(f) \sqsubseteq_{\text{EM}} Bd(f)$.

Lemma 4. Let $f: \mathbf{Tapes} \rightarrow \mathbf{D}$ generate X . If A is a finite subset of $B_{\mathbf{D}}$ such that $A \sqsubseteq_{\text{EM}} X$ then for some n , $A \sqsubseteq_{\text{EM}} Bd_n(f)$. Further if $Bd_n(f) \sqsubseteq_{\text{EM}} Y$ for all n then $X \sqsubseteq_{\text{EM}} Y$.

Proof. As $A \sqsubseteq_{\text{EM}} X$ for every infinite w in $\mathbf{Tapes}$ there is a finite $v \sqsubseteq w$ with $a \sqsubseteq f(v)$ for some a in A . Hence by König's Lemma there is an n_0 such that $|v| \geq n_0$ implies $a \sqsubseteq f(v)$ for some a in A . Equally as $A \sqsubseteq_{\text{EM}} X$ for every a in A there is an infinite w with $a \sqsubseteq f(w)$ and hence a finite $v_a \sqsubseteq w$ with $a \sqsubseteq f(v_a)$. Now put $n = \max(n_0, \max_{a \in A}(|v_a|))$ and then $A \sqsubseteq Bd_n(f)$. The rest now follows using from Theorem 4 that (4) $\Rightarrow$ (2) for fg X and (1) $\Leftrightarrow$ (2) for finite subsets of finite elements. ■

Now assuming a similar property holds of $\sqsubseteq$ we can show it must be $\sqsubseteq_{\text{EM}}$ at least as regards fg sets. Specifically, assume that:

$$\forall f: \mathbf{Tapes} \rightarrow \mathbf{D}. \forall Y \subseteq \mathbf{D}. (\forall n. Bd_n(f) \sqsubseteq Y) \Rightarrow Bd(f) \sqsubseteq Y.$$

Now suppose $X \sqsubseteq_{\text{EM}} Y$ with X fg. Then by Lemma 3 there is an $f: \mathbf{Tapes} \rightarrow \mathbf{D}$ with $X = Bd(f)$ and $Bd_n(f) \subseteq B_{\mathbf{D}}$ for every n . Then for any n we have $Bd_n(f) \sqsubseteq_{\text{EM}} Y$; so by Theorem 4 we have $Bd_n(f) \sqsubseteq_{\text{EM}} Y$ (since $Bd_n(f)$ is a finite set of finite elements) and so $Bd_n(f) \sqsubseteq Y$. Therefore by the assumption $X = Bd(f) \sqsubseteq Y$. Thus we have established, under reasonable assumptions, that $\sqsubseteq$ is $\sqsubseteq_{\text{EM}}$.

Our proposed powerdomain according to this analysis is therefore $\langle \mathcal{F}_g(\mathbf{D}), \sqsubseteq_{\text{EM}} \rangle / \sqsubseteq_{\text{EM}}$. Since we already have a proposal from Theorem 3 we must now prove that the two possibilities are isomorphic.

Lemma 5. *Let $A_0 \sqsubseteq_{\text{EM}} A_1 \sqsubseteq_{\text{EM}} \dots$ be an increasing sequence of elements of $\mathcal{F}(\mathbf{D})$. Then there is an $f: \mathbf{Tapes} \rightarrow \mathbf{D}$ with every A_n being some $Bd_m(f)$ (with $m \geq n$). Further $Bd(f) = \{\bigcup a_n \mid \forall n. a_n \in A_n, a_0 \sqsubseteq a_1 \sqsubseteq \dots\}$.*

Proof. Define a finitary tree $T = \{\langle a_0, \dots, a_n \rangle \mid a_i \in A_i \text{ and } a_0 \sqsubseteq \dots \sqsubseteq a_n\}$ and label $\langle a_0, \dots, a_n \rangle$ by a_n for $n > 0$ and by $\perp$ for $n = 0$. Then A_n is the set of labels of the set of nodes of depth $n + 1$. Finally pad out T so that it is (isomorphic to) $B_{\mathbf{Tapes}}$ and choose the labelling so that every A_n is the set of labels of some $\mathbf{Tapes}_m$ and use the labels to construct f . The last part is an easy König's Lemma argument on $\mathbf{T}$. ■

We also need a lemma about ω -algebraic cpos.

Lemma 6. *Let P be a partial order with a least element. Suppose $B \subseteq P$ is a countable subset of P such that:*

- (1) *Every element of P is the lub of an increasing sequence of elements of B .*
- (2) *Every increasing sequence of elements of B has a lub in P .*
- (3) *If $b \sqsubseteq \bigcup_p b_i$ where $b_0 \sqsubseteq b_1 \sqsubseteq \dots$ and b and the b_i are all in B then $b \sqsubseteq$ some b_n .*

Then P is ω -algebraic and B its set of finite elements.

Proof. To see P is a cpo let $d_0 \sqsubseteq d_1 \sqsubseteq \dots$ be an increasing sequence in P . Let $X = \{b \in B \mid b \sqsubseteq \text{some } d_i\}$. Then $\perp$ is in X . Then one sees from (1) and (3) that X is directed. Hence as X is countable we can take a cofinal chain in X to show by (2) that $d = \bigcup X$ exists. By (1) we have that d is an upper bound of every d_n and the definition of X shows it must be the least one.

Next take b in B and let $d_0 \sqsubseteq d_1 \sqsubseteq \dots$ be an increasing sequence and suppose $b \sqsubseteq \bigcup_n d_n$. Define X as above. We know $b \sqsubseteq \bigcup X$ and hence as X is directed and countable it follows from (3) that $b \sqsubseteq$ some element of $X \sqsubseteq$ some d_n . Hence b is finite and so $B_0 \subseteq B$.

Therefore by Chapter 6 Lemma 1, $\mathbf{D}$ is ω -algebraic. ■

It follows from Lemma 6 that $\mathcal{F}_g(\mathbf{D}) / \sqsubseteq_{\text{EM}}$ is ω -algebraic and its set of finite elements $\{A \downarrow \mid A \in \mathcal{F}(\mathbf{D})\}$ (where, of course, $A \downarrow = \{X \in \mathcal{F}_g(\mathbf{D}) \mid X \sqsubseteq_{\text{EM}} A\}$). For condition (1) of that lemma is immediate from Lemma 3 and Lemma 4; condition (2) follows from Lemma 4 and Lemma 5; condition (3) from Lemma 5 and Lemma 4. But now we note

$$\begin{aligned} \langle B_{\mathcal{F}_g(\mathbf{D})} / \sqsubseteq_{\text{EM}}, \subseteq \rangle &\cong \langle \mathcal{F}(\mathbf{D}), \sqsubseteq_{\text{EM}} \rangle / \sqsubseteq_{\text{EM}} && \text{since for } A, B \text{ in } \mathcal{F}(\mathbf{D}), A \downarrow \subseteq B \downarrow \text{ iff } A \sqsubseteq_{\text{EM}} B \\ &= \langle \mathcal{F}(\mathbf{D}), \sqsubseteq_{\text{EM}} \rangle / \sqsubseteq_{\text{EM}} && \text{by remark after Theorem 4.} \end{aligned}$$

Therefore $\mathcal{F}_g(\mathbf{D}) / \sqsubseteq_{\text{EM}} = \overline{\mathcal{F}(\mathbf{D})} = \mathcal{P}(\mathbf{D})$ showing as promised that the two possibilities are indeed isomorphic.

Exercise 69. Show that we should define $\{\cdot\}_P: \mathbf{D} \rightarrow \mathcal{F}_g(\mathbf{D}) / \sqsubseteq_{\text{EM}}$ and $\cup_P: (\mathcal{F}_g(\mathbf{D}) / \sqsubseteq_{\text{EM}})^2 \rightarrow \mathcal{F}_g(\mathbf{D}) / \sqsubseteq_{\text{EM}}$ by $\{d\}_P = \{d\} \downarrow$ and $A \downarrow \cup_P B \downarrow = (A \cup B) \downarrow$ respectively. No particularly revealing formula for function extension is known.

Thus we have now greatly increased our understanding of powerdomains: the sets are the finitely generable sets, modulo $\sqsubseteq_{\text{EM}}$ and the ordering is $\sqsubseteq_{\text{EM}}$; further the singleton and union functions have natural definitions and Lemma 5 shows us what lubs of increasing chains of finite elements look like.

Canonical Representatives and Scott-Compact Sets

We now explore the equivalence relation $\simeq_{\text{EM}}$ in order to obtain canonical (actually maximal) elements of the equivalence classes. Following Theorem 3(3) there is a very natural closure operation. For any set X define X^* by:

$$X^* = \{y \mid (\exists x \in X. x \sqsubseteq y) \wedge (\forall b \in B_{\mathbf{D}}. b \sqsubseteq y \Rightarrow \exists x \in X. b \sqsubseteq x)\}.$$

Proposition 1. For any set $X \subseteq \mathbf{D}$ we have

- (1) $X \subseteq X^*$,
- (2) $X^{**} = X^*$,
- (3) $X \simeq_{\text{EM}} Y$ iff $X^* = Y^*$

(and so (4) $X \simeq_{\text{EM}} X^*$).

Proof. Exercise 70. ■

Thus $(\cdot)^*$ is a closure operation and X^* is the maximal member of its equivalence class. What we would like to prove now is that X^* is fg; however it is *open* whether this is true in general but we can proceed further by considering the *Scott-compact* sets which, as we will see, generalise fg sets (Proposition 2) but are the fg sets, to within $\simeq_{\text{EM}}$. (It should be noted that better results will be obtained later for *special kinds* of ω -algebraic cpos, the so-called 2/3 SFP ones.)

Exercise 71. This exercise looks at the convexity part of $\simeq_{\text{EM}}$ as illustrated by $A_5 = A_6$.

Definition 10. Let P be a preorder. A subset X of P is *convex* iff $\forall x, z \in X. \forall y \in P. x \sqsubseteq y \sqsubseteq z \Rightarrow y \in X$. The least convex set containing a given $X \subseteq P$ is:

$$\text{Con}(X) = \{y \in P \mid \exists x, z \in X. x \sqsubseteq y \sqsubseteq z\}.$$

Show that $\text{Con}(X^*) = X^* \supseteq \text{Con}(X)$ but equality does not always hold. Show that $\mathcal{F}(\mathbf{D})/\sqsubseteq_{\text{EM}} \cong \langle \{\text{Con}(A) \mid A \in \mathcal{F}(\mathbf{D})\}, \sqsubseteq_{\text{EM}} \rangle$ and conclude that for *finite* $\mathbf{D}$, we have $\mathcal{P}(\mathbf{D}) \cong \langle \{\text{Con}(A) \mid A \subseteq \mathbf{D}, A \neq \emptyset\}, \sqsubseteq_{\text{EM}} \rangle$. Note that Con is *not* a topological closure operator (that is we do not always have $\text{Con}(X \cup Y) = \text{Con}(X) \cup \text{Con}(Y)$) and conclude the same for $(\cdot)^*$.

Definition 11. A set $C \subseteq B_{\mathbf{D}}$ *covers* $X \subseteq \mathbf{D}$ iff $\forall x \in X. \exists c \in C. c \sqsubseteq x$. A set $X \subseteq \mathbf{D}$ is *Scott-compact* iff whenever C covers X so does a finite subset of C .

The idea is to use compactness to replace König's Lemma arguments.

Exercise 72. Show that X is Scott-compact iff it is compact in the Scott topology. Show that whenever C is finite and covers X then $C \cup X$ is Scott-compact. Conclude there are Scott-compact sets which are not fg; how many are there?

Proposition 2. Every finitely generable set is Scott-compact.

Proof. Let X be fg with generating function $f: \mathbf{Tapes} \rightarrow \mathbf{D}$. Let C cover X . Then for every infinite w in $\mathbf{Tapes}$ $f(w) \sqsupseteq$ some c in C . So $f(v) \sqsupseteq c$ for some finite $v \sqsubseteq w$. Applying König's Lemma we obtain a cross-section of such v 's and the corresponding c 's provide the needed finite subset of C . ■

This proposition makes it plain why sets like $\mathbf{N} \subseteq \mathbf{N}_{\perp}$ are not fg.

Exercise 73. Show that if $X \simeq_{\text{EM}} Y$ then X is Scott-compact iff Y is.

So we know that for a fg X , X^* is Scott-compact and so by Proposition 1 it is the maximal member of the equivalence class $\{Y \subseteq \mathbf{D} \mid Y \text{ is Scott-compact and } Y \simeq_{\text{EM}} X\}$. This suggests that

$$\mathcal{P}(\mathbf{D}) \cong \langle (\cdot)^*\text{-closed nonempty Scott-compact subsets of } \mathbf{D}, \sqsubseteq_{\text{EM}} \rangle$$

and to establish this it only remains to show that every Scott-compact set is $\simeq_{\text{EM}}$ -equivalent to a fg set, which we now do.

Lemma 7. Let $C \subseteq \mathbf{D}$ be nonempty and Scott-compact. Then $I(C) \stackrel{\text{def}}{=} \{A \in \mathcal{F}(\mathbf{D}) \mid A \sqsubseteq_{\text{EM}} C\}$ is a directed ideal and has $\sqsubseteq_{\text{EM}}$ -lub C (that is if for some $X \subseteq \mathbf{D}$ we have $\forall A \in I(C). A \sqsubseteq_{\text{EM}} X$ then $C \sqsubseteq X$).

Proof. To see $I(C)$ is an ideal the only nontrivial point is to show that it is directed. Clearly $\{\perp\}$ is in $I(C)$ so it is nonempty. Suppose A and B are in $I(C)$. Then C is covered by A and B and so as C is compact it is covered by $\{x \in B_{\mathbf{D}} \mid \exists a \in A, b \in B, c \in C. a \sqsubseteq x \wedge b \sqsubseteq x \wedge x \sqsubseteq c\}$ and so by some finite subcollection, X , of the latter set. Thus we have an X in $\mathcal{F}(\mathbf{D})$ which is covered by both A and B and with $X \sqsubseteq_{\text{EM}} C$.

For any a in A there is a c in C with $c \sqsupseteq a$ and for that c there is an x in X so that $c \sqsupseteq x$. But then there is an a' such that $a, x \sqsubseteq a' \sqsubseteq c$. Let A' be the finite collection of these a' 's. Then $A \sqsubseteq_{\text{EM}} X \cup A' \sqsubseteq_{\text{EM}} C$ and B covers $X \cup A'$. Defining B' similarly we find that $A, B \sqsubseteq_{\text{EM}} X \cup A' \cup B' \sqsubseteq_{\text{EM}} C$, concluding the proof that $I(C)$ is directed.

Next suppose that $\forall A \in I(C). A \sqsubseteq_{\text{EM}} X$. We will show that $C \sqsubseteq X$. Take any $f: \mathbf{D} \rightarrow \mathbf{O}$ to show that $f(C) \sqsubseteq_{\text{EM}} f(X)$. If $\top \in f(C)$ then for some c in C and finite $a \sqsubseteq c$ we have $\top = f(a)$ and so considering $\{a, \perp\}$ in $I(C)$ we find that $\top \in f(X)$. finally suppose $\perp \notin f(C)$. Then $\{a \in B_{\mathbf{D}} \mid f(a) = \top \wedge \exists c \in C. a \sqsubseteq c\}$ covers C and so as C is compact it has a finite subcover, A , say. Then $A \sqsubseteq_{\text{EM}} C$ and $\perp \notin f(A)$. So as $A \sqsubseteq_{\text{EM}} X$ we have $\perp \notin f(X)$ either. ■

(Compare this last argument to the König's Lemma argument in the last part of theorem).

Theorem 5. *Every Scott-compact set is $\simeq_{\text{EM}}$ -equivalent to some fg set.*

Proof. Immediate from Lemmas 7, 4 and Theorem 4. ■

This establishes our desired characterisation of $\mathcal{P}(\mathbf{D})$ as the collection of $(\cdot)^*$ -closed Scott-compact sets partially ordered by $\sqsubseteq_{\text{EM}}$. So we have our first characterisation of $\mathcal{P}(\mathbf{D})$ as a collection of subsets; the open problem is still to see if all $(\cdot)^*$ -closed Scott-compact sets are actually finitely generable.

Exercise 74. Show that in terms of the latest characterisation of $\mathcal{P}(\mathbf{D})$ we have for $x \in \mathbf{D}$, $X, Y \in \mathcal{P}(\mathbf{D})$, $f: \mathbf{D} \rightarrow \mathcal{P}(\mathbf{E})$, $g: \mathbf{D} \rightarrow \mathcal{P}(\mathbf{E})$:

$$\begin{aligned} \{x\}_{\mathbf{P}} &= \{x\}, \\ X \cup_{\mathbf{P}} Y &= (X \cup Y)^*, \\ f^\dagger(X) &= \left(\bigcup_{x \in X} f(x) \right)^*, \\ \mathcal{P}(g)(X) &= (g(X))^*. \end{aligned}$$

The Lawson Topology

Our understanding of powerdomain is still incomplete in that we have not systematically considered the identification $A_7 = A_8$. What seems to be wanted is a notion of limit of a sequence so that, for example:

$$\lim_{n \rightarrow \infty} 0^n 1^\omega = 0^\omega.$$

Attending to this desire will have several pleasant consequences: $(\cdot)^*$ -closure will be explained in terms of these limits and convex closure; the X^* will be fg; $\sqsubseteq_{\text{EM}}$ will just be $\sqsubseteq_{\text{EM}}$ on the X^* . (But all this will only work for certain ω -algebraic cpos, remember.)

So let us say when $\langle x_n \rangle$ has *limit* x (or *converges* to x) writing:

$$\begin{aligned} \lim_n x_n = x \quad &\text{iff} \quad \forall b \in B_{\mathbf{D}}. [b \sqsubseteq x \Leftrightarrow \forall_\infty n. b \sqsubseteq x_n] \wedge [b \not\sqsubseteq x \Leftrightarrow \forall_\infty n. b \not\sqsubseteq x_n] \\ &(\text{iff} \quad \forall b \in B_{\mathbf{D}}. [b \sqsubseteq x \Rightarrow \forall_\infty n. b \sqsubseteq x_n] \wedge [b \not\sqsubseteq x \Rightarrow \forall_\infty n. b \not\sqsubseteq x_n]) \\ &(\text{iff} \quad \forall b \in B_{\mathbf{D}}. [b \sqsubseteq x \Rightarrow \forall_\infty n. b \sqsubseteq x_n] \wedge [\exists_\infty n. b \sqsubseteq x_n \Rightarrow b \sqsubseteq x]). \end{aligned}$$

(Here the notations $\forall_\infty n. P(n)$, $\exists_\infty n. P(n)$ mean that P holds for all but finitely many n , respectively infinitely many n .)

The elementary properties of these limits form an exercise.

Exercise 75.

- (1) Let $\langle x_n \rangle$ and $\langle y_n \rangle$ be sequences with respective limits x and y . Show that if $x_n \sqsubseteq y_n$ for all n then $x \sqsubseteq y$.
- (2) Show that if $\langle x_n \rangle$ is an increasing sequence then it has limit, $\bigsqcup_n x_n$. Show that if $\langle x_n \rangle$ is a decreasing sequence then it has limit x iff $x = \bigcap_n x_n$.
- (3) Show that if a sequence has limit x then any infinite subsequence has the same limit.
- (4) Suppose $\lim_n x_{mn} = x_m$ for each m and $\lim_m x_m = x$. Then there are functions r, r' such that

$$\lim_m x_{r(m)r'(m)} = x.$$

[Hint Choose r and r' so that for any $1 \leq m$, $b \sqsubseteq x$ iff $b \sqsubseteq x_{r(m)r'(m)}$.]

Because of part (1) of this exercise, $\lim_n x_n$ is determined. Because of (2) and (4) if we put for any $X \subseteq \mathbf{D}$

$$\mathcal{Cl}(X) \stackrel{\text{def}}{=} \{\lim_n x_n \mid x_n \text{ is a sequence of elements of } X \text{ with a limit}\}$$

then $\mathcal{Cl}(X)$ is the least set of elements of $\mathbf{D}$ containing X and closed under limits. Because of (3) we have that $\mathcal{Cl}(X \cup Y) = \mathcal{Cl}(X) \cup \mathcal{Cl}(Y)$ and so we have a *topological* closure operator. This topology is called the *Lawson topology* and the next few exercises discuss it (they will not be used in the main development).

Exercise 76.

- (1) Show that a subbasis of the topology is provided by the sets $P_b = \{x \in \mathbf{D} \mid x \sqsupseteq b\}$ and $N_b = \mathbf{D} \setminus P_b$ ($b \in B_{\mathbf{D}}$).
- (2) Show one can define a *metric* as follows. Let $b_0, b_1, \dots$ be an enumeration of $B_{\mathbf{D}}$. Define

$$d(x, y) = \begin{cases} 0 & (x = y), \\ \frac{1}{1 + \mu n. b_n \in B_x \oplus B_y} & (x \neq y) \end{cases}$$

(where, of course, $X \oplus Y = (X \setminus Y) \cup (Y \setminus X)$).

Indeed show that this is even an *ultrametric* in the sense that the triangle inequality can be strengthened to

$$d(x, z) \leq \max(d(x, y), d(y, z)).$$

Show that $\lim_n x_n = x$ iff this is true in the metric. Show that $\langle x_n \rangle$ is a *Cauchy sequence* iff $\forall m. (\forall_{\infty} n. b_m \sqsubseteq x_n) \vee (\forall_{\infty} n. b_m \not\sqsubseteq x_n)$.

To proceed further we need to know that the Lawson topology is *compact*. What this means in terms of limits is that every sequence has a convergent subsequence. Strangely it turns out that an ω -algebraic cpo has compact Lawson topology iff it obeys 2 out of the 3 conditions needed to make it strongly algebraic!

Lemma 8. *Every sequence $\langle x_m \rangle$ contains a subsequence, $\langle y_m \rangle$, such that for all finite b either $(\forall_{\infty} m. b \sqsubseteq y_m)$ or else $\forall_{\infty} m. b \not\sqsubseteq y_m$.*

Proof. Let $b_0, b_1, \dots$ enumerate the finite elements of $\mathbf{D}$. Put $B_n = \{b_i \mid 0 \leq i \leq n\}$. Say a partition $\langle P, Q \rangle$ of B_n is *good* if $\exists m \geq n. (\forall b \in P. b \sqsubseteq x_m) \wedge (\forall b \in Q. b \not\sqsubseteq x_m)$ and clearly every B_n has a good partition and so by König's Lemma there is an infinite sequence $\langle P_0, Q_0 \rangle, \langle P_1, Q_1 \rangle, \dots$ of good partitions of $B_0, B_1, \dots$ with $P_0 \subseteq P_1 \subseteq \dots$ and $Q_0 \subseteq Q_1 \subseteq \dots$. Now choose a subsequence $\langle y_m \rangle$ of $\langle x_n \rangle$ so that $(\forall b \in P_m. b \sqsubseteq y_m) \wedge (\forall b \in Q_m. b \not\sqsubseteq y_m)$. ■

Recall that in Chapter 6 we defined $U(X)$ as the set of minimal upper bounds of X ; we say $U(X)$ is *complete* if every upper bound of X is greater than or equal to some element of $U(X)$. Recall that if X is in $\mathcal{F}(\mathbf{D})$ then every element of $U(X)$ is finite.

Theorem 6 The 2/3 SFP Theorem. *The Lawson topology on $\mathbf{D}$ is compact iff every $U(\{a, b\})$ with a, b finite is both complete and finite.*

Proof. $\Leftarrow$) Suppose every such $U(\{a, b\})$ is both complete and finite. Let $\langle x_n \rangle$ be a sequence and choose an infinite subsequence $\langle y_m \rangle$ according to Lemma 7. Let $X = \{b \in B_{\mathbf{D}} \mid \forall_{\infty} m. b \sqsubseteq y_m\}$. We claim that X is directed. For if a, b are in X then since $U(\{a, b\})$ is complete we have that almost all y_m are greater than both a and b and hence some minimal upper bound of a and b . Hence as $U(\{a, b\})$ is finite it has an element c with $c \sqsubseteq y_m$ for infinitely many y_m . But then by the construction of $\langle y_m \rangle$ we have $c \sqsubseteq y_m$ for almost all y_m . Thus X is indeed directed and then one easily proves that $\lim_m y_m = \bigsqcup X$ (show this!).

$\Rightarrow$) We know from Exercise 75 that every decreasing sequence has a glb. Now take finite a, b and suppose x is an upper bound of a and b . Let C be $\{c \in B_{\mathbf{D}} \mid x \sqsupseteq c, c \sqsupseteq a, c \sqsupseteq b\}$. Clearly $C \neq \emptyset$. Let $c_0, c_1, \dots$ be an enumeration of C and let $c_{r(n)}$ be defined by $c_{r(0)} = c_0, r(n+1) = \mu m > r(n). c_{r(n)} \sqsupseteq c_m$ if that exists and $r(n)$ otherwise. Then $c = \bigsqcap c_{r(n)}$ exists. By construction c is less than x , a lower bound of C and an upper bound of the finite elements a and b . It is also in $U(\{a, b\})$ as otherwise there is a finite c_m an upper bound of a, b and $\not\sqsubseteq c$ and m is some $r(n)$, so $c \sqsubseteq c_m$ a contradiction. So $U(\{a, b\})$ is complete.

Next suppose $U(\{a, b\})$ is infinite for the sake of contradiction. Let $c_0, c_1, \dots$ be an enumeration without repetitions of an infinite subset of $U(\{a, b\})$. We can assume $\lim_n c_n = x$ exists by compactness. Now since a is less than all the c_n 's we have $a \sqsubseteq x$ and similarly $b \sqsubseteq x$. Therefore some c in $U(\{a, b\})$ is less than or equal to x and so $c \sqsubseteq$ almost all the c_n and choosing $c_m \neq c$ we get $c \sqsubseteq c_m$ contradicting the assumption that c_m is a minimal upper bound of a and b and concluding the proof. ■

Exercise 77.

- (1) Show that the assumption of the 2/3 SFP Theorem are equivalent to saying that $U(A)$ is finite and complete for any finite subset, A , of $B_{\mathbf{D}}$; or equally that the intersection of Scott-compact sets is Scott-compact.
- (2) Show that the metric of Exercise 76 is complete iff the Lawson topology is compact.

From now on we assume the Lawson topology is compact.

Lemma 9. *Let X be a subset of $\mathbf{D}$. Then $\text{Con}(Cl(X))$ is the least convex, Lawson closed set containing X .*

Proof. It is enough to show $Y = \text{Con}(Cl(X))$ Lawson-closed.

Let $\langle y_n \rangle$ be a convergent sequence in Y . Then there are $\langle x_n \rangle, \langle z_n \rangle$ in $Cl(X)$ with $x_n \sqsubseteq y_n \sqsubseteq z_n$. By compactness we can assume $\langle x_n \rangle$ and $\langle z_n \rangle$ to be convergent. Then by Exercise 75(1) $\lim_n x_n \sqsubseteq \lim_n y_n \sqsubseteq \lim_n z_n$ and so $\lim_n y_n$ is in Y showing that Y is Lawson-closed as required. ■

Now we can see how well we can understand $(\cdot)^*$ -closed and $\sqsubseteq_{\text{EM}}$.

Lemma 10. *Let X be Scott-compact and let $\langle x_n \rangle$ be a convergent sequence in X . Then for some y in X , $y \sqsubseteq \lim_n x_n$.*

Proof. Suppose, for the sake of contradiction that no y in X is less than $x \stackrel{\text{def}}{=} \lim_n x_n$. Then for any such y we have a finite $a \sqsubseteq y$ with $a \not\sqsubseteq x$. By compactness, we can assume a finite collection $\{a_1, \dots, a_n\}$ covering X with no $a_i \sqsubseteq x$. But now some a_i is less than infinitely many x_n and so less than x , being the required contradiction. ■

Again the reader will appreciate how compactness replaces König's Lemma if she proves Lemma 10, but restricted to fg sets.

Theorem 7.

- (1) For any $X \subseteq \mathbf{D}$, $X^* = \{y \mid \exists \text{ convergent sequence } x_n \in X, z \in X, z \sqsubseteq y \sqsubseteq \lim x_n\}$.
- (2) For Scott-compact X , $X^* = \text{Con}(Cl(X))$, but not for arbitrary X .
- (3) For Scott-compact Y , $X^* \sqsubseteq_{\text{EM}} Y^*$ iff $X^* \sqsubseteq_{\text{EM}} Y^*$, but not for arbitrary Y .

Proof. (1) Let $Y = \{y \mid \exists \text{ convergent sequence } \langle x_n \rangle \subseteq X, z \in X, z \sqsubseteq y \sqsubseteq \lim_n x_n\}$. Clearly $Y \subseteq X^*$. For the converse suppose $y \in X^*$. There $y \sqsupseteq$ some z in X and taking $b_0 \sqsubseteq b_1 \sqsubseteq \dots$ in B_0 with $\text{lub } y$ we obtain $\langle x_n \rangle$ in X with $b_n \sqsubseteq x_n$; by compactness we can assume the $\langle x_n \rangle$ converges and clearly then $y \sqsubseteq \lim_n x_n$.

(2) Part (1) shows that $X^* \subseteq \text{Con}(Cl(X))$; Lemma 10 and the easily established convexity of X^* shows the converse conclusion. Finally a counterexample for arbitrary X is given by taking $\mathbf{D} = \mathbf{Tapes}$, $X = \{0^n 1^\omega \mid n \in \omega\}$.

(3) Clearly $\sqsubseteq_{\text{EM}}$ is included in $\sqsubseteq_{\text{EM}}$. Conversely suppose $X^* \sqsubseteq_{\text{EM}} Y^*$. If $y \in Y^*$ then by Theorem 4, some x in X^* is less than y . If $x \in X^*$, let $b_0 \sqsubseteq b_1 \sqsubseteq \dots$ be an increasing sequence of finite elements with $\text{lub } x$. Then by Theorem 4 we find $y_0, y_1, \dots$ in Y^* with $a_n \sqsubseteq y_n$. By compactness we can assume the $\langle y_n \rangle$ converges, say to y . By part (2) we have y in Y^* and clearly $x \sqsubseteq y$. For a counterexample for general Y take $\mathbf{D} = \mathbf{Tapes}$, $Y = \{0^n 1^\omega \mid n \in \omega\}$, $X = Y \cup \{0^\omega\}$. ■

Proposition 2. *Every Lawson-closed set is Scott-compact.*

Proof. This follows from general topological principles. For since the Lawson topology is compact any Lawson-closed set is Lawson-compact and so compact also in the coarser Scott topology. Alternatively suppose C is Lawson-closed and is covered by $\{x_0, x_1, \dots\} \subseteq B_{\mathbf{D}}$. If C is not Scott-compact then for any n there is a c_n in C such that $c_n \not\sqsupseteq x_0, \dots, c_n \not\sqsupseteq x_n$. Taking c as the limit of a convergent subsequence of the c_n we find $c \not\sqsupseteq$ no x_n , the required contradiction. ■

Now we know that the nonempty $(\cdot)^*$ -closed Scott-compact sets are just the nonempty convex-closed, Lawson closed sets (use Theorem 7(2) and Lemma 9 for one half on the proof and Proposition 2 and Theorem 7(2) for the other half). But putting this and Theorem 7(3) together with the above characterisation of $\mathcal{P}(\mathbf{D})$ we get another one:

$$\mathcal{P}(\mathbf{D}) = \langle \text{nonempty, convex and Lawson-closed sets, } \sqsubseteq_{\text{EM}} \rangle$$

and this is the clearest picture we can obtain of our sets.

As promised we now show that the X^* are finitely generable.

Lemma 11. *Every nonempty Lawson-closed set is fg (and so every $(\cdot)^*$ -closed Scott-compact nonempty set is too).*

Proof. Let X be a nonempty Lawson-closed set. Let $b_0, \dots, b_n, \dots$ be an enumeration without repetitions of $B_{\mathbf{D}}$ (if $\mathbf{D}$ is finite the whole lemma is trivial). Consider the tree with nodes those nonempty sequences $\langle \langle P_0, Q_0, a_0 \rangle, \dots, \langle P_k, Q_k, a_k \rangle, \dots, \langle P_m, Q_m, a_m \rangle \rangle$ where $\langle P_k, Q_k \rangle$ is a partition of $\{b_i \mid 0 \leq i < k\}$, $a_k \in U(P_k)$ and $a_k \sqsubseteq x$ for some x in X and for all b in Q_k , $b \not\sqsubseteq x$ and furthermore $P_0 \subseteq \dots \subseteq P_k \subseteq \dots$ and $Q_0 \subseteq \dots \subseteq Q_k \subseteq \dots$ and $a_0 \sqsubseteq \dots \sqsubseteq a_k \sqsubseteq \dots$; label such a node by a_m . The nodes are to be ordered by the subsequence ordering (note that the root is $\langle \langle \emptyset, \emptyset, \perp \rangle \rangle$). Now prune off all nodes not dominated by infinitely many nodes. This gives a generating tree for X .

Exercise 78. Prove this! ■

Exercise 79. Improve previous formulae for 2/3 SFP ω -algebraic $\mathbf{D}$.

- (1) Show that for $X, Y \subseteq \mathbf{D}$ we have

$$X \cup_P Y = \text{Con}(X \cup Y).$$

Can the formula: $\mathcal{P}(g)(X) = g(X)^*$ for $g: \mathbf{D} \rightarrow \mathbf{E}$ be improved similarly?

- (2) Define the “big union” function $\bigcup_P: \mathcal{P}(\mathcal{P}(\mathbf{D})) \rightarrow \mathcal{P}(\mathbf{D})$ to be $(id_{\mathcal{P}(\mathbf{D})})^\dagger$. Is it true that for any χ in $\mathcal{P}(\mathcal{P}(\mathbf{D}))$ the formula

$$\bigcup_P \chi = \text{Con}\left(\bigcup_{X \in \chi} X\right)$$

holds?

- (3) Let $X_0 \sqsubseteq X_1 \sqsubseteq \dots$ be an increasing chain in $\mathcal{P}(\mathbf{D})$. Show that

$$\bigsqcup X_n = \{\bigsqcup_n x_n \mid x_n \in X_n, x_0 \sqsubseteq x_1 \sqsubseteq \dots\}^*.$$

Can this be improved to Con , even for X_n finite?

General Powerdomains: The Smyth Powerdomain

Now we consider the general form of the Smyth powerdomain; this will be dealt with rather briefly, leaving many points as exercises. The idea here is that the *more* elements a set contains, the *smaller* it should be in the order — just the dual of the relational case. In terms of semilattices we have the following natural observations:

Exercise 80. Let $\langle \mathbf{D}, \sqsubseteq, \cup \rangle$ be a continuous semilattice. Show that the following statements are equivalent:

- (1) $\forall x, y. x \sqsubseteq y \Rightarrow x \sqsupseteq y$,
- (2) $\forall x, y. x \sqsubseteq y \Leftrightarrow x \sqsupseteq y$,
- (3) $\forall x, y. x \cup y \sqsubseteq x$,
- (4) $\forall x, y. x \cup y = x \sqcap y$

and all imply that $x \cup \perp = \perp$ (but not conversely).

Thus we have to do with cpos with a continuous meet operation and the linear maps are the multiplication ones.

As an order on sets one naturally defines the appropriate half of the Egli-Milner order for any cpo $\mathbf{D}$.

$$X \sqsubseteq_S Y \text{ iff } \forall y \in Y. \exists x \in X. x \sqsubseteq y$$

(and the associated equivalence is $=_S$). (Note that X covers C just means that $X \sqsubseteq_S C$.) The contextual variant is

$$X \sqsubseteq_S Y \text{ iff } \forall f: \mathbf{D} \rightarrow \mathbf{O}. f(X) \sqsubseteq_S f(Y)$$

(and the associated equivalence is $\simeq_S$). There is no controversy over choice of order since:

Exercise 81. Show that $\sqsubseteq_S$ and $\sqsubseteq_S$ are identical. Show that when $\mathbf{D}$ is ω -algebraic, $X \sqsubseteq_S Y$ implies that:

$$\forall A \subseteq B_{\mathbf{D}}. (A \text{ a finite subset of } B_{\mathbf{D}} \wedge A \sqsubseteq_S X) \Rightarrow A \sqsubseteq_S Y$$

and the converse holds if X is finitely generable (or, more generally, Scott compact). [Plunder Theorem 4.]

For a quick construction of the Smyth powerdomain for an ω -algebraic $\mathbf{D}$, put:

$$\mathcal{F}_S(\mathbf{D}) = \langle \{\text{finite nonempty subsets of } B_{\mathbf{D}}\}, \sqsubseteq_S \rangle.$$

This is a preorder with least element $\{\perp\}$. Put

$$\begin{aligned} \mathcal{P}_S(\mathbf{D}) &= \overline{\mathcal{F}_S(\mathbf{D})}, \\ \{d\}_S &= \{\{b_1, \dots, b_n\} \mid \exists i. b_i \sqsubseteq d\}, \\ I \cup_S J &= \{A \cup B \mid A \in I, B \in J\}. \end{aligned}$$

You should verify that $\{\cdot\}_S$ and $\cup_S$ are well-defined and continuous and that $\{b\}_S = [\{b\}]$ for b in $B_{\mathbf{D}}$ and $[A] \cup_S [B] = [A \cup B]$ for A, B in $\mathcal{F}_S(\mathbf{D})$. Clearly too $\langle \mathcal{P}_S(\mathbf{D}), \cup_S \rangle$ is a semilattice and you should verify that $\cup_S$ is the meet operation on the cpo $\mathcal{P}_S(\mathbf{D})$, as $I \cup_S J = I \cap J$.

Exercise 82. Obtain the following universal characterisation of $\mathcal{P}_S(\mathbf{D})$ for ω -algebraic $\mathbf{D}$:

If $f: \mathbf{D} \rightarrow \mathbf{P}$ is any (strict) continuous function to a cpo $\mathbf{P}$ with a continuous meet then there is a unique (strict) multiplicative continuous function $f^\dagger: \mathcal{P}_S(\mathbf{D}) \rightarrow \mathbf{P}$ such that the following diagram commute:

$$\begin{array}{ccc} \mathbf{D} & & \\ \downarrow \{\cdot\}_S & \searrow f & \\ \mathcal{P}_S(\mathbf{D}) & \xrightarrow{f^\dagger} & \mathbf{P} \end{array}$$

Show further that $(\cdot)^\dagger$ is an isomorphism of the continuous semilattice of (strict) continuous $f: \mathbf{D} \rightarrow \mathbf{P}$ and the continuous semilattice of (strict) multiplicative continuous $g: \mathcal{P}_S(\mathbf{D}) \rightarrow \mathbf{P}$. [Plunder Theorem 3.]

This exercise indicates that we can use relations $t \sqsubseteq u$ as well as equalities $t = u$ when constructing free continuous algebras, since the Smyth case depends on the relation $x \cup y \sqsubseteq x$ (note, as a joke, that this is inconsistent with the relational powerdomain requirement that $x \cup y \sqsupseteq x$ as then we would have $x = y$).

Anyway, now we can view $\mathcal{P}_S$ as a locally continuous functor over $\omega\text{-}\mathcal{ALG}$ putting:

$$\mathcal{P}_S(f) = (\{\cdot\} \circ f)^\dagger.$$

The usual arguments then show that $\mathcal{P}_S$ is a functor over $\mathcal{SFP}$ and so we can solve domain equations involving all the functors we know. In fact, however, as long as we don't want $\mathcal{P}_{\mathbf{P}}$ too we can work with consistent completeness as that is preserved by $\mathcal{P}_S$; indeed as the next exercise demonstrates the situation is somewhat curious:

Exercise 83. Let $\mathbf{D}$ be ω -algebraic. Show that $\mathcal{P}_S(\mathbf{D})$ is consistently complete iff $\mathbf{D}$ is 2/3 SFP, demonstrating the formula

$$[A] \cup [B] = [\bigcup_{a \in A} \bigcup_{b \in B} U(\{a, b\})]$$

which should be interpreted as implying that the left-hand-side exists when the set on the right is nonempty.

We now look for nicer description of $\mathcal{P}_S(\mathbf{D})$ as we did for the Egli-Milner case. From now on we fix an ω -algebraic $\mathbf{D}$.

Lemma 12. Let $f: \mathbf{Tapes} \rightarrow \mathbf{D}$ generate X . If A is a finite subset of $B_{\mathbf{D}}$ such that $A \sqsubseteq_S X$ then for some n , $A \sqsubseteq_S Bd_n(f)$. Further if $Bd_n(f) \sqsubseteq_S Y$ for all n then $X \sqsubseteq_S Y$ (any $Y \subseteq \mathbf{D}$).

Proof. **Exercise 84.** [Plunder Lemma 4.] ■

Lemma 13. Let $A_0 \sqsubseteq_S A_1 \sqsubseteq_S \dots$ be an increasing sequence in $\mathcal{P}_S(\mathbf{D})$. Then there is an $f: \mathbf{Tapes} \rightarrow \mathbf{D}$ with every A_n being $\sqsubseteq_S$ some $Bd_n(f)$.

Proof. **Exercise 85.** [Plunder Lemma 5.] ■

It follows much as before that

$$\mathcal{P}_S(\mathbf{D}) = \langle \mathcal{F}_g(\mathbf{D}), \sqsubseteq_S \rangle / \sqsubseteq_S$$

and the finite elements have the form $A \downarrow$ for A in $\mathcal{P}_S(\mathbf{D})$.

Passing on to canonical representatives for any $X \subseteq \mathbf{D}$ we define:

$$X^\dagger = \{y \mid \exists x \in X. y \sqsupseteq x\}$$

and note that:

- (1) $X \subseteq X^\dagger$,
- (2) $X^{\dagger\dagger} = X^\dagger$,
- (3) $(X \cup Y)^\dagger = X^\dagger \cup Y^\dagger$,
- (4) $X =_S Y \iff X \simeq_S Y \iff X^\dagger = Y^\dagger$,
- (5) $X \sqsubseteq_S Y \iff X \sqsubset_S Y \iff X^\dagger \supseteq Y^\dagger$.

Thus $(\cdot)^\dagger$ is a *topological* closure operation (the topology is called the *lower* topology as the open sets are the downwards closed ones). Also $X^\dagger$ is the maximal member of the equivalence class of X and $\sqsubseteq_S$ is just $\supseteq$ on these maximal members. It would now be nice to know that $X^\dagger$ is fg if X is; however this too is an open problem and we consider the Scott compact subsets again.

Lemma 14. *Let $C \subseteq \mathbf{D}$ be nonempty and Scott-compact. Then $I_S(C) \stackrel{\text{def}}{=} \{A \in \mathcal{F}_S(\mathbf{D}) \mid A \sqsubseteq_S C\}$ is a directed ideal and has $\sqsubseteq_S$ -lub C .*

Proof. Exercise 86. [Plunder Lemma 7.] ■

Together with Theorem 5 above this establishes that

$$\mathcal{P}_S(\mathbf{D}) = \langle \text{nonempty upper-closed Scott compact sets}, \supseteq \rangle$$

which is not too bad a characterisation of the powerdomain. When $\mathbf{D}$ is 2/3 SFP we know, of course, that the nonempty upper-closed Scott-compact sets are exactly the nonempty upper-closed, Lawson-closed sets and they in turn are just the upper-closed finitely generable sets.

Exercise 87. Prove these last few assertions!

Exercise 88. Establish the following formulae for $x \in \mathbf{D}$, $X, Y \in \mathcal{P}_S(\mathbf{D})$, $f: \mathbf{D} \rightarrow \mathcal{P}_S(\mathbf{E})$ and $g: \mathbf{D} \rightarrow \mathbf{E}$:

$$\begin{aligned} \{x\}_S &= \{y \in \mathbf{D} \mid y \sqsupseteq x\}, \\ X \cup_S Y &= X \cup Y, \\ f^\dagger(X) &= \left(\bigcup_{x \in X} f(x) \right). \end{aligned}$$

Show that if $X_0 \supseteq X_1 \supseteq \dots$ is an increasing chain in $\mathcal{P}_S(\mathbf{D})$ then:

$$\bigsqcup_S X = \bigcap_n X_n.$$

[Hint Consider $Y = \bigsqcup_S X_n$, which must exist as $\mathcal{P}_S(\mathbf{D})$ is a cpo.]

Duality

It was realised by M. Smyth that the duality theory connecting Smyth powerdomain and Dijkstra's predicate transformers can be carried far beyond the simple case of discrete cpos to arbitrary ω -algebraic cpos (for example).

Definition 12. Let $\mathbf{D}$ and $\mathbf{E}$ be ω -algebraic. A *predicate transformer* from $\mathbf{E}$ to $\mathbf{D}$ is any strict continuous and multiplicative map

$$p: \mathcal{O}(\mathbf{E}) \rightarrow \mathcal{O}(\mathbf{D})$$

Exercise 89. Suppose $m: \mathbf{D} \rightarrow \mathcal{P}_S(\mathbf{E})$. Define the weakest precondition function by putting for any open V of $\mathbf{E}$:

$$wp(m, V) = \{x \in \mathbf{D} \mid m(x) \subseteq V\}.$$

Show that $\Omega(m) \stackrel{\text{def}}{=} \lambda V. wp(m, V)$ is a predicate transformer. [Hint Use Scott-compactness.] Show too that Ω is monotonic and that the following properties hold:

$$\begin{aligned} \Omega(\{\cdot\}_S) &= id, \\ \Omega(m; m') &= \Omega(m) \circ \Omega(m'), \\ \Omega(m \cup_S m') &= \Omega(m) \cap \Omega(m') \end{aligned}$$

(where, of course, $m; m' = m^\dagger \circ m$ and the other new operations are defined pointwise).

Exercise 90. Let $\mathcal{ST}(\mathbf{D}, \mathbf{E})$ be the partial order $\mathbf{D} \rightarrow \mathcal{P}_S(\mathbf{E})$ of continuous functions from $\mathbf{D}$ to $\mathcal{P}_S(\mathbf{E})$. Let $\mathcal{PT}(\mathbf{E}, \mathbf{D})$ be the partial order of predicate transformers with the pointwise ordering. Show that

$$\Omega: \mathcal{ST}(\mathbf{D}, \mathbf{E}) = \mathcal{PT}(\mathbf{E}, \mathbf{D})$$

is an isomorphism of partial orders with inverse given by

$$\Omega^{-1}(p)(x) = \bigcap \{V \in \mathcal{O}(\mathbf{E}) \mid x \in p(V)\}.$$

[Hint Show first that $\{A^\dagger \mid A \in \mathcal{F}_S(\mathbf{D}) \wedge x \in p(A^\dagger)\}$ is $\sqsubseteq_S$ -directed and has lub $\Omega^{-1}(p)(x)$.]

Now show that Ω is an isomorphism of continuous semilattices and can be viewed as a duality of categories:

$$\Omega: \mathcal{ST}^{op} \cong \mathcal{PT}.$$

It is to be hoped that this duality will (together with that of Exercise 33 above) lay the foundations for a systematic investigation of the connection between denotational and axiomatic semantics. This however is a substantial programme of investigation which largely remains to be undertaken. This is further underlined by:

Exercise 91. Let $\mathcal{DPT}(\mathbf{E}, \mathbf{D})$ be the additive transformers. Show that Ω cuts down to an isomorphism

$$\Omega: \omega\text{-}\mathcal{ALG} \cong \mathcal{DPT}$$

of categories.

Exercise 92. The above does not yet generalise the case of a flat cpo since $\mathcal{ST}(X, Y)$ corresponds to $X_\perp \rightarrow_\perp \mathcal{P}_S(Y_\perp)$ rather than $X_\perp \rightarrow \mathcal{P}_S(Y_\perp)$. So put $\mathcal{ST}_\perp(\mathbf{D}, \mathbf{E}) = \mathbf{D} \rightarrow_\perp \mathcal{P}_S(\mathbf{E})$ and find an appropriate *strict* version of the above exercises.

One can object that the above treatment of the duality is incomplete since the objects of $\mathcal{PT}$ should really be the complete lattices $\mathcal{O}(\mathbf{D})$ rather than the $\mathbf{D}$ themselves, and we do not yet know what complete lattices we are talking about. We now fill in this gap.

Definition 13. Let P be a partial order. An element x of P is an ω -prime (prime) iff whenever $x \sqsubseteq \bigsqcup Y$ where Y is a countable (finite) nonempty set then $x \sqsubseteq$ some element of Y . (Note that $\perp$ is not a prime!) Then P is said to be ω -prime algebraic if every element of P is the lub of the ω -primes less than it and there are ω such ω -primes altogether.

Exercise 93. Show that not all ω -algebraic cpos are ω -prime algebraic and neither are all ω -prime algebraic cpos ω -algebraic.

But, for example, all ω -prime algebraic consistently complete cpos are ω -algebraic and then the ω -prime are exactly the finite primes and every finite element is a finite lub of ω -primes. Furthermore the ω -prime algebraic consistently complete cpos are *completely distributive* meaning that:

$$x \sqcap (\bigsqcup Y) = \bigsqcup_{y \in Y} (x \sqcap y)$$

in that when $\bigsqcup Y$ exists then so does the right-hand side and then both sides are equal. Finally any ω -algebraic consistently complete cpo in which every finite element is a lub of primes is ω -prime algebraic.

Exercise 94.

- (1) Show that if $\mathbf{D}$ is ω -algebraic then $\mathcal{O}(\mathbf{D})$ is an ω -prime algebraic complete lattice with ω -prime top element.
- (2) Let L be an ω -prime algebraic complete lattice with ω -prime top element. Define

$$Pr(L) = \langle \omega\text{-primes of } L, \sqsubseteq \rangle$$

and prove that:

- (a) $\mathbf{D} \cong \overline{Pr(\mathcal{O}(\mathbf{D}))^{op}}$,
- (b) $L \cong \mathcal{O}(\overline{Pr(L)^{op}})$

(where for any preorder $\langle P, \sqsubseteq \rangle$, P^{op} is $\langle P, \supseteq \rangle$ and the ideal completion of Chapter 6 is intended).

- (3) Reformulate Exercise 89 in terms of an *equivalence* between the categories $\mathcal{ST}^{op}$ and the category of ω -prime algebraic complete lattices with ω -prime top elements and with morphisms the strict, continuous, additive functions. (We also call this a duality result.)

Exercise 95. Show that the duality of Exercise 94 cuts down to one of the 2/3 $\mathcal{SFP}$ objects and the ω -prime algebraic, ω -arithmetic complete lattices with ω -prime top element. (An ω -arithmetic complete lattice is one which is ω -algebraic and in which the glb of any two finite elements is finite.)

Exercise 96. Show that the duality of Exercise 94 cuts down to one of the consistently complete ω -algebraic cpos and the ω -prime algebraic, complete lattices in which the meet of any two ω -primes is an ω -prime.

We should also mention that much of the above has been inspired by the voluminous

A Compendium of Continuous Lattices by G. Gierz, J. D. Lawson, K. H. Hofmann, M. Mislove, K. Keimel, D. S. Scott.

which contains a great deal of further potentially computationally relevant material (Open Problem!!).

The Vietoris Topology

As long ago as 1921 Vietoris investigated nondeterministic functions in a topological setting. That this amounts to an alternative treatment of powerdomains was realised by M. Smyth and this includes predicate transformers both of the weakest precondition and (the dual of) the weakest liberal precondition variety. We now indicate some of the connections.

Definition 14.

- (1) Let X and Y be sets and suppose $f: X \rightarrow \mathcal{P}(Y)$ is a multi-function from X to Y (i.e. from X to the collection of all subsets of Y). Then the *strong inverse* of f is $\Omega(f): \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$ where:

$$\Omega(f)(B) = \{a \in X \mid f(a) \subseteq B\}$$

and the *weak inverse* of f is $\Lambda(f): \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$ where:

$$\Lambda(f)(B) = \{a \in X \mid f(a) \cap B \neq \emptyset\}.$$

- (2) Let X and Y be topological spaces in the above. Then f is *upper semi-continuous* if $\Omega(f)(V)$ is open when V is; it is *lower semi-continuous* if $\Lambda(f)(V)$ is open when V is; it is *continuous* if it is both upper and lower semi-continuous.

Definition 15. Let X be a topological space. We define three topologies on $\mathcal{P}(X)$.

- (1) The *upper topology* has basis $\{U_O \mid O \text{ is open in } X\}$ where $U_O \stackrel{\text{def}}{=} \{A \subseteq X \mid A \subseteq O\}$. Note that $U_O \cap U_{O'} = U_{O \cap O'}$.
- (2) The *lower topology* has subbasis $\{L_O \mid O \text{ is open in } X\}$ where $L_O \stackrel{\text{def}}{=} \{A \subseteq X \mid O \cap A \neq \emptyset\}$.
- (3) The *Vietoris topology* is the common refinement of the lower and upper topologies. It has a basis $\{V_{O_1, \dots, O_n} \mid O_1, \dots, O_n \text{ are open in } X\} \cup \{\emptyset\}$ where

$$V_{O_1, \dots, O_n} = \{A \subseteq X \mid A \subseteq \bigcup_i O_i, \forall i. A \cap O_i \neq \emptyset\}$$

(where a *basis* for a topology is a collection of open sets such that every set is a union of basis sets; it need not be closed under intersections).

Exercise 97. Prove the above does define a basis for the Vietoris topology. Show the only topology contained in both the upper and lower topologies is the trivial one.

These topologies *classify* the various kinds of continuity possible for multi-functions in the following sense:

Exercise 98. Show that for a multi-function $f: X \rightarrow \mathcal{P}(X)$

- (1) f is upper semi-continuous iff it is continuous (in the usual topological sense) relative to the upper topology.
- (2) f is lower semi-continuous iff it is continuous relative to the lower topology.
- (3) f is continuous iff it is continuous relative to the Vietoris topology.

The standard connection between topologies and preorders is given by:

Definition 16. Let X be a topological space. A preorder is defined on X by putting for all x, y in X :

$$x \sqsubseteq y \quad \text{iff} \quad \forall \text{ open } V. x \in V \Rightarrow y \in V.$$

The preorders defined on $\mathcal{P}(X)$ relative to the upper, lower, Vietoris topologies will be denoted by $\sqsubseteq_U, \sqsubseteq_L, \sqsubseteq_V$.

Recall that in a cpo we recover the original partial order in the above way from the Scott topology. Now here is the connection between Vietoris' ideas and ours:

Exercise 99. Let $\mathbf{D}$ be a cpo. Let $\sqsubseteq_S, \sqsubseteq_R, \sqsubseteq_{EM}$ be defined as above on the collection of $\mathcal{P}(\mathbf{D})$ of all subsets of $\mathbf{D}$ removing any restriction to nonempty sets or ω -algebraic cpos. Regarding $\mathbf{D}$ as also being a topological space via the Scott topology show that:

- (1) $\sqsubseteq_U = \sqsubseteq_S$,
- (2) $\sqsubseteq_L = \sqsubseteq_R$,
- (3) $\sqsubseteq_V = \sqsubseteq_{EM}$.

Perhaps we should speak of the Egli-Milner-Vietoris ordering!

Just as we did with cpos, one normally works with the various topologies on $\mathcal{P}(X)$ by concentrating on particular kinds of sets e.g. closed or compact. For example metric spaces have been proposed by Nivat and his co-workers as a substitute for cpos e.g.

G. Comyn, M. Dauchet. Approximations of Infinitary Objects. ICALP '82

and de Bakker and Zucker have used Vietoris topologies on compact subsets of metric spaces as a substitute for powerdomain in

Denotational Semantics of Concurrency. SIGACT, 1982.

We now make some connection with our powerdomains. We already know that, for example, $\mathcal{P}_R(\mathbf{D})$ is the partial order on the Scott-closed subsets of $\mathbf{D}$ derived from the lower topology. So in the sense we can derive our powerdomain from Vietoris' ideas applied to ω -algebraic $\mathbf{D}$'s and a clever choice of $\mathbf{D}$'s. The converse also holds:

Exercise 100. Show that the Scott topology on $\mathcal{P}_S(\mathbf{D}), \mathcal{P}_R(\mathbf{D}), \mathcal{P}_P(\mathbf{D})$ are just the restrictions (to appropriate kinds of subsets) of the upper, lower and Vietoris topologies.

Exercises

Finally here are some exercises on miscellaneous topics.

Exercise 101 The Empty Set. This is a variation on powerdomains where one asks for an empty set, considering semilattices with a constant $\emptyset$ satisfying the law:

$$\emptyset \cup x = x$$

and considering linear maps preserving $\emptyset$. Clearly in semilattices satisfying the “relational inequality”: $x \cup y \sqsupseteq x$, we must have $\emptyset = \perp$ and $\mathcal{P}_R(\mathbf{D})$ is the powerdomain we want, as shown by Theorem 2 restricted to strict functions, noting that here the strict linear maps also preserve $\emptyset$.

- (1) **The Plotkin Powerdomain.** The situation here is a little curious. First show that for any ω -algebraic $\mathbf{D} \xrightarrow{\text{inro}} \{\emptyset\}_\perp + \mathcal{P}_P(\mathbf{D})$ is a strict continuous function such that any strict continuous $f: \mathbf{D} \rightarrow \mathbf{A}$ (where $\mathbf{A}$ is a continuous semilattice with $\emptyset$) has a unique strict, linear, continuous extension $f^\dagger: \{\emptyset\}_\perp + \mathcal{P}_P(\mathbf{D}) \rightarrow \mathbf{A}$. Second show that there is *no* continuous $\varepsilon: \mathbf{D} \rightarrow \mathbf{P}$ (a continuous semilattice with $\emptyset$) such that any other continuous $f: \mathbf{D} \rightarrow \mathbf{A}$ (a continuous semilattice with $\emptyset$) has a unique linear continuous extension $f^\dagger: \mathbf{P} \rightarrow \mathbf{A}$. [Hint Begin by proving that if it exists $\mathbf{P}$ must be $\{\emptyset\}_\perp + \{1\}_\perp$ when $\mathbf{D} = \{1\}_\perp$.] You should also see why the natural candidate $\{\emptyset\}_\perp + \mathcal{P}_P(\mathbf{D})_\perp$ does not work.
- (2) **The Smyth Powerdomain.** Here everything goes smoothly. In semilattices satisfying “Smyth inequality”: $x \cup y \sqsubseteq x$ we clearly have $\emptyset$ as the top element (since $x = \emptyset \cup x \sqsubseteq \emptyset$). For any cpo $\mathbf{D}$ let $\mathbf{D}_\top$ be $\mathbf{D}$ with a top element added and let $\mathbf{D} \rightarrow \mathbf{D}_\top$ be the natural (anonymous) inclusion.

Show that for any ω -algebraic $\mathbf{D} \xrightarrow{\{\cdot\}_S} \mathcal{P}_S(\mathbf{D}) \rightarrow \mathcal{P}_S(\mathbf{D})_\top$ is a (strict) continuous function such that any (strict) continuous $f: \mathbf{D} \rightarrow \mathbf{A}$ (where $\mathbf{A}$ is a cpo with a continuous meet and a top element) has a unique (strict) continuous, multiplicative, top-preserving extension $f^\dagger: \mathcal{P}_S(\mathbf{D})_\top \rightarrow \mathbf{A}$.

Exercise 102 Bistrict Union. This is a variation where we ask for union to be *bistrict* so that

$$x \cup \perp = \perp \cup x = \perp.$$

Clearly this is inconsistent with the relational cases, as then $\perp = x \cup \perp = x$. Also it is implied in the Smyth case as $x \cup \perp = x \sqcap \perp = \perp$. So only the Egli-Milner case remains and variations with and without the empty set are possible.

- (1) Show that for any ω -algebraic $\mathbf{D}$ there is a strict continuous $\varepsilon: \mathbf{D} \rightarrow \mathbf{P}$ (a continuous semilattice with bistrict union) such that any strict continuous $f: \mathbf{D} \rightarrow \mathbf{A}$ (a continuous semilattice with bistrict union) has a unique strict continuous linear extension $f^\dagger: P \rightarrow \mathbf{A}$. Note what $\mathbf{P}$ is when $\mathbf{D}$ is flat.

Show that all of this becomes false if the strictness qualifications are omitted.

[Hint Consider $\mathbf{D} = \mathbb{P}$ and prove that if it exists then $P = \mathbb{P}$.]

- (2) Show that for any ω -algebraic $\mathbf{D}$ there is a strict continuous $\varepsilon: \mathbf{D} \rightarrow \mathbf{P}$ (a continuous semilattice with $\emptyset$ and bistrict union) such that any strict continuous $f: \mathbf{D} \rightarrow \mathbf{A}$ (a continuous lattice with $\emptyset$ and bistrict union) has a unique strict continuous linear extension $f^\dagger: P \rightarrow \mathbf{A}$. And show all of this becomes false if the strictness condition is removed.

Exercise 103 Multisets — an Open Problem. Investigate multiset constructions $\mathcal{M}(\mathbf{D})$ where the appropriate laws are:

Associativity

$$x \cup (y \cup z) = (x \cup y) \cup z,$$

Commutativity

$$x \cup y = y \cup x$$

and one is considering $\mathcal{M}(\mathbf{D})$ as a free abelian semigroup. In this setting the convexity phenomena in powerdomains do not arise and the theory may well be very pleasant. This should be an interesting and straightforward problem.

Exercise 104 Cartesian Products and Multilinear Functions. Show the Cartesian product gives a continuous bilinear (= linear in each argument) function $\times: \mathcal{P}_P(\mathbf{D}) \times \mathcal{P}_P(\mathbf{E}) \rightarrow \mathcal{P}_P(\mathbf{D} \times \mathbf{E})$ and that satisfies the formula

$$\{d\}_P \times \{e\}_P = \{ \langle d, e \rangle \}_P.$$

[Hint Establish that the product of fg sets is fg and that the formula $X^* \times Y^* = (X \times Y)^*$ holds for arbitrary sets.] Now show that this classifies the bilinear continuous functions in that to any such $f: \mathcal{P}_P(\mathbf{D}) \times \mathcal{P}_P(\mathbf{E}) \rightarrow \mathbf{A}$ such that the following diagram commutes:

$$\begin{array}{ccc} \mathcal{P}_P(\mathbf{D}) \times \mathcal{P}_P(\mathbf{E}) & & \\ \downarrow \times & \searrow f & \\ \mathcal{P}_P(\mathbf{D} \times \mathbf{E}) & \xrightarrow{g} & \mathbf{A} \end{array}$$

Also conclude that any continuous $f: \mathbf{D} \times \mathbf{E} \rightarrow \mathbf{A}$ has a unique bilinear continuous extension $f^\dagger: \mathcal{P}_P(\mathbf{D}) \times \mathcal{P}_P(\mathbf{E}) \rightarrow \mathbf{A}$.

Now carry this exercise on the Smyth powerdomain and redo Exercise 34 in the same style.

Exercise 105 Collapsing Morphisms. Find formulae for the natural functions $\alpha_S: \mathcal{P}_P(\mathbf{D}) \rightarrow \mathcal{P}_S(\mathbf{D})$ and $\alpha_R: \mathcal{P}_P(\mathbf{D}) \rightarrow \mathcal{P}_R(\mathbf{D})$ (the extensions of $\{\cdot\}_S, \{\cdot\}_R$ to strict linear function on $\mathcal{P}_P(\mathbf{D})$). Note that α_S and α_R are onto. Show that there are no such extensions $\beta_S: \mathcal{P}_S(\mathbf{D}) \rightarrow \mathcal{P}_P(\mathbf{D})$, $\beta_R: \mathcal{P}_R(\mathbf{D}) \rightarrow \mathcal{P}_P(\mathbf{D})$ of $\{\cdot\}_P$.

Exercise 106 Computability. Show that $\mathcal{P}_S$ is a computable functor over the effectively given domains of Chapter 7 and show along the way that all associated functions are computable including Cartesian product. The same can be done for $\mathcal{P}_P$ but first one must set up a computability theory for $\mathcal{SFP}$. This has been done by A. Kanda.

[¶] Editorial Note: These seem to be intentionally left blank by the author.

Exercise 107 Arbitrary CPOs. It is possible to find powerdomains of *arbitrary* cpos. For example let $\mathbf{D}$ be a cpo. Let P be the preorder given by:

$$P = \langle \text{finite nonempty subsets of } \mathbf{D}, \sqsubseteq_{\text{EM}} \rangle.$$

Then we can take $\mathcal{P}(\mathbf{D}) = \tilde{P}$ the completion keeping existing lubs given in Exercise 7, p. 63, Chapter 6. Show that in this way one obtains the free continuous semilattice over $\mathbf{D}$. This would have provided another approach to powerdomains that would have been much more general but would have been even more abstract than the approach adopted.